Reading Guide: LLM vs SAST vs DL for Vulnerability Detection

LLM-based vs Traditional Static Analysis and Deep Learning Approaches for Source Code Vulnerability Detection: A Comparative Review (24 papers, 2026-11-12)

Generated 2026-11-12

Overview

What this research is about

Software often contains security bugs. A security bug that an attacker can use is called a **vulnerability**. Examples are SQL injection (an attacker smuggles database commands into an input field) and buffer overflows (a program writes past the end of a memory area).

There are three main ways to find vulnerabilities in source code without running it:

- **SAST tools.** SAST means static application security testing. These are programs such as CodeQL, Semgrep or Flawfinder. They scan code with hand-written rules.
- **Deep-learning (DL) detectors.** These are neural networks, such as LineVul or a fine-tuned CodeBERT, that learn from many examples of code labelled "vulnerable" or "safe".
- **Large language models (LLMs).** These are general AI models such as GPT-4, Claude or Llama. Since about 2023 people simply ask them whether a piece of code is vulnerable, or let them act as "agents" that explore a whole project.

A fourth group, **hybrids**, combines an LLM with a SAST tool. For example, the LLM can tell the SAST tool what to look for, or it can check the SAST tool's warnings and throw away false ones.

Published studies disagree about which approach works best. This project collected 24 papers and compared their evidence side by side. The goal was to find out which approach finds more real vulnerabilities, with fewer false alarms, at what cost, and how far the published numbers can be trusted.

The five research questions in plain words

A **research question (RQ)** is the exact question a review tries to answer. This project has five.

1. **RQ1. Who wins?** When LLMs, SAST tools and DL detectors are tested on the same code, which one finds more vulnerabilities with fewer false alarms?
2. **RQ2. Why do studies disagree?** How do the test setups differ (test data, programming languages, vulnerability types, real or artificial code, data leaks), and do these differences explain the disagreements?
3. **RQ3. Do hybrids help?** Does combining an LLM with a SAST tool beat either one alone, and what does it cost in money, time and explainability?
4. **RQ4. Where does each approach work?** Which vulnerability types, programming languages and code sizes suit each approach?
5. **RQ5. What is still missing?** Which open problems block reliable, deployable LLM-based vulnerability detection?

How to use this guide

The guide has five parts:

1. This **Overview**: the main findings, a map of all 24 papers, five papers to read first, and where studies disagree.
2. **Paper Summaries**: one brief of one to two pages per paper, in the same order as the paper map below. Each brief starts with a one-sentence takeaway.
3. **Research Gaps and Opportunities**: six open problems that a new study could address, each with a study a beginner could run.
4. **Glossary**: plain-language definitions of more than 30 terms.
5. **Appendices**: the full research report, the methodology comparison and the evidence matrix, for readers who want every detail.

Evidence labels. Every important statement carries a label:

- **[V] Verified**: we checked it against the paper's own tables or results.
- **[A] Author claim**: the authors say it, but we cannot check it ourselves.
- **[I] Interpretation**: our own reasoning across papers. Treat it as an informed opinion.

Numbers without a label are **[V]** from the cited page.

Citations. A citation looks like [deininger2026open, p. 8]. The part after the @ is the paper's key: first author's surname, year and the first word of the title. "p. 8" is the page of the PDF file we read, which can differ from the page number printed in the journal.

Evidence strength. Strong means several independent studies agree and their designs are sound. Moderate means a sound design but one study or few data. Weak means small or unclear samples, design problems, or unreplicated preprints.

Peer review and preprints. A **peer-reviewed** paper was checked by independent experts before publication. A **preprint** was posted online without that check. Points to keep in mind:

- zhou2024comparison is a **preprint, not peer-reviewed** [zhou2024comparison, p. 1].
- lu2025lab is published in the Journal of Systems and Software (2026). Our copy is the arXiv accepted manuscript, so its page numbers follow that copy [lu2025lab, p. 1].
- li2024iris (IRIS) and guo2025repoaudit (RepoAudit) appear as arXiv preprints in our database because their conferences issue no DOI. They are the peer-reviewed ICLR 2025 and ICML 2025 papers [li2024iris, p. 1; guo2025repoaudit, p. 1].
- ahmed2026secvuleval is published at the AIware 2026 conference. Our copy is its arXiv version, which has a different title ("Benchmarking LLMs for Real-World C/C++ Vulnerability Detection"). We confirmed by hand that the authors match.
- khare2025understanding is the ICST 2025 paper. Our metadata check flags a year mismatch only because its arXiv version dates from 2023.
- qin2026review is a single-author narrative review in a low-visibility venue that does not document peer review [qin2026review, p. 1]. We treat it as weak and use it only to show how our review differs from it.
- Most other papers were read from arXiv copies of the published version. Numbers in the final published version may differ slightly.

Naming of lists. Three numbered lists appear in this guide. They come from different project files, so we renamed them to avoid confusion:

- **Agreement 1 to Agreement 11** are points where several studies agree. In the evidence matrix (Appendix C) they are called G1 to G11.

- **Contradiction C1 to C8** are points where studies disagree. They keep their names from the evidence matrix.
- **Gap 1 to Gap 6** are the research gaps in the back part of this guide. In the project's gap report they are also called G1 to G6, but they are different from the agreements.
- Weaknesses W1 to W11 refer to the methodology comparison (Appendix B).

Five terms you need first

- **Precision**: of all the warnings a tool raises, the share that are real vulnerabilities. Low precision means many false alarms.
- **Recall**: of all the real vulnerabilities, the share the tool finds. Low recall means many missed bugs.
- **F1**: a single score that combines precision and recall. It is high only when both are high.
- **Prevalence**: the share of vulnerable samples in the test data. Real code has few vulnerable functions, roughly 2% in two of our studies [deininger2026open, p. 8; lu2025lab, p. 4]. Many test sets are instead **balanced** (50% vulnerable).
- **Trivial baseline**: a "detector" that ignores the code, for example one that flags everything as vulnerable ("flag-all"). On a balanced test set, flag-all already reaches an F1 of 0.667. A real detector should clearly beat such baselines **[I]**.

Main findings per research question

RQ1. Who wins? No approach wins everywhere

Evidence strength: weak to moderate.

- **On small, self-contained code, frontier LLMs beat SAST.** On 10 small C# projects, LLMs reached a mean F1 of 0.75–0.80 against 0.26–0.55 for SAST tools [gnieciak2025large, p. 10]. On 250 tiny C programs, the best LLM scored 977 against 661 for the best non-LLM tool [dubniczky2025castle, p. 11]. That first study, however, counted LLM findings as correct even when they pointed to the wrong line, a leniency it did not give SAST tools [gnieciak2025large, p. 6].
- **On artificial test suites, CodeQL beats LLMs.** CodeQL had higher accuracy than the best LLM on all three synthetic datasets in one study [khare2025understanding, p. 9].
- **On whole projects, LLM agents find more but warn too often.** General coding agents found far more known vulnerabilities than SAST (recall 59.55% against 0–34.29% on C/C++) [li2026llm, p. 7]. Yet most LLM detectors and SAST tools produced mostly false warnings [li2026llm, p. 11].
- **On the same functions, the winner depends on the score.** One study tested all three families on the same 1549 functions. Seven of eight LLMs beat lightweight SAST on balanced accuracy [deininger2026open, p. 7; deininger2026open, p. 10]. But at a realistic prevalence of 1 vulnerable function in 45 (written 1:44), Flawfinder and Semgrep were more precise than every LLM. LLM precision was 2.3–4.6%, barely above the 2.21% of flag-all [deininger2026open, p. 8; deininger2026open, p. 17].

- **Against fine-tuned DL, the ranking depends on data, class balance and threshold.** A small fine-tuned CodeBERT beat all eight LLMs on those functions (balanced accuracy 0.765 against at most 0.711) [deininger2026open, p. 11; deininger2026open, p. 17]. On pairs of vulnerable and fixed code, all families were near or below chance [ding2025vulnerability, p. 9; tan2026think, p. 6].
- **On brand-new 2025 vulnerabilities, everything struggles.** Trained DL detectors reached F1 3.47–8.65 and four open LLMs 0.79–5.36 [lu2025lab, p. 13; lu2025lab, p. 14]. By our calculation, flag-all would reach about 3.87 there **[I]**.
- **The most robust pattern [I]**: LLMs tend to have higher recall and lower precision than SAST. At realistic prevalence, their margin over trivial baselines is small.

RQ2. Why do studies disagree? Mostly because of the test setup

Evidence strength: strong for label noise and for the collapse under realistic testing. **Moderate** for prompt and threshold effects and for trivial baselines.

- **Wrong labels.** When experts re-checked "vulnerable" labels in popular datasets, only 39–65% were correct [risse2025top, p. 8]. In a PHP dataset, 48.8% were [tashenova2026real, p. 11].
- **Data leaks.** Up to 18.9% of test functions also appeared in the training data [ding2025vulnerability, p. 4]. A model can then score well by remembering, not by understanding.
- **Balanced versus realistic test sets.** The ranking of models flips when the test set is imbalanced like real code [shu2026evaluating, p. 40; yildiz2025benchmarking, p. 7; deininger2026open, p. 8].
- **Missing trivial baselines.** They are rarely reported, but often competitive [tashenova2026real, p. 8; deininger2026open, p. 7; lu2025lab, p. 14].
- **New code and other datasets.** Performance drops sharply on code from a different source or a later time [lu2025lab, p. 12; lu2025lab, p. 13].
- **Prompt wording and decision threshold.** Rewording the question to the LLM, or moving the cut-off between "vulnerable" and "safe", can change the winner [deininger2026open, p. 11; deininger2026open, p. 20].
- **Artificial versus real code** [khare2025understanding, p. 5], and **scoring a whole function versus the exact faulty line** [ahmed2026secvuleval, p. 8].
- **Home-made datasets with inconsistent splits** are common [kaniewski2026systematic, p. 35]. The same model on the same named dataset can differ by 28 F1 points between two papers [lu2025lab, p. 12; ding2025vulnerability, p. 7].

RQ3. Do hybrids help? Yes for filtering false alarms, with a price

- **Filtering SAST warnings works, but loses real bugs** (moderate). LLMs removed most false alarms for under $0.20 per alarm, but also discarded 12–25% of real bugs [du2026reducing, p. 8; xiong2026sifting, p. 18].
- **Hybrid detectors are contested** (weak). IRIS and RepoAudit beat SAST in their authors' own tests [li2024iris, p. 8; guo2025repoaudit, p. 8], but not in an independent

project-scale study [li2026llm, p. 7; li2026llm, p. 12]. IRIS also needs a strong LLM [li2024iris, p. 10].

- **LLM cost varies enormously.** It ranges from cents per alarm to $1,247.66 per project, while SAST runs in minutes with no token cost [li2026llm, p. 16].
- **Cost per real vulnerability found** (moderate, one study): USD 0.003–0.212 and 3.7–119.6 kJ of energy per true positive at 1:44 prevalence. This is computing cost only. It leaves out the human time needed to check the 25.6–41.9 false alarms that come with each real one [deininger2026open, p. 8]. Small open models matched a frontier model at 1/66 of the cost [deininger2026open, p. 17].
- **Explanations are unreliable.** LLMs point to the wrong place in the code [gnieciak2025large, p. 11], and a fluent explanation does not mean the answer is right [tan2026think, p. 9].

RQ4. Where does each approach work? Patchy evidence

Evidence strength: weak (small samples per vulnerability type, results that diverge, and a single PHP study).

- **Vulnerability types.** LLMs do best on self-contained injection bugs. They do worst on bugs that need wider context, on weak cryptography and on pinpointing memory bugs [khare2025understanding, p. 7; xiong2026sifting, p. 18; shu2026evaluating, p. 30]. CodeQL is better on integer overflow (CWE-190) [khare2025understanding, p. 8].
- **Code size.** LLM performance falls as functions and files get longer [ahmed2026secvuleval, p. 13; li2026llm, p. 9].
- **Projects.** One DL detector found 76% of Chromium vulnerabilities but none in Linux, while all LLMs covered Linux better [lu2025lab, p. 13].
- **Languages.** Evidence is concentrated on C/C++ and Java [kaniewski2026systematic, p. 28]. The only PHP study tested no LLM. Its home-made SAST tool found 1.7–1.8% of vulnerabilities [tashenova2026real, p. 9].

RQ5. What is still missing?

Evidence strength: moderate, as a description of recurring limitations **[I]**.

- Realistic, verified test data at repository level, with surrounding code as context [zhou2025large, p. 19; sheng2025llms, p. 25].
- Trivial baselines and realistic prevalence in every evaluation [tashenova2026real, p. 13; deininger2026open, p. 8].
- Better labels and control of data leaks [tashenova2026real, p. 12; lu2025lab, p. 17].
- Comparisons of all three families on the same data. Only two studies do this, each with clear limits [khare2025understanding, p. 7; deininger2026open, p. 10].
- Independent labelling, statistical tests and checks of prompt sensitivity [li2026llm, p. 11; deininger2026open, p. 20].
- Cost figures that include human checking time [deininger2026open, p. 8], and control of how many real bugs filtering throws away [xiong2026sifting, p. 19].
- Integration into developers' daily work [zhou2025large, p. 19].

The authors of the broadest three-family study conclude that no method they tested is ready to sort vulnerable from safe functions on its own [deininger2026open, p. 26] **[A]**. The six research gaps at the back of this guide turn these problems into concrete study ideas.

Where studies agree

Agreement	What several studies agree on	Strength	Key sources
1	Performance collapses when the test is realistic (real code, later code, realistic prevalence)	strong	[chakraborty2024revisiting, p. 7; ding2025vulnerability, p. 7; lu2025lab, p. 13]
2	Labels in vulnerability datasets are often wrong	strong	[risse2025top, p. 8; ding2025vulnerability, p. 4; tashenova2026real, p. 11]
3	Prompted LLMs over-flag and cannot tell vulnerable code from its fixed version	moderate	[ding2025vulnerability, p. 9; yildiz2025benchmarking, p. 8; tan2026think, p. 7]
4	SAST finds few vulnerabilities in real code and does well only on artificial suites	moderate	[zhou2024comparison, p. 6; li2026llm, p. 7; khare2025understanding, p. 9]
5	CodeQL's results on the OWASP test suite were reproduced by two independent groups	moderate	[khare2025understanding, p. 9; xiong2026sifting, p. 11]
6	Missing context (code outside the function) is the shared cause of failure	moderate	[risse2025top, p. 9; ahmed2026secvuleval, p. 9; li2026llm, p. 15]
7	LLM filtering of SAST warnings removes many false alarms but also some real bugs	moderate	[du2026reducing, p. 8; xiong2026sifting, p. 18]
8	Prompted-only LLMs trail models adapted to the task	moderate	[fu2023chatgpt, p. 4; zhou2024comparison, p. 10; shu2026evaluating, p. 17]
9	LLM cost far exceeds SAST cost and varies by orders of magnitude	moderate	[li2026llm, p. 16; du2026reducing, p. 8; shu2026evaluating, p. 42]
10	Trivial baselines are surprisingly competitive	moderate	[tashenova2026real, p. 8; deininger2026open, p. 7; lu2025lab, p. 14]
11	Computing cost per real vulnerability found is cents or less, but excludes human checking	moderate, one study	[deininger2026open, p. 8]

Paper map

All 24 papers, grouped by the kind of comparison they make. Many papers fit more than one group; each appears once, under its main group. The group letters follow the methodology comparison (Appendix B). The Paper Summaries follow the same order.

A. LLMs against SAST tools on the same code (6 papers)

Key	Short title	Year and venue	Status	What it adds
gnieciak2025large	Large Language Models Versus Static Code Analysis Tools	2025, IEEE Access	Peer-reviewed; arXiv copy read	3 SAST tools against 3 LLMs on 10 small C# projects; LLMs win, mainly through recall [gnieciak2025large, p. 10]
dubniczky2025castle	CASTLE benchmark	2025, LNCS book chapter	Peer-reviewed; arXiv copy read	250 tiny C programs; 13 SAST and 2 formal tools against 10 LLMs; top LLMs score highest [dubniczky2025castle, p. 11]
khare2025understanding	Understanding the Effectiveness of LLMs in Detecting Security Vulnerabilities	2025, ICST	Peer-reviewed; arXiv copy read	16 LLMs against CodeQL (artificial code only) and DL detectors; CodeQL wins on artificial suites [khare2025understanding, p. 9]
zhou2024comparison	Comparison of SAST Tools and LLMs for Repo-level Vulnerability Detection	2024, arXiv	Preprint, not peer-reviewed	15 SAST tools against 12 open LLMs on whole repositories; LLMs find more but flag up to 77.4% of functions [zhou2024comparison, p. 7]
li2026llm	LLM-based Vulnerability Detection at Project Scale	2026, IEEE TSE	Peer-reviewed; arXiv copy read	LLM detectors and coding agents against 4 SAST tools on real projects; agents find most, most warnings are false, cost up to $1,247.66 per project [li2026llm, p. 7; li2026llm, p. 16]
deininger2026open	Open or Frontier? A Cost- and Energy-Aware Benchmark	2026, Computers	Peer-reviewed; publisher PDF	8 LLMs, 3 SAST tools and 2 CodeBERT detectors on the same 1549 functions, with cost and energy per real vulnerability found [deininger2026open, p. 8]

B. LLMs against fine-tuned deep-learning detectors (4 papers)

Key	Short title	Year and venue	Status	What it adds
fu2023chatgpt	ChatGPT for Vulnerability Detection, Classification, and Repair	2023, APSEC	Peer-reviewed; arXiv copy read	Early test: fine-tuned models far beat prompted ChatGPT on Big-Vul (F1 94% against 29%) [fu2023chatgpt, p. 4]
ding2025vulnerability	Vulnerability Detection with Code Language Models: How Far are We? (PrimeVul)	2025, ICSE	Peer-reviewed; arXiv copy read	New cleaner dataset PrimeVul; models collapse under realistic testing; GPT-4 below random on vulnerable/fixed pairs [ding2025vulnerability, p. 7; ding2025vulnerability, p. 9]
shu2026evaluating	Evaluating LLMs for Multilingual Vulnerability Detection at Dual Granularities	2026, Empirical Software Engineering	Peer-reviewed; arXiv copy read	8 smaller pre-trained models against 5 LLMs in 7 languages; the winner flips with class balance [shu2026evaluating, p. 40]
lu2025lab	From Lab to Reality	2025 (arXiv); published 2026, Journal of Systems and Software	Peer-reviewed; our copy is the arXiv accepted manuscript	DL and open LLMs on vulnerabilities disclosed from May 2025; LineVul F1 falls from 91 to 4.39–6.87 [lu2025lab, p. 12; lu2025lab, p. 13]

B2. SAST against classical machine learning, no LLM (1 paper)

Key	Short title	Year and venue	Status	What it adds
tashenova2026real	A Real CVE-Backed Benchmark for WordPress Plugin Vulnerability Detection	2026, Big Data and Cognitive Computing	Peer-reviewed; publisher PDF	Only PHP study; home-made taint analyser finds 1.7–1.8% of vulnerabilities; no LLM tested [tashenova2026real, p. 9; tashenova2026real, p. 13]

C1. Hybrids: the LLM works inside the analyser (2 papers)

Key	Short title	Year and venue	Status	What it adds
li2024iris	IRIS:	2024	Peer-reviewed	LLM tells CodeQL what to

	LLM-Assisted Static Analysis	(arXiv); ICLR 2025	conference paper; recorded as preprint	look for; 55 against 27 of 120 Java vulnerabilities found [li2024iris, p. 8]
guo2025repoaudit	RepoAudit: An Autonomous LLM-Agent for Repository-Level Code Auditing	2025 (arXiv); ICML 2025	Peer-reviewed conference paper; recorded as preprint	LLM agent audits C/C++ repositories; 40 real and 11 false findings (78.43% precision) in the authors' test [guo2025repoaudit, p. 8]

C2. Hybrids: the LLM filters SAST warnings (2 papers)

Key	Short title	Year and venue	Status	What it adds
du2026reducing	Reducing False Positives in Static Bug Detection with LLMs: An Empirical Study in Industry	2026, ICSE-SEIP	Peer-reviewed; arXiv copy read	At Tencent, a hybrid removes 94–98% of false alarms and keeps 75–88% of real bugs [du2026reducing, p. 8]
xiong2026sifting	Sifting the Noise: LLM Agents in Vulnerability False Positive Filtering	2026, PACMSE (ISSTA)	Peer-reviewed	Best agent cuts the false-alarm rate from 98.3% to 6.3% but drops 22.25% of real vulnerabilities [xiong2026sifting, p. 12; xiong2026sifting, p. 18]

D. Agents and reasoning LLMs (2 papers)

Key	Short title	Year and venue	Status	What it adds
tan2026think	To Think or Not to Think	2026, ASE	Peer-reviewed; publisher PDF	Controlled test of "thinking" mode and multi-agent setups; thinking raises F1 mainly via recall at 2.7–4.4 times the energy [tan2026think, p. 6; tan2026think, p. 10]
yildiz2025benchmarking	Benchmarking LLMs and LLM-based Agents in Practical Vulnerability Detection for Code Repositories	2025, ACL	Peer-reviewed	Repository-context benchmark of 879 vulnerabilities; GPT-4o methods stay at or below 20.17% pairwise accuracy [yildiz2025benchmarking, p. 6]

E. How realistic are the benchmarks? (3 papers)

Key	Short title	Year and venue	Status	What it adds
chakraborty2024revisiting	Revisiting the Performance of DL-Based Vulnerability Detection on Realistic Datasets	2024, IEEE TSE	Peer-reviewed; arXiv copy read	On whole-project test data, DL precision falls to 1–10% [chakraborty2024revisiting, p. 7]
risse2025top	Top Score on the Wrong Exam	2025, PACMSE (FSE)	Peer-reviewed; arXiv copy read	Only 39–65% of "vulnerable" labels are correct; counting words alone gives F1 0.86 [risse2025top, p. 8; risse2025top, p. 11]
ahmed2026secvuleval	SecVulEval: Context-Aware Benchmarking of LLMs	2026, AIware	Peer-reviewed; our copy is the arXiv version under a different title	Line-level scoring: the best LLM reaches only 23.83% F1 [ahmed2026secvuleval, p. 8]

F. Surveys and reviews (4 papers)

Key	Short title	Year and venue	Status	What it adds
kaniewski2026systematic	A Systematic Literature Review on Detecting Software Vulnerabilities with LLMs	2026, ACM TOSEM	Peer-reviewed systematic review; arXiv copy read	263 studies; field dominated by C/C++ function-level tests on noisy data [kaniewski2026systematic, p. 8; kaniewski2026systematic, p. 28]
zhou2025large	Large Language Model for Vulnerability Detection and Repair: Literature Review	2025, ACM TOSEM	Peer-reviewed systematic review; arXiv copy read	58 studies to March 2024; only one of 40 detection studies worked at repository level [zhou2025large, p. 18]
sheng2025llms	LLMs in Software Security: A Survey of Vulnerability Detection Techniques	2025, ACM Computing Surveys	Peer-reviewed narrative survey; arXiv copy read	Overview of techniques and datasets; names data leaks and label errors as key challenges [sheng2025llms, p. 25]
qin2026re	A Review of Source	2026,	Weak: peer	Single-author narrative

view	Code Vulnerability Detection Methods Based on LLMs	Computers and Artificial Intelligence	review not documented; positioning only	review of 32 references with the same "evaluation reliability" framing as ours, but no search protocol [qin2026review, p. 14]

Five papers to read first

If you only have time for five papers, read these, in this order:

1. **deininger2026open** (2026). The only study that runs LLMs, SAST tools and fine-tuned DL on exactly the same real functions, and then re-scores them at a realistic prevalence. It shows in one paper how the winner changes with the score you choose, and it gives the first cost per real vulnerability found [deininger2026open, p. 8; deininger2026open, p. 11]. Look at its tables on precision at 1:44 and on cost.
2. **ding2025vulnerability** (2025, PrimeVul). The key paper on why many published numbers are too good. It audits labels and data leaks, builds a cleaner dataset, and shows models collapse on it [ding2025vulnerability, p. 4; ding2025vulnerability, p. 7]. Many later studies build on PrimeVul.
3. **li2026llm** (2026). The largest independent test at the scale of whole projects. It compares LLM detectors, coding agents and SAST tools, explains why warnings are false, and measures cost [li2026llm, p. 7; li2026llm, p. 16]. It is also the independent check on IRIS and RepoAudit.
4. **risse2025top** (2025). Short and readable. It shows that many "vulnerable" labels are wrong and that a classifier counting words scores almost as high as the best published models [risse2025top, p. 8; risse2025top, p. 11]. It will change how you read every number in this field.
5. **kaniewski2026systematic** (2026). The most recent systematic review, covering 263 studies [kaniewski2026systematic, p. 8]. Read it for the big picture of datasets and methods, and to see what our review adds (it does not compare LLMs with SAST).

Where studies disagree

These eight contradictions come from the evidence matrix. In each case we compared the data, method, metric and timing. Our explanations are interpretations **[I]**, not settled facts.

- **C1. Fine-tuned models or GPT-4?** Fine-tuned models far beat GPT-4 on Big-Vul (F1 94% and 87% against 29%) [fu2023chatgpt, p. 4]. On vulnerable/fixed pairs from PrimeVul, GPT-4 beats every fine-tuned model, although all stay below a random guesser (12.94% against at most 3.01%, random 22.70%) [ding2025vulnerability, p. 9]. On single PrimeVul functions, a fine-tuned CodeBERT beats all eight LLMs again [deininger2026open, p. 11]. **[I]** The benchmark and the score, not the model family, drive the reversal.
- **C2. Does RepoAudit work?** Its authors report 78.43% precision and all 21 known bugs found [guo2025repoaudit, p. 8]. An independent study found 0 of 110 known C/C++ vulnerabilities and 92.77% false warnings [li2026llm, p. 7; li2026llm, p. 11]. The two studies differ in targets, code versions, metric and who labelled the results. Unresolved.

- **C3. LLMs or SAST?** LLMs win on small programs [gnieciak2025large, p. 10; dubniczky2025castle, p. 11], CodeQL wins on artificial suites [khare2025understanding, p. 9], and at repository level both have problems [li2026llm, p. 11]. On identical functions, LLMs lead on balanced accuracy but SAST is more precise at realistic prevalence [deininger2026open, p. 8; deininger2026open, p. 10]. **[I]** LLMs over-flag and SAST finds little, so the ranking depends on the metric.
- **C4. Does "reasoning" reduce false alarms?** One paper says reasoning models made fewer false alarms [dubniczky2025castle, p. 14], but its own table shows DeepSeek R1 with more false alarms than GPT-4o [dubniczky2025castle, p. 11]. A controlled study finds thinking raises F1 mostly through recall [tan2026think, p. 6]. Reasoning lowered one model's ability to recognise safe code [deininger2026open, p. 17]. **[I]** The effect depends on the model.
- **C5. Balanced or realistic test sets?** On a balanced test, fine-tuned GPT-4o wins; on an imbalanced test, smaller pre-trained models win [shu2026evaluating, p. 17; shu2026evaluating, p. 40]. F1 drops from about 49–65% on balanced pairs to 5.41–12.21% on realistic data [yildiz2025benchmarking, p. 7]. **[I]** Rankings on balanced data do not carry over to real use.
- **C6. Does IRIS beat CodeQL?** In the authors' test, IRIS finds 55 of 120 vulnerabilities against CodeQL's 27 [li2024iris, p. 8]. On new real-world Java projects, 97.80% of IRIS warnings were false against 77.00% for CodeQL [li2026llm, p. 12]. The setups are not like-for-like. Open.
- **C7. Same model, same dataset, different score.** LineVul on Big-Vul scores F1 91.00 in one paper [lu2025lab, p. 12] and the same model type 62.88 in another [ding2025vulnerability, p. 7]. Neither paper explains the 28-point gap. **[I]** Probably different data splits and preprocessing.
- **C8. Does a detector trained on time-split data survive new code?** A CodeBERT trained on PrimeVul keeps balanced accuracy 0.765 on PrimeVul's later test split [deininger2026open, p. 11]. DL detectors trained on older datasets fall to F1 3.47–8.65 on 2025 vulnerabilities [lu2025lab, p. 13]. The datasets, training data and thresholds differ. Unresolved.

Paper Summaries

One brief per paper, in the order of the paper map. Each starts with a one-line takeaway and a fact table, then explains in plain words what the authors did, what they found (with evidence labels and page numbers), the weaknesses, the gaps the paper leaves open, and why it matters for your research.

Large Language Models Versus Static Code Analysis Tools: A Systematic Benchmark for Vulnerability Detection

> **Takeaway:** On ten small C# projects with planted security flaws, three chatbot-style LLMs found far more flaws than three standard code scanners (mean F1 0.75–0.80 against 0.26–0.55). The test set is tiny and the scoring was more lenient for the LLMs, so treat this as weak evidence.

Item	Detail
Authors	Damian Gnieciak, Tomasz Szandala
Year	2025
Venue	IEEE Access (journal)
Status	Peer-reviewed. Our copy is the arXiv version (2508.04448, v1), so numbers may differ slightly from the published article.
Identifier	DOI 10.1109/access.2025.3635168
Paper type	Empirical benchmark (head-to-head comparison)
Relevance to us	High — RQ1, RQ2, RQ3, RQ4

In plain words

A security vulnerability is a mistake in code that an attacker can exploit. One classic way to find such mistakes is static application security testing (SAST). SAST tools read the source code without running it and apply fixed rules, such as "user input must not flow into a database query". A newer way is to ask a large language model (LLM), a chatbot-style AI trained on huge amounts of text and code, to review the code.

This paper puts three well-known SAST tools (SonarQube, CodeQL and Snyk Code) against three LLMs (GPT-4.1, Mistral Large and DeepSeek V3). All six look at the same ten small C# projects. The LLMs win clearly on the authors' headline numbers. But the paper also shows two practical problems: the LLMs never gave the right line number, and they sometimes reported problems that did not exist.

How they did it

The authors built a tool called ProjectAnalyzer. It pastes every file of a project into one prompt and asks the LLM to act as "a static code analysis engine" [gnieciak2025large, p. 7]. The prompt lists ten vulnerability types and asks for a structured answer. This was zero-shot: the models got no examples and no extra training. The SAST tools ran with their default settings [gnieciak2025large, p. 8]. Each tool or model ran once per project. The authors then checked every reported finding by hand against a list of the known flaws [gnieciak2025large, p. 6].

The main measures are precision, recall and F1. Precision is the share of a tool's warnings that are real flaws. Recall is the share of real flaws the tool finds. F1 combines the two into one score between 0 and 1. The authors computed these per project and then took the plain average over the ten projects, which is called a macro average [gnieciak2025large, p. 10].

Data and setup

- The test set is 10 C# projects with 3–10 files each, 3,145–5,317 characters each, and 63 vulnerabilities in total [gnieciak2025large, p. 6].
- The flaws come from six categories: SQL injection, cross-site scripting, hard-coded secrets, command injection, weak encryption and outdated libraries [gnieciak2025large, p. 5].
- The paper says the projects were "prepared" with "a random number of vulnerabilities" [gnieciak2025large, p. 5], but the abstract calls them "real-world" projects [gnieciak2025large, p. 1]. **[I]** The sizes suggest small purpose-built examples, not real software.
- Temperature (a setting that controls how random the LLM's answers are), model versions and actual run costs are not reported. List prices per million tokens are given [gnieciak2025large, p. 5].
- The harness code is public [gnieciak2025large, p. 1].

Key findings

- **[V]** Mean F1: GPT-4.1 0.797, Mistral Large 0.753, DeepSeek V3 0.750, against Snyk Code 0.546, CodeQL 0.386 and SonarQube 0.260 [gnieciak2025large, p. 10].
- **[V]** The gap comes mainly from recall. The LLMs reach mean recall 0.783–0.877, the SAST tools 0.187–0.523. Mean precision is closer: 0.723–0.783 for LLMs, 0.567–0.686 for SAST [gnieciak2025large, p. 10].
- **[I]** Pooling the raw counts instead of averaging per project gives SAST precision of about 0.74–0.87 against 0.74–0.79 for the LLMs, and Snyk's F1 rises to about 0.70 (our arithmetic from the raw tables [gnieciak2025large, p. 8]). The averaging method widens the gap.
- **[V]** LLMs were fast. CodeQL took 185.9–213.1 seconds per project and GPT-4.1 took 3.5–17.8 seconds [gnieciak2025large, p. 9].
- **[A]** All LLMs named the right file but always gave the wrong line and column [gnieciak2025large, p. 11].
- **[V]** LLMs invented findings. For example, GPT-4.1 flagged a current version of a library (Newtonsoft.Json 13.0.3) as outdated [gnieciak2025large, p. 12].
- **[A]** The authors recommend LLMs for early, broad checks during development and SAST for high-assurance audits [gnieciak2025large, p. 12].

Limitations and weaknesses

- **[I]** The benchmark is tiny: 63 flaws in projects small enough to fit easily in one prompt [gnieciak2025large, p. 6]. This suits LLMs and says little about real repositories.
- **[V]** Wrong line locations were not counted against the LLMs [gnieciak2025large, p. 6]. **[I]** SAST tools got no such leniency, so the comparison is tilted towards the LLMs.

- **[V]** When a tool reported nothing on a project, its precision was recorded as 0.000 and averaged in [gnieciak2025large, p. 9]. **[I]** This lowers SAST precision.
- **[I]** Each model ran once, with no repeat runs, no statistical tests and no confidence intervals. LLM answers can vary between runs.
- **[I]** Default SAST settings were used, and "outdated libraries" is normally the job of a different tool type. Both may disadvantage the SAST tools.
- **[I]** There are no results per vulnerability type, and no actual cost, although prices are listed.

Gaps and future work this paper leaves open

- **[A]** The authors call for studying prompt design and building a hybrid in which classic tools and LLMs prioritise and verify findings [gnieciak2025large, p. 11].
- **[I]** The lenient location rule for LLMs is one of the scoring differences behind gap G1 (no shared evaluation protocol across studies).
- **[I]** The study uses small, synthetic-looking projects in one language, so it does not touch gap G2 (a fair three-way test on real, recent code with repository context).
- **[I]** It lists prices but reports no cost per real flaw found, which is the subject of gap G6.

Why it matters for your research

This is one of the few head-to-head tests of commercial-grade SAST against frontier LLMs, and the only one in our set on C#. It supports the claim that LLMs beat SAST on small, self-contained code. But it is weak evidence. Use it as one data point in the "LLMs win on small code" group, and always pair it with the larger real-code studies that point the other way. It is also a good teaching example of how a scoring choice, such as averaging per project, can change a headline result.

Connections to other papers

- It is one side of contradiction C3 in the evidence matrix (LLM vs SAST rankings reverse between settings). CASTLE also finds LLMs ahead on small C programs [dubniczky2025castle, p. 11]. CodeQL beats LLMs on accuracy on synthetic test suites [khare2025understanding, p. 9].
- At repository scale, LLM tools flag up to 77.4% of functions (preprint) [zhou2024comparison, p. 7], and most LLM detectors and SAST tools produce mostly false warnings [li2026llm, p. 11].
- **[I]** The "LLMs find more but are noisier" pattern also appears in deininger2026open, where SAST tools are more precise than every LLM at a realistic vulnerability rate [deininger2026open, p. 8].

CASTLE: Benchmarking Dataset for Static Code Analyzers and LLMs Towards CWE Detection

> **Takeaway:** On CASTLE, a new set of 250 small hand-written C programs, the best LLMs (top score 977 for GPT-o3-Mini) beat all 13 static analysers and 2 formal verification tools (best 661). The programs are tiny and the result table has counting problems, so the win may not carry over to real code. The authors' own test on a 400-line program suggests it does not.

Item	Detail
Authors	Richard A. Dubniczky, Krisztofer Zoltan Horvát, Tamás Bisztray, Mohamed Amine Ferrag, Lucas C. Cordeiro, Norbert Tihanyi
Year	2025
Venue	Lecture Notes in Computer Science (proceedings book chapter)
Status	Peer-reviewed (published chapter). Our copy is the arXiv version (2503.09433, v2), so numbers may differ from the published chapter.
Identifier	DOI 10.1007/978-3-031-98208-8_15
Paper type	Benchmark dataset paper with a comparative evaluation
Relevance to us	High — RQ1, RQ2, RQ3, RQ4, RQ5

In plain words

A benchmark is a fixed test set that lets different tools be scored in the same way. This paper builds a new benchmark for the C language called CASTLE. It then runs three kinds of tools on it. The first kind is static analysers (SAST), which read code and apply rules. The second is formal verification (FV) tools, which try to prove mathematically that a program cannot reach a bad state. The third is LLMs, chatbot-style AI models asked to review the code.

Each program has at most one flaw, and each flaw is tied to a CWE. A CWE (Common Weakness Enumeration) is a standard catalogue number for a type of weakness, for example CWE-476 for "null pointer dereference". On these short programs the large LLMs come out on top.

How they did it

Security experts wrote the 250 programs by hand and marked the exact vulnerable line [dubniczky2025castle, p. 6]. The authors ran 13 static analysers, 2 FV tools and 10 LLMs [dubniczky2025castle, p. 11]. The LLMs were prompted zero-shot (no examples, no extra training) to act as a "professional security analyst". The prompt also told them the scoring rule: "5 points for each vulnerability you find, but get -1 for all false positives" [dubniczky2025castle, p. 9].

Results are given as true and false positives (correct and wrong warnings), precision, recall and accuracy. The authors also define a new metric, the CASTLE Score. It gives points for each correct find, a bonus for more dangerous CWEs, points for staying quiet on clean programs, and penalties for each false alarm [dubniczky2025castle, p. 10]. The maximum is 1,250 [dubniczky2025castle, p. 12].

Data and setup

- 25 CWEs × 10 programs = 250 programs: 150 vulnerable and 100 clean. They are 7–164 lines long, with a mean of 42 [dubniczky2025castle, p. 6].
- The CWEs come mostly from the MITRE Top-25 list of most dangerous weaknesses [dubniczky2025castle, p. 7].
- Tool versions are reported, for example CodeQL 2.20.1 and ESBMC 7.8.1 [dubniczky2025castle, p. 11]. LLM temperature, model snapshots and costs are not reported.
- The dataset and tool wrappers are public [dubniczky2025castle, p. 1].

Key findings

- **[V]** CASTLE Scores: GPT-o3-Mini 977, GPT-o1 962, DeepSeek R1 956, GPT-4o 954. The best non-LLM tools are ESBMC 661 and CodeQL 634 [dubniczky2025castle, p. 11].
- **[V]** Large LLMs have high recall: GPT-o3-Mini 78% with 63% precision, DeepSeek R1 90% with 47% precision. Small open models are weaker, for example Llama 3.1 8B with 20% precision [dubniczky2025castle, p. 11].
- **[A]** CodeQL is the best SAST tool and finds 29% of the flaws, against a SAST average of 23% [dubniczky2025castle, p. 12]. **[I]** The paper writes this as "45/150", which is 30%; 29% matches a different denominator in the table (see Limitations).
- **[V]** Some SAST tools are swamped by noise. Splint raised 1,027 false alarms and CodeThreat 1,101, which gave them negative scores [dubniczky2025castle, p. 11].
- **[V]** Combining two non-LLM tools helps: ESBMC plus CodeQL scores 814, up from 661 for ESBMC alone [dubniczky2025castle, p. 13]. No LLM-plus-tool combination was tested.
- **[A]** The authors say reasoning models "consistently produced fewer false positives" [dubniczky2025castle, p. 14]. **[I]** Their own table shows this only for OpenAI's o-series: DeepSeek R1 made 166 false alarms against 116 for GPT-4o [dubniczky2025castle, p. 11].
- **[A]** FV tools raise almost no false alarms but cannot detect issues such as SQL injection or hard-coded passwords [dubniczky2025castle, p. 12].
- **[A]** In a preliminary test on one program of more than 400 lines, LLMs raised more false alarms and most missed the single hidden flaw. No numbers are given [dubniczky2025castle, p. 14].

Limitations and weaknesses

- **[I]** Counting problem: there are 150 vulnerable programs, but true positives plus missed flaws add up to 154–179 for all 10 LLMs and 155–157 for four SAST tools [dubniczky2025castle, p. 11]. Recall and accuracy therefore use unclear denominators. Quote exact numbers with this caveat.

- **[V]** LLMs returned the content of the flagged line so that small line-number errors could be corrected [dubniczky2025castle, p. 8]. **[I]** No such help is described for the SAST tools, and the prompt reveals the scoring rule. Both favour the LLMs.
- **[A]** The programs are tiny and single-file. Real flaws often span many files [dubniczky2025castle, p. 14].
- **[I]** The CASTLE Score weights are the authors' own choice and are not checked against any outside standard.
- **[I]** There are no per-CWE results, no statistical tests, and no costs. The authors say repeated runs differ by under 3%, but do not say how many repeats there were [dubniczky2025castle, p. 14].
- **[A]** Because the tests are public, tool vendors could tune for them; the authors suggest refreshing the dataset [dubniczky2025castle, p. 14].

Gaps and future work this paper leaves open

- **[A]** The authors call for refreshing the tests and studying scale-up to large codebases [dubniczky2025castle, p. 14].
- **[I]** The prompt that reveals the scoring scheme and the new CASTLE Score both feed gap G1: studies use incompatible metrics and no shared protocol.
- **[I]** Single-file synthetic programs leave gap G2 open (a fair three-way test on real, recent code with repository context).
- **[I]** No hybrid LLM-plus-tool run is tested, and no cost is reported, so the paper says nothing on gap G4 (independent tests of hybrids) or gap G6 (cost per real flaw found).

Why it matters for your research

CASTLE has the widest tool panel in our set: 13 SAST tools including commercial ones, 2 FV tools and 10 LLMs, all on one controlled benchmark with line-level labels. It is the cleanest evidence that frontier LLMs beat SAST on small, self-contained C code. For a comparative review it is just as useful as a contrast case. Its own large-file test, together with the real-code studies, shows why "who wins" depends on the size and realism of the code.

Connections to other papers

- It sits on the "LLMs win" side of contradiction C3, together with gnieciak2025large [gnieciak2025large, p. 10]. On synthetic Java and C/C++ suites, CodeQL beats LLMs on accuracy [khare2025understanding, p. 9]. At project scale, most tools produce mostly false warnings [li2026llm, p. 11].
- It is the "for" side of contradiction C4 (does reasoning reduce false alarms?). In deininger2026open, reasoning lowers DeepSeek-V3.2's specificity, its ability to correctly clear safe code, from 0.52 to 0.37 [deininger2026open, p. 20].
- Its CodeQL recall of 29% supports agreement G4 (SAST recall is low outside synthetic suites), as do zhou2024comparison (preprint) [zhou2024comparison, p. 6] and li2026llm [li2026llm, p. 7].

Understanding the Effectiveness of Large Language Models in Detecting Security Vulnerabilities

> **Takeaway:** Across 5,000 short code samples, 16 prompted LLMs reached only modest accuracy, and CodeQL beat the best LLM on accuracy on all three synthetic test suites. LLMs beat CodeQL on some vulnerability types, but on real code neither the LLMs nor the deep-learning tools clearly beat simply calling every sample vulnerable.

Item	Detail
Authors	Avishree Khare, Saikat Dutta, Ziyang Li, Alaia Solko-Breslin, Rajeev Alur, Mayur Naik
Year	2025
Venue	2025 IEEE Conference on Software Testing, Verification and Validation (ICST)
Status	Peer-reviewed. Our copy is the arXiv version (2311.16169, v3, October 2024). The arXiv record starts in 2023 while the conference is 2025; this year difference is normal and harmless.
Identifier	DOI 10.1109/icst62969.2025.10988968
Paper type	Empirical study
Relevance to us	High — RQ1, RQ2, RQ3, RQ4, RQ5

In plain words

This paper asks how good off-the-shelf LLMs (chatbot-style AI models) are at spotting security flaws in a single function. It covers all three families in our review. The LLMs are compared with CodeQL, a widely used static analysis (SAST) tool that searches code with hand-written rules. They are also compared with two deep-learning (DL) detectors, DeepDFA and LineVul. A DL detector is a smaller neural network trained specifically on labelled examples of vulnerable and safe code.

The answer is sobering. The LLMs are only somewhat better than guessing. CodeQL wins overall on artificial test suites. The LLMs do better than CodeQL on a few specific vulnerability types.

How they did it

The authors prompted 16 LLMs, from small open models to GPT-4, with three prompts [khare2025understanding, p. 4]. The Basic prompt asks whether there is any vulnerability. The CWE-specific prompt names the target weakness type; a CWE is a standard catalogue number for a type of weakness. The CWE-DF prompt asks the model to trace dataflow first. That means following data from a source (where untrusted input enters), through any sanitizer (code that

cleans it), to a sink (where it could do harm). The models got no extra training, and temperature was set to 0 to make answers as repeatable as possible [khare2025understanding, p. 5].

Each answer is a yes/no verdict per code sample. Results are reported as accuracy, precision, recall and F1. Accuracy is the share of all verdicts that are correct. Precision is the share of "vulnerable" verdicts that are right. Recall is the share of real flaws found. F1 combines precision and recall.

Data and setup

- Five datasets with 1,000 samples each, half vulnerable and half safe, covering 25 CWEs [khare2025understanding, p. 5].
- Three are synthetic, meaning artificial test suites written to contain textbook flaws: OWASP Benchmark (Java), Juliet Java and Juliet C/C++. Two are real-world: CVEFixes Java and CVEFixes C/C++, taken from fixes of publicly reported vulnerabilities [khare2025understanding, p. 3].
- Comments and revealing names were removed, so the models could not read the answer from the code [khare2025understanding, p. 5].
- CodeQL ran on the synthetic datasets only, because real projects must be built (compiled) first [khare2025understanding, p. 7].
- The DL tools were tested on CVEFixes C/C++, after training on Juliet, CVEFixes or another dataset called BigVul [khare2025understanding, p. 9].
- Each setup ran once [khare2025understanding, p. 10].

Key findings

- **[V]** Taking the best model and prompt for each dataset, average accuracy is 62.8%. Best accuracy on synthetic data is 10.5% higher on average than on real-world data [khare2025understanding, p. 5].
- **[V]** CodeQL beats the best LLM on accuracy on all three synthetic suites: OWASP 0.63 vs 0.60, Juliet Java 0.94 vs 0.76, Juliet C/C++ 0.72 vs 0.65 [khare2025understanding, p. 9].
- **[V]** LLMs beat CodeQL on some CWEs, for example path traversal (CWE-22), command injection (CWE-78), null pointer (CWE-476) and use-after-free (CWE-416). CodeQL is better on integer overflow (CWE-190) by 11–21% [khare2025understanding, p. 8].
- **[V]** The dataflow prompt raises real-world F1 by up to 0.18 [khare2025understanding, p. 5]. In a manual check, the model found the right sources and sinks in 18 of 20 real Java samples [khare2025understanding, p. 6].
- **[A]** Bigger models are not reliably better. Qwen-2.5 with 14 or 32 billion parameters beat GPT-4 on real-world data [khare2025understanding, p. 5].
- **[V]** On CVEFixes C/C++, DL tools reach F1 0.02–0.64 depending on the training data, and LineVul trained on BigVul collapses to 0.02. The best LLM, Qwen-2.5-32B, reaches 0.65 [khare2025understanding, p. 10].
- **[I]** On a half-and-half dataset, answering "vulnerable" every time gives F1 0.667. Qwen's 0.65 and every DL score are below that. Several LLM setups behave exactly like this always-vulnerable rule [khare2025understanding, p. 16].

- **[A]** Missing wider context (code outside the function) is the main reason for poor results on real C/C++ [khare2025understanding, p. 7].

Limitations and weaknesses

- **[I]** The best of 48 model-and-prompt combinations was picked after seeing the test results [khare2025understanding, p. 5]. This favours the LLMs against a single CodeQL setup.
- **[I]** CodeQL was never run on real code, where its rules are less well tuned.
- **[I]** Half-and-half datasets are far from reality, where few functions are vulnerable. Precision would be much lower at a realistic rate.
- **[I]** The public datasets predate the models' training data cutoff, so models may have seen them (data contamination). This is not discussed beyond renaming identifiers.
- **[A]** Single runs, and a possible bias from the choice of models and datasets [khare2025understanding, p. 10].
- **[I]** There are no statistical tests, no costs, and no released code or samples in our version.

Gaps and future work this paper leaves open

- **[A]** The authors propose "neuro-symbolic" hybrids, in which an LLM infers the sources, sinks and sanitizers and a static analyser then reasons over them [khare2025understanding, p. 11].
- **[I]** CodeQL on synthetic data only is a named limitation in gap G2: there is still no fair three-family test on real, recent code with repository context.
- **[I]** Always-vulnerable rows among the best F1 scores are part of the evidence for gap G1: published results are rarely checked against trivial baselines or realistic rates.
- **[I]** The proposed hybrid is close to how IRIS works (LLM-inferred sources and sinks plus CodeQL) [li2026llm, p. 4]. Whether such hybrids hold up in independent tests is gap G4.

Why it matters for your research

This is a peer-reviewed study that covers all three families: LLMs, SAST and DL. It is the main counterweight to the "LLMs beat SAST" papers: on synthetic suites CodeQL still wins overall. It is also the best source in our set for which vulnerability types suit LLMs (RQ4): self-contained flaws go well, context-dependent ones go badly. Finally, it shows how a balanced dataset can make near-trivial behaviour look like a decent F1 score.

Connections to other papers

- It is the "SAST wins" side of contradiction C3, against gnieciak2025large and dubniczky2025castle [gnieciak2025large, p. 10; dubniczky2025castle, p. 11].
- Agreement G1 (realistic evaluation collapses performance): its 10.5% synthetic-to-real drop matches the larger drops in ding2025vulnerability [ding2025vulnerability, p. 7] and deininger2026open [deininger2026open, p. 8].

- Agreement G3 (LLMs over-flag): its always-vulnerable rows match ding2025vulnerability, where GPT-4 labels both versions of a vulnerable/fixed pair as vulnerable in 71.63% of pairs [ding2025vulnerability, p. 9].
- Agreement G5: its CodeQL OWASP result (precision 0.58, recall 0.96) is replicated by xiong2026sifting (60.3%, 97.0%) [xiong2026sifting, p. 11].
- Agreement G6 (missing context causes failures) links it to risse2025top and li2026llm [risse2025top, p. 9; li2026llm, p. 7].

Comparison of Static Application Security Testing Tools and Large Language Models for Repo-level Vulnerability Detection

> **Takeaway:** In whole Java, C and Python repositories, free SAST tools found few known vulnerabilities (at best 30–44%) but flagged few functions, while fine-tuned open LLMs found many more (up to 100%) but flagged up to 77.4% of all functions. This is a preprint built on only about 8–10 test vulnerabilities per language, so the evidence is weak.

Item	Detail
Authors	Xin Zhou, Duc-Manh Tran, Thanh Le-Cong, Ting Zhang, Ivana Clairine Irsan, Joshua Sumarlin, Bach Le, David Lo
Year	2024
Venue	arXiv
Status	Preprint, not peer-reviewed. The "ESEC/FSE'23" page header is a template leftover, not a sign of peer review.
Identifier	arXiv 2407.16235 (DOI 10.48550/arxiv.2407.16235)
Paper type	Empirical comparison study
Relevance to us	High — RQ1, RQ2, RQ3, RQ4, RQ5

In plain words

A preprint is a paper posted online before, or without, peer review, so no outside experts have checked it. This one asks a practical question. If you scan a whole software repository (all of a project's code), which works better: SAST tools or LLMs? SAST tools read code and apply fixed rules. LLMs here means language models trained on code, from small ones such as CodeBERT to larger ones such as CodeLlama-7B.

The paper finds a clear trade-off. SAST tools miss most vulnerabilities but raise few alarms. LLMs catch more but mark a large share of all functions as suspicious, which a developer would then have to check by hand.

How they did it

Each repository is split into functions, and every function is classified as vulnerable or not [zhou2024comparison, p. 3]. The authors ran 15 free SAST tools and 12 open LLMs [zhou2024comparison, p. 4]. Four small models were fully fine-tuned, which means they got extra training on labelled examples from this task. Eight larger models (3.8–8 billion parameters) were either fine-tuned with LoRA, a cheap method that trains only a small add-on, or prompted with no extra training [zhou2024comparison, p. 4]. A small side test also prompted ChatGPT (GPT-3.5) [zhou2024comparison, p. 9].

Because there may be unknown flaws in the code, the authors do not compute precision or F1 [zhou2024comparison, p. 5]. Instead they use two measures. The detection ratio is the share of known vulnerabilities found. In Scenario 1 one flagged function counts as a find; in Scenario 2 all affected functions must be flagged. The marked function ratio is the share of all functions flagged, used as a rough stand-in for the false alarm rate [zhou2024comparison, p. 5].

Data and setup

- 270 known vulnerabilities in 134 repositories: Java (50 projects, 87 vulnerabilities), C (6 projects, 85) and Python (78 projects, 98). Only 0.008–0.18% of functions are vulnerable [zhou2024comparison, p. 3].
- The Python dataset is new, built from the US National Vulnerability Database [zhou2024comparison, p. 3].
- Vulnerabilities were split randomly 8:1:1 into training, validation and test sets [zhou2024comparison, p. 5].
- **[I]** Test-set sizes are not reported. The steps between scores imply about 8, 9 and 10 test vulnerabilities for Java, C and Python [zhou2024comparison, p. 6].
- SAST tool versions and settings are not reported. Runtime and cost are not reported.

Key findings

- **[V]** SAST detection is low. Many tools found nothing. The best single tools found 37.5% in Java (Horusec), 44.4% in C (CodeQL) and 30.0% in Python (DevSkim) in Scenario 1, while marking only 0.2–5.2% of functions [zhou2024comparison, p. 7].
- **[V]** LLMs found up to 100% in Java and C and 90% in Python, but marked 3.5–77.4% of functions depending on model and language [zhou2024comparison, p. 6; zhou2024comparison, p. 7].
- **[V]** Fine-tuning beats prompting. In Java, LoRA models found 73.4% (Scenario 1) while marking 19.4% of functions; prompted models marked 52.1–55.0% [zhou2024comparison, p. 7].
- **[V]** Plain ChatGPT prompting found only 3.7–23.2%, against 89.2% for a fine-tuned CodeBERT in the same test [zhou2024comparison, p. 10].
- **[V]** Combining all SAST tools helps at little cost. In Python, Scenario 1 detection rose from 30.0% to 50.0% while marked functions rose from 4.5% to 7.7% [zhou2024comparison, p. 8].
- **[V]** A majority vote across the 12 fine-tuned LLMs cut the marked ratio by 40.9–74.6%, with detection close to the best single LLM [zhou2024comparison, p. 9].
- **[V]** By weakness type, CWE-119 (memory buffer errors) was found best; CWE-835 (infinite loop) and CWE-89 (SQL injection) were missed by both groups [zhou2024comparison, p. 8].
- **[A]** Combined LLMs work best for Java, combined SAST tools for C and Python [zhou2024comparison, p. 9].

Limitations and weaknesses

- **[I]** With only about 8–10 test vulnerabilities per language, one vulnerability moves a score by 10 or more points. There are no repeated runs or confidence intervals.
- **[I]** Several cells in the main results table lost their first digit in typesetting (for example 7.5 where the text says 37.5) [zhou2024comparison, p. 6; zhou2024comparison, p. 7]. We quote the text values.
- **[I]** The abstract says combining SAST tools and LLMs reduces both families' drawbacks [zhou2024comparison, p. 1]. But the experiments only combine SAST with SAST and LLM with LLM [zhou2024comparison, p. 8].
- **[I]** The LLMs were trained on the same data source as the test set, possibly even the same projects, while SAST tools ran untuned. The comparison is uneven.
- **[I]** The marked function ratio is not a true false alarm rate, so results cannot be pooled with studies that report precision.
- **[A]** Models were capped at 8 billion parameters for compute reasons, and pre-training data leakage is possible [zhou2024comparison, p. 4; zhou2024comparison, p. 10].

Gaps and future work this paper leaves open

- **[A]** The authors call for repository-level evaluation, fewer flagged functions (filtering, repository context, better prompts) and better ways to combine LLMs with SAST [zhou2024comparison, p. 10].
- **[I]** Its detection ratio is one of five incompatible headline metrics across LLM-vs-SAST studies, which is part of gap G1.
- **[I]** It has no classic deep-learning detector and uses older, pre-cutoff code, so it does not close gap G2.
- **[I]** It is one of the few Python sources, but with about 10 test vulnerabilities, and it has no PHP or Go. This is gap G5.
- **[I]** No runtime or cost is reported, so it adds nothing to gap G6.

Why it matters for your research

This is one of the few studies that compares many SAST tools with many LLMs on the same real repositories in three languages. Its pattern recurs across the review: SAST is quiet but misses most flaws, while LLMs catch more but bury real findings in alarms. Because it is a preprint with very small test sets, cite it as supporting evidence for that pattern, not as proof on its own.

Connections to other papers

- Contradiction C3 (LLM vs SAST rankings reverse): its "LLMs detect more but flag up to 77.4%" result sits between the small-code wins in gnieciak2025large [gnieciak2025large, p. 10] and the project-scale study li2026llm, where most tools' real-world warnings are mostly false [li2026llm, p. 11].
- Agreement G4 (SAST recall is low on real code): it agrees with CASTLE [dubniczky2025castle, p. 12] and li2026llm [li2026llm, p. 7].

- Agreement G8 (prompt-only LLMs trail adapted models): its ChatGPT result matches fu2023chatgpt [fu2023chatgpt, p. 4] and shu2026evaluating [shu2026evaluating, p. 17].
- **[I]** Its huge marked ratios foreshadow deininger2026open, where every LLM's precision is barely above flagging everything at a realistic vulnerability rate [deininger2026open, p. 8].

LLM-based Vulnerability Detection at Project Scale: An Empirical Study

> **Takeaway:** When whole projects are scanned, general coding agents (Codex, Claude Code) find far more known vulnerabilities than SAST tools (59.55% average recall on C/C++ against 0–34.29%). Specialised LLM detectors often find almost nothing, most tools' warnings on real projects are 77–100% false, and LLM runs can cost up to USD 1,247.66 per project.

Item	Detail
Authors	Fengjie Li, Jiajun Jiang, Dongchi Chen, Yingfei Xiong
Year	2026
Venue	IEEE Transactions on Software Engineering (journal)
Status	Peer-reviewed. Our copy is the arXiv version (2601.19239, v2, September 2026), not the IEEE version of record; page numbers and some numbers may differ.
Identifier	DOI 10.1109/tse.2026.3742453
Paper type	Comparative empirical study
Relevance to us	High — RQ1, RQ2, RQ3, RQ4, RQ5

In plain words

Most studies test detectors on single functions. This one tests them where developers actually use them: on whole projects, some with millions of lines of code. It compares three groups of tools. Specialised LLM detectors are research tools built around an LLM for one job, such as RepoAudit, KNighter, IRIS, LLMDFA and INFERROI. General coding agents are AI assistants that can open files and search code on their own; here Codex and Claude Code. SAST tools are rule-based scanners: CodeQL, Semgrep, SpotBugs and the Clang Static Analyzer (CSA).

The general agents clearly find the most known flaws. But on real, current projects nearly every tool, LLM or SAST, raises mostly false alarms, and some LLM tools are slow and costly.

How they did it

The study has two parts. First, a controlled benchmark of known vulnerabilities measures recall (share of known flaws found) and precision (share of warnings that are right) [li2026llm, p. 5]. Second, the tools scan 24 real projects where the true flaws are unknown. There the main measure is SFDR, the sampled false discovery rate: the share of a random sample of a tool's warnings that turn out to be false [li2026llm, p. 6].

Each specialised tool ran with its original model and again with GPT-5.4, to separate the effect of the workflow from the effect of the model [li2026llm, p. 4]. SAST rule sets were picked by hand

to match the eight CWEs studied; a CWE is a standard catalogue number for a type of weakness [li2026llm, p. 7]. Codex labelled the sampled real-world warnings as true or false, and humans checked a stratified sample [li2026llm, p. 11]. The authors also hand-coded why tools raise false alarms [li2026llm, p. 13].

Data and setup

- Controlled set: 265 known vulnerabilities, 110 in C/C++ and 155 in Java, across 8 CWEs [li2026llm, p. 6].
- Real-world set: 24 actively maintained projects or modules, from 1,715 to 6,033,855 lines of code [li2026llm, p. 6]. 21 of the 24 code versions are newer than the August 2025 knowledge cutoff of the agents' models [li2026llm, p. 5]. The knowledge cutoff is the date after which a model has seen no training data.
- 6,442 warnings were sampled and 5,896 false positives classified by cause [li2026llm, p. 11].
- No deep-learning detectors were included [li2026llm, p. 4]. Each setup ran once [li2026llm, p. 18].
- Scripts, prompts and labels are public [li2026llm, p. 2].

Key findings

- **[V]** On 110 known C/C++ vulnerabilities, the general agents average 59.55% recall and 32.19% precision. CodeQL's recall is 6.00–34.29% across the three CWEs, while Semgrep and CSA find none. KNighter finds none, and RepoAudit finds 0 of 110 with its original model [li2026llm, p. 7].
- **[V]** On 155 known Java vulnerabilities, Claude Code's recall is 81.94% and Codex's 63.87%, against CodeQL 14.19%, Semgrep 6.67% and SpotBugs 14.84% [li2026llm, p. 9].
- **[V]** On real C/C++ projects, SFDR is 92.77% for RepoAudit, 86.01% for CodeQL, 85.51% for CSA and 100% for Semgrep. Codex is lowest at 9.09%, but on about two warnings per project [li2026llm, p. 11].
- **[V]** On real Java projects, IRIS (an LLM plus CodeQL hybrid) has 97.80% SFDR, worse than plain CodeQL at 77.00% [li2026llm, p. 12].
- **[V]** Weak API modelling explains all 108 RepoAudit misses with GPT-5.4 [li2026llm, p. 7]. API modelling means knowing which library functions allocate memory, read input and so on. Given perfect API lists for memory leaks (CWE-401), recall rises sharply: CodeQL from 6% to 56%, RepoAudit from 0% to 48% [li2026llm, p. 8].
- **[V]** Across tools, 63.33% of false alarms come from misjudging what a program point does, and 14.60% from missing context in other functions [li2026llm, p. 15].
- **[V]** Cost per project: RepoAudit averages USD 172.77 with a maximum of USD 1,247.66; Codex averages USD 5.38 and 14.80 minutes; SAST tools run in minutes with no token cost [li2026llm, p. 16].
- **[A]** The authors conclude that the tool's workflow, not the model, limits recall [li2026llm, p. 9].

Limitations and weaknesses

- **[V]** Codex, which is also evaluated, labelled the real-world warnings; humans found its labels 94.36% accurate on 621 checked warnings [li2026llm, p. 11]. **[I]** A judge that favours its own reports is only partly ruled out, and Codex scores best on SFDR.
- **[I]** Codex's low SFDR rests on roughly 22 warnings per language, so it is imprecise.
- **[I]** Controlled-set precision counts false alarms only inside known vulnerable files, so it is not comparable with precision in other papers [li2026llm, p. 6].
- **[I]** The known vulnerabilities likely predate model training, so agent recall may partly reflect memory. Recall on the newer real-world code is not measured.
- **[I]** SAST rules were chosen by the authors; other rules could change SAST results.
- **[A]** Two languages, 8 CWEs, single runs, and several modules from one Linux version [li2026llm, p. 18].

Gaps and future work this paper leaves open

- **[A]** The authors call for combining agents with program analysis, better API specifications, cost-aware designs, and more languages and CWEs [li2026llm, p. 17; li2026llm, p. 18].
- **[I]** Gap G4: this is the only independent re-test of the hybrids RepoAudit and IRIS in our set, and its labeller is an evaluated tool, so target, metric and labeller effects are still mixed.
- **[I]** Gap G2: it uses code newer than the training cutoff with repository context, but has no deep-learning arm and no recall on that newer code.
- **[I]** Gap G6: it reports cost per project, not cost per confirmed vulnerability including human checking time.
- **[I]** Gap G1: SFDR is yet another headline metric that cannot be pooled with others. Gap G5: no PHP or Go.

Why it matters for your research

This is the most realistic LLM-vs-SAST comparison in our set: real projects, current code, full tools rather than single functions. It shows that "LLM vs SAST" depends heavily on the LLM workflow. Flexible agents beat SAST on recall, while several published specialised hybrids do not. It also gives concrete cost data for RQ3 and a shared list of false-alarm causes for RQ5.

Connections to other papers

- Contradiction C2: RepoAudit's own paper reports 78.43% precision and USD 2.54 per project [guo2025repoaudit, p. 7], against 0 of 110 here. Targets, code versions, metrics and labellers differ, so neither paper refutes the other.
- Contradiction C6: IRIS beat CodeQL on 120 known Java vulnerabilities in its own paper [li2024iris, p. 8], but is noisier than CodeQL on new Java projects here.
- Contradiction C3: the agents' recall lead echoes the small-code LLM wins [gnieciak2025large, p. 10; dubniczky2025castle, p. 11], and the high false alarm rates echo the repository-level preprint zhou2024comparison [zhou2024comparison, p. 7].

- Agreement G4 (low SAST recall), agreement G6 (missing context causes failures) and agreement G9 (LLM cost far exceeds SAST) all draw on this paper.

Open or Frontier? A Cost- and Energy-Aware Benchmark of Large Language Models for Software Vulnerability Detection

> **Takeaway:** On the same 1,549 real C/C++ functions, a small fine-tuned CodeBERT beat all eight zero-shot LLMs, and at the real-world rate of about one vulnerable function per 44 safe ones every LLM's precision (2.3–3.8%, or 2.8–4.6% in a budget-matched re-run) was barely above flagging everything (2.21%). Open models matched or beat frontier models at a small fraction of the cost.

Item	Detail
Authors	Patrick Deininger, Wolfgang Slany
Year	2026
Venue	Computers (journal)
Status	Peer-reviewed
Identifier	DOI 10.3390/computers15090623
Paper type	Empirical benchmark with cost and energy measurement
Relevance to us	High — RQ1, RQ2, RQ3, RQ5

In plain words

Companies can use frontier LLMs (the newest paid models, such as Claude, GPT and Gemini) or open-weight LLMs (models whose files are public, so anyone can run them). This paper asks which is better for finding vulnerable code, counting money and electricity as well as accuracy. It also tests three classic static analysis (SAST) tools and two CodeBERT detectors on the very same functions. CodeBERT is a small deep-learning model that can be fine-tuned, meaning given extra training on labelled examples.

Note that the paper does include SAST (Flawfinder, Cppcheck and Semgrep). That makes it one of very few studies that test all three families on identical code.

How they did it

Each model gets one C/C++ function and must answer YES or NO: does it contain a vulnerability? The LLMs were used zero-shot (no examples, no extra training), at temperature 0, which makes answers as repeatable as possible [deininger2026open, p. 6]. The main score is balanced accuracy, the average of the share of vulnerable functions caught and the share of safe functions correctly cleared. It is used because both trivial rules, "always safe" and "always vulnerable", score exactly 0.5 on it [deininger2026open, p. 6].

The authors then re-scored everything at the natural prevalence of 1:44. Prevalence is how common vulnerable code is; here about one vulnerable function for every 44 safe ones

[deininger2026open, p. 8]. Differences are tested with a paired bootstrap (resampling to see whether a gap could be chance) and corrected for running many comparisons [deininger2026open, p. 6].

Data and setup

- The data is PrimeVul, a C/C++ dataset with carefully cleaned labels. Its test split has 24,788 functions, 549 of them vulnerable [deininger2026open, p. 4].
- The sample is all 549 vulnerable plus 1,000 random safe functions, 1,549 in total [deininger2026open, p. 4]. **[I]** That is 35.4% vulnerable, not 50/50, although the text once calls it "balanced by construction" [deininger2026open, p. 8].
- Eight LLMs were tested: three frontier (Claude Sonnet 5, GPT-5.1, Gemini 3.1 Pro) and five open-weight (DeepSeek-V3.2, GLM-5, Llama-3.3-70B, Qwen3-Coder-30B, Gemma-3-4B) [deininger2026open, p. 4].
- SAST: Flawfinder 2.0.20, Cppcheck 2.21 and Semgrep 1.175 [deininger2026open, p. 10]. CodeQL could not run, because isolated functions cannot be built (compiled) [deininger2026open, p. 10].
- Energy was measured on a GPU for some open models and estimated from computation counts (FLOP estimates) for the rest [deininger2026open, p. 5].

Key findings

- **[V]** LLM balanced accuracy is 0.511–0.676 in the first run, with DeepSeek-V3.2 best [deininger2026open, p. 7], and 0.568–0.711 in a second, budget-matched run seven weeks later [deininger2026open, p. 17].
- **[V]** A fine-tuned CodeBERT (125 million parameters) beats every LLM, with balanced accuracy 0.765 [deininger2026open, p. 11]. With the default decision rule it looked like an "always safe" classifier, despite a ROC-AUC (ranking quality, independent of the cut-off) of 0.845; the authors nearly reported this artefact as a real result [deininger2026open, p. 11].
- **[V]** Precision is the share of warnings that are real flaws. At 1:44 prevalence, every LLM's precision is 2.3–3.8%, against 2.21% for flagging everything. Each true finding comes with 25.6–41.9 false alarms [deininger2026open, p. 8]. In the budget-matched run, precision is 2.8–4.6% [deininger2026open, p. 17].
- **[V]** SAST is more precise but finds little. At 1:44, Semgrep's precision is 26.0% (95% confidence interval 5.7–66.9%) and Flawfinder's is 6.2–13.4%; Cppcheck (2.46–3.77%) sits in the LLM range [deininger2026open, p. 10]. Semgrep flagged only 19 of 1,549 functions [deininger2026open, p. 11].
- **[V]** The open model DeepSeek-V3.2 significantly beats all three frontier models, by 0.054–0.060 balanced accuracy [deininger2026open, p. 27]. In the second run, the small Gemma-3-4B matches Claude's quality at 1/66 of the cost [deininger2026open, p. 17].
- **[V]** Rewording the prompt moves balanced accuracy by up to 0.088 and produces three different winners [deininger2026open, p. 20].
- **[V]** Compute cost per true positive (a correctly flagged vulnerable function) at 1:44 is USD 0.003–0.212, and energy is 3.7–119.6 kJ [deininger2026open, p. 8]. **[V]** Caveat: the kJ values for Qwen and Llama come from FLOP estimates, not from the measured 88 J and 738 J per task [deininger2026open, p. 7; deininger2026open, p. 14].

- **[V]** Recall is 7–15 points higher on pre-2020 vulnerabilities than on 2021–2022 ones, which the authors call "consistent with some memorization" [deininger2026open, p. 24].

Limitations and weaknesses

- **[I]** The SAST arm is lightweight, function-level tools only. There is no CodeQL or other whole-program analyser, so this is not a test against the strongest SAST.
- **[A]** All vulnerabilities date from 2008–2022 and predate the models' training cutoff, so scores may be optimistic [deininger2026open, p. 24].
- **[A]** CodeBERT was trained on PrimeVul's own training data, while the LLMs had none; the authors say the two need different prerequisites [deininger2026open, p. 11].
- **[A]** One dataset, one language family, function level only [deininger2026open, p. 25].
- **[V]** LLM inputs were cut at 8,000 characters, which affects 15.8% of the vulnerable functions [deininger2026open, p. 4].
- **[I]** Cost per true positive leaves out human checking time. With 25.6–41.9 false alarms per real find, analyst time would plausibly dominate the cost of a few cents.
- **[I]** Revision left some prose inconsistencies, so cite table values rather than the text.

Gaps and future work this paper leaves open

- **[A]** The authors plan tests on vulnerabilities newer than the models' training cutoff, a second dataset, other languages and line-level localisation [deininger2026open, p. 24; deininger2026open, p. 25].
- **[I]** Gap G1: it reports trivial baselines and realistic-prevalence scores for its own results, but does not re-score other studies.
- **[I]** Gap G2 is narrowed by this paper. A function-level C/C++ three-family test now exists. Still missing: code newer than the training cutoff, file or repository context, and a semantic SAST tool such as CodeQL.
- **[I]** Gap G6 is narrowed too. Compute cost per true positive now exists for eight LLMs. Still missing: human triage time and a costed SAST and DL arm.
- **[I]** Gap G5: no PHP or Go.

Why it matters for your research

This is the strongest single answer in our set to RQ1, because all three families face identical functions with proper statistics. It shows that the winner depends on the metric. On balanced accuracy, fine-tuned CodeBERT leads and LLMs beat lightweight SAST. On precision at a realistic vulnerability rate, Flawfinder and Semgrep beat every LLM. The paper's own verdict is that none of the methods is ready for independent use [deininger2026open, p. 26]. It is also the only paper here that prices each true finding.

Connections to other papers

- Contradiction C1 (fine-tuned vs GPT-4): its CodeBERT win and decision-rule artefact may help explain why fine-tuned models look near chance in ding2025vulnerability [ding2025vulnerability, p. 9].

- Contradiction C3: on balanced accuracy LLMs beat lightweight SAST; on realistic precision they lose. This fits the small-code LLM wins [gnieciak2025large, p. 10] and the project-scale noise in li2026llm [li2026llm, p. 11].
- Contradiction C4: reasoning helps frontier models but lowers DeepSeek's specificity from 0.52 to 0.37 [deininger2026open, p. 20], against CASTLE's claim [dubniczky2025castle, p. 14].
- Contradiction C8: its time-split CodeBERT holds up, while DL detectors collapse on 2025 vulnerabilities in lu2025lab [lu2025lab, p. 13].
- Agreement G1 and agreement G10: performance collapses at realistic prevalence, and trivial baselines are competitive, as in lu2025lab and tashenova2026real [lu2025lab, p. 14; tashenova2026real, p. 8].
- Agreement G11 (cost per true positive) rests on this single study.

ChatGPT for Vulnerability Detection, Classification, and Repair: How Far Are We?

> **Takeaway:** In this early, short study, prompted ChatGPT (gpt-4 F1 29%, gpt-3.5 F1 10%) loses clearly to small fine-tuned models (AIBugHunter F1 94%) on the Big-Vul benchmark, and it is worst on all four tasks [fu2023chatgpt, p. 4; fu2023chatgpt, p. 5]. Big-Vul was split at random and has noisy labels, which later studies show favours fine-tuned models, so part of the gap may be a benchmark artefact.

Item	Detail
Authors	Michael Fu, Chakkrit Kla Tantithamthavorn, Van Nguyen, Trung Le
Year	2023
Venue	2023 30th Asia-Pacific Software Engineering Conference (APSEC)
Status	Peer-reviewed (conference short paper). Our copy is the arXiv version (2310.09810v1), so page numbers follow the arXiv PDF
Identifier	DOI 10.1109/apsec60848.2023.00085; arXiv 2310.09810
Paper type	Short empirical comparison (5 pages; the authors call it a pilot study)
Relevance to us	Medium (RQ1, RQ2, RQ4)

In plain words

ChatGPT is OpenAI's chat-based large language model (LLM), a big general AI model trained on huge amounts of text and code. The authors ask how well it handles four security tasks. It must find vulnerable functions and lines, name the weakness type, estimate how severe the flaw is, and repair it. ChatGPT is compared with much smaller models that were fine-tuned, meaning given extra training on exactly this data. ChatGPT itself could not be fine-tuned because its model is closed, so it was only prompted [fu2023chatgpt, p. 1]. Prompting means giving the model instructions in plain text.

How they did it

- Models: gpt-3.5-turbo and gpt-4, called through OpenAI's paid service with task prompts [fu2023chatgpt, p. 2; fu2023chatgpt, p. 3].
- Detection: ChatGPT is asked whether a function is vulnerable. For vulnerable functions it is then asked for the 10 most suspicious lines [fu2023chatgpt, p. 2]. **[I]** From the description, the detection prompt appears to give no examples (zero-shot).
- Weakness type: ChatGPT is told the function is vulnerable and given a list of possible CWE IDs [fu2023chatgpt, p. 2]. A CWE is a standard category of software weakness.
- Repair: the prompt includes three example fixes [fu2023chatgpt, p. 3].

- Comparison models: AIBugHunter (about 120 million parameters), CodeBERT and GraphCodeBERT, all fine-tuned, plus VulExplainer for weakness types [fu2023chatgpt, p. 3; fu2023chatgpt, p. 4].
- No static analysis tool was included [fu2023chatgpt, p. 3].
- Scores: F1, precision and recall for detection; top-10 line accuracy; accuracy for weakness type; error size for severity; and exact-match patches for repair [fu2023chatgpt, p. 3; fu2023chatgpt, p. 4; fu2023chatgpt, p. 5].

Data and setup

- Big-Vul: about 188,000 C/C++ functions from 348 GitHub projects, with 3,754 vulnerabilities of 91 types. There is about 1 vulnerable function for every 20 safe ones [fu2023chatgpt, p. 3].
- Split: 80% train, 10% validation and 10% test, chosen at random "similar to previous studies". No duplicate removal or time-based split is reported [fu2023chatgpt, p. 3].
- Repair data: 5.5k vulnerable/fixed function pairs from Big-Vul and CVEFixes [fu2023chatgpt, p. 3].
- Not reported: how many test functions were sent to ChatGPT, the temperature setting, the model version dates, and the number of runs.
- Data, prompts and results are released [fu2023chatgpt, p. 1].

Key findings

- **[V]** Detection F1: AIBugHunter 94%, CodeBERT 87%, GraphCodeBERT 35%, gpt-4 29%, gpt-3.5-turbo 10% [fu2023chatgpt, p. 4]. F1 balances precision (share of alarms that are real) and recall (share of real flaws found).
- **[V]** Precision: AIBugHunter 98%, gpt-4 25%, gpt-3.5-turbo 11%. Recall: AIBugHunter 90%, gpt-4 33%, gpt-3.5-turbo 9% [fu2023chatgpt, p. 4].
- **[V]** Finding the right lines (top-10 accuracy): AIBugHunter and CodeBERT 99%, gpt-4 65%, gpt-3.5-turbo 25% [fu2023chatgpt, p. 4].
- **[V]** Naming the weakness type: fine-tuned models 62–65% correct, gpt-4 20%, gpt-3.5-turbo 13% [fu2023chatgpt, p. 4].
- **[V]** Severity on the CVSS scale, a standard 0–10 severity score: mean squared error 1.8–1.86 for fine-tuned models against 5.4 (gpt-3.5) and 5.85 (gpt-4); lower is better [fu2023chatgpt, p. 4].
- **[V]** Exact-match repairs: AIBugHunter's repair model 30%, GraphCodeBERT 9%, CodeBERT 7%, both ChatGPT models 0% [fu2023chatgpt, p. 5].
- **[A]** Despite about 14,000 times more parameters than CodeBERT, ChatGPT needs fine-tuning on security data to do these tasks well, the authors argue [fu2023chatgpt, p. 1; fu2023chatgpt, p. 5].

Limitations and weaknesses

- **[A]** The paper has no limitations section. The only constraint it mentions is that ChatGPT cannot be fine-tuned [fu2023chatgpt, p. 1].

- **[I]** Method details are thin, as listed above under "not reported".
- **[I]** The random Big-Vul split favours fine-tuned models. Another study found 12.7% of Big-Vul test functions copied from training [ding2025vulnerability, p. 4]. An expert audit found only 39% of its vulnerable labels correct [risse2025top, p. 8]. A word-count model reaches F1 0.86 on it [risse2025top, p. 11]. The 94% F1 is therefore likely inflated.
- **[I]** The comparison is uneven: prompting against fine-tuning on the same data. It answers "prompting vs fine-tuning", not "LLMs vs deep learning" in general.
- **[I]** The first two authors also built the strongest comparison models. This is not a funding conflict, but readers should note it.
- **[I]** The large gap between GraphCodeBERT (35%) and CodeBERT (87%) is not explained [fu2023chatgpt, p. 4]. There are single runs, no confidence intervals, and 2023 model versions only.

Gaps and future work this paper leaves open

- Gap G1 (no shared protocol and no simple baselines): no chance-level or trivial baseline is reported, so the 94% cannot be put in context.
- Gap G2 (no same-benchmark LLM vs static analysis vs deep-learning comparison on clean data): this paper has no static analysis arm and uses leaky, randomly split data. Its ranking reverses on cleaner data (see C1 below).
- **[A]** The authors see fine-tuning on security data as "crucial" for ChatGPT [fu2023chatgpt, p. 5].

Why it matters for your research

This paper is often cited as proof that LLMs lose to fine-tuned models. That is true for its setup (RQ1), but the setup is exactly what RQ2 warns about. Its weakness-type results also help RQ4. Cite it together with the later, cleaner studies, never alone.

Connections to other papers

- Contradiction C1: on cleaned, time-split PrimeVul pairs, GPT-4 with step-by-step reasoning beats fine-tuned models (12.94% vs at most 3.01% pair-wise correct), although all are below random guessing (22.70%) [ding2025vulnerability, p. 9]. Ding's own Big-Vul F1 rows agree with this paper's direction [ding2025vulnerability, p. 7].
- Agreement G8 (prompt-only LLMs trail trained models): zhou2024comparison, a preprint, finds GPT-3.5 prompting detects 3.7–23.2% against 89.2% for fine-tuned CodeBERT [zhou2024comparison, p. 10]; shu2026evaluating finds prompt-only LLMs near chance [shu2026evaluating, p. 17]; deininger2026open finds a fine-tuned CodeBERT ahead of 8 zero-shot LLMs [deininger2026open, p. 11].
- risse2025top explains why fine-tuned models can score high on Big-Vul through shortcuts [risse2025top, p. 11] **[I]**.

Vulnerability Detection with Code Language Models: How Far are We?

> **Takeaway:** Popular test sets for AI vulnerability detectors contain many wrong labels and copied test examples, so AI models that look strong on them (F1 up to 68%) mostly fail on a cleaned, time-split benchmark. Even GPT-4 scores below random guessing at telling vulnerable code apart from its fixed version [ding2025vulnerability, p. 7; ding2025vulnerability, p. 9].

Item	Detail
Authors	Yangruibo Ding, Yanjun Fu, Omniyyah Ibrahim, Chawin Sitawarin, Xinyun Chen, Basel Alomair, David Wagner, Baishakhi Ray, Yizheng Chen
Year	2025
Venue	2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE)
Status	Peer-reviewed (ICSE 2025). Our copy is the arXiv version (2403.18624v2), so page numbers follow the arXiv PDF
Identifier	DOI 10.1109/icse55347.2025.00038; arXiv 2403.18624
Paper type	Dataset audit, new benchmark (PrimeVul) and experiments with 7 code language models
Relevance to us	High (RQ1, RQ2, RQ3, RQ5)

In plain words

A vulnerability is a flaw in code that an attacker can exploit. Researchers train AI models to spot such flaws. They train and test them on benchmarks: large collections of functions labelled "vulnerable" or "safe". A function is a small, named block of code. This paper asks whether those benchmarks can be trusted. The answer is mostly no. Many labels are wrong, and many test functions also appear in the training data. The authors build a cleaner benchmark called PrimeVul and test the models again. Scores collapse. They also test GPT-3.5 and GPT-4. These are large language models (LLMs): very big AI models trained on huge amounts of text and code.

How they did it

- Label audit: three people checked 50 random "vulnerable" functions per benchmark and took a majority vote [ding2025vulnerability, p. 3].
- Leakage check: data leakage means a test example also sits in the training data, so a model can simply remember it. The authors compared every test function with the training set after removing spaces and line breaks [ding2025vulnerability, p. 4].

- PrimeVul: they merged four existing datasets. They kept only functions with stronger evidence that they really were the vulnerable code. They removed exact copies. They split the data by time: the oldest 80% of code changes for training, the next 10% for tuning, the newest 10% for testing [ding2025vulnerability, p. 5; ding2025vulnerability, p. 6].
- Models: five open code models were fine-tuned, which means given extra training on this task. GPT-3.5 and GPT-4 were prompted with two examples ("two-shot") or asked to reason step by step ("chain-of-thought"). GPT-3.5 was also fine-tuned [ding2025vulnerability, p. 7].
- New scores: VD-S is the share of real vulnerabilities a model misses when its false-alarm rate is held at 0.5% or less. Pair-wise scores test a vulnerable function together with its fixed (patched) version. "P-C" means both were labelled correctly [ding2025vulnerability, p. 6].
- No static analysis tool was tested [ding2025vulnerability, p. 7]. Static analysis (SAST) tools scan code with fixed rules without running it.

Data and setup

- PrimeVul holds 6,968 vulnerable and 228,800 safe C/C++ functions from 755 projects, covering 140 CWEs [ding2025vulnerability, p. 6]. A CWE is a standard category of weakness, such as a buffer overflow.
- The test set has 695 vulnerable and 25,216 safe functions, about 1 vulnerable for every 32 safe [ding2025vulnerability, p. 6; ding2025vulnerability, p. 8].
- Because of cost, the GPT models were run only on 564 test pairs [ding2025vulnerability, p. 9].
- The number of repeated runs and random seeds is not reported. Data, code and prompts are released [ding2025vulnerability, p. 3].

Key findings

- **[V]** In the authors' own audit, only 24% (CodeXGLUE/Devign) and 36% (VulnPatchPairs) of "vulnerable" labels were correct, against 86–92% for PrimeVul [ding2025vulnerability, p. 4]. The BigVul figure of 25% in the same table comes from an earlier study, not from this audit **[A]**.
- **[V]** Exact copies from training into test reach 12.7% in BigVul and 18.9% in CVEFixes; PrimeVul has 0.0% [ding2025vulnerability, p. 4].
- **[V]** F1 is a 0–100% score that balances precision (how many alarms are real) and recall (how many real flaws are found). Fine-tuned models score F1 62.88–68.26% on BigVul, 18.05–21.43% when trained and tested on PrimeVul, and 1.91–5.82% when trained on BigVul and tested on PrimeVul [ding2025vulnerability, p. 7].
- **[I]** The often-quoted drop from 68.26% to 3.09% mixes two effects: a cleaner test set and a switch to a different dataset. The fair within-PrimeVul figure is 18.05% [ding2025vulnerability, p. 7].
- **[V]** At a false-alarm rate of 0.5% or less, PrimeVul-trained models miss 88.78–91.51% of vulnerabilities [ding2025vulnerability, p. 7].

- **[V]** Pair-wise correctness on 564 pairs: GPT-4 chain-of-thought 12.94%, GPT-3.5 chain-of-thought 6.21%, fine-tuned GPT-3.5 1.24%, fine-tuned open models 1.06–3.01%. Random guessing scores 22.70% [ding2025vulnerability, p. 7; ding2025vulnerability, p. 9].
- **[V]** The two families fail in opposite ways. GPT-4 two-shot calls both versions vulnerable in 71.63% of pairs, while fine-tuned open models call both safe in 84.22–88.30% [ding2025vulnerability, p. 7; ding2025vulnerability, p. 9].

Limitations and weaknesses

- **[A]** The authors note that PrimeVul labels are still not 100% correct, that VD-S depends on the chosen 0.5% threshold, and that OpenAI models ran with default settings. They also say a function without its surrounding code may be "an impossible question" even for humans [ding2025vulnerability, p. 10].
- **[I]** GPT models and fine-tuned models are never scored on the same metric. GPT models were run on pairs only, so there is no F1 or VD-S for them [ding2025vulnerability, p. 9].
- **[I]** Fine-tuning of GPT-3.5 was weak (one training pass on a 1:3 subsample), so its poor result says little about well-tuned LLMs [ding2025vulnerability, p. 9].
- **[I]** Only exact copies were removed; near-copies with renamed variables may remain. Audits used 50 functions each, which leaves wide uncertainty (about plus or minus 14 points).
- **[I]** The text gives CodeBERT accuracy as 96.86% on BigVul, but the table shows 95.57% [ding2025vulnerability, p. 7; ding2025vulnerability, p. 8]. There are no repeated runs or confidence intervals.

Gaps and future work this paper leaves open

- Gap G1 (no shared evaluation protocol): the paper proposes VD-S because accuracy and F1 mislead [ding2025vulnerability, p. 2], and its random-guess row shows why every result needs a chance reference.
- Gap G2 (no same-benchmark LLM vs SAST vs deep-learning comparison): PrimeVul has no SAST arm, and the GPT models were tested on pairs only.
- **[A]** The authors call for wider code context, teaching models security concepts, and hybrids that join language models with program-analysis tools [ding2025vulnerability, p. 10].

Why it matters for your research

This is the key paper for RQ2. It shows that much of the "progress" on old benchmarks comes from wrong labels and leakage. It also gives the only comparison in our set of GPT-4 against fine-tuned models on a de-duplicated, time-split benchmark (RQ1). For your own work, PrimeVul and the pair-wise test are ready-made tools. Never cite a BigVul F1 alone.

Connections to other papers

- Contradiction C1: fu2023chatgpt finds fine-tuned models far ahead of GPT-4 on BigVul (F1 94% vs 29%) [fu2023chatgpt, p. 4], while here GPT-4 leads on pairs. Ding's own BigVul rows agree with Fu, so the reversal reflects metric and data **[I]**.
- Agreement G2 (noisy labels): risse2025top independently finds only 39–65% of labels correct [risse2025top, p. 8].
- Agreement G1 (realistic tests collapse scores): see chakraborty2024revisiting, where precision falls to 1–10% [chakraborty2024revisiting, p. 7], and lu2025lab.
- Agreement G3 (LLMs over-flag): yildiz2025benchmarking finds "both vulnerable" is 40–95% of LLM failures on pairs [yildiz2025benchmarking, p. 8].
- Contradiction C7: lu2025lab reports LineVul F1 91.00 on BigVul [lu2025lab, p. 12], far above Ding's 62.88 for CodeBERT.
- deininger2026open uses PrimeVul without pairs and finds a fine-tuned CodeBERT ahead of all 8 zero-shot LLMs [deininger2026open, p. 11]. qin2026review repeats this paper's table [qin2026review, p. 12].

Evaluating large language models for multilingual vulnerability detection at dual granularities

> **Takeaway:** Across seven programming languages, only one setup (GPT-4o, fine-tuned and given retrieved examples) clearly beat smaller fine-tuned code models, and only on balanced test data. On realistic, imbalanced data the smaller models won. No static-analysis tool was tested.

Item	Detail
Authors	Honglin Shu, Michael Fu, Junji Yu, Dong Wang, Chakkrit Tantithamthavorn, Junjie Chen, Yasutaka Kamei
Year	2026
Venue	Empirical Software Engineering (journal)
Status	Peer-reviewed journal article. Our copy is the arXiv version 2506.07503v2 (March 2026), not the typeset journal version.
Identifier	DOI 10.1007/s10664-026-10832-4; arXiv 2506.07503
Paper type	Empirical comparison (LLMs against fine-tuned pre-trained code models)
Relevance to us	High — RQ1, RQ2, RQ3, RQ4, RQ5

In plain words

This paper compares two kinds of AI models for finding security flaws in code. The first kind are PLMs (pre-trained language models for code), such as CodeBERT or CodeT5. These are smaller models that are fine-tuned, meaning trained further on labelled examples of vulnerable and safe code. In our project they count as deep-learning (DL) detectors. The second kind are LLMs (large language models), such as GPT-4o, which are much bigger and are often just given instructions in a prompt.

The paper tests seven languages: C, C++, C#, Go, Java, JavaScript and Python. It asks two questions. Can a model tell whether a whole function (a named block of code) is vulnerable? And can it point to the vulnerable lines inside that function? The main lesson is that the winner depends on how the test data is built **[V]** [shu2026evaluating, p. 17; shu2026evaluating, p. 40].

How they did it

The authors used five LLMs and several ways of using them **[V]** [shu2026evaluating, p. 10; shu2026evaluating, p. 12; shu2026evaluating, p. 14]:

- zero-shot prompting: the model gets only the question;
- few-shot prompting: the model also gets three similar solved examples, picked from the training data by a text-search method (BM25);

- instruction tuning: the LLM is itself fine-tuned on the task, then prompted. The best setup combined instruction tuning with few-shot examples.

They compared these with eight PLMs and embedding models, and with "dummy" baselines that call everything vulnerable or everything safe **[V]** [shu2026evaluating, p. 9; shu2026evaluating, p. 11]. A baseline is a simple reference method; beating it shows a model learned something real.

Main metrics were accuracy (share of correct answers) and F1 (a single score that balances precision, the share of flagged items that are truly vulnerable, against recall, the share of real flaws found) **[V]** [shu2026evaluating, p. 14; shu2026evaluating, p. 15].

Data and setup

- The data is REEF: real vulnerability fixes from 2016–2023. The version before the fix is labelled vulnerable, the version after the fix safe **[V]** [shu2026evaluating, p. 6].
- Function level: 20,165 functions, about half vulnerable, with 2,026 in the test set **[V]** [shu2026evaluating, p. 7].
- Line level: 8,134 vulnerable functions, 810 in the test set **[V]** [shu2026evaluating, p. 8].
- Functions longer than 512 tokens (word pieces) were removed, so all code is short **[V]** [shu2026evaluating, p. 6].
- Extra tests: newer 2024–2025 code, to check whether models just memorised old data, and an imbalanced test set where only about 17% of functions are vulnerable **[V]** [shu2026evaluating, p. 38; shu2026evaluating, p. 39].
- The split into training and test data was random by function **[V]** [shu2026evaluating, p. 7]. There are no significance tests and no repeated runs **[I]**.

Key findings

- On balanced data, the best LLM setup (GPT-4o, instruction-tuned plus few-shot) reached accuracy 0.7196 against 0.6037 for the best PLM, CodeT5P. Their F1 scores were essentially tied (0.7069 vs 0.7075) **[V]** [shu2026evaluating, p. 17].
- The "flag everything" dummy reached F1 0.6693, so the best F1 is only a little above this trivial level **[V]** [shu2026evaluating, p. 17].
- LLMs used only with prompts were near chance at function level, with accuracy around 0.48–0.51 for zero-shot **[V]** [shu2026evaluating, p. 17].
- At line level, the best LLM setup reached F1 0.6641 against 0.4841 for CodeT5P **[V]** [shu2026evaluating, p. 23; shu2026evaluating, p. 26].
- On the imbalanced test, the ranking flipped. UniXcoder (a PLM) had the best F1, 0.4626, against 0.4010 for the best GPT-4o setup **[V]** [shu2026evaluating, p. 40; shu2026evaluating, p. 41].
- On 2024–2025 code, function-level accuracy held (0.7874), but line-level F1 fell from 0.6641 to 0.3450 **[V]** [shu2026evaluating, p. 39].
- Memory-safety flaws were hardest to locate. For CWE-787 (writing outside a memory buffer; a CWE is a standard weakness category), only 6.25% were located correctly at line level **[V]** [shu2026evaluating, p. 30].

- Fine-tuning GPT-4o cost $86.07 for the function task, about 15 times more than a local PLM training run **[V]** [shu2026evaluating, p. 42].

Limitations and weaknesses

- No static-analysis tool (SAST) was tested. SAST tools are rule-based programs that scan code for known bad patterns. So this paper only supports LLM-versus-DL claims, not LLM-versus-SAST claims **[I]**.
- The headline result rests on one configuration. Every other LLM setup was near chance **[I]** [shu2026evaluating, p. 17].
- Possible leakage: each vulnerable function and its fixed twin can land on opposite sides of the random split. Few-shot search could then hand the model a near-copy with the opposite label. This is our hypothesis. The steady accuracy on 2024–2025 code argues partly against it **[I]** [shu2026evaluating, p. 7; shu2026evaluating, p. 39].
- Line-level accuracy may be inflated, because metrics seem to be computed over padded empty line slots. This is our arithmetic, not confirmed by the authors **[I]** [shu2026evaluating, p. 8; shu2026evaluating, p. 23].
- The text claims GPT-4o beat all models in all seven languages, but in JavaScript CodeT5P scored higher (0.7062 vs 0.6642) **[V]** [shu2026evaluating, p. 21; shu2026evaluating, p. 22].
- The authors note only one dataset, short functions and only exact-copy leakage filtering **[A]** [shu2026evaluating, p. 43; shu2026evaluating, p. 44].

Gaps and future work this paper leaves open

The authors ask for larger multilingual benchmarks and better learning strategies **[A]** [shu2026evaluating, p. 47]. In our gap report, this paper is evidence for gap G1 (results are rarely checked against trivial baselines and realistic class ratios), gap G2 (rankings reverse with class balance, and no SAST arm), gap G5 (the only multi-language LLM study in our corpus, including Go, has no SAST arm) and gap G6 (it is one of the few studies that report money cost).

Why it matters for your research

It is our clearest example that the class ratio of the test set can flip which model family wins (RQ2). It also gives per-language and per-CWE results (RQ4) and cost figures (RQ3). For RQ1, use it for LLM-versus-DL only **[I]**.

Connections to other papers

- Contradiction C5 in the evidence matrix (class balance flips rankings): yildiz2025benchmarking and deininger2026open show the same effect [yildiz2025benchmarking, p. 7; deininger2026open, p. 8].
- Agreement G8 (prompt-only LLMs trail adapted models): fu2023chatgpt and zhou2024comparison agree [fu2023chatgpt, p. 4; zhou2024comparison, p. 10].

- Agreement G10 (trivial baselines are competitive): tashenova2026real and lu2025lab also report them [tashenova2026real, p. 8; lu2025lab, p. 14].
- Contradiction C4 (does reasoning help?): here, reasoning LLMs were near chance [shu2026evaluating, p. 37].
- Agreement G9 (LLM cost far exceeds the alternatives) uses this paper's cost figure.

From Lab to Reality: A Practical Evaluation of Deep Learning Models and LLMs for Vulnerability Detection

> **Takeaway:** On real Linux and Chromium vulnerabilities disclosed from May 2025, deep-learning detectors that scored F1 above 90 in the lab fall to F1 3.47–8.65, and four open LLMs score 0.79–5.36 [lu2025lab, p. 12; lu2025lab, p. 13; lu2025lab, p. 14]. By our calculation, simply flagging every function would score about 3.87, so both families sit close to that "flag everything" level.

Item	Detail
Authors	Chaomeng Lu, Bert Lagaisse
Year	2025 (arXiv); 2026 (journal)
Venue	Journal of Systems and Software (2026); first posted on arXiv
Status	Peer-reviewed (JSS 2026); our copy is the arXiv accepted manuscript (v4), so page numbers follow arXiv v4
Identifier	DOI 10.1016/j.jss.2026.113185 (journal, checked via Crossref on 2026-11-12); arXiv 2512.10485
Paper type	Empirical evaluation with a new benchmark (VentiVul)
Relevance to us	High (RQ1, RQ2, RQ4, RQ5)

In plain words

Vulnerability detectors often look excellent on standard test sets. This paper asks whether that holds in "the real world". The authors collect brand-new vulnerabilities, published after the models were built. This is called temporal separation: the test data comes from a later time than the training data. Data from a different source or period is called out-of-distribution. They then test two kinds of model. Deep-learning (DL) detectors are neural networks trained on older labelled code. Large language models (LLMs) are big general AI models, here used without any extra training.

How they did it

- Stage 1: check whether the models' internal view of code separates vulnerable from safe code, using t-SNE, a method that draws complex data as a 2D picture [lu2025lab, p. 6; lu2025lab, p. 8].
- Stage 2: train four DL models on each of four datasets and test each on all four. This compares "same dataset" with "different dataset" results [lu2025lab, p. 7].
- Stage 3: test on the new VentiVul set. Only two DL models (ReVeal, LineVul) were taken to this stage [lu2025lab, p. 7].

- LLMs: Qwen2.5-Coder-32B, DeepSeek-Coder-V2-Lite, Qwen3-30B-A3B (thinking turned off) and Mistral-Small-24B. All are open-weight (downloadable) and were released before May 2025 [lu2025lab, p. 5; lu2025lab, p. 8].
- Prompts: zero-shot (instructions only), few-shot (with worked examples) and chain-of-thought (asked to reason step by step) [lu2025lab, p. 8].
- Every model sees one function at a time, with no surrounding code [lu2025lab, p. 7; lu2025lab, p. 8].
- Scores: F1 balances precision (share of alarms that are real) and recall (share of real flaws found). CDR is the share of vulnerabilities where at least one affected function is flagged. PACC is the share of pairs where the vulnerable version is flagged and its fixed version is not [lu2025lab, p. 9].
- For LLMs, "conservative" scoring counts unreadable answers as wrong [lu2025lab, p. 8].

Data and setup

- Training sets, all C/C++: Juliet (synthetic, 253,002 functions, 37% vulnerable), Devign (27,318, 46%), BigVul (188,636, 6%) and ICVul (15,396, 41%) [lu2025lab, p. 3]. ICVul is the authors' own dataset **[I]**.
- VentiVul: 100 CVEs each from Linux and Chromium, disclosed from 1 May 2025 and checked by hand [lu2025lab, p. 4]. A CVE is a public ID for a known vulnerability.
- It holds 13,326 functions, 200 CVEs, 39 CWEs and 263 vulnerable–fixed pairs. Other functions from the same files are labelled not vulnerable, to keep a realistic mix [lu2025lab, p. 4]. A CWE is a standard weakness category.
- **[I]** By our calculation, 263 of 13,326 functions (1.97%) are vulnerable.
- LLMs ran with temperature 0 and seed 42, so each answer is repeatable [lu2025lab, p. 8]. Data and code are public [lu2025lab, p. 2].

Key findings

- **[V]** In the lab, LineVul reaches F1 94.52 on Juliet and 91.00 on BigVul [lu2025lab, p. 11; lu2025lab, p. 12].
- **[V]** Accuracy can hide failure. On BigVul, ReVeal, Devign and VulBERTa reach accuracy 87.45, 94.09 and 94.39 but F1 only 23.58, 16.90 and 0.19 [lu2025lab, p. 11].
- **[V]** Across datasets, LineVul trained on BigVul scores F1 0 on Juliet, 1.90 on Devign and 13.05 on ICVul [lu2025lab, p. 12].
- **[V]** On VentiVul, the 8 DL setups score F1 3.47–8.65. The best is ReVeal trained on BigVul (precision 5.25%, recall 24.43%) [lu2025lab, p. 13].
- **[V]** The LLMs score conservative F1 0.79–5.36. The best is Mistral few-shot (F1 5.36, CDR 46%, PACC 5.75%) [lu2025lab, p. 13; lu2025lab, p. 14]. LLM precision and recall are not reported.
- **[I]** Flagging every function would give F1 about 3.87. Seven of the 12 LLM results fall below that, and all DL results are at most 4.8 points above it (our calculation from [lu2025lab, p. 4; lu2025lab, p. 13; lu2025lab, p. 14]).

- **[V]** Finding a CVE is not the same as understanding it. Some DL setups reach CDR 94.59–100% but PACC 0–0.76% [lu2025lab, p. 13]. **[I]** All PACC values (5.75% or less) are far below the up to 25% a random labeller could reach.
- **[V]** Prompting effects vary: few-shot helps only Mistral [lu2025lab, p. 13].

Limitations and weaknesses

- **[A]** The authors note: few models and no commercial LLMs; release dates cannot rule out that LLMs saw this code during training; C/C++ only; "unrelated" functions may hide flaws; and VentiVul does not include every vulnerability in those versions [lu2025lab, p. 17].
- **[I]** The paper reports no simple baseline, such as "flag everything", so its weak results are never compared with chance.
- **[I]** The DL vs LLM comparison is uneven. DL models were trained; the LLMs were mid-size open models with no extra training and Qwen3's thinking mode turned off [lu2025lab, p. 8]. Nothing here speaks to frontier or fine-tuned LLMs.
- **[I]** There are single runs and no confidence intervals, and the families were scored on slightly different function sets [lu2025lab, p. 7; lu2025lab, p. 8].
- **[I]** No static analysis tool is tested, although the introduction frames learning methods as a complement to static analysis [lu2025lab, p. 1]. We read the arXiv copy, not the paywalled journal version.

Gaps and future work this paper leaves open

- Gap G1 (no shared protocol with trivial baselines): the "flag everything" comparison above is ours, not the paper's. **[A]** The authors ask for cross-dataset and time-separated testing plus pair-based scores [lu2025lab, p. 16].
- Gap G2 (no same-benchmark LLM vs static analysis vs DL comparison on new code): this is post-cutoff evidence, but without a static analysis arm. The gap report suggests a low-cost study that reuses VentiVul and adds a static analysis arm, an LLM with file context and a "flag everything" baseline **[I]**.
- Gap G5 (missing languages): VentiVul is C/C++ only [lu2025lab, p. 17].

Why it matters for your research

This is peer-reviewed evidence that lab scores do not survive new, real vulnerabilities (RQ1, RQ2). It also varies by project, which matters for RQ4: LineVul trained on BigVul finds 76% of Chromium CVEs but none in Linux [lu2025lab, p. 13]. Its public data makes it a good base for a first study of your own.

Connections to other papers

- Agreement G1 (realistic tests collapse scores): see ding2025vulnerability [ding2025vulnerability, p. 7] and chakraborty2024revisiting [chakraborty2024revisiting, p. 7].
- Agreement G10 (trivial baselines are competitive): deininger2026open finds an always-vulnerable F1 of 0.523 against LLM F1 0.405–0.615 [deininger2026open, p. 7]; tashenova2026real finds a similar result in PHP [tashenova2026real, p. 8].

- Contradiction C7: LineVul scores F1 91.00 on BigVul here, but ding2025vulnerability reports 62.88 for CodeBERT [ding2025vulnerability, p. 7]. **[I]** Lab BigVul numbers are not comparable across papers.
- Contradiction C8: a fine-tuned detector in deininger2026open holds balanced accuracy 0.765 on later PrimeVul data [deininger2026open, p. 11], unlike the collapse here. Unresolved.
- ding2025vulnerability also finds pair scores far below random (22.70%) [ding2025vulnerability, p. 9].

A Real CVE-Backed Benchmark for WordPress Plugin Vulnerability Detection: Re-Evaluating Static and Learning-Based Detectors Under Leakage-Controlled Evaluation

> **Takeaway:** On real PHP vulnerabilities in WordPress plugins, with copies removed, no detector does well (best macro-F1 0.601), a one-number "function length" baseline almost matches the neural models, and only 48.8% of cleaned "vulnerable" labels are genuine [tashenova2026real, p. 8; tashenova2026real, p. 11]. Its static analyser is an unnamed in-house tool that misses about 98% of vulnerabilities, so it says nothing about standard PHP static analysis tools, and no LLM was tested [tashenova2026real, p. 13].

Item	Detail
Authors	Zhuldyz Tashenova, Aisultan Aitmagambetuly, Altyn Urynbassarova, Askhat Zhetkerbay, Shirin Amanzholova, Akerke Kerim
Year	2026
Venue	Big Data and Cognitive Computing
Status	Peer-reviewed (journal article)
Identifier	DOI 10.3390/bdcc10090303
Paper type	Benchmark construction and comparative evaluation (in-house static analyser, classical machine learning, neural models, trivial baselines; no LLMs)
Relevance to us	Medium (RQ2 mainly; RQ1 for static analysis vs machine learning only; RQ4; RQ5)

In plain words

WordPress is a widely used website system. Its plugins are add-ons written in PHP, a web programming language. Many plugins have publicly reported vulnerabilities. The same authors earlier reported about 93% accuracy on 600 synthetic code snippets [tashenova2026real, p. 2]. This paper re-checks that claim on real vulnerabilities, after carefully removing duplicate code. It is partly a self-correction after reviewers audited the first data release [tashenova2026real, p. 2; tashenova2026real, p. 14]. The result is sobering: real performance is far lower.

How they did it

- Ten detector setups were tested with five-fold cross-validation grouped by plugin [tashenova2026real, p. 5; tashenova2026real, p. 6]. Cross-validation splits the data into five parts; each part serves once as the test set. Grouping by plugin keeps all code from one plugin in the same part.
- Static analysis arm: a "two-pass intra-procedural analyzer" that does taint tracking [tashenova2026real, p. 5]. Taint tracking follows untrusted user input to see if it reaches a

dangerous operation without being cleaned. Intra-procedural means it looks inside one function only. The tool is unnamed and unversioned; the authors call it "intra-procedural and lexical" [tashenova2026real, p. 13].

- Classical machine learning: Random Forest and a linear SVM, fed counts of short character sequences (TF-IDF) [tashenova2026real, p. 5; tashenova2026real, p. 6].
- Neural models: LSTM variants, a type of network that reads code as a sequence, plus one model that combines the taint verdict with an LSTM [tashenova2026real, p. 5].
- Three trivial baselines: always predict the most common class, a simple source/sink rule, and a one-feature model using only function length in tokens [tashenova2026real, p. 5].
- No LLMs or transformer models were run. CodeBERT was skipped because the machine had no GPU [tashenova2026real, p. 13].

Data and setup

- 1757 CVEs in 1281 plugins could be rebuilt as before-fix and after-fix code. About 54% of indexed CVEs were lost because old versions were deleted [tashenova2026real, p. 3]. A CVE is a public ID for a known vulnerability.
- Labels come from the fix: functions changed by the fix are "vulnerable". Variant A uses the fixed version as the safe example; Variant B uses untouched functions from the same plugin [tashenova2026real, p. 3].
- Copies and near-copies were removed. Final folds share no code hashes, no clone families and no plugins. This left 15,294 (A) and 24,917 (B) functions from 30,860 each [tashenova2026real, p. 4; tashenova2026real, p. 5].
- **[I]** By our calculation, 74.5% (A) and 46.2% (B) of functions are vulnerable, far more than in real code.
- There is no time-based split [tashenova2026real, p. 3]. Everything ran on CPU with one random seed [tashenova2026real, p. 6; tashenova2026real, p. 7]. Labels and folds are released [tashenova2026real, p. 14].

Key findings

- **[V]** The best binary macro-F1 is 0.563 (A, SVM) and 0.601 (B, stacked model). No neural model leads in any of six settings [tashenova2026real, p. 7; tashenova2026real, p. 8]. Macro-F1 averages the F1 score of the "safe" and "vulnerable" classes; F1 balances false alarms and misses.
- **[V]** The function-length baseline matches or beats the Random Forest in 4 of 6 settings. On Variant B it scores 0.572, against 0.573 for the Bi-LSTM and 0.578 for the best neural model [tashenova2026real, p. 8].
- **[V]** The in-house taint analyser finds only 1.7–1.8% of vulnerabilities [tashenova2026real, p. 9]. Its macro-F1 is 0.366 against 0.350 for always guessing the common class on Variant B, and below that baseline on Variant A [tashenova2026real, p. 8].
- **[A]** The authors say this low recall "reflects that implementation rather than taint analysis in general" [tashenova2026real, p. 13].

- **[V]** Aggregate scores hide broken models. On Variant A the Random Forest calls 15,068 of 15,294 functions vulnerable; its accuracy (0.747) barely beats always guessing (0.745) [tashenova2026real, p. 9].
- **[V]** An expert review of 400 functions found only 48.8% of cleaned "vulnerable" labels genuine [tashenova2026real, p. 11].
- **[V]** Before cleaning, 166 (A) and 151 (B) groups of duplicate code crossed folds, and 730 and 456 groups had conflicting labels. Cleaning cut the data by 50.4% (A) and 19.3% (B) [tashenova2026real, p. 4; tashenova2026real, p. 5].

Limitations and weaknesses

- **[A]** The authors note: copies are controlled, but not provably all removed; lost CVEs may bias the sample; rare classes are unreliable; neural models were small and CPU-trained; no transformer was tested; labels are only about half right; one annotator did the expert review [tashenova2026real, p. 11; tashenova2026real, p. 13].
- **[I]** The static analysis arm is too weak to stand for PHP static analysis in general. No established PHP tool was run. Do not quote the "misses about 98%" figure as a statement about static analysis tools.
- **[I]** Label accuracy of the full test data is not reported and is probably below 48.8%, since that figure is for the cleaned subset [tashenova2026real, p. 11].
- **[I]** Macro-F1 cannot be compared with the vulnerable-class F1 used by most other papers. Testing is per function, never at realistic prevalence (the true share of vulnerable code).
- **[I]** One seed and overlapping confidence intervals mean many model gaps may be noise [tashenova2026real, p. 8]. The earlier ~93% claim should not be cited [tashenova2026real, p. 2].

Gaps and future work this paper leaves open

- Gap G1 (no shared protocol with trivial baselines): **[A]** the authors say future detectors "should be required to clear such trivial baselines" [tashenova2026real, p. 13].
- Gap G2 (no fair three-family comparison): this is PHP evidence without an LLM or a standard static analysis tool.
- Gap G5 (no real-CVE LLM vs standard static analysis evidence for PHP and Go): this is the only PHP study in our corpus. Its released expert-checked subset of 125 vulnerable and 247 safe functions is a ready data source [tashenova2026real, p. 11]. **[I]** The gap report notes LLM-only WordPress evidence elsewhere that we know only from an abstract.

Why it matters for your research

For RQ2 it shows that leakage, label noise and model stacking shape scores more than the model type does. For RQ4 it adds PHP, where learned models find XSS (script injection) much better than rare remote-code-execution flaws [tashenova2026real, p. 9]. It is also the natural starting point if you study gap G5.

Connections to other papers

- Agreement G2 (noisy labels): matches risse2025top [risse2025top, p. 8] and ding2025vulnerability [ding2025vulnerability, p. 4].
- Agreement G10 (trivial baselines are competitive): see deininger2026open [deininger2026open, p. 7], lu2025lab [lu2025lab, p. 14] and shu2026evaluating [shu2026evaluating, p. 11]; risse2025top's word-count model makes the same point [risse2025top, p. 11].
- Contradiction C5 (class balance flips results): here, changing how safe examples are chosen flips which way the same Random Forest fails [tashenova2026real, p. 9].
- Agreement G1 (realistic tests collapse scores): **[I]** its best macro-F1 of 0.601 fits the pattern seen in C/C++ [tashenova2026real, p. 8].

IRIS: LLM-Assisted Static Analysis for Detecting Security Vulnerabilities

> **Takeaway:** In the authors' own test, letting GPT-4 help the CodeQL static analyser roughly doubled the number of known Java vulnerabilities found (55 vs 27 of 120), but most warnings were still false and the gain needed a strong model. A later independent study found IRIS much noisier than CodeQL on new code, so the benefit is contested.

Item	Detail
Authors	Ziyang Li, Saikat Dutta, Mayur Naik
Year	2024 (our copy is arXiv version 3, April 2025)
Venue	arXiv. Our copy's header reads "Published as a conference paper at ICLR 2025"
Status	Preprint in our records, because we hold the arXiv copy. The text is the ICLR 2025 conference version, so it is peer-reviewed in substance
Identifier	doi:10.48550/arxiv.2405.17238 (arXiv 2405.17238)
Paper type	Tool proposal with benchmark evaluation
Relevance to us	High: RQ1, RQ2, RQ3, RQ4, RQ5

In plain words

A static analysis tool reads source code without running it and looks for known bug patterns. Security-focused ones are called SAST (static application security testing) tools; CodeQL is a widely used one. Many use taint analysis: they follow untrusted data from a "source" (for example, input from a web request) to a "sink" (a dangerous place, such as a shell command or a file path). If the data reaches a sink without being cleaned, the tool raises a warning.

This only works if someone has listed which library functions are sources and sinks (the specifications). Missing specifications mean missed bugs; missing context means false alarms (false positives: warnings about safe code).

IRIS is a hybrid tool. It asks a large language model (LLM, an AI model trained on huge amounts of text and code, such as GPT-4) to write the missing source and sink lists. A second LLM step then judges each warning as real or false **[A]** [li2024iris, p. 2].

How they did it

IRIS works in four stages **[V]** [li2024iris, p. 4]:

1. CodeQL extracts candidate library functions and internal function parameters.
2. An LLM labels them as sources or sinks for a given weakness type.
3. These labels become CodeQL rules, and a taint query runs over the whole project.

4. An LLM looks at each reported data path and decides whether it is a true or false positive ("contextual analysis").

The comparison tools were CodeQL 2.15.3 with all its built-in security queries, plus Facebook Infer, SpotBugs and Snyk **[V]** [li2024iris, p. 7]. Seven LLMs were tried, plus ablations (removing one part at a time to see what it contributes) **[V]** [li2024iris, p. 10]. A vulnerability counted as "detected" if at least one reported path passed through the method that the official fix changed **[V]** [li2024iris, p. 6]. No LLM-only detector and no deep-learning detector was tested **[V]** [li2024iris, p. 7].

Data and setup

The authors built a new benchmark, CWE-Bench-Java. It holds 120 real vulnerabilities, one per Java project **[V]** [li2024iris, p. 7]. Each is a CVE, a public ID for a known vulnerability. Each belongs to a CWE, a standard category of weakness. There are four CWEs: path traversal (CWE-22, 55 cases), OS command injection (CWE-78, 13), cross-site scripting (CWE-79, 31) and code injection (CWE-94, 21). Projects average 300,000 lines of code, the largest about 7 million **[V]** [li2024iris, p. 7]. The 120 cases were filtered from 1,065 candidates and checked by two of the authors **[V]** [li2024iris, p. 19].

Models ran at temperature 0 (the most repeatable setting), and the OpenAI queries ran in April–May 2024 **[V]** [li2024iris, p. 20]. Code and data are public **[V]** [li2024iris, p. 2].

Key findings

- **[V]** IRIS with GPT-4 detected 55 of 120 vulnerabilities; CodeQL alone detected 27. The average false discovery rate (FDR, the share of warnings that are not the known bug) was 84.82% for IRIS vs 90.03% for CodeQL [li2024iris, p. 8].
- **[V]** Other SAST tools found fewer: Infer 0, SpotBugs 4, Snyk 23 of 120 [li2024iris, p. 8].
- **[V]** Gains differ by weakness type: code injection 0 to 8, cross-site scripting 4 to 13, path traversal 22 to 31, command injection 1 to 3 (CodeQL vs IRIS with GPT-4) [li2024iris, p. 8].
- **[V]** Only GPT-4 lowered the FDR below CodeQL. Smaller models reached FDRs of about 95%, and the LLM warning check made two small models slightly worse [li2024iris, p. 8; li2024iris, p. 10].
- **[V]** Both LLM-written sources and sinks were needed: 27 detections with CodeQL's lists, 36 with GPT-4 sources only, 24 with GPT-4 sinks only, 55 with both [li2024iris, p. 10].
- **[V]** A manual check of 50 random IRIS warnings found 27 with potential attack surfaces, giving an estimated FDR of 46% [li2024iris, p. 7].
- **[V]** On the latest versions of 30 projects, IRIS found 4 previously unknown vulnerabilities that CodeQL missed. Whether developers confirmed them is not reported [li2024iris, p. 8].
- **[V]** Runtime with a small open model ranged from 43 seconds to 4 hours per project. Dollar or token cost is not reported [li2024iris, p. 22].

Limitations and weaknesses

The authors say many vulnerabilities are still missed, that IRIS makes many LLM calls and so costs more, and that only Java was tested **[A]** [li2024iris, p. 10]. They also note that the FDR is an upper bound, because unknown real bugs count as false alarms **[A]** [li2024iris, p. 7].

Our own concerns **[I]**:

- The CVEs and fixes are public and older than the 2024 queries, so GPT-4 may have seen them in training (contamination). The paper does not discuss this.
- An 84.82% FDR still means most warnings are not the known bug. The friendlier 46% estimate rests on only 50 warnings.
- IRIS ran on a patched CodeQL engine [li2024iris, p. 20]. It is unclear whether the CodeQL baseline used the same patched engine.
- "Detected" is lenient: any path touching the fixed method counts.
- There is one run per setting, with no significance tests and no cost in dollars.

Gaps and future work this paper leaves open

- Gap G4 (independent replication of hybrid tools): the authors' result and the independent result disagree, and no study yet separates the causes. The gap report proposes an IRIS replication on public CWE-Bench-Java data [li2024iris, p. 2].
- Gap G5 (other languages): the authors list other languages as future work **[A]** [li2024iris, p. 10].
- Gap G2 (same-benchmark comparison of all three families on recent code): IRIS has no LLM-only or deep-learning baseline and no contamination control **[I]**.
- Gap G6 (cost): no token or dollar cost is given **[I]**.

Why it matters for your research

IRIS is the central "LLM inside SAST" study in our corpus, with a real SAST baseline on large real-world Java projects. It shows the hybrid idea can raise recall on known CVEs. It also shows that precision stays poor and that the gain depends on model strength **[I]**. CWE-Bench-Java is a reusable benchmark for RQ2.

Connections to other papers

- li2026llm is an independent, peer-reviewed study and the main counter-evidence (contradiction C6 in the evidence matrix). On new real-world Java projects it measured a sampled FDR (SFDR) of 97.80% for IRIS vs 77.00% for CodeQL **[V]** [li2026llm, p. 12]. The two studies are not like-for-like: known CVEs vs new code, detections vs SFDR, and different labelling. li2026llm also labels warnings with Codex, an LLM it evaluates itself **[V]** [li2026llm, p. 11]. Neither side settles the question **[I]**.
- CodeQL finding only 27 of 120 fits agreement G4 in the evidence matrix: SAST recall is low on real code [li2024iris, p. 8; li2026llm, p. 7].
- guo2025repoaudit, the other hybrid tool, is contested in the same way (contradiction C2).

- du2026reducing excluded IRIS from its industrial study because IRIS needs a compilable repository for CodeQL **[V]** [du2026reducing, p. 5].

RepoAudit: An Autonomous LLM-Agent for Repository-Level Code Auditing

> **Takeaway:** In its authors' evaluation, the RepoAudit LLM agent found 40 real memory bugs with 78.43% precision in 15 C/C++ projects at about $2.54 per project, far ahead of two industrial analysers. An independent later study found it missed almost all known vulnerabilities on different targets and cost much more, so the result is contested.

Item	Detail
Authors	Jinyao Guo, Chengpeng Wang, Xiangzhe Xu, Zian Su, Xiangyu Zhang
Year	2025 (our copy is arXiv version 3, 29 May 2025)
Venue	arXiv. Our copy carries the footer of the ICML 2025 proceedings (PMLR 267)
Status	Preprint in our records, because we hold the arXiv copy. It appears to be the ICML 2025 camera-ready text, which we did not compare
Identifier	doi:10.48550/arxiv.2501.18160 (arXiv 2501.18160)
Paper type	Tool paper with empirical evaluation
Relevance to us	High: RQ1, RQ3, RQ4, RQ5

In plain words

Code auditing means reviewing code to find bugs. A large language model (LLM, an AI model trained on huge amounts of text and code) can read code without compiling it. But a whole repository (all the code of a project) is too big to fit into one prompt, and LLMs sometimes "hallucinate", that is, state things that are not true **[A]** [guo2025repoaudit, p. 1].

RepoAudit is an LLM agent. An agent is an LLM that works in several steps and decides by itself what to read next. RepoAudit follows how a value moves through the program (data flow), one function at a time. It stores what it has learned in a memory and then checks its own findings before reporting them **[V]** [guo2025repoaudit, p. 5]. It targets three memory bugs in C/C++: null pointer dereference (NPD, using a pointer that points to nothing), memory leak (MLK, memory that is never freed) and use-after-free (UAF, using memory after it has been released) **[V]** [guo2025repoaudit, p. 7].

How they did it

RepoAudit has three parts **[V]** [guo2025repoaudit, p. 5]:

1. An initiator finds starting points, such as places where a value is set to NULL, using a code parser.

2. An explorer asks the LLM about one function at a time and moves to calling or called functions only when needed.
3. A validator checks that the reported steps happen in a possible order and asks the LLM whether the path conditions can all be true together.

It follows a bug across at most four functions **[V]** [guo2025repoaudit, p. 9]. The default model was Claude 3.5 Sonnet at temperature 0 (the most repeatable setting) **[V]** [guo2025repoaudit, p. 7].

Comparison tools were two industrial static analysers, Meta Infer and Amazon CodeGuru, plus simple LLM prompting and another LLM agent, LLMDFA **[V]** [guo2025repoaudit, p. 8; guo2025repoaudit, p. 13]. No open-source SAST tool (static application security testing, such as CodeQL or Semgrep) and no deep-learning detector was tested **[I]**.

Data and setup

The benchmark is 15 C/C++ projects, 5 per bug type, taken from bug reports in earlier papers. They range from about 2,000 to 1.4 million lines of code **[V]** [guo2025repoaudit, p. 7]. These projects contain 21 previously reported bugs **[V]** [guo2025repoaudit, p. 17]. A field study then ran RepoAudit on 9 further open-source projects **[V]** [guo2025repoaudit, p. 8].

Precision is the share of reports that are real bugs. "Recall" here means only the share of the 21 known bugs that RepoAudit found again. Recall against all bugs in the projects was not measured **[V]** [guo2025repoaudit, p. 17]. Who judged RepoAudit's own reports as true or false is not reported; Infer and CodeGuru reports were checked manually **[V]** [guo2025repoaudit, p. 14]. The code is public **[V]** [guo2025repoaudit, p. 1].

Key findings

- **[V]** RepoAudit found all 21 known bugs plus 19 new ones: 40 true positives and 11 false positives, a precision of 78.43% [guo2025repoaudit, p. 7; guo2025repoaudit, p. 8].
- **[V]** Average cost per project was 0.44 hours and $2.54, or $0.95 per true bug. The most expensive project cost $18.15 [guo2025repoaudit, p. 7; guo2025repoaudit, p. 8].
- **[V]** Meta Infer could analyse only 8 projects (others failed to build or crashed) and found 7 true bugs with 2 false positives. Amazon CodeGuru found 0 true bugs and 18 false positives [guo2025repoaudit, p. 8; guo2025repoaudit, p. 15].
- **[V]** Simple prompting found far fewer bugs: 1 with single-function prompting and 10 with multi-function prompting [guo2025repoaudit, p. 8].
- **[V]** Removing the validator raised false positives by 245.45%. Removing program abstraction (the step that keeps only the statements relevant to the bug type) cut true positives by 47.50% [guo2025repoaudit, p. 8; guo2025repoaudit, p. 15].
- **[V]** With reasoning models, precision was 82.35–88.46% with 42–46 true positives (DeepSeek R1: $0.57 per project, OpenAI o3-mini: $0.30) [guo2025repoaudit, p. 16].
- **[A]** In the field study, RepoAudit reported 185 new bugs, of which 174 were confirmed or fixed by developers [guo2025repoaudit, p. 9].

Limitations and weaknesses

The authors name hallucinated false positives and missed bugs, cost that grows with the number of starting points, the four-function limit, and the risk of sending private code to LLM providers **[A]** [guo2025repoaudit, p. 9; guo2025repoaudit, p. 10].

Our own concerns **[I]**:

- The tool's own authors evaluated it, and the judging procedure for its reports is not described.
- True recall is unknown; only 21 known bugs were checked.
- The industrial baselines were handicapped by build failures and crashes. No build-free open-source SAST tool was compared.
- The benchmark bugs are public, so the model may have seen them in training. This is not discussed.
- One run per setting, no significance test, and small inconsistencies ("ten" vs "nine" field-study projects).
- The bugs are memory-safety defects, not necessarily exploitable security vulnerabilities.

Gaps and future work this paper leaves open

- Gap G4 (independent replication of hybrid tools): the authors' and the independent results disagree, and no study separates target, metric and labeller effects.
- Gap G5 (other languages): the authors plan to extend RepoAudit to Python and JavaScript **[A]** [guo2025repoaudit, p. 10].
- Gap G6 (full cost comparison): this paper reports cost, but per project and for small projects only; human checking time is not included **[I]**.

Why it matters for your research

RepoAudit is the main agent-style detector in our corpus with explicit cost data and a direct comparison against industrial analysers (RQ1, RQ3). The clash between its self-reported results and an independent evaluation shows how strongly the evaluation setup shapes reported numbers (RQ2) **[I]**.

Connections to other papers

- li2026llm, an independent peer-reviewed study, is the main counter-evidence (contradiction C2 in the evidence matrix). There, RepoAudit found 0 of 110 known C/C++ vulnerabilities, had a sampled false discovery rate (SFDR, the share of sampled warnings that are false) of 92.77% on real projects, and cost $172.77 per project on average and up to $1,247.66 **[V]** [li2026llm, p. 7; li2026llm, p. 11; li2026llm, p. 16].
- The two studies differ in targets, code versions, bug types, project size, metric (precision vs recall and SFDR) and labeller (li2026llm uses the Codex LLM, then checks manually) **[I]**. li2026llm traces all 108 misses of one RepoAudit configuration to incomplete coverage of library functions **[V]** [li2026llm, p. 7]. Our reading is that this coverage may explain part of the gap, but the question is unresolved **[I]**.

- Agreement G6 in the evidence matrix (missing context is a shared failure cause) and agreement G9 (LLM cost varies by orders of magnitude) both draw on this pair of papers.
- li2024iris is the other contested hybrid tool (contradiction C6). Unlike IRIS, RepoAudit needs no build **[V]** [guo2025repoaudit, p. 15].
- On contradiction C4 (does reasoning reduce false positives?), RepoAudit's higher precision with reasoning models counts "for"; tan2026think finds reasoning mainly raises recall [tan2026think, p. 6].

Reducing False Positives in Static Bug Detection with LLMs: An Empirical Study in Industry

> **Takeaway:** On 433 real alarms from Tencent's in-house static analyser, hybrid methods that combine static analysis with an LLM removed 94–98% of false alarms while keeping 75–88% of real bugs, at cents per alarm; plain prompting and off-the-shelf deep-learning detectors did far worse. The winning method is the authors' own, four authors work at Tencent, the data are private, and the bugs are general bugs rather than security vulnerabilities, so treat it as indirect, unreplicated evidence.

Item	Detail
Authors	Xueying Du, Jiayi Feng, Yi Zou, Wei Xu, Jie Ma, Wei Zhang, Sisi Liu, Xin Peng, Yiling Lou
Year	2026
Venue	Proceedings of the IEEE/ACM 48th International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)
Status	Peer-reviewed (conference). Our copy is the arXiv version 1 preprint (26 January 2026), not the camera-ready, so final numbers may differ
Identifier	doi:10.1145/3786583.3786910 (arXiv 2601.18844)
Paper type	Industrial empirical study (interviews plus offline evaluation)
Relevance to us	Medium: RQ1, RQ3, RQ4, RQ5

In plain words

A static analysis tool (SAT) reads code without running it and raises warnings, called alarms, about possible bugs. Many alarms are false positives: warnings about code that is actually fine. Developers must check each alarm by hand, which costs time.

This study asks whether large language models (LLMs, AI models trained on huge amounts of text and code) can sort the alarms, so that developers only look at the likely real ones **[A]** [du2026reducing, p. 1]. It is set inside Tencent, a large Chinese IT company, using its own analyser, BkCheck **[V]** [du2026reducing, p. 1].

Important scope note: the three bug types are null pointer dereference (using a pointer that points to nothing), out-of-bounds access (reading or writing outside an array) and divide-by-zero **[V]** [du2026reducing, p. 4]. These are general reliability bugs, not classic security vulnerabilities. They overlap with some security weakness types, so the study is only indirect evidence for our topic **[I]**.

How they did it

First, the authors interviewed 5 developers and reviewers about how alarms are handled **[V]** [du2026reducing, p. 4]. Then they tested several ways to sort alarms on historical data **[V]** [du2026reducing, p. 5]:

1. Plain prompting of four LLMs (GPT-4o, Claude-Opus-4, Qwen-3-Coder, DeepSeek-R1), with basic, chain-of-thought (asking the model to reason step by step), few-shot (showing a few solved examples) and bug-type-specific prompts.
2. Two hybrid methods that feed static-analysis information to the LLM. LLM4SA gives the LLM the warning plus surrounding code. LLM4PFA adds path conditions from static analysis and lets an LLM agent (an LLM that works in several steps) reason about whether the bug path can really happen.
3. Four deep-learning (DL) detectors, Devign, ReGVD, LLMAO and LineVul, used as originally trained, without retraining **[V]** [du2026reducing, p. 6].

Each LLM query was run three times and the majority answer was used **[V]** [du2026reducing, p. 6].

Data and setup

The data are 433 alarms from Tencent's advertising and marketing code, mostly C/C++, from September 2024 to June 2025 **[V]** [du2026reducing, p. 4]. Of these, 328 were false alarms and 105 real bugs, as judged by Tencent's routine two-round code review **[V]** [du2026reducing, p. 4]. The dataset is confidential; methods and results are in a public replication package **[V]** [du2026reducing, p. 2].

Key measures: precision (share of alarms called real that are real), recall (share of real bugs kept), and the share of false alarms removed.

Key findings

- **[V]** The analyser alone: 76% of alarms in the dataset were false (328 of 433) [du2026reducing, p. 6].
- **[A]** Developers review every alarm, spending 10–20 minutes per alarm [du2026reducing, p. 6].
- **[V]** The hybrid LLM4PFA removed 94–98% of false alarms and kept 75–88% of real bugs, with precision 0.83–0.93 across the four LLMs [du2026reducing, p. 8].
- **[V]** The other hybrid, LLM4SA, removed 84–96% of false alarms with recall 0.65–0.86 [du2026reducing, p. 8].
- **[V]** Plain prompting was much weaker: accuracy 0.40–0.55 with a basic prompt and precision 0.26–0.29 [du2026reducing, p. 8].
- **[V]** Off-the-shelf DL detectors failed. LineVul reached accuracy 0.76 only by calling every alarm false [du2026reducing, p. 7; du2026reducing, p. 8].
- **[V]** LLM4PFA cost 12.6–58.3 seconds and $0.0025–$0.12 per alarm [du2026reducing, p. 8].
- **[V]** Cases that every method got wrong had functions about 95.6 lines longer than average, with 21 more conditional statements [du2026reducing, p. 9].

Limitations and weaknesses

The authors say recall is below the 90% that the company expects, so manual review is still needed **[A]** [du2026reducing, p. 9]. LLMs can only filter alarms; they cannot find bugs the analyser misses **[A]** [du2026reducing, p. 10]. The data cover one company, 433 cases over 10 months, mostly C/C++ **[A]** [du2026reducing, p. 10].

Our own concerns **[I]**:

- Conflict of interest. Four of the nine authors are Tencent employees, and the study evaluates Tencent's own analyser on Tencent data [du2026reducing, p. 1]. LLM4PFA, the winning method, is the authors' own earlier work. Funding is not reported. With private data, nobody outside can check the result.
- The DL comparison is unfair. These detectors were built for a different task and were not retrained.
- Plain-prompting precision (0.26–0.29) is barely above the share of real bugs in the data (about 0.24, our arithmetic).
- Model names in the text do not match the cited references, so the exact models are unclear.
- Whether the few-shot examples came from the test data is not reported.
- Our copy is the preprint, not the peer-reviewed final version.

Gaps and future work this paper leaves open

- Gap G3 (true-positive loss when LLMs filter alarms): this paper loses 12–25% of real bugs while filtering, on non-security bugs. The gap report notes that loss per security weakness type on real alerts is still not controlled.
- Gap G5 (other languages): the authors plan more languages and business lines **[A]** [du2026reducing, p. 10].
- Gap G6 (full cost comparison): cost is given per alarm but not per project, and human checking time is not costed against it **[I]**.

Why it matters for your research

This is the only industrial, real-world study in our corpus of LLMs sorting static-analysis alarms (RQ3). It shows hybrids beating both plain prompting and off-the-shelf DL detectors on the same alarms, with cost per alarm **[I]**. Its weight is limited by the conflict of interest, the private data and the non-security bug types.

Connections to other papers

- xiong2026sifting is the closest independent study, on real security alerts. Together they form agreement G7 in the evidence matrix: LLM filtering removes most false alarms but loses some real ones [du2026reducing, p. 8; xiong2026sifting, p. 18].
- Both papers report filtering costs below $0.20 per alarm, part of agreement G9 on LLM cost [du2026reducing, p. 8; xiong2026sifting, p. 17].

- The authors excluded IRIS (li2024iris) because it needs a compilable repository for CodeQL **[V]** [du2026reducing, p. 5].
- The finding that long functions defeat all methods matches li2026llm, where LLM performance falls with file length [li2026llm, p. 9].

Sifting the Noise: A Comparative Study of LLM Agents in Vulnerability False Positive Filtering

> **Takeaway:** LLM agents that check SAST warnings can remove most false alarms (on a synthetic Java benchmark the best setup cut the false-positive rate from 98.3% to 6.3%), but the same setup threw away 22.25% of the real vulnerabilities, with the highest miss rates in cryptography-related weakness types. The authors conclude that agents should support human reviewers, not silently suppress warnings.

Item	Detail
Authors	Yunpeng Xiong, Ting Zhang
Year	2026
Venue	Proceedings of the ACM on Software Engineering (ISSTA 2026 issue)
Status	Peer-reviewed (journal-style proceedings). Our copy is arXiv version 3 (23 July 2026), formatted as the published article
Identifier	doi:10.1145/3832100 (arXiv 2601.22952)
Paper type	Comparative empirical study
Relevance to us	High: RQ1, RQ2, RQ3, RQ4, RQ5

In plain words

SAST tools (static application security testing) read source code without running it and warn about possible vulnerabilities. Most of their warnings are false positives: warnings about code that is actually safe. Checking them by hand is slow.

This paper tests whether LLM agents can filter out the false warnings. A large language model (LLM) is an AI model trained on huge amounts of text and code. An agent is an LLM that works in several steps: it can open files, search the code and run tools before giving an answer. The authors compare agents with "vanilla" prompting, meaning one single question to the LLM **[A]** [xiong2026sifting, p. 1].

How they did it

They tested three open-source agent frameworks (Aider, OpenHands, SWE-agent), each with three LLMs: Claude Sonnet 4, DeepSeek Chat and GPT-5 **[V]** [xiong2026sifting, p. 7]. Each agent received the SAST warning, access to the code and the instruction to decide whether the finding is real or a false positive **[V]** [xiong2026sifting, p. 7].

The SAST tools were CodeQL, Semgrep, SonarQube and Joern, run with default rules **[V]** [xiong2026sifting, p. 6]. On a recent C/C++ set, the authors also ran stronger single-prompt

baselines and ablations, meaning versions with one agent ability switched off **[V]** [xiong2026sifting, p. 10]. No deep-learning filter was tested **[I]**.

Data and setup

Three datasets were used:

1. OWASP Benchmark v1.2: 2,740 synthetic Java test programs with planted flaws, 1,415 vulnerable and 1,325 safe, across 11 weakness types **[V]** [xiong2026sifting, p. 6]. Synthetic means written for testing, not taken from real projects.
2. Vul4J: 50 randomly sampled CodeQL warnings from real Java projects, labelled 31 false and 19 true, mostly by the first author **[V]** [xiong2026sifting, p. 8; xiong2026sifting, p. 9].
3. OSS-Fuzz: 50 CodeQL warnings from 5 C/C++ projects whose fixes came 10–12 months after Claude Sonnet 4's training cutoff (the date after which a model has seen no data) **[V]** [xiong2026sifting, p. 9]. Two raters labelled them, with Cohen's kappa 0.725, a measure of agreement beyond chance where 1 is perfect **[V]** [xiong2026sifting, p. 10].

Temperature was 0 (the most repeatable setting) **[V]** [xiong2026sifting, p. 19]. The false-positive rate (FPR) is the share of safe cases that are still flagged. Scripts and data are stated to be public **[V]** [xiong2026sifting, p. 21].

Key findings

- **[V]** SAST alone on OWASP: CodeQL had precision 60.3% and recall 97.0%, with an FPR of 68.2%; SonarQube's FPR was 94.6% [xiong2026sifting, p. 11].
- **[V]** All four tools together flagged 98.3% of safe cases. After filtering by SWE-agent with Claude, the remaining FPR was 6.3%, vs 23.0% with vanilla Claude [xiong2026sifting, p. 12].
- **[V]** Agents do not always help. With DeepSeek, vanilla prompting (11.2%) beat all three agents (13.1–15.8%) [xiong2026sifting, p. 12].
- **[V]** The best setup wrongly dismissed 314 real vulnerabilities, a miss rate of 22.25% [xiong2026sifting, p. 18].
- **[V]** Misses depend on weakness type: at most 2.38% for injection types such as SQL injection and cross-site scripting, but 77.17% for weak cryptography and 84.50% for weak hashing [xiong2026sifting, p. 18].
- **[V]** On real Java warnings, the best F1 (a score combining precision and recall) was 83.3% [xiong2026sifting, p. 13].
- **[V]** On the post-cutoff C/C++ set, SWE-agent reached F1 95.5% vs 51.6% for vanilla prompting. Giving the vanilla model the same files did not help (51.6%) [xiong2026sifting, p. 15].
- **[V]** Cost per warning ranged from $0.0028 to $0.1867 depending on agent and model [xiong2026sifting, p. 17].

Limitations and weaknesses

The authors note LLM randomness, small real-world samples (50 warnings each), mostly single-author labels for Vul4J, and real-world tests based mainly on CodeQL **[A]** [xiong2026sifting,

p. 19; xiong2026sifting, p. 20]. They warn that agents are "unsuitable for unconditional, automatic suppression of SAST warnings" **[A]** [xiong2026sifting, p. 19].

Our own concerns **[I]**:

- The headline numbers are inconsistent: the abstract says an initial false-positive rate of "over 92%", the results say 98.3%.
- The main comparison uses synthetic code. The real-vulnerability loss was measured for one configuration only.
- With 50 warnings per real-world set, one or two cases change percentages a lot, and no confidence intervals are given. Some runs that did not finish seem to be silently dropped.
- From the real Java table we estimate that the GPT-5 settings dismissed roughly half to two-thirds of the true vulnerabilities in that small sample; the authors do not report this share.
- No significance tests are reported, although the abstract says agents "significantly" outperform vanilla prompting.

Gaps and future work this paper leaves open

- Gap G3 (true-positive loss per weakness type): this paper is the main evidence for this gap. The authors plan "CWE-aware, cost-adaptive agent strategies" and human-in-the-loop review **[A]** [xiong2026sifting, p. 20]. No study yet tests a filter that is only applied to weakness types with low miss rates, on real security alerts.
- Gap G5 (other languages and SAST tools): listed as future work **[A]** [xiong2026sifting, p. 20].
- Gap G2 (recent, contamination-free comparisons): this is one of only three papers in our corpus that test on code newer than the models' training data, but it runs no deep-learning detector **[I]**.

Why it matters for your research

It gives the clearest numbers in our corpus on the main trade-off of SAST-plus-LLM pipelines (RQ3): far fewer false alarms in exchange for lost real vulnerabilities. Its per-weakness results feed RQ4, and its SAST-only figures give a clean RQ1 baseline **[I]**.

Connections to other papers

- du2026reducing finds the same trade-off in industry on non-security bugs. Together they form agreement G7 in the evidence matrix [du2026reducing, p. 8; xiong2026sifting, p. 18].
- CodeQL's OWASP numbers here closely match khare2025understanding (precision 0.58, recall 0.96), a rare cross-group replication (agreement G5) [khare2025understanding, p. 9; xiong2026sifting, p. 11].
- Its finding that injection-type warnings are easy fits khare2025understanding, where LLM detectors do best on self-contained weakness types such as command injection and SQL injection [khare2025understanding, p. 7].
- The authors argue that agents gain by gathering evidence step by step rather than from more context **[A]** [xiong2026sifting, p. 15]. This fits agreement G6 in the evidence matrix, that

missing context is a shared cause of failure; for example, 14.60% of false positives in li2026llm come from truncated context across functions [li2026llm, p. 15] **[I]**.

To Think or Not to Think: Evaluating LLM Reasoning and Agents in Vulnerability Detection

> **Takeaway:** In a carefully controlled experiment with four open LLMs, "thinking" modes and multi-agent designs mostly made models flag more code as vulnerable, which raises F1 without making them better at telling vulnerable code from its fixed version. They also cost several times more energy, and the quality of an explanation did not reveal whether the answer was right.

Item	Detail
Authors	Hua Beng Tan, Ratnadira Widyasari, Merve Astekin, Arda Goknil, Hasan Sözer, Yan Naing Tun, Erik Johannes Husom, Lwin Khin Shar, Lingxiao Jiang
Year	2026
Venue	Proceedings of the 41st IEEE/ACM International Conference on Automated Software Engineering (ASE 2026)
Status	Peer-reviewed (conference). Our copy is the publisher's version of record, supplied by you
Identifier	doi:10.1145/3832783.3837521
Paper type	Controlled (factorial) empirical study
Relevance to us	Medium: RQ1 (LLM-only), RQ2, RQ3, RQ5

In plain words

Large language models (LLMs, AI models trained on huge amounts of text and code) are now often used in two newer ways. Some have a "thinking" or reasoning mode: the model writes out a long chain of reasoning before answering. Others are used as agents: several LLM "roles" (for example, an author and a reviewer) talk to each other before deciding.

Both are claimed to help in finding vulnerabilities (security flaws in code). Earlier studies tested them separately and under different settings. This paper tests them together in one controlled design, so that each effect can be separated **[A]** [tan2026think, p. 1; tan2026think, p. 2]. It also asks whether "thinking" produces better explanations.

How they did it

The design is factorial: every combination of factors is tested **[V]** [tan2026think, p. 3]. The factors were:

1. Four open models: Nemotron-Nano-8B, Nemotron-Super-49B, Qwen3-4B and Qwen3-30B-A3B.
2. Thinking mode on or off.
3. Zero-shot (no examples) or few-shot (a few solved examples in the prompt).

4. Four designs: no agent, a single agent, a dual agent (one proposes, one reviews and decides) and a multi-agent team with four roles **[V]** [tan2026think, p. 4].

That gives 64 configurations. Extra "call-matched" runs gave a single model the same number of calls as the agent teams, for example by letting it revise its own answer **[V]** [tan2026think, p. 11]. Explanations were rated by two blinded humans (who did not know which mode produced them) and by an LLM judge **[V]** [tan2026think, p. 5].

There are no SAST tools (static analysers) and no fine-tuned deep-learning detectors **[I]**.

Data and setup

The data are PrimeVul Pair: 435 vulnerable C/C++ functions, each paired with its fixed (benign) version, 870 functions in total **[V]** [tan2026think, p. 3]. The model sees one function at a time, never the pair.

Two scores matter. F1 combines precision (share of flagged functions that are vulnerable) and recall (share of vulnerable functions flagged). Pair-wise correctness (P-C) is the share of pairs where both the vulnerable and the fixed version are labelled correctly. It tests whether the model really sees the flaw **[V]** [tan2026think, p. 5].

Models ran at temperature 0 (the most repeatable setting) with one run per configuration **[V]** [tan2026think, p. 4]. Code, prompts and data are public **[V]** [tan2026think, p. 12].

Key findings

- **[V]** Without agents, thinking raised mean F1 from 0.31 to 0.41 (Wilcoxon test $p = 0.023$; Cohen's $d = 1.37$, a large effect). Accuracy barely changed (0.518 vs 0.523), so the gain came from flagging more code [tan2026think, p. 6].
- **[V]** The best single-model F1 was 0.620, and the best pair-wise correctness was only 22.5% [tan2026think, p. 6].
- **[V]** The dual-agent design had the highest mean F1 (0.634) because it flagged 88.2% of all functions; its false-positive rate was 0.880 and its pair-wise correctness only 6.2% [tan2026think, p. 7].
- **[V]** F1 and P-C ranked designs differently: the multi-agent team had the best P-C in 11 of 16 settings, the dual agent in none [tan2026think, p. 7; tan2026think, p. 8].
- **[V]** In the dual-agent and team designs, thinking lowered F1 (by 0.054 and 0.066), while pair-wise correctness still rose slightly [tan2026think, p. 8].
- **[V]** A single model that revised its own answer four times reached mean P-C 24.7, vs 21.9 for the multi-agent team. The authors conclude that much of the agent benefit is simply extra computing budget [tan2026think, p. 11].
- **[V]** Thinking used 2.7–4.4 times more energy. All 64 configurations together cost about $2,448 and 481 kWh [tan2026think, p. 10].
- **[V]** In blinded ratings, explanations of correct and wrong answers scored the same (all $p > 0.3$) [tan2026think, p. 9].

Limitations and weaknesses

The authors note: one run per configuration, prompt and design choices that are representative rather than optimal, a rating scheme that measures perceived quality rather than truthfulness, and function-level data with no project context **[A]** [tan2026think, p. 11].

Our own concerns **[I]**:

- On this 50/50 dataset, a classifier that flags everything scores F1 0.667. No configuration beats it (best 0.665–0.666), so the "best" F1 is at the flag-everything level.
- A random guesser can reach at most 25% pair-wise correctness. Almost all results are at or below that level.
- Training-data contamination is not discussed; PrimeVul is public.
- Tests on 8 or 16 configuration averages have little statistical power.

Gaps and future work this paper leaves open

- Gap G1 (trivial baselines and realistic prevalence): this paper shows why F1 on balanced data must be compared with a flag-everything baseline, which it does not report.
- Gap G2 (same-benchmark comparison with SAST and deep learning): no traditional or deep-learning baseline is included **[I]**. The authors call for repository-level settings with dependency resolution, retrieval and tools **[A]** [tan2026think, p. 11].
- Gap G6 (cost): it gives detailed energy and dollar costs, but not cost per confirmed vulnerability **[I]**.

Why it matters for your research

It is the most rigorous test in our corpus of whether reasoning and agents help, and one of only two primary studies in our corpus (with deininger2026open) that test their main comparison for statistical significance **[I]**. For RQ3 it shows that higher F1 can be bought by flagging more, at real energy cost. For RQ5 it warns that a convincing explanation is not evidence of a correct answer [tan2026think, p. 9].

Connections to other papers

- yildiz2025benchmarking finds the same pattern with GPT models: decent F1 but near-random pair-wise accuracy. Both feed agreement G3 in the evidence matrix (LLMs over-flag and cannot separate vulnerable from fixed code) [yildiz2025benchmarking, p. 8; tan2026think, p. 7].
- On whether reasoning reduces false positives (contradiction C4), this paper is evidence "against": thinking raises recall, not discrimination. dubniczky2025castle reports the opposite, and deininger2026open finds the effect depends on the model [dubniczky2025castle, p. 14; deininger2026open, p. 17].
- ding2025vulnerability, which introduced the pair-wise metric used here, also finds all model families near or below chance on PrimeVul pairs [ding2025vulnerability, p. 9].

Benchmarking LLMs and LLM-based Agents in Practical Vulnerability Detection for Code Repositories

> **Takeaway:** On a new benchmark of 879 real C/C++ vulnerabilities, each paired with its fixed version, GPT-4o and GPT-4o-mini reached F1 scores of about 48–66% but correctly labelled both versions of a pair in at most about 20% of cases. On realistic, mostly-safe data their F1 fell to about 5–12%, so F1 on balanced data greatly overstates how useful these detectors are.

Item	Detail
Authors	Alperen Yildiz, Sin G Teo, Yiling Lou, Yebo Feng, Chong Wang, Dinil Mon Divakaran
Year	2025
Venue	Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (ACL 2025, Volume 1: Long Papers)
Status	Peer-reviewed (conference). Our copy is the published ACL Anthology version
Identifier	doi:10.18653/v1/2025.acl-long.1490
Paper type	Benchmark paper with comparative evaluation
Relevance to us	Medium: RQ1 (LLM-only), RQ2, RQ5

In plain words

Real vulnerabilities often depend on code outside the function where they appear. A practical detector should therefore look at related functions too. It should also tell a vulnerable function apart from its fixed version, which usually looks almost the same.

The authors build a benchmark, JITVul, for "just-in-time" detection: checking code at the moment a change (commit) is made **[A]** [yildiz2025benchmarking, p. 1]. They use it to compare plain large language models (LLMs, AI models trained on huge amounts of text and code) with LLM agents. An agent is an LLM that can call tools, here tools that look up the functions that call or are called by the target function **[V]** [yildiz2025benchmarking, p. 6].

How they did it

Three detection methods were compared **[V]** [yildiz2025benchmarking, p. 5; yildiz2025benchmarking, p. 6]:

1. Plain LLM: sees only the target function.
2. Dependency-augmented LLM: also sees the five most similar calling or called functions, chosen by word overlap.
3. ReAct agent: an LLM that alternates between reasoning and tool calls to fetch callers, callees and definitions on demand.

Each method was tried with four prompting styles: plain, chain-of-thought (asking the model to reason step by step), few-shot (adding 10 hand-made example pairs) and both together **[V]** [yildiz2025benchmarking, p. 6]. The main models were GPT-4o-mini and GPT-4o at temperature 0 (the most repeatable setting); Llama-3.1-8B and DeepSeek-Coder-V2-16B appear in an appendix **[V]** [yildiz2025benchmarking, p. 6; yildiz2025benchmarking, p. 15].

No SAST tools (static analysers) and no fine-tuned deep-learning detectors were tested **[V]** [yildiz2025benchmarking, p. 5].

Data and setup

JITVul contains 879 CVEs (public IDs for known vulnerabilities) from the PrimeVul dataset, covering 91 weakness types (CWEs), with at most 10 per type **[V]** [yildiz2025benchmarking, p. 4]. Each CVE gives two samples: the function as it was when the vulnerability was introduced, and the function after the fix. That makes 1,758 samples, half vulnerable **[V]** [yildiz2025benchmarking, p. 5]. The repositories are large, on average about 989,000 lines of code, and 91% C **[V]** [yildiz2025benchmarking, p. 5].

Two scores were used. F1 combines precision (share of flagged samples that are vulnerable) and recall (share of vulnerable samples flagged). Pairwise accuracy (pAcc) is the share of pairs where both versions are labelled correctly **[V]** [yildiz2025benchmarking, p. 6]. Code and data are released **[V]** [yildiz2025benchmarking, p. 2].

Key findings

- **[V]** With GPT-4o and GPT-4o-mini, F1 ranged from about 48% to 66%, but pairwise accuracy only from 1.02% to 20.17% [yildiz2025benchmarking, p. 6].
- **[V]** ReAct agents had the highest pairwise accuracy for every prompting style with the GPT models, by up to 16.61 points, while the non-agent methods usually had higher F1 [yildiz2025benchmarking, p. 7].
- **[V]** Some settings labelled over 90% of samples vulnerable, and precision was about 50% for all methods, so F1 rewards over-flagging [yildiz2025benchmarking, p. 7].
- **[V]** On the original PrimeVul test set, which is mostly safe code (695 vulnerable vs 25,216 safe functions), GPT-4o-mini's F1 fell to 5.41–12.21% [yildiz2025benchmarking, p. 7].
- **[V]** Adding related functions by word overlap did not consistently help and sometimes lowered pairwise accuracy [yildiz2025benchmarking, p. 7].
- **[V]** The most common error was calling both versions vulnerable: about 40–95% of errors for plain LLMs and 35–50% for agents [yildiz2025benchmarking, p. 8].
- **[V]** Agents called tools far more often on the vulnerable version than on the fixed one (5.85 vs 1.79 calls), so they analysed pairs inconsistently [yildiz2025benchmarking, p. 9].
- **[V]** With DeepSeek-Coder-V2, the dependency-augmented LLM reached F1 82.09 and pairwise accuracy 69.24, the best in the paper, while its agent stayed at 21.39 or below [yildiz2025benchmarking, p. 15].

Limitations and weaknesses

The authors say the point where a vulnerability was introduced may be traced imperfectly, the balanced dataset may bias F1, and some statistics (such as how many vulnerabilities span several functions) are missing **[A]** [yildiz2025benchmarking, p. 9].

Our own concerns **[I]**:

- Pairwise accuracy of 20% or less is below the 25% a random guesser can reach at most, so "agents are better" means "less bad".
- The agent advantage does not hold for DeepSeek-Coder-V2, so it is not robust across models.
- Training-data contamination is not discussed, although PrimeVul is public and probably in GPT-4o's training data.
- There are no repeated runs, variance or significance tests, and no cost data.
- The number of pairs per setting is not stated. Some values look like they were computed on about 119 pairs rather than 879, so compare settings with caution.

Gaps and future work this paper leaves open

- Gap G1 (trivial baselines and realistic prevalence): this paper is key evidence. It shows the F1 collapse from balanced to realistic data, and the authors call for evaluation guidelines for this kind of benchmark **[A]** [yildiz2025benchmarking, p. 2].
- Gap G2 (same-benchmark comparison with SAST and deep learning): no traditional or deep-learning baseline is included, and contamination is not controlled **[I]**.
- The authors also suggest combining LLMs with constraint solving or program analysis **[A]** [yildiz2025benchmarking, p. 9].

Why it matters for your research

This paper is mainly methodological evidence for RQ2. It explains why reported LLM results vary so much: F1 on balanced data looks moderate, while pairwise accuracy and realistic class ratios show near-random discrimination **[I]**. It does not compare LLMs with traditional tools.

Connections to other papers

- tan2026think finds the same pattern with open models on PrimeVul pairs: higher F1 by flagging more, with weak pair-wise accuracy. Both feed agreement G3 in the evidence matrix (LLMs over-flag and cannot separate vulnerable from fixed code) [tan2026think, p. 7; yildiz2025benchmarking, p. 8].
- Its F1 drop on realistic data is part of contradiction C5 (class balance flips rankings), together with shu2026evaluating [shu2026evaluating, p. 40; yildiz2025benchmarking, p. 7].
- The 22.70% random-guessing reference for pairs that it cites comes from ding2025vulnerability, which also finds GPT-4 below it [ding2025vulnerability, p. 9].
- Its finding that extra context alone does not help echoes xiong2026sifting, where giving a model more files did not help but step-by-step evidence gathering did [xiong2026sifting, p. 15].

Revisiting the Performance of Deep Learning-Based Vulnerability Detection on Realistic Datasets

> **Takeaway:** When four well-known deep-learning detectors scan whole real projects instead of hand-picked functions, their precision falls to 1–10%, so almost every alarm is false. Retraining on realistic data does not fix this, and the small comparison with two fine-tuned LLMs favours neither side clearly [chakraborty2024revisiting, p. 7; chakraborty2024revisiting, p. 9; chakraborty2024revisiting, p. 12].

Item	Detail
Authors	Partha Chakraborty, Krishna Kanth Arumugam, Mahmoud Alfadel, Meiyappan Nagappan, Shane McIntosh
Year	2024
Venue	IEEE Transactions on Software Engineering
Status	Peer-reviewed (journal). Our copy is the arXiv author pre-print (2407.03093v1), so page numbers follow the arXiv PDF
Identifier	DOI 10.1109/tse.2024.3423712; arXiv 2407.03093
Paper type	Replication and re-evaluation study with a new dataset (Real-Vul)
Relevance to us	High (RQ1, RQ2, RQ4, RQ5)

In plain words

Deep-learning (DL) detectors are neural networks trained on many examples of vulnerable and safe code. Some reported scores up to 99%. But they were tested on functions taken from code changes that fixed flaws, with roughly as many vulnerable as safe examples. Real use is different. A tool scans every function in a project, and vulnerable ones are rare. The share of truly vulnerable code is called prevalence. This paper builds a test set, Real-Vul, that mimics a whole-project scan. It then re-tests four detectors on it.

How they did it

- Detectors: LineVul, which reads code as a sequence of words using the CodeBERT model, and three graph-based models (DeepWukong, ReVeal, IVDetect) [chakraborty2024revisiting, p. 4; chakraborty2024revisiting, p. 5]. Graph-based models turn code into a network of statements and data flows.
- Step 1: re-run each model on its original dataset with a random 80/20 split [chakraborty2024revisiting, p. 5; chakraborty2024revisiting, p. 6].
- Step 2: test those models on Real-Vul (RQ1). Step 3: retrain them on Real-Vul, with and without balancing the classes (RQ2) [chakraborty2024revisiting, p. 6; chakraborty2024revisiting, p. 8].

- Step 4: side analyses, including "dead-code" augmentation, results per weakness group and severity, and two fine-tuned LLMs [chakraborty2024revisiting, p. 10; chakraborty2024revisiting, p. 12]. Dead-code augmentation adds harmless unused code to training examples so the model learns to ignore surface details.
- No static analysis tool was tested [chakraborty2024revisiting, p. 13].

Data and setup

- Real-Vul uses the 10 projects with the most vulnerabilities in the BigVul dataset, including Chromium, Linux, FFmpeg and OpenSSL. They cover 73% of BigVul's vulnerable samples [chakraborty2024revisiting, p. 3; chakraborty2024revisiting, p. 4].
- It has 5,528 vulnerable functions (4,418 train, 1,110 test) and 1,682,713 "uncertain" functions (769,464 train, 913,249 test) [chakraborty2024revisiting, p. 4].
- "Uncertain" means every other function in the project snapshot that does not match a known vulnerable function. These are not proven safe [chakraborty2024revisiting, p. 4].
- Time split: vulnerable functions are sorted by fix date; the first 80% train the model and the last 20% test it [chakraborty2024revisiting, p. 4].
- Training was not repeated; the authors say repeats were not feasible [chakraborty2024revisiting, p. 13]. LLM fine-tuning settings are not reported [chakraborty2024revisiting, p. 12]. Data and scripts are released [chakraborty2024revisiting, p. 2].

Key findings

- **[V]** On their original data, DeepWukong reaches F1 93% and LineVul 90% [chakraborty2024revisiting, p. 6]. F1 balances precision (share of alarms that are real) and recall (share of real flaws found).
- **[V]** On Real-Vul, precision falls to 1% (DeepWukong, LineVul), 2% (IVDetect) and 10% (ReVeal), while recall stays at 80–90%. F1 is 2–17% [chakraborty2024revisiting, p. 7].
- **[V]** Precision drops by up to 95 percentage points and F1 by up to 91 points [chakraborty2024revisiting, p. 1; chakraborty2024revisiting, p. 7].
- **[V]** Retrained on Real-Vul as is, DeepWukong and LineVul label everything as not vulnerable: F1 0% and AUC 50% [chakraborty2024revisiting, p. 8]. AUC measures how well a model ranks vulnerable above safe code; 50% is coin-flip level.
- **[V]** Retrained on balanced data, F1 is 2–36% and AUC 51.3–52.3% [chakraborty2024revisiting, p. 9].
- **[V]** Dead-code augmentation lifts LineVul F1 from 20% to 46% and AUC from 51.4% to 81.8% [chakraborty2024revisiting, p. 10].
- **[V]** Fine-tuned CodeLlama reaches F1 42% and AUC 67%; Mixtral reaches F1 46% and AUC 71%. **[A]** The authors conclude that LineVul outperforms general LLMs [chakraborty2024revisiting, p. 12].
- **[V]** Information-leak flaws are found best (F1 up to 33.6%). High-severity flaws are hardest (F1 9.06–10.89%) [chakraborty2024revisiting, p. 11].

Limitations and weaknesses

- **[A]** The authors note that some BigVul labels may be wrong, only 10 projects are used, "uncertain" functions may hide real flaws, balancing used one random sample, and only DL models were tested [chakraborty2024revisiting, p. 13].
- **[I]** Some numbers do not fit together. IVDetect shows precision 39% and recall 63% but F1 24% [chakraborty2024revisiting, p. 6]. Project sizes total 270,919 functions, while the text gives 1,682,713 uncertain functions [chakraborty2024revisiting, p. 4; chakraborty2024revisiting, p. 13]. The story holds, but exact figures are less reliable.
- **[I]** The LLM comparison is thin. Mixtral's F1 (46%) equals augmented LineVul's; only AUC favours LineVul. The LLM results also seem to come from a less imbalanced test set, which is not described [chakraborty2024revisiting, p. 10; chakraborty2024revisiting, p. 12].
- **[I]** Vulnerable labels come from BigVul, where an independent audit found only 39% correct [risse2025top, p. 8].
- **[I]** There are no confidence intervals for the main results. False-positive counts use different units per model, so they cannot be compared directly [chakraborty2024revisiting, p. 7].

Gaps and future work this paper leaves open

- Gap G2 (no same-benchmark comparison of LLMs, static analysis and DL on realistic code): **[A]** the authors themselves ask for static and dynamic analysis tools to be tested on Real-Vul [chakraborty2024revisiting, p. 13]. Real-Vul could serve as that benchmark.
- Gap G1 (results not re-scored at realistic prevalence): this paper shows why prevalence matters. Its precision figures are the kind of number most studies never report **[I]**.
- **[A]** Other author directions: better detection of high-severity flaws and better LLM specificity, meaning fewer false alarms on safe code [chakraborty2024revisiting, p. 11; chakraborty2024revisiting, p. 12].

Why it matters for your research

This is the strongest evidence in our set that realistic prevalence destroys precision (RQ2). It also gives results per weakness group and severity (RQ4). If you compare tools, always report precision at a realistic share of vulnerable code, not only F1 on balanced data.

Connections to other papers

- Agreement G1 (realistic evaluation collapses performance): ding2025vulnerability finds the same with cleaner labels [ding2025vulnerability, p. 7]; lu2025lab finds ReVeal and LineVul at F1 3.47–8.65 on 2025 vulnerabilities [lu2025lab, p. 13].
- Agreement G2 (noisy labels): this paper's 15% inconsistency in ReVeal [chakraborty2024revisiting, p. 3] sits beside risse2025top's 39–65% correct labels [risse2025top, p. 8].
- ding2025vulnerability focuses on wrong labels and copies; this paper focuses on prevalence. Together they explain most of the gap between lab and real scores **[I]**.

Top Score on the Wrong Exam: On Benchmarking in Machine Learning for Vulnerability Detection

> **Takeaway:** In three popular C/C++ benchmarks, only 39–65% of functions labelled "vulnerable" really are, and every truly vulnerable function needed code outside it to judge. A simple word-counting model gets close to top published scores, so high benchmark scores are weak evidence that a detector understands vulnerabilities [risse2025top, p. 8; risse2025top, p. 9; risse2025top, p. 11].

Item	Detail
Authors	Niklas Risse, Jing Liu, Marcel Böhme
Year	2025
Venue	Proceedings of the ACM on Software Engineering (FSE 2025)
Status	Peer-reviewed. Our copy is the arXiv version (2408.12986v2), so page numbers follow the arXiv PDF
Identifier	DOI 10.1145/3728887; arXiv 2408.12986
Paper type	Literature survey (81 papers), manual expert labelling study, and a simple-model probe experiment
Relevance to us	High (RQ2, RQ3, RQ5)

In plain words

Machine learning for vulnerability detection (ML4VD) trains models to find security flaws in code. Most of this research asks one question: "given one function, does it contain a flaw?" A function is a small, named block of code. This paper asks whether that question even makes sense. Two security experts checked samples from popular benchmarks by hand. Benchmarks are the shared test collections everyone reports scores on. The authors also built a deliberately simple model that only counts words. If such a model scores well, the benchmark is rewarding shortcuts rather than real understanding.

How they did it

- Survey: they searched 2020–2024 papers at top software engineering and security venues. Over 2,500 candidates were narrowed to 81 papers; one co-author did the screening [risse2025top, p. 3; risse2025top, p. 4].
- Expert labelling: two security researchers (3 and 5 years of experience) judged each function with a 15-minute limit. They read the code change that fixed the flaw and, where available, the CVE report [risse2025top, p. 6; risse2025top, p. 7]. A CVE is a public ID for a known vulnerability.

- Agreement was 82% (Cohen's kappa 0.64) on vulnerable vs secure and 98% (kappa 0.96) on context dependence [risse2025top, p. 6]. Cohen's kappa measures agreement beyond chance: 0 is chance level, 1 is perfect.
- Context test: for functions labelled "secure", they tried to write calling code that would make the function vulnerable [risse2025top, p. 10].
- Probe: a Gradient Boosting classifier, a standard simple method that combines many small decision trees, trained on word counts only [risse2025top, p. 7; risse2025top, p. 11].
- No LLM or static analysis tool was tested [risse2025top, p. 7].

Data and setup

- Benchmarks: BigVul, Devign and DiverseVul, all C/C++ [risse2025top, p. 6].
- 100 "vulnerable" functions per benchmark (300 in total) from 80 projects, plus 30 "secure" functions per benchmark (90 in total) [risse2025top, p. 6; risse2025top, p. 10].
- Share of vulnerable functions: Devign 47.9%, BigVul 4.5%, DiverseVul 5.4% [risse2025top, p. 7].
- Labelling took over 150 hours; the probe ran in 30 minutes on a laptop [risse2025top, p. 6; risse2025top, p. 11]. Probe settings and exact data splits are not reported in the paper text. Labels and scripts are released [risse2025top, p. 13].

Key findings

- **[V]** 88% of the 81 surveyed papers frame the task as a yes/no decision on one function, rising to 95% in 2024 [risse2025top, p. 5].
- **[V]** Only 39% (BigVul), 47% (Devign) and 65% (DiverseVul) of sampled "vulnerable" functions really are vulnerable [risse2025top, p. 8].
- **[V]** A common reason for wrong labels is that a function was changed only as a side effect of the fix (48%, 38% and 66% of wrong labels) [risse2025top, p. 8].
- **[V]** All 151 truly vulnerable functions needed context from outside the function. Most often this was other functions they call (43–52%) or the values passed in (40–47%) [risse2025top, p. 9].
- **[V]** For 82 of 90 "secure" functions, the authors could write a caller that makes the function vulnerable [risse2025top, p. 10].
- **[V]** The word-count model reaches F1 0.86 on BigVul against a best published 0.96, and 0.62 on Devign against 0.68. On DiverseVul it reaches only 0.11 against 0.48 [risse2025top, p. 11]. F1 is a 0–1 score balancing false alarms and missed flaws.
- **[A]** The authors conclude "there is currently no evidence that ML4VD techniques are actually capable of identifying security vulnerabilities at the function-level" [risse2025top, p. 2].
- **[A]** They argue that static analysis limited to one function faces the same context problem, and that combining static analysis with machine learning is promising [risse2025top, p. 12].

Limitations and weaknesses

- **[A]** The authors list: random samples of 100 per benchmark, subjective labelling, only three mainly-C benchmarks, only the function-level question, and only one kind of shortcut (word counts) [risse2025top, p. 11; risse2025top, p. 12]. The probe shows the data can be exploited, not that top models actually exploit it [risse2025top, p. 11].
- **[A]** "Secure" functions were not checked to be truly safe [risse2025top, p. 10].
- **[I]** The context rule is very broad. Almost any function that uses a pointer can be made vulnerable by a hostile caller (for example one passing NULL) [risse2025top, p. 10]. So the 151-of-151 and 82-of-90 results say more about how the question is framed than about any detector.
- **[I]** The best published scores were copied from other papers, not re-run on the same splits [risse2025top, p. 7].
- **[I]** The shortcut argument is weak for DiverseVul, where word counts do poorly [risse2025top, p. 11].
- **[I]** With 100 functions per benchmark, label figures carry about plus or minus 10 points of uncertainty. A kappa of 0.64 is only "substantial" agreement. A single person screened the survey.

Gaps and future work this paper leaves open

- Gap G1 (no shared evaluation protocol with trivial baselines): a word-count model near the top score shows why every result needs a simple baseline [risse2025top, p. 11].
- Gap G2 (no fair, context-aware three-family comparison): missing context is the root problem here. The gap report's study design borrows this paper's two-rater manual check with Cohen's kappa **[I]**.
- **[A]** Authors' future work: let models abstain, use other units (lines, files, slices across functions), give the whole repository as context, and combine static analysis with ML [risse2025top, p. 12; risse2025top, p. 13].

Why it matters for your research

This paper explains why a high F1 on BigVul or Devign proves little. For RQ2, it is a central source on label noise and on missing context. For RQ3 and RQ5, it argues for context-aware and hybrid methods, but tests neither. If you build your own test set, verify labels by hand with two raters and report a simple baseline.

Connections to other papers

- Agreement G2 (noisy labels): ding2025vulnerability finds 24–36% correct labels in its own audit [ding2025vulnerability, p. 4]; tashenova2026real finds 48.8% genuine in PHP [tashenova2026real, p. 11]; chakraborty2024revisiting finds 15% of ReVeal's vulnerable samples also listed as non-vulnerable [chakraborty2024revisiting, p. 3].
- Agreement G6 (missing context): ahmed2026secvuleval finds LLMs identify the needed context type at most 41.33% of the time [ahmed2026secvuleval, p. 9]; li2026llm traces all 108 RepoAudit misses to API modelling [li2026llm, p. 7].

- Contradiction C1: this paper's shortcut finding helps explain why fine-tuned models in fu2023chatgpt score 94% F1 on BigVul [fu2023chatgpt, p. 4] **[I]**.
- tashenova2026real reaches a similar result with a function-length baseline in PHP [tashenova2026real, p. 8].

SecVulEval: Context-Aware Benchmarking of LLMs for Vulnerability Detection

> **Takeaway:** Five LLMs had to point to the exact faulty lines in real C/C++ code and give the right reason. The best one reached only 23.83% F1, less than half its score on the easier "is this function vulnerable?" question. The study tests no static-analysis tool and no trained deep-learning detector, and our copy is the arXiv preprint, which was not peer-reviewed.

Item	Detail
Authors	Md Basim Uddin Ahmed, Nima Shiri Harzevili, Jiho Shin, Hung Viet Pham, Song Wang
Year	2026
Venue	Proceedings of the 3rd ACM International Conference on AI-Powered Software (AIware 2026)
Status	Peer-reviewed conference paper. Our copy is the earlier arXiv preprint 2505.19828v1 (May 2025, "Preprint. Under review."), with a different title. The authors match, but rlab could not confirm the identity automatically because the PDF has no DOI. Numbers in the published version may differ.
Identifier	DOI 10.1145/3805760.3814932; arXiv 2505.19828
Paper type	New benchmark plus an evaluation of 5 LLMs
Relevance to us	High — RQ2, RQ4, RQ5 (indirectly RQ1 and RQ3)

In plain words

A large language model (LLM) is an AI system trained on huge amounts of text and code, such as GPT-4.1 or Claude. Many papers ask an LLM a simple yes/no question: "Is this function vulnerable?" A function is one named block of code. A vulnerability is a flaw an attacker could exploit. This paper asks a harder question. Can the LLM point to the exact statements (lines of code) that cause the flaw, and explain why correctly?

To test this, the authors built a new dataset called SecVulEval. A benchmark or dataset like this is a fixed collection of examples with known answers, so tools can be scored and compared. The answer is sobering. When the reason must be right, scores drop by more than half **[V]** [ahmed2026secvuleval, p. 8].

How they did it

The authors collected real vulnerabilities from the NVD, the US public database of known security flaws. Each flaw there has a CVE number, a public ID for one specific vulnerability. They kept only CVEs linked to a fix in the project's own code history, and only fixes made in a single change **[A]** [ahmed2026secvuleval, p. 4; ahmed2026secvuleval, p. 5].

They then tested five LLMs inside a "multi-agent" pipeline. This means several LLM prompts work in sequence, each with its own job. One plans what to look for. One asks for missing code context, such as definitions of other functions the code calls. One gives the verdict and names the faulty statements. One double-checks the answer **[V]** [ahmed2026secvuleval, p. 7]. The models were three open models (22–33 billion parameters), GPT-4.1 and Claude-3.7-Sonnet [ahmed2026secvuleval, p. 8].

The scoring used precision, recall and F1. Precision is the share of flagged items that are really vulnerable. Recall is the share of real vulnerabilities that were found. F1 combines both into one number between 0 and 100%. Humans judged the statement-level answers by hand, using four rules. These rules are fairly lenient. For example, naming only the "sink" statement (where the harm happens) with the right reason counted as correct **[A]** [ahmed2026secvuleval, p. 8].

Data and setup

- SecVulEval holds 25,440 C/C++ functions (10,998 vulnerable, 14,442 not vulnerable) from 5,867 CVEs, 707 projects and 145 CWE types. A CWE is a standard category of weakness, such as "use after free" **[V]** [ahmed2026secvuleval, p. 6].
- The dataset has no duplicate functions. The authors cite earlier figures of up to 18.9% duplicates in older datasets **[V]** [ahmed2026secvuleval, p. 3].
- The detection test used only 300 functions, 120 of them vulnerable, drawn from the 25 most common CWE types **[V]** [ahmed2026secvuleval, p. 8; ahmed2026secvuleval, p. 13].
- Each model ran once, at a low randomness setting. No confidence intervals or significance tests are given for the main scores **[V]** [ahmed2026secvuleval, p. 8].
- Dataset, code and full prompts are public **[V]** [ahmed2026secvuleval, p. 3].

Key findings

- Statement-level F1 was low for every model: Claude 23.83%, GPT-4.1 22.38%, and down to 7.14% for the weakest open model **[V]** [ahmed2026secvuleval, p. 8].
- Function-level F1 was at most 53.89% (Claude), and function-level precision was 47.41% or lower for all five models **[V]** [ahmed2026secvuleval, p. 8].
- The two commercial models flagged far more functions: 214 and 215 of 300, against 116–124 for the open models. This gave them higher recall (about 73–76%) but lower precision **[V]** [ahmed2026secvuleval, p. 8; ahmed2026secvuleval, p. 13].
- Statement-level precision was only 5.65–15.35%, so most named lines or reasons were wrong **[V]** [ahmed2026secvuleval, p. 8].
- All models found short functions and missed long ones: the median length was 14–33 lines for found flaws against 64–71 lines for missed ones **[V]** [ahmed2026secvuleval, p. 13].
- LLMs were poor at saying which extra context they needed: at best 41.33% for one context type, and 0–5.31% for function arguments **[V]** [ahmed2026secvuleval, p. 9].
- The authors report over-flagging of null-pointer, use-after-free and integer-overflow flaws, for example flagging a null pointer even when the code checks for it. No counts are given **[A]** [ahmed2026secvuleval, p. 13].

Limitations and weaknesses

- There is no non-LLM comparison at all: no static-analysis tool, no trained deep-learning detector, and no simple single-prompt LLM. So the paper cannot tell us how LLMs compare with traditional tools **[I]**.
- Our own check of the results table suggests the statement-level recall was not computed as the paper defines it. Cases with the right function but the wrong reason were not counted as misses. Under the stated rule, Claude's recall would be about 27.7% instead of 53.23%, and its F1 about 19.8% instead of 23.83%. This is our arithmetic, not the authors' statement, and it means the headline numbers are, if anything, too optimistic **[I]** [ahmed2026secvuleval, p. 7; ahmed2026secvuleval, p. 8; ahmed2026secvuleval, p. 13].
- 300 test functions is small. Gaps of one or two points between models are probably within noise **[I]**.
- 40% of the test functions are vulnerable, far more than in real code. Precision would likely be lower in practice **[I]** [ahmed2026secvuleval, p. 13].
- The "correct context" answers were themselves generated by GPT-4.1, which is also one of the tested models. This makes the context test partly circular **[I]** [ahmed2026secvuleval, p. 6].
- Data contamination, meaning the models may have seen these public CVEs and fixes during training, is not discussed **[I]**.
- The authors themselves note subjective manual judging and labels checked on only 100 samples **[A]** [ahmed2026secvuleval, p. 12].

Gaps and future work this paper leaves open

The authors call for better statement-level detection and for training models on their labels **[A]** [ahmed2026secvuleval, p. 9; ahmed2026secvuleval, p. 10]. In our gap report, this paper is evidence for gap G1: published scores are rarely checked against realistic conditions, and stricter scoring halves F1 [ahmed2026secvuleval, p. 8]. It also relates to gap G2, the missing same-benchmark comparison of LLMs, static analysis and deep learning with file or repository context. This paper gives LLMs some context but runs no other tool family **[I]**.

Why it matters for your research

This paper shows that a "yes, this function is vulnerable" answer can hide wrong reasoning. That matters for RQ2: how you score changes the result. It also gives CWE-level and code-length weaknesses for RQ4. Do not use it to answer RQ1 (LLM against other tools), because it has no comparison arm **[I]**.

Connections to other papers

- Agreement G1 in the evidence matrix (realistic evaluation lowers scores): this paper's statement-level drop sits beside similar drops in chakraborty2024revisiting, ding2025vulnerability and shu2026evaluating [chakraborty2024revisiting, p. 7; ding2025vulnerability, p. 7; shu2026evaluating, p. 39].

- Agreement G6 (missing context is the shared cause of failure): risse2025top finds that all audited truly vulnerable functions need outside context [risse2025top, p. 9], and li2026llm also traces misses to missing context [li2026llm, p. 15].
- The finding that LLMs fail on longer code matches li2026llm at project scale [li2026llm, p. 9].
- kaniewski2026systematic includes this paper in its review **[I]**.

A Systematic Literature Review on Detecting Software Vulnerabilities with Large Language Models

> **Takeaway:** The largest review in our project (263 studies, up to November 2025) shows that LLM vulnerability-detection research is fragmented. Most studies use old, noisy, imbalanced C/C++ datasets with their own splits, so their numbers cannot be compared fairly. It does not compare LLMs with static-analysis tools or other detectors.

Item	Detail
Authors	Sabrina Kaniewski, Fabian Schmidt, Markus Enzweiler, Michael Menth, Tobias Heer
Year	2026
Venue	ACM Transactions on Software Engineering and Methodology (TOSEM)
Status	Peer-reviewed journal article. Our copy is the arXiv version 2507.22659v2 (December 2025), not the typeset journal version.
Identifier	DOI 10.1145/3815425; arXiv 2507.22659
Paper type	Systematic literature review (mapping study, no pooled performance numbers)
Relevance to us	High — RQ2, RQ5, RQ4 (qualitative only for RQ3; not RQ1)

In plain words

A systematic literature review (SLR) collects all studies on a question by following a written plan, so that others can repeat it. This SLR looks at how researchers use large language models (LLMs) to find vulnerabilities, meaning security flaws in source code. Here "LLM" is used broadly and even includes smaller models such as CodeBERT **[A]** [kaniewski2026systematic, p. 2].

It does not ask "which tool is best?" Instead it maps how studies set up their tests, and which datasets they use. A dataset (or benchmark) is a fixed set of code examples with known answers. The review's main message is that studies use such different datasets and data splits that their results are hard to compare **[A]** [kaniewski2026systematic, p. 2].

How they did it

The authors followed the Kitchenham guidelines, a standard method for software-engineering reviews **[A]** [kaniewski2026systematic, p. 5]. They searched three databases (IEEE Xplore, ACM Digital Library and arXiv) with a printed search string, then followed references forward and backward ("snowballing") **[A]** [kaniewski2026systematic, p. 7].

They kept studies that build and test LLM-based detectors. They excluded studies that only use an LLM to post-process the output of other tools **[A]** [kaniewski2026systematic, p. 8]. This matters for us. SAST tools (static application security testing: rule-based scanners that read code without

running it) often produce many false alarms. Studies where an LLM filters those alarms are therefore outside this review.

They also re-analysed 20 popular datasets themselves, measuring size, share of vulnerable samples and CWE coverage. A CWE is a standard category of weakness, such as "buffer overflow" **[V]** [kaniewski2026systematic, p. 28; kaniewski2026systematic, p. 30].

Data and setup

- 4,155 database hits; 263 studies included, of which 185 are peer-reviewed and 78 (29.7%) are arXiv preprints **[V]** [kaniewski2026systematic, p. 8].
- Time window: January 2020 to November 2025 **[A]** [kaniewski2026systematic, p. 2].
- A quality check is described, but its scoring and threshold are not reported. The number of reviewers and their agreement are also not reported **[I]**.
- The list of studies and other materials are public on GitHub **[A]** [kaniewski2026systematic, p. 3].

Key findings

- Most studies treat detection as a yes/no question: 195 studies use binary classification **[V]** [kaniewski2026systematic, p. 11].
- The most-used datasets are old, function-level C/C++ sets: Devign in 69 studies, Big-Vul in 63, ReVeal in 40 **[V]** [kaniewski2026systematic, p. 26].
- Class balance differs hugely between datasets. "Class balance" means the share of vulnerable examples. It is 4.5% in Big-Vul and 2.9% in PrimeVul, but 48.0% in Devign **[V]** [kaniewski2026systematic, p. 28].
- 87 studies use or create custom datasets, and only nine datasets are reused in more than ten studies **[V]** [kaniewski2026systematic, p. 35]. Few datasets come with fixed training and test splits **[V]** [kaniewski2026systematic, p. 39].
- 555 CWE types that may be used for mapping flaws appear in none of the analysed datasets **[V]** [kaniewski2026systematic, p. 34].
- The authors warn of data leakage: popular datasets are probably in LLM training data, so high scores may reflect memory rather than skill **[A]** [kaniewski2026systematic, p. 30; kaniewski2026systematic, p. 39].
- They note that models often perform worse outside curated benchmarks **[A]** [kaniewski2026systematic, p. 39; kaniewski2026systematic, p. 40].
- They recommend combining SAST and LLMs as a path to industry use: SAST finds candidates and the LLM filters false alarms, or the reverse. No numbers support this **[A]** [kaniewski2026systematic, p. 42].

Limitations and weaknesses

- It is a mapping study, not a performance synthesis. It pools no precision, recall or F1 numbers, so it cannot answer RQ1 **[I]**.
- The hybrid it recommends (SAST then LLM filtering) is excluded from its own corpus by design **[I]** [kaniewski2026systematic, p. 8; kaniewski2026systematic, p. 42].

- Only three databases were searched **[I]**.
- Screening and quality checks are not transparent **[I]**.
- Counting BERT-size models as LLMs blurs the line between "LLM" and "deep learning", which matters for our three-way comparison **[I]**.
- Nearly 30% of the included studies are preprints, which were not peer-reviewed **[I]** [kaniewski2026systematic, p. 8].
- The authors themselves name possible missed papers, selection bias and subjective classification **[A]** [kaniewski2026systematic, p. 43].

Gaps and future work this paper leaves open

The authors list seven open themes. These include repository-level detection and richer context, languages beyond C/C++, verified labels, datasets built after model training cutoffs, documented splits and workflow integration **[A]** [kaniewski2026systematic, p. 38; kaniewski2026systematic, p. 40; kaniewski2026systematic, p. 41]. In our gap report this paper supports gap G2 (benchmarks probably sit in training data, and post-cutoff datasets are recommended), gap G3 (it recommends SAST-then-LLM filtering but does not evaluate it) and gap G5 (evidence is concentrated on C/C++ and Java). None of its directions asks for same-benchmark comparisons of LLMs with SAST and DL detectors **[I]**.

Why it matters for your research

This is the strongest source for RQ2 (why numbers differ across studies). Its dataset statistics were recomputed by the authors, which makes them more reliable than claims copied from abstracts. It is also a main "competing review" to position against. It covers LLMs only, while our review covers LLMs, SAST, DL and hybrids together **[I]**.

Connections to other papers

- Agreement G1 in the evidence matrix (realistic evaluation lowers scores): this review agrees with the primary studies there [kaniewski2026systematic, p. 39].
- zhou2025large and sheng2025llms are the two other large reviews. All three find a C/C++, function-level focus. This one is the newest and the most systematic **[I]**.
- qin2026review shares the "benchmarks inflate results" message but is a much weaker narrative review **[I]**.
- It includes several of our primary studies, among them ahmed2026secvuleval, chakraborty2024revisiting, ding2025vulnerability, khare2025understanding, li2024iris, yildiz2025benchmarking and zhou2024comparison **[I]**.
- The alarm-filtering studies du2026reducing and xiong2026sifting supply the hybrid evidence this review leaves out [du2026reducing, p. 8; xiong2026sifting, p. 18].

Large Language Model for Vulnerability Detection and Repair: Literature Review and the Road Ahead

> **Takeaway:** Up to March 2024, LLM vulnerability detection was mostly small fine-tuned models judging single functions on heuristically labelled data, and no study was built into developers' daily tools. The review's only evidence on LLMs versus static-analysis tools is the authors' own earlier study.

Item	Detail
Authors	Xin Zhou, Sicong Cao, Xiaobing Sun, David Lo
Year	2025
Venue	ACM Transactions on Software Engineering and Methodology (TOSEM)
Status	Peer-reviewed journal article. Our copy is the arXiv version 2404.02525v3 (October 2024), not the typeset journal version.
Identifier	DOI 10.1145/3708522; arXiv 2404.02525
Paper type	Systematic literature review (descriptive, no pooled performance numbers)
Relevance to us	Medium — RQ2, RQ5 (marginally RQ1 and RQ4)

In plain words

This is a systematic literature review (SLR): a study that collects all relevant papers by a written, repeatable plan. It covers how large language models (LLMs) are used for two tasks. The first is detecting vulnerabilities (security flaws) in source code. The second is repairing them. It describes which models people used, how they adapted them, and what data they tested on. It then proposes a research roadmap.

It is a snapshot of the field up to March 2024. That is before most of the newer "agent" and whole-project studies in our project appeared **[I]**.

How they did it

The authors searched 13 top conferences and 4 journals by hand, then searched 7 databases automatically, then followed references (snowballing) **[A]** [zhou2025large, p. 5]. Each candidate paper was scored from 0 to 3 on five quality criteria and needed at least 12 points to stay in **[A]** [zhou2025large, p. 6]. One criterion is "published in a prestigious venue", which mixes venue fame with method quality **[I]**.

They sorted the studies into a taxonomy, that is, a structured list of categories. Models were grouped by architecture. Adaptation methods were grouped as fine-tuning (further training on

labelled examples), prompt engineering (careful instructions, with or without examples) and retrieval augmentation (fetching related information to add to the prompt) **[A]** [zhou2025large, p. 9].

Studies that used only older deep-learning models, such as graph neural networks, were excluded **[A]** [zhou2025large, p. 6].

Data and setup

- 1,535 records screened; 58 studies included **[V]** [zhou2025large, p. 4; zhou2025large, p. 7].
- 40 detection studies and 19 repair studies **[V]** [zhou2025large, p. 10; zhou2025large, p. 16].
- Time window: January 2018 to March 2024 **[A]** [zhou2025large, p. 5].
- The number of screeners, their agreement and the per-study quality scores are not reported **[I]**.

Key findings

- Encoder-only models dominated detection. These are models that read code and produce a label, rather than generating text. CodeBERT alone accounted for 26.1% of model uses **[V]** [zhou2025large, p. 7; zhou2025large, p. 8].
- Fine-tuning was by far the most common approach for detection (about 73% of studies), ahead of prompting (about 17%) and retrieval (about 10%) **[V]** [zhou2025large, p. 9].
- Most detection studies worked on single functions. Seven worked at line level, and only one at repository (whole-project) level **[V]** [zhou2025large, p. 18].
- Detection datasets mostly use heuristic labels: code changed by a security fix is called vulnerable and everything else safe. Earlier work showed these labels are noisy **[A]** [zhou2025large, p. 18].
- None of the 58 studies was built into developers' workflows or tools. All were tested offline on old data **[V]** [zhou2025large, p. 19].
- The only comparison of LLMs with SAST tools came from one study. SAST (static application security testing) tools are rule-based scanners. That study found that SAST finds few vulnerabilities with few false alarms, while LLMs find more but raise many false alarms **[A]** [zhou2025large, p. 22]. It is the survey authors' own study (zhou2024comparison).
- Cited studies report that LLMs are not robust to small code changes and struggle with rare flaw types and with flaws spread across several functions **[A]** [zhou2025large, p. 20; zhou2025large, p. 21].

Limitations and weaknesses

- The corpus is dated. It ends in March 2024, before most agent-based, project-scale and LLM-plus-SAST hybrid work **[I]**.
- The LLM-versus-SAST statement is effectively self-citation, not a synthesis of independent studies **[I]**.
- Excluding older deep-learning detectors means it cannot say whether LLMs beat them **[I]**.
- Screening details and quality scores are not public **[I]**.

- Half the scope is repair, so detection gets less depth **[I]**.
- Minor counting slips exist: by our count the tables list 57 unique studies, not 58 **[I]**.
- The authors discuss only one threat to validity: missing papers because of keyword choice **[A]** [zhou2025large, p. 24].

Gaps and future work this paper leaves open

The authors propose a small, manually verified, "living" test set; repository-level detection with long-context LLMs; agents and tool use; benchmarks that do not overlap with model training data; and integration into developer tools **[A]** [zhou2025large, p. 20; zhou2025large, p. 22; zhou2025large, p. 23]. In our gap report this paper supports gap G2. Surveys ask for verified, contamination-free, repository-level benchmarks, but not for a controlled comparison of LLMs, SAST and deep learning on the same data **[I]**.

Why it matters for your research

It sets the baseline for RQ2 and RQ5. Already in early 2024, label noise, small input units and missing workflow integration were known problems. It also shows how thin head-to-head LLM-versus-SAST evidence was at that time. For that comparison, cite the primary study zhou2024comparison, not this survey **[I]**.

Connections to other papers

- Contradiction C3 in the evidence matrix (LLM vs SAST) includes the authors' primary study, zhou2024comparison, where LLMs flagged up to 77.4% of functions [zhou2024comparison, p. 7].
- kaniewski2026systematic and sheng2025llms are later reviews. Both report that decoder-only models (text-generating models such as GPT-4) now dominate, which shows how fast this survey's picture dated **[I]**.
- Its call for agents, tool use and repository-level work has since been taken up by guo2025repoaudit, li2024iris and li2026llm. The independent study li2026llm, however, finds most such tools still produce mostly false warnings [li2026llm, p. 11].

LLMs in Software Security: A Survey of Vulnerability Detection Techniques and Insights

> **Takeaway:** A broad but loosely run survey of 58 LLM studies (2019–2024). Its most useful messages are that most work tests isolated code snippets and that, on whole codebases, LLMs usually assist static-analysis tools rather than replace them. Its numbers contain inconsistencies, so cite primary studies for any performance figure.

Item	Detail
Authors	Ze Sheng, Zhicheng Chen, Shuning Gu, Heqing Huang, Guofei Gu, Jeff Huang
Year	2025
Venue	ACM Computing Surveys
Status	Peer-reviewed journal article. Our copy is the arXiv version 2502.07049v2 (February 2025), not the typeset journal version. The published version may cover a different time window.
Identifier	DOI 10.1145/3769082; arXiv 2502.07049
Paper type	Narrative survey (not a systematic review)
Relevance to us	Medium — RQ2, RQ3, RQ4, RQ5

In plain words

A survey collects and describes many studies on one topic. This one covers large language models (LLMs), such as GPT-4, used to find vulnerabilities (security flaws) in software. It describes which models are used, which datasets, which techniques, and which problems remain.

It is a narrative survey. That means the authors chose papers by their own judgement, not by a written, repeatable search plan. Such surveys are useful for an overview but are less reliable for counting things than a systematic review **[I]**.

How they did it

The authors started from top security conferences and journals, then searched with key terms such as "vulnerability detection" and "LLM" every three weeks for two months **[A]** [sheng2025llms, p. 3; sheng2025llms, p. 4]. They excluded studies using older machine-learning models, such as recurrent neural networks, and studies of malware or network attacks **[A]** [sheng2025llms, p. 3].

No search string, database list, formal inclusion table or quality check is reported **[I]**. They screened roughly 500–600 papers and kept 58 highly relevant ones **[A]** [sheng2025llms, p. 3; sheng2025llms, p. 4].

Data and setup

- 58 studies from 2019–2024 **[A]** [sheng2025llms, p. 3].
- The corpus includes smart-contract (Solidity) studies, which our project treats as a separate topic **[I]** [sheng2025llms, p. 4].
- The authors estimate that about 60% of research in this area appears only as arXiv preprints, which are not peer-reviewed **[A]** [sheng2025llms, p. 26]. The preprint share of their own corpus is not reported **[I]**.
- They catalogue 42 datasets in one table **[V]** [sheng2025llms, p. 13].

Key findings

- Half of the studies (50%) target C/C++, then Java (21.1%) and Solidity (11.8%) **[V]** [sheng2025llms, p. 11].
- Only two of the 42 catalogued datasets work at the level of whole repositories (entire code projects). The rest work on single functions or files **[V]** [sheng2025llms, p. 13].
- About 40 studies (stated as 83%) test isolated code snippets. The authors argue that results on snippets look better than real-world detection **[A]** [sheng2025llms, p. 20].
- For whole codebases, "LLMs typically assist static analysis tools or support fuzzing tasks" rather than work alone **[A]** [sheng2025llms, p. 20]. Static analysis means tools that inspect code without running it. Fuzzing means feeding a program many random inputs to make it crash.
- 41.3% of studies add code-processing steps, such as syntax trees or retrieval of related code. The authors judge the gains as only modest, shrinking for flaws that span several files [V for the share, A for the judgement] [sheng2025llms, p. 16].
- Reported F1 scores range widely by dataset. F1 is a 0-to-1 score that balances found flaws against false alarms. The survey cites 0.21 on PrimeVul for one study, and CodeBERT at 0.099 on PrimeVul but 0.66 on another dataset. These are second-hand figures **[A]** [sheng2025llms, p. 18; sheng2025llms, p. 19].
- Named challenges include data leakage (test data that the model already saw in training, which inflates scores), complex context, high false-positive rates and weak performance on new, unknown flaws **[A]** [sheng2025llms, p. 14].
- LLMs give inconsistent explanations between runs, even when the detection is right **[A]** [sheng2025llms, p. 23].

Limitations and weaknesses

- The review method is weak: no search protocol, no quality appraisal, no flow counts. Its counts about the field are less reliable than those of kaniewski2026systematic **[I]**.
- Several numbers do not add up. It reports 58 studies but uses 56 for the language statistics. "About 40 studies (83%)" implies about 48 studies, not 58 **[I]** [sheng2025llms, p. 8; sheng2025llms, p. 10; sheng2025llms, p. 20].
- It defines precision as "true positive rate", which is actually the definition of recall **[I]** [sheng2025llms, p. 14].

- One claim about CWE-787 versus CWE-862 is cited to a reference that seems to be the wrong paper. Do not reuse that claim without checking the original **[I]** [sheng2025llms, p. 22].
- It lists F1 scores from different datasets side by side, which are not comparable **[I]** [sheng2025llms, p. 18].
- It offers no quantified comparison of LLMs with static-analysis tools or other deep-learning detectors **[I]**.

Gaps and future work this paper leaves open

The authors call for moving beyond snippets to whole projects and code changes, for better datasets (accurate labels, little leakage, repository-level annotations), and for better combined "neuro-symbolic" tools that pair LLMs with program analysis **[A]** [sheng2025llms, p. 3; sheng2025llms, p. 20; sheng2025llms, p. 25]. In our gap report this paper supports gap G2: surveys ask for verified, repository-level benchmarks, but not for a controlled comparison of LLMs, SAST and DL on the same data **[I]**.

Why it matters for your research

Use it for background on RQ2 and RQ5: its dataset catalogue and its list of challenges. Its point that LLMs assist static analysis in practice is relevant context for RQ3 on hybrids. Because of its weak method and inconsistent numbers, cite primary studies for any figure **[I]**.

Connections to other papers

- Agreement G1 in the evidence matrix (realistic evaluation lowers scores): this survey agrees with the primary studies there [sheng2025llms, p. 20].
- zhou2025large is an earlier, more systematic review with a similar scope. kaniewski2026systematic is newer and larger and also classifies this survey as non-systematic **[I]**.
- qin2026review relies on this survey as one of its main sources, so qin is not independent of it **[I]**.
- The survey cites ding2025vulnerability (PrimeVul) and li2024iris (IRIS, an LLM plus static-analysis hybrid), both analysed in full in our project **[I]**.

A Review of Source Code Vulnerability Detection Methods Based on Large Language Models: Models, Techniques, and Evaluation

> **Takeaway:** This is the review closest to ours: it also argues that benchmark design inflates LLM results. But it is a small, single-author narrative review in a venue with undocumented peer review, it leaves static-analysis tools out of scope, and it mostly repeats sources we already have. Use it to position our work, not as independent evidence.

Item	Detail
Authors	Letong Qin
Year	2026
Venue	Computers and Artificial Intelligence (Zeus Press)
Status	Listed as a published journal article, but peer review is not documented. The PDF has no received or accepted dates and no peer-review statement, and the venue is low-visibility. We weight it like grey literature. Our copy is the publisher PDF (DOI printed in it).
Identifier	DOI 10.70267/cai.v3n5.122137
Paper type	Narrative literature review (no search protocol)
Relevance to us	High for positioning (closest competing review); weak as evidence — RQ2, RQ5 (conceptual only for RQ1 and RQ3)

In plain words

This paper reviews how researchers use AI language models to find vulnerabilities (security flaws) in source code. It covers smaller "code Transformer" models, such as CodeBERT, and large language models (LLMs), such as GPT-4 **[A]** [qin2026review, p. 1].

Its main message is about trust in results. High scores on benchmarks are easily inflated by duplicate samples, wrong labels, random data splits and "temporal leakage", meaning that test examples come from the same period the model learned from **[A]** [qin2026review, p. 1]. A benchmark is a fixed test set with known answers. This framing is very close to our own review, which is why the paper matters to us.

How they did it

It is a narrative review: the author picked and discussed papers without a written search plan. No databases, search strings, screening steps, study counts or quality checks are reported **[I]** [qin2026review, p. 4]. It has 32 references in total **[I]**.

The author sorts methods by how they acquire knowledge about vulnerabilities, for example fine-tuning (further training on labelled examples), direct prompting, or combining models **[A]** [qin2026review, p. 4]. SAST tools (static application security testing: rule-based scanners that check

code without running it) are explicitly not reviewed. They appear only as baselines or helper tools **[A]** [qin2026review, p. 4].

Data and setup

- No new data or experiments; it re-reports numbers from cited papers **[I]**.
- Its main numerical example repeats a results table from the PrimeVul study. Five code models scored F1 62.88–68.26% on the older Big-Vul dataset but only 1.91–5.82% when moved to PrimeVul **[A]** [qin2026review, p. 12]. F1 is a 0-to-100% score that balances found flaws against false alarms. We checked these ranges against the original study, and they match [ding2025vulnerability, p. 7].
- Its quantitative claims rest mainly on three sources already in our corpus: ding2025vulnerability, gnieciak2025large and sheng2025llms **[I]** [qin2026review, p. 15].

Key findings

- Scores from a random data split should be called "within-benchmark performance", not a real-world detection rate **[A]** [qin2026review, p. 10].
- Testing on later code ("out-of-time" testing) should be the basic reliability check **[A]** [qin2026review, p. 10].
- Training-data contamination should be audited as one possible explanation for good scores, not assumed **[A]** [qin2026review, p. 10].
- Different LLM methods cannot be ranked against each other, because their datasets, model versions and test rules differ **[A]** [qin2026review, p. 12].
- On LLMs versus SAST, the review says SAST is "usually more stable" within its rules, LLMs are better at explaining, and the two are complementary, not replacements **[A]** [qin2026review, p. 12]. No numbers are given.
- It recommends a workflow in which SAST proposes candidates, an LLM reviews or explains them, and a human confirms. It says such a workflow should be judged by audit time and missed flaws, not only by F1 **[A]** [qin2026review, p. 8; qin2026review, p. 13].

Limitations and weaknesses

- It is not systematic. Its selection of studies cannot be checked for bias, so it cannot count as a systematic review **[I]**.
- The corpus is small and selective. It misses the project-scale and alarm-filtering studies that we have, such as li2026llm, du2026reducing and xiong2026sifting **[I]**.
- The LLM-versus-SAST conclusion is under-supported. SAST is out of scope, and the only head-to-head study it cites (gnieciak2025large) found LLMs above SAST on mean F1 (0.75–0.80 vs 0.26–0.55) [gnieciak2025large, p. 10]. The review does not mention this pro-LLM result **[I]**.
- It is not independent evidence. Its numbers come from studies already in our corpus. Counting it as extra support would count the same evidence twice **[I]**.
- It does not line up conflicting results study by study against their test setups **[I]**.

- There are editorial loose ends. Three opening questions are not matched to its four formal research questions, and a "C2" label is used but never defined **[I]** [qin2026review, p. 2; qin2026review, p. 4; qin2026review, p. 12].
- The venue is uncertain: there is no peer-review statement, no submission dates and no DOAJ listing. We did not confirm that it is predatory **[I]**.

Gaps and future work this paper leaves open

The author recommends out-of-time and project-separated testing, data cards (documents that describe how a dataset was built), recall at a fixed false-alarm budget, checks of each part of an explanation, studies that record audit time per real vulnerability, and SAST-plus-LLM systems judged by total audit cost **[A]** [qin2026review, p. 13]. In our gap report it appears under gap G1: it recommends better reporting but re-scores no published results itself. It also appears under gap G6: it calls for audit-time and total-cost measurement but supplies no figures **[I]**.

Why it matters for your research

Reviewers of our work are likely to point to this paper. It shares our framing, but it lacks what we add: coverage of LLMs, SAST, deep learning and hybrids together, plus a study-by-study audit that explains contradictions through test setup. So it strengthens our novelty claim rather than weakening it. Cite it for positioning only, not as evidence **[I]**.

Connections to other papers

- It relies on ding2025vulnerability for benchmark inflation. Agreement G1 in the evidence matrix (realistic evaluation lowers scores) should cite ding directly [ding2025vulnerability, p. 7].
- It relies on gnieciak2025large for LLM versus SAST, but leaves out that study's pro-LLM result. Contradiction C3 in the evidence matrix gives the full picture [gnieciak2025large, p. 10; khare2025understanding, p. 9].
- It also leans on sheng2025llms. kaniewski2026systematic is the stronger, systematic review with a similar "evaluation pitfalls" message **[I]**.
- Its suggested SAST-then-LLM workflow is tested in du2026reducing and xiong2026sifting, which find large false-alarm cuts but lose 12–25% of real bugs [du2026reducing, p. 8; xiong2026sifting, p. 18].

Research Gaps and Opportunities

A **research gap** is a question that the existing studies leave open and that a new study could answer. We list a gap only when two conditions hold. First, evidence from the analysed papers supports it: authors' own future-work statements, repeated limitations, or contradictions. Second, a **novelty check** found no paper that already fills it. A novelty check is a set of targeted literature searches run specifically to look for such a paper.

All novelty checks were run on 2026-11-12. We used 35 database queries on OpenAlex, arXiv and Crossref, limited to 2024 onwards or 2025 onwards, plus 12 web searches for very recent work. In the second pass, the arXiv service timed out on several queries. We covered arXiv instead with one further query and by reading the abstract pages of the two papers we most needed to check. "We found no study" therefore always means "we found none in these searches on that date", never "this has never been done".

The gaps are called **Gap 1 to Gap 6** here. In the project's gap report they are G1 to G6. They are not the same as Agreements 1 to 11 in the Overview.

The six gaps at a glance

The ranking combines how strong the evidence is with how feasible a study would be for a beginner.

Gap	Short name	Evidence	Feasibility for a beginner	Confidence that it is still open	Second-pass verdict
Gap 1	No common yardstick: published results are not re-scored against trivial baselines and realistic prevalence across studies	strong (9 or more papers)	easy	medium to high	unchanged
Gap 2	No like-for-like test of LLMs, full SAST and fine-tuned DL on new, real code with context	strong (6 or more papers)	moderate	medium	narrowed
Gap 3	Real bugs lost when LLMs filter SAST warnings are not controlled per vulnerability type	moderate (2 papers plus author future work)	easy to moderate	low to medium	unchanged
Gap 4	Hybrid tools (IRIS, RepoAudit) lack an independent replication that explains the conflicting results	moderate	moderate to hard	medium	unchanged
Gap 5	No real-vulnerability comparison of LLMs with standard SAST tools	moderate	moderate	low to medium	narrowed

	for PHP and Go				
Gap 6	No cost per confirmed vulnerability that includes human checking time and costs all three families	moderate	moderate	low to medium	narrowed

"Narrowed" means a paper found in the second pass covers part of the gap. The rest remains open and is described below.

Gap 1: No common yardstick for comparing published results

> **In one sentence:** many published LLM, SAST and DL results sit at or near the score of a "detector" that flags everything, yet nobody has systematically re-scored the published numbers across studies against such trivial baselines and at realistic prevalence.

Evidence that the gap exists

- **Results sit at the flag-everything level.** On a balanced test set, flag-all scores F1 0.667 **[I]**. One study's flag-all baseline scored 0.6693 against a best model of 0.7075 [shu2026evaluating, p. 11; shu2026evaluating, p. 17] **[V]**. Another study's best F1 was 0.665–0.666 [tan2026think, p. 7]. A third reports GPT F1 values of 48–66 [yildiz2025benchmarking, p. 6]. In a fourth, "always vulnerable" rows appear among the best F1 values [khare2025understanding, p. 16].
- **The same pattern appears at other prevalences.** On a sample with 35.4% vulnerable functions, LLM F1 was 0.405–0.615 against 0.523 for always-vulnerable [deininger2026open, p. 7] **[V]**. At realistic 1:44 prevalence, every LLM's precision was 2.3–3.8% against 2.21% for flag-all [deininger2026open, p. 8] **[V]**. In PHP, a baseline that only counts tokens matched or beat a random-forest model in 4 of 6 settings [tashenova2026real, p. 8] **[V]**. On 2025 vulnerabilities, by our calculation, 7 of 12 LLM F1 values fell below the flag-all level of about 3.87% [lu2025lab, p. 14] **[I]**.
- **Changing the balance changes the winner.** Fine-tuned GPT-4o wins on balanced data, smaller models win on the imbalanced test [shu2026evaluating, p. 17; shu2026evaluating, p. 40] **[V]**.
- **Scoring is unequal between families.** One study gives only LLMs a lenient rule for where a bug is located [gnieciak2025large, p. 6] **[V]**. One benchmark's prompt tells the LLM how it will be scored [dubniczky2025castle, p. 9] **[V]**. Scoring the exact line instead of the whole function halves F1 [ahmed2026secvuleval, p. 8] **[V]**.
- **Headline LLM-against-SAST numbers use five incompatible metrics** [gnieciak2025large, p. 10; dubniczky2025castle, p. 10; khare2025understanding, p. 9; zhou2024comparison, p. 5; li2026llm, p. 11] **[I]**.
- **Simple tricks score well.** A word-count classifier reaches F1 0.86 on Big-Vul [risse2025top, p. 11] **[V]**. GPT-4's pairwise score is below a random reference [ding2025vulnerability, p. 9] **[V]**.

- **The decision rule and the prompt can flip conclusions.** With the default cut-off, a good fine-tuned detector looked like it always said "safe"; the authors nearly concluded that trained detectors are no better than LLMs [deininger2026open, p. 11] **[V]**. Rewording the prompt changed the winning LLM [deininger2026open, p. 20] **[V]**.
- **Authors ask for better evaluation rules** [yildiz2025benchmarking, p. 2; ding2025vulnerability, p. 2; lu2025lab, p. 16] **[A]**.

Novelty verdict and closest prior work

Still open (unchanged after the second pass). Several studies now report trivial baselines for their own results: shu2026evaluating, deininger2026open and tashenova2026real; for lu2025lab, the baseline can be calculated. Each applies it only to its own numbers, and none re-scores other studies **[I]**. qin2026review recommends better reporting but recomputes no published results [qin2026review, p. 13; qin2026review, p. 12]. General "pitfalls" catalogues (CACM 2024, doi 10.1145/3643456; TOSEM 2025, doi 10.1145/3748647) re-score no vulnerability-detection results. A 2026 study re-scores models against a simple reference, but for warning triage, not vulnerability detection, and in a venue of unknown quality (doi 10.66308/air.e80134906). We found no study that recomputes published LLM, SAST and DL results across studies against trivial baselines and realistic prevalence (searches on 2026-11-12). **Confidence: medium to high.**

What filling it would contribute

A "re-scored results table" for the field. It would show which reported LLM advantages survive once each result is compared with flag-all and evaluated at realistic prevalence **[I]**. It would also give a short reporting checklist: full error counts, prevalence, trivial baselines, scoring unit, number of runs. A new study should credit the existing within-study baselines rather than claim to be first to use them.

A study a beginner could run

- **Type:** secondary analysis of published numbers. No models are run.
- **Data:** the results tables of the 20 primary studies in this project. Extract every row (study, tool, dataset, setting) that reports enough numbers to rebuild the counts of correct and wrong answers.
- **Method:** for each row compute MCC, Youden's J (recall minus false-positive rate), the F1 margin over flag-all, and the precision the tool would have at 1%, 5% and 17% prevalence. The flag-all F1 at prevalence p is 2p divided by (1 + p).
- **Validity check:** where a study published its own baseline, check that your calculation reproduces it.
- **Analysis:** within each study, rank tools before and after re-scoring and count how often the ranking changes. Do not pool datasets that the methodology comparison marks "not comparable". Report how many rows fail to beat flag-all.

Feasibility

Easy [I]. A spreadsheet or a short Python script is enough; no GPU and no API cost. We estimate 2–4 weeks. It can go straight into the comparative review as an added-value table.

Risks

Many rows lack the numbers needed (true negatives or false-positive rate), which can bias the sample; list those rows instead of dropping them silently. Rounding in published tables adds error. Lenient scoring rules cannot be undone afterwards. Copying errors are likely, so extract twice and spot-check against the PDFs.

Gap 2: No like-for-like test of LLMs, full SAST and fine-tuned DL on new, real code with context

> **In one sentence:** one study now compares all three families on the same functions, but with lightweight SAST tools, isolated functions and old vulnerabilities; nobody has done it with a full semantic SAST tool such as CodeQL, on vulnerabilities published after the models were trained, and with surrounding file or repository context.

Evidence that the gap exists

- **Two studies run all three families, each in a limited setting.** khare2025understanding runs CodeQL on artificial data only [khare2025understanding, p. 7] **[V]**. deininger2026open runs 8 LLMs, Flawfinder, Cppcheck, Semgrep and two CodeBERT detectors on the same 1549 C/C++ functions [deininger2026open, p. 6; deininger2026open, p. 10] **[V]**. Its limits define what is left: CodeQL could not run because isolated functions cannot be built [deininger2026open, p. 10] **[V]**; all vulnerabilities date from 2008–2022, before the models' training cutoffs, and recall is 7–15 points higher on pre-2020 ones [deininger2026open, p. 24] **[V]**; it is function level, one dataset and C/C++ only [deininger2026open, p. 25] **[A]**.
- **That study shows why the gap matters.** Fine-tuned CodeBERT leads on balanced accuracy, Semgrep and Flawfinder lead on precision at realistic prevalence, and every LLM is at 2.3–3.8% precision [deininger2026open, p. 8; deininger2026open, p. 10] **[V]**.
- **Other studies miss a family.** zhou2024comparison has no classic DL detector [zhou2024comparison, p. 4] **[V]**. lu2025lab tests DL and LLMs on 2025 vulnerabilities but has no SAST tool [lu2025lab, p. 4] **[V]**. tashenova2026real has no LLM [tashenova2026real, p. 13] **[V]**.
- **Results reverse across setups**, both for LLMs against SAST [gnieciak2025large, p. 10; khare2025understanding, p. 9; li2026llm, p. 7] and for LLMs against DL [fu2023chatgpt, p. 4; ding2025vulnerability, p. 9; shu2026evaluating, p. 40] **[V]**.
- **Data leaks and training-data overlap are rarely controlled.** Exact copies between training and test data reach 18.9% [ding2025vulnerability, p. 4] **[V]**. Benchmarks are probably in LLM training data [kaniewski2026systematic, p. 30] **[A]**. Only three studies test on code

published after the models were trained [xiong2026sifting, p. 15; li2026llm, p. 5; lu2025lab, p. 4], and none of them runs all three families **[I]**.
- **Authors call for it** [chakraborty2024revisiting, p. 13; ding2025vulnerability, p. 10] **[A]**.

Novelty verdict and closest prior work

Narrowed. deininger2026open fills the function-level, older-vulnerability, C/C++ version with lightweight SAST. A second three-family comparison exists for Python, Java and JavaScript at function level: Zhang et al. 2025 (arXiv 2503.01449, preprint, read as abstract only). It mentions no time separation and does not name its SAST tools. Dahiya et al. 2026 (arXiv 2605.23243, preprint, abstract only) has no SAST and no DL arm, so it does not fill the gap. Other near misses lack DL (RealVuln, arXiv 2604.13764) or SAST (VulEval, arXiv 2404.15596). We found no study that runs semantic SAST, fine-tuned DL and LLMs on the same real, de-duplicated, post-cutoff targets with file or repository context and a shared scoring unit (searches on 2026-11-12). **Confidence: medium.** Near misses now lack only one or two conditions, so re-check before publishing.

What filling it would contribute

The first like-for-like answer to RQ1 on realistic, recent code. Comparing results on older and newer vulnerabilities would also estimate how much of the LLM advantage comes from having seen the test data during training. It would test whether deininger's finding (DL best on balanced accuracy, SAST best on precision) still holds with context and with CodeQL.

A study a beginner could run

- **Data:** Python or Java vulnerabilities from GitHub Security Advisories or OSV, split into about 100 disclosed after the newest model's training cutoff and about 100 from before. Use each vulnerable function and its fixed version, and add unchanged functions from the same files to reach a realistic ratio such as 1:20. Have two people re-check 50 positive labels.
- **Arms:** Semgrep and CodeQL; a CodeBERT or UniXcoder fine-tuned on older data with project-separated splits; two or three open 7–14B LLMs, plain and step-by-step prompts, three runs each.
- **Baselines:** flag-all, random, and a function-length classifier.
- **Metrics:** recall, precision and MCC at the natural ratio; pairwise correctness on pairs; cost.
- **Analysis:** McNemar tests between arms on identical functions; bootstrap confidence intervals by project; older-versus-newer difference per arm.
- **Cheaper variant:** reuse the public VentiVul dataset of 2025 vulnerabilities [lu2025lab, p. 2] and add a SAST arm, an LLM with file context, and a flag-all baseline.

Feasibility

Moderate [I]. All tools are free and a 7B model runs on a laptop GPU or Colab. The hard parts are building the dataset and mapping SAST warnings to functions. Expect 6–10 weeks. A

CodeQL arm on C/C++ projects such as Linux or Chromium requires building them, which is hard for a beginner.

Risks

Labels from fix commits are noisy. Mapping warnings to functions can disadvantage SAST, so also report file-level results. Fine-tuning data for Python may be small. LLM results depend on the prompt. Some advisories are public before the official vulnerability date, so leakage may remain. Results may not generalise beyond one language.

Gap 3: Real bugs lost when LLMs filter SAST warnings

> **In one sentence:** letting an LLM throw away SAST false alarms works, but it also throws away real vulnerabilities, unevenly across vulnerability types, and no study has tested a filter that is only applied where it is known to be safe.

Evidence that the gap exists

- **One study measures the loss per vulnerability type.** The best agent cuts the false-alarm rate on the OWASP test suite from 98.3% to 6.3% [xiong2026sifting, p. 12] but drops 22.25% of true vulnerabilities [xiong2026sifting, p. 18] **[V]**. The miss rate is low for injection bugs (1.10% for SQL injection, 0.41% for cross-site scripting) but 77.17% for weak cryptography and 84.50% for weak hashing [xiong2026sifting, p. 18] **[V]**. The authors plan vulnerability-type-aware strategies [xiong2026sifting, p. 20] **[A]**.
- **An industrial study shows the same trade-off.** It removes 94–98% of false alarms while keeping 75–88% of real bugs, on general bug types [du2026reducing, p. 8] **[V]**.
- **Recommended but not reviewed.** The largest systematic review recommends SAST-then-LLM filtering pipelines [kaniewski2026systematic, p. 42] **[A]**, but excludes such studies from its corpus [kaniewski2026systematic, p. 8].
- **Real-world samples are small:** 50 alerts per set [xiong2026sifting, p. 13] **[V]**.

Novelty verdict and closest prior work

Still open (unchanged). Several 2025–2026 preprints measure or reduce the loss, mostly on artificial suites: ZeroFalse (arXiv 2510.02534), an adjudication study reaching high recall on Juliet and similar suites (arXiv 2607.09979), QASecClaw (arXiv 2605.01885), a study of the same trade-off for dynamic scanners (arXiv 2610.08406) and Trusting the Tool (arXiv 2610.32876). We found no study that tests a filter applied only to vulnerability types with proven low miss rates, evaluated on held-out real-world security alerts with full trade-off curves (searches on 2026-11-12). **Confidence: low to medium**, because this area moves fast.

What filling it would contribute

A practical view of filtering: how many real vulnerabilities are lost per type for a given reduction in false alarms, plus a simple rule that practitioners can adopt.

A study a beginner could run

- **Data:** the OWASP Benchmark (Java, 2,740 labelled cases with an official scorer) for development, and about 100 SAST alerts from recent open-source Java projects, labelled by two people, for testing.
- **Method:** run Semgrep and CodeQL; ask two open 7–8B LLMs whether each alert is real, with a confidence score, three runs.
- **Policies compared:** no filter; filter everything; filter only the vulnerability types whose miss rate was below 5% on one half of OWASP, then test on the other half and on the real set.
- **Metrics:** share of false alarms removed, share of real bugs kept, and an expected checking cost for cost ratios from 1:10 to 1:100 between a false alarm and a missed bug.

Feasibility

Easy to moderate [I]. Labels and the scorer are public, Java SAST runs easily on OWASP, and 7B models run locally. Expect 4–6 weeks. The main effort is labelling the real-world alerts.

Risks

OWASP is artificial and probably in LLM training data. Groups per vulnerability type are small. Human labellers can be biased.

Gap 4: No independent replication of hybrid tools that explains the conflicting results

> **In one sentence:** IRIS and RepoAudit look strong in their authors' papers and weak in the one independent study, and no one has re-run them in a way that shows which difference (targets, metric or labeller) causes the gap.

Evidence that the gap exists

- **RepoAudit.** Authors: 78.43% precision and all 21 known bugs found, one run per setting [guo2025repoaudit, p. 8; guo2025repoaudit, p. 17] **[V]**. Independent study: 0 of 110 known C/C++ vulnerabilities and 92.77% false warnings [li2026llm, p. 7; li2026llm, p. 11] **[V]**, at up to $1,247.66 per project [li2026llm, p. 16] **[V]**.
- **IRIS.** Authors: 55 against 27 of 120 vulnerabilities compared with CodeQL [li2024iris, p. 8] **[V]**. Independent: 97.80% against 77.00% false warnings on new Java projects [li2026llm, p. 12] **[V]**.

- **The independent study's labeller is not independent.** It labels warnings with Codex, a tool it also evaluates [li2026llm, p. 11] **[V]**.
- **Gains need a strong LLM.** IRIS improves on CodeQL only with GPT-4; smaller LLMs make the false-discovery rate worse [li2024iris, p. 10] **[V]**.

Novelty verdict and closest prior work

Still open (unchanged). li2026llm is itself the only independent re-evaluation in our searches. A 2026 preprint re-runs software-engineering technique papers in general (arXiv 2609.00468), not these tools. New agent tools (arXiv 2606.22263, arXiv 2604.11950) are not replications. We found no study that re-runs IRIS or RepoAudit on the original benchmark and on new targets with the same blinded human labelling (searches on 2026-11-12). **Confidence: medium**; replications may exist in artifact-evaluation reports that search engines do not index.

What filling it would contribute

It tells practitioners whether published hybrid gains carry over to new code, and tells researchers which evaluation choices drive the reported differences.

A study a beginner could run

- **Design:** 2 by 2. Target set (original benchmark subset against recent Java vulnerabilities) crossed with metric (the original "vulnerabilities detected" against the share of false warnings), on the same runs.
- **Data:** 30 vulnerabilities from the IRIS benchmark across 4 types plus 10–15 recent Java vulnerabilities. IRIS code and data are public [li2024iris, p. 2].
- **Arms:** IRIS with one affordable model and one open model; CodeQL alone as baseline.
- **Labelling:** at least 100 warnings labelled by two people who do not know which tool produced them; report agreement (Cohen's kappa).

Feasibility

Moderate to hard [I]. Building Java projects for CodeQL and running a hybrid pipeline is demanding. A reduced IRIS-only replication on 20–30 vulnerabilities with an open model is feasible in about 8 weeks. RepoAudit (C/C++, high API cost) is not recommended for a beginner.

Risks

The original GPT-4 versions are being retired. CodeQL versions change. Builds can fail. API costs can grow.

Gap 5: No comparison of LLMs with standard SAST tools on real PHP and Go vulnerabilities

> **In one sentence:** almost all evidence is about C/C++ and Java; for PHP, the only study in our set tests no LLM and uses a home-made analyser, and for Go we found nothing comparing LLMs with SAST.

Evidence that the gap exists

- **The field is concentrated on C/C++ and Java** [kaniewski2026systematic, p. 28] **[V]**. 555 vulnerability types appear in no analysed dataset [kaniewski2026systematic, p. 34] **[V]**. The review calls for Python, Java, Rust and multi-language benchmarks [kaniewski2026systematic, p. 40] **[A]**.
- **Many authors list other languages as future work** [guo2025repoaudit, p. 10; li2024iris, p. 10; du2026reducing, p. 10; xiong2026sifting, p. 20; li2026llm, p. 18; deininger2026open, p. 25; lu2025lab, p. 17] **[A]**.
- **The multi-language studies are thin.** The 7-language study has no SAST arm [shu2026evaluating, p. 7] **[V]**. zhou2024comparison covers Python with only about 8–10 test vulnerabilities per language [zhou2024comparison, p. 6] **[I]**.
- **The only PHP study cannot answer the question.** tashenova2026real built a benchmark of 1757 real vulnerabilities in 1281 WordPress plugins [tashenova2026real, p. 3] **[V]**. Its SAST arm is an unnamed, home-made analyser that finds only 1.7–1.8% of vulnerabilities [tashenova2026real, p. 9] **[V]**; the authors blame that implementation, not the technique [tashenova2026real, p. 13] **[A]**. No LLM was run [tashenova2026real, p. 13] **[V]**.

Novelty verdict and closest prior work

Narrowed. Neef et al. 2026 (arXiv 2606.21397; to appear at an ARES 2026 workshop; read as abstract only) tests six LLMs on real WordPress-plugin vulnerabilities. According to its abstract, the best model detected 63%, and it names no SAST baseline. deininger2026open cites it as a comparison of open and frontier LLMs on WordPress plugins [deininger2026open, p. 3]. Zhang et al. 2025 (arXiv 2503.01449, abstract only) covers Python, Java and JavaScript at function level. A PHP SAST benchmarking framework exists without LLMs (Entropy 2025, doi 10.3390/e27090926). No Go-specific study turned up. We found no study comparing LLMs with an established SAST tool on real vulnerabilities in PHP or Go, and only function-level, snippet-level or artificial-repository evidence for JavaScript and Python (searches on 2026-11-12). **Confidence: low to medium**; the Neef et al. full text may contain a tool baseline its abstract does not mention.

What filling it would contribute

A test of whether "LLMs have higher recall and lower precision" holds in web languages, where injection bugs (which LLMs handle best [xiong2026sifting, p. 18]) are common **[I]**.

A study a beginner could run

- **PHP data:** the expert-checked subset released with tashenova2026real (125 vulnerable and 247 safe functions) [tashenova2026real, p. 11], plus the plugin versions for file-level runs,

rebuilt with the released loader [tashenova2026real, p. 14]. If Neef et al. released their plugin set, reuse it so results can be compared.
- **Go data:** about 60 Go vulnerabilities from the Go vulnerability database (OSV).
- **Arms:** established SAST (Semgrep; for PHP also a taint tool such as Psalm or Progpilot; for Go, gosec); two open LLMs at function level and at file level; the token-count baseline [tashenova2026real, p. 8]; and flag-all.
- **Metrics:** recall on known vulnerabilities; precision on a hand-labelled sample of warnings; cost; per-type recall with confidence intervals.

Feasibility

Moderate [I]. The tools are free and need no build step for PHP and Go, and PHP labels already exist. The main cost is labelling warnings that fall outside the labelled subset.

Risks

Label quality: only 48.8% of cleaned positive PHP labels were genuine [tashenova2026real, p. 11]; using the expert-checked subset reduces this. 125 positives give wide confidence intervals. Tools cover different vulnerability types. Old plugin versions may be gone; about 54% of indexed vulnerabilities could not be recovered in the PHP study [tashenova2026real, p. 3]. Older vulnerabilities are likely in LLM training data.

Gap 6: No cost per confirmed vulnerability that includes human checking time

> **In one sentence:** we now know the computing cost of an LLM per real vulnerability found, but not the far larger cost of the human time spent checking the many false alarms, and not the matching cost for SAST and DL on the same code.

Evidence that the gap exists

- **Most studies report no money cost.** 13 of our 20 primary studies report none [I, our count]. The seven that do are du, guo, li2026llm, shu, tan, xiong and deininger [du2026reducing, p. 8; guo2025repoaudit, p. 7; li2026llm, p. 16; shu2026evaluating, p. 42; tan2026think, p. 10; xiong2026sifting, p. 17; deininger2026open, p. 7].
- **Where measured, cost is decisive.** Per project, cost ranges from $2.54 in the RepoAudit paper to an average of $172.77 and a maximum of $1,247.66 in the independent study [guo2025repoaudit, p. 7; li2026llm, p. 16] **[V]**. Reasoning mode costs 4.5–12.1 times more per task [deininger2026open, p. 18] **[V]**.
- **Compute cost per true positive exists only for LLMs.** It is USD 0.003–0.212 and 3.7–119.6 kJ per true positive at 1:44 prevalence [deininger2026open, p. 8] **[V]**. Flawfinder is given as about zero; no figure is given for CodeBERT, Cppcheck or Semgrep [deininger2026open, p. 8; deininger2026open, p. 11] **[V]**.

- **Human checking is left out.** Each LLM true positive comes with 25.6–41.9 false positives [deininger2026open, p. 8] **[V]**, but analyst time is not costed **[I]**. At such ratios, analyst time would plausibly dominate a compute cost of a few cents **[I]**.
- **Part of the energy figures are estimates.** Some energy values per true positive come from calculation estimates, not measurement [deininger2026open, p. 14] **[V]**.
- **Authors call for audit-time measurement** [li2026llm, p. 18; qin2026review, p. 13] **[A]**.

Novelty verdict and closest prior work

Narrowed. deininger2026open gives compute cost per true positive for LLMs but no human checking time and no costed SAST or DL arm. A 2026 preprint argues that checking capacity, not bug finding, is the bottleneck, but its abstract reports no measured time (arXiv 2605.24632). Vendor benchmarks report dollars per true positive for their own products, but they are not peer-reviewed, have conflicts of interest and exclude analyst time. We found no peer-reviewed study reporting cost per confirmed vulnerability that includes human checking time, for LLM, SAST and DL arms on the same targets (searches on 2026-11-12). **Confidence: low to medium.**

What filling it would contribute

A decision-ready number for practitioners: dollars and analyst minutes per real vulnerability found, for each family **[I]**.

A study a beginner could run

Add it to the Gap 2 study as a second outcome. Record compute cost per arm; for SAST and DL use processor time times a stated hourly price, so they are not left at "about zero". Time two people checking a random 50 warnings per arm. Cost per confirmed vulnerability equals compute cost plus checking minutes times an hourly wage, divided by the number of confirmed vulnerabilities. Report it at realistic prevalence and for a range of wages.

Feasibility

Moderate **[I]**. Cheap as an add-on to Gap 2; on its own it needs the same set-up.

Risks

Checking time depends on the checker's skill and on the order in which warnings are shown. API prices change, and one model's price varied up to 14 times between providers [deininger2026open, p. 5].

Is our own review new? (positioning check)

Besides the six gaps, we checked whether a review like ours already exists. No separate follow-up topic novelty checks have been saved in the reports folder yet; this positioning check is recorded in the gap report.

Verdict. We found no review that both (1) combines evidence across all three families plus both kinds of hybrid, and (2) explains specific contradictions between studies through their test setup (searches on 2026-11-12). **Confidence: medium.**

- kaniewski2026systematic covers LLM detection only and excludes studies that filter tool output [kaniewski2026systematic, p. 8].
- zhou2025large predates almost all hybrid, agent and project-scale evidence; its only LLM-against-SAST study is the authors' own preprint [zhou2025large, p. 22].
- sheng2025llms is a narrative survey that does not compare families on matched results **[I]**.
- qin2026review is the closest competitor. It shares our "evaluation reliability" framing [qin2026review, p. 10], but it has no search protocol, puts SAST out of scope [qin2026review, p. 4] and reuses studies we already cover [qin2026review, p. 15], so it is not independent evidence **[I]**.

[I] Our review should present itself as a critical comparative review, not a full systematic review, because it has 24 papers. Its distinct contribution is the cross-family, contradiction-explaining angle. The Gap 1 re-scoring table would make that contribution concrete.

Next steps checklist

1. Decide whether to run an optional third pass that adds the two high-priority papers below (Zhang et al. 2025 and Neef et al. 2026).
2. If yes, find a legal open-access copy or ask your library, then analyse both papers from full text and re-check Gap 2 and Gap 5.
3. Screen the Information and Software Technology 2026 paper (doi 10.1016/j.infsof.2026.108213); we could not retrieve its abstract.
4. Build the Gap 1 re-scored results table. It is the easiest gap and adds the most value to the review.
5. Frame the paper as a critical comparative review and cite qin2026review and kaniewski2026systematic as the nearest reviews.
6. Re-run the novelty searches for Gap 2, Gap 3 and Gap 5 shortly before submitting, because these areas move fast.
7. Where a paper was read from its arXiv copy, check key numbers against the published version before quoting them.
8. If you want a follow-up empirical study, pick Gap 3 (4–6 weeks) or the cheaper variant of Gap 2 that reuses VentiVul.

Recommended papers not yet included

These were found during novelty checks. They are **not analysed**; anything we say about them comes from abstracts or titles only.

1. **Zhang et al. 2025**, Benchmarking Large Language Models for Multi-Language Software Vulnerability Detection (arXiv 2503.01449, preprint). High priority. A second same-data comparison of LLMs, fine-tuned small models and SAST, and the only one in Python,

Java and JavaScript. Relevant to RQ1, Gap 2 and Gap 5. Our first-pass searches returned it, but we missed it.

2. **Neef et al. 2026**, Evaluating LLMs for Real-World Web Vulnerability Detection (arXiv 2606.21397; AI&CCPS Workshop at ARES 2026, doi 10.1007/978-3-032-35576-8_3). High priority. Six LLMs on real WordPress-plugin vulnerabilities, with code and data released. The closest work to Gap 5; the full text must show whether it has a tool baseline.
3. **[SoK] Large Language Models in Security Code Review and Testing** (Journal of Systems Research 2026, doi 10.5070/sr3.62177). A second review competitor that compares LLMs with traditional tools.
4. **RealVuln** (arXiv 2604.13764, preprint). SAST against LLMs on Python repositories. The authors evaluate their own company's scanner, a conflict of interest.
5. **Leveraging Large Language Models for advanced static code analysis** (Information and Software Technology 2026, doi 10.1016/j.infsof.2026.108213). Screen first; relevance unknown.
6. **Dahiya et al. 2026**, Are Frontier LLMs Ready for Cybersecurity? (arXiv 2605.23243, preprint). Low priority; no SAST or DL arm in its abstract.
7. Optional: ZeroFalse (arXiv 2510.02534) and Trusting the Tool (arXiv 2610.32876) for the filtering discussion; Out of Distribution, Out of Luck (arXiv 2507.21817, ICSE 2026) for RQ2; and a web-snippet study comparing against an all-vulnerable classifier (doi 10.66571/tsarka-3134-6057-11, venue quality unknown) for Gap 1.

Glossary

Term	Plain-language meaning
Vulnerability	A security bug that an attacker can exploit.
CVE	Common Vulnerabilities and Exposures: a public ID for one known vulnerability, such as CVE-2025-1234.
CWE	Common Weakness Enumeration: a catalogue of vulnerability types; for example CWE-89 is SQL injection.
SQL injection, XSS	Injection bugs: an attacker smuggles commands into a database query (SQL injection) or scripts into a web page (cross-site scripting, XSS).
Static analysis	Examining code without running it.
SAST	Static application security testing: tools that scan source code with hand-written rules to find vulnerabilities.
CodeQL	A semantic SAST tool that turns code into a database and runs queries on it; it usually needs a buildable project.
Semgrep, Flawfinder, Cppcheck	Lighter SAST tools that match patterns in code; they work on single files or functions without building.

Taint analysis	A SAST technique that follows untrusted input (a "source") to a dangerous operation (a "sink").
Large language model (LLM)	A very large AI model trained on text and code, such as GPT-4 or Llama, that answers questions in natural language.
Frontier model	One of the most capable commercial LLMs, used through a paid API.
Open-weight model	An LLM whose model files are public, so it can run on your own computer.
Deep-learning (DL) detector	A neural network trained on labelled examples to classify code as vulnerable or safe, such as LineVul or Devign.
Pre-trained language model (PLM)	A smaller model such as CodeBERT that is pre-trained on code and then fine-tuned for one task.
Fine-tuning	Further training a pre-trained model on task-specific labelled data.
Prompting, zero-shot, few-shot	Asking an LLM directly; zero-shot gives no examples, few-shot gives a few solved examples in the prompt.
Chain-of-thought	A prompt that asks the LLM to reason step by step before answering.
Reasoning or thinking mode	An LLM setting that generates long internal reasoning before the answer; it costs more.
Agent	An LLM that can use tools, such as opening files or running searches, over several steps.
Hybrid	A system that combines an LLM with a SAST tool.
Alarm filtering (triage)	Sorting tool warnings into real and false ones; here, done by an LLM.
True positive / false positive	A correct warning about a real bug / a warning about code that is actually safe (a false alarm).
False negative	A real vulnerability that the tool missed.
Precision	Share of warnings that are real vulnerabilities.
Recall	Share of real vulnerabilities that the tool finds.
F1 score	One number combining precision and recall; high only when both are high.
Accuracy	Share of all answers that are correct; misleading when vulnerable code is rare.
Balanced accuracy	The average of recall on vulnerable code and recall on safe code; any trivial baseline scores 0.5.
MCC	Matthews correlation coefficient: a score from -1 to 1 that stays fair when classes are imbalanced; 0 means no better than chance.
False-positive rate (FPR)	Share of safe code that is wrongly flagged.
False discovery rate (FDR, SFDR)	Share of warnings that are false; SFDR estimates it from a random sample of warnings.

ROC-AUC	A score from 0.5 (chance) to 1 that measures how well a model ranks vulnerable above safe code, regardless of the cut-off.
Decision threshold	The cut-off score above which a model says "vulnerable"; moving it trades precision against recall.
Prevalence	Share of vulnerable samples in the test data; real code has few.
Balanced test set	A test set with 50% vulnerable samples; easier than reality.
Trivial baseline	A "detector" that ignores the code, for example flag-all (always vulnerable), always safe, or a rule based on function length.
Pairwise correctness (P-C, pAcc, PACC)	Share of vulnerable/fixed code pairs where both versions are classified correctly; tests whether a model notices the fix.
VD-S	A PrimeVul metric: the share of vulnerabilities missed when false alarms are held at 0.5% or lower.
Benchmark	A shared test dataset used to compare tools.
Synthetic benchmark	Artificial test code written to contain known bugs, such as Juliet or the OWASP Benchmark.
Big-Vul, Devign, PrimeVul	Popular C/C++ vulnerability datasets; PrimeVul is a cleaner, de-duplicated and time-split version built from older ones.
Label noise	Wrong labels in a dataset, such as "vulnerable" code that is not vulnerable.
Data leakage	Test examples that also appear in the training data, which inflates scores.
Data contamination, training cutoff	An LLM may have seen the test code during training; the training cutoff is the date after which it saw no data. Post-cutoff code is safer to test on.
Temporal split, out-of-distribution	Testing on code from a later time or another source than the training data.
Function, statement and repository level	Judging a whole function, the exact faulty line, or a whole project.
Systematic literature review (SLR)	A review that searches, selects and appraises studies with a written, repeatable protocol.
Narrative review	A review without a fixed search protocol; more open to selective reading.
Peer review, preprint	Expert checking before publication; a preprint is posted without it.
Novelty check	Targeted searches to see whether a proposed gap has already been filled.
Cohen's kappa	A measure of how well two human labellers agree beyond chance.
Bootstrap confidence interval	A range of plausible values found by re-sampling the data many times.
McNemar test, Holm	A test for whether two classifiers differ on the same items; Holm

correction	correction adjusts for running many tests.

About this guide

What it is. A plain-language guide to the project "LLM-based vs Traditional Static Analysis and Deep Learning Approaches for Source Code Vulnerability Detection: A Comparative Review". It was produced on 2026-11-12.

How the evidence was gathered.

1. **Scoping.** The topic, five research questions and inclusion rules were fixed first. The time window was 2020–2026, with peer-reviewed work preferred and preprints allowed only if central and flagged.
2. **Search.** Queries on six themes were run on OpenAlex, Crossref and arXiv, followed by snowballing (checking the references and citations of key papers). The project log records 54 database searches, including the novelty checks.
3. **First pass.** 20 papers were selected and analysed.
4. **Second pass.** Four more papers were added to test the gaps and to cover the closest competing review: qin2026review, lu2025lab, tashenova2026real and deininger2026open. The final set is 24 papers: 20 primary studies and 4 reviews.
5. **Reading.** All 24 papers were read in full text; none is abstract-only. Each analysis records findings with page references and the labels **[V]**, **[A]** and **[I]**.
6. **Checking.** Four critical reviews (A to D) re-checked every analysis against the paper text, and all findings were fixed before analyses were marked complete.
7. **Synthesis.** An evidence matrix (221 rows), a methodology comparison, answers to RQ1–RQ5 and a gap report with recorded novelty checks were built from the analyses. This guide is written only from those files.

Limitations.

- 24 papers is a focused set, not a complete systematic review. Relevant studies may be missing, and two recommended papers are not yet analysed.
- Most papers were read from arXiv copies rather than the final published version, so a few numbers may differ.
- Analyses and critical reviews were written by AI research agents (Claude) under the user's direction. No second human extractor re-checked them.
- Several results come from single studies with one run and no statistical tests; evidence strength is stated for each finding.
- In the second-pass novelty checks the arXiv service timed out on several queries; we compensated with other sources, but coverage of very recent arXiv preprints is weaker.
- Novelty verdicts are valid as of 2026-11-12. The field publishes very quickly, so re-check before relying on them.
- Papers found during novelty checks were judged from abstracts only.

Appendix A: Full Research Report

- Project: 2026-11-12-llm-based-vs-traditional-static-analysis
- Date: pass 2 (24 papers), 2026-11-12
- **What changed since the pass-1 report** (research-report-2026-11-12.md, 20 papers, kept unchanged): four papers were added and read in full (deininger2026open, lu2025lab, tashenova2026real, qin2026review). As a result, all five research-question answers, two new contradictions (C7, C8), the gap list (G2, G5 and G6 narrowed) and the recommended next steps have been updated.
- Based on: 24 full-text analyses, evidence-matrix.md, reports/methodology-comparison.md, gap-report.md, four critical reviews (A and B for pass 1, C and D for pass 2, all 2026-11-12), search/screening.md, search/search-log.md.
- How to read the labels: **[V]** means we checked the number in the paper's own tables or results, **[A]** means the authors say it but we cannot check it, and **[I]** is our own interpretation. Unlabelled numbers are **[V]** from the cited page. Citations give the paper's folder name and the PDF page, e.g. [risse2025top, p. 8].

1. Summary in plain language

We asked whether large language models (LLMs, such as GPT-4 or Claude) find security bugs in source code better than two older approaches: rule-based scanners (SAST tools such as CodeQL or Semgrep) and neural networks trained on labelled vulnerable code (deep-learning or "DL" detectors). After reading 24 papers in full, the short answer is still that **no approach wins everywhere**: LLMs come out ahead on small, self-contained programs [gnieciak2025large, p. 10; dubniczky2025castle, p. 11], CodeQL on synthetic test suites [khare2025understanding, p. 9], and on large real projects both miss many bugs and raise many false alarms [li2026llm, p. 7; li2026llm, p. 11]. The only study that tests all three families on the same real functions finds that the winner depends on the score you choose: a fine-tuned DL model has the best overall score, two of the scanners are the most precise (their warnings are least often false alarms), and the LLMs sit in between [deininger2026open, p. 8; deininger2026open, p. 11]. The most consistent pattern **[I]** is that LLMs find more of the real bugs (higher recall) but also flag much more harmless code (lower precision); when vulnerable code is as rare as in real software (about 2% of functions), their precision is barely better than a rule that flags everything, and on vulnerabilities from 2025 trained DL detectors do little better [deininger2026open, p. 8; lu2025lab, p. 13; lu2025lab, p. 14]. A large part of the disagreement between papers comes from **how** they test, not from the tools themselves: label quality, overlap between training and test data, the share of vulnerable samples, and how lenient the scoring is [risse2025top, p. 8; ding2025vulnerability, p. 4; shu2026evaluating, p. 40; gnieciak2025large, p. 6]. Letting an LLM filter a scanner's alarms removes most false alarms cheaply but also throws away 12–25% of real bugs [du2026reducing, p. 8; xiong2026sifting, p. 18]. For your comparative review, this "setup explains the contradictions" angle, applied across all three families, is still the contribution we could not find in existing reviews (medium confidence). The easiest concrete addition is to re-score the published results against a "flag everything" baseline, which needs only a spreadsheet.

2. What we did

- **Scope.** Source-code vulnerability detection by LLMs, SAST tools, DL detectors and LLM+SAST hybrids, published 2020–2026. Binary analysis, fuzzing and repair-only work were excluded (RESEARCH.md).
- **Searches.** Three search agents covered six facets: head-to-head LLM vs SAST, LLM vs DL, hybrids, evaluation realism, reasoning/agentic LLMs, and surveys. They ran about 19 searches on OpenAlex, Crossref and arXiv, plus *snowballing*, which means following the references and citations of central papers. Every search is logged in search/search-log.md, and every screening decision with its reason is in search/screening.md.
- **Two passes.** Pass 1 analysed 20 papers. The pass-1 gap check found four papers that bore directly on our gaps and on the positioning of the review; with your approval they were added in pass 2.
- **Included papers.** 24 papers in total: 20 primary studies and 4 reviews (kaniewski2026systematic, zhou2025large, sheng2025llms and qin2026review). rlab status counts 4 of the 24 as preprints:
- IRIS [li2024iris, p. 1] and RepoAudit [guo2025repoaudit, p. 1] appeared at ICLR 2025 and ICML 2025. Both are peer-reviewed, but rlab records them as arXiv preprints because those venues issue no DOIs.
- lu2025lab is recorded as an arXiv preprint, but it is published in the *Journal of Systems and Software* (2026, doi:10.1016/j.jss.2026.113185), so we treat it as peer-reviewed. We read the arXiv v4 accepted manuscript, and its page references follow that version [lu2025lab, p. 1].
- Only zhou2024comparison is a true preprint, meaning it is **not peer-reviewed** [zhou2024comparison, p. 1].
- qin2026review is published, but in a low-visibility venue that does not document peer review [qin2026review, p. 1]. It is a weak narrative review (single author, no search protocol), so we give it the weight of *grey literature* (material outside the normal peer-reviewed channels) and use it only to position our review.
- One pass-1 candidate (VulnLLM-R) was dropped to keep to the 20-paper target, because it is a preprint with no venue.
- **Full text.** 24 of 24 papers were read in full and none were abstract-only (rlab status).
- Pass 1: 18 PDFs were fetched automatically from legal open-access sources, SecVulEval came from its arXiv preprint, and tan2026think came from a publisher PDF that you supplied.
- Pass 2: lu2025lab and qin2026review were fetched automatically. tashenova2026real and deininger2026open are open access, but the publisher blocked automated download, so you supplied the PDFs.
- 15 published papers, plus lu2025lab, were read from their arXiv versions, not the publisher's final text (the *version of record*). Numbers could differ slightly between the two (see §7).
- **Quality control.** Four independent critical reviews checked every analysis against the paper text.

- Pass 1 (reviews A and B): 36 findings, all fixed. Most were small: labels, wording, or relative vs absolute percentages. None changed a paper's headline result.
- Pass 2 (reviews C and D): 9 findings for qin2026review, 16 for lu2025lab, 10 plus minor notes for tashenova2026real and 12 for deininger2026open, all fixed. Several were substantive, e.g. separating measured from estimated energy figures and limiting tashenova2026real's "SAST misses ~98%" result to the authors' own in-house analyser.
- **Gap checks.** Before any gap was claimed we ran targeted novelty searches, as our standards require (gap-report.md): 26 searches and 8 web searches in pass 1, and 9 more searches (Q27–Q35), 4 web searches and 2 directly read arXiv abstract pages in pass 2, all on 2026-11-12. The arXiv service timed out on 6 of the pass-2 searches (see §7).

3. Answers to the research questions

RQ1 — Which family detects vulnerabilities better in head-to-head tests?

Answer. It depends on the setting and on which score you use:

- **Small, self-contained code.** Frontier LLMs beat SAST: macro F1 0.75–0.80 vs 0.26–0.55 on ten C# projects [gnieciak2025large, p. 10], and a CASTLE score of 977 for GPT-o3-mini vs 661 for the best non-LLM tool [dubniczky2025castle, p. 11].
- **Synthetic suites.** CodeQL beats the best LLM's accuracy on OWASP, Juliet Java and Juliet C/C++ [khare2025understanding, p. 9].
- **Whole projects.** General coding agents reach much higher recall than SAST on C/C++ (59.55% vs 0–34.29%). Most LLM detectors and SAST tools produce mostly false warnings [li2026llm, p. 7; li2026llm, p. 11].
- **All three families on the same real functions (new in pass 2).** deininger2026open tests 8 LLMs, 3 lightweight SAST tools and fine-tuned CodeBERT detectors on the same 1549 C/C++ functions from the PrimeVul dataset.
- Seven of the eight LLMs, asked without examples (*zero-shot*), beat the SAST tools on *balanced accuracy*, a score that averages how well a tool handles vulnerable and safe code [deininger2026open, p. 7; deininger2026open, p. 10].
- A fine-tuned CodeBERT beats all eight LLMs (balanced accuracy 0.765 vs ≤ 0.711) [deininger2026open, p. 11; deininger2026open, p. 17].
- When the functions are re-scored at the dataset's natural mix (about 1 vulnerable function for every 44 safe ones, "1:44"), Flawfinder and Semgrep are more precise than every LLM. LLM precision is 2.3–4.6%, barely above the 2.21% of a rule that flags everything [deininger2026open, p. 8; deininger2026open, p. 17].
- **Against fine-tuned DL elsewhere.** The ranking depends on the dataset, the share of vulnerable samples and the *decision threshold* (the score above which a model says "vulnerable") [fu2023chatgpt, p. 4; shu2026evaluating, p. 40; deininger2026open, p. 11]. On PrimeVul pairs of vulnerable and fixed code, every family scores near or below chance [ding2025vulnerability, p. 9; tan2026think, p. 6].
- **Recent code (new in pass 2).** On Linux and Chromium vulnerabilities disclosed from May 2025, where 1.97% of functions are vulnerable, trained DL detectors reach F1

3.47–8.65 and four open LLMs of up to 32B parameters reach 0.79–5.36 [lu2025lab, p. 13; lu2025lab, p. 14]. By our calculation, a rule that flags everything would score about 3.87, so both families sit close to it **[I]**.

- **PHP (new in pass 2).** No LLM has been tested on PHP in our corpus. The authors' own in-house analyser detects only 1.7–1.8% of vulnerabilities in WordPress plugins [tashenova2026real, p. 9], which says nothing about established PHP SAST tools **[I]**.

Evidence strength: weak to moderate. The studies are mostly single runs, use incompatible headline metrics, and cannot be pooled **[I]** (methodology comparison §3.5). Only one study compares all three families on identical real functions, and its SAST arm is lightweight: CodeQL could not be run on isolated functions [deininger2026open, p. 10]. The most robust pattern **[I]** is that LLMs have higher recall and lower precision, and that at realistic prevalence their margin over trivial baselines is small.

What would change it. A study that runs all three families, including a *semantic* SAST tool such as CodeQL, on the same real code released after the models' training cutoff, with file or repository context and one shared metric (gap G2). khare2025understanding runs all three, but its SAST arm is on synthetic data only [khare2025understanding, p. 7]; deininger2026open uses pre-2023 CVEs and single functions [deininger2026open, p. 24]. A preprint comparing LLMs, fine-tuned small models and SAST in Python, Java and JavaScript (Zhang et al. 2025, arXiv 2503.01449; abstract only, not analysed) could extend the answer beyond C/C++ if added (§8).

RQ2 — How do evaluation setups differ, and do they explain the disagreements?

Answer. Yes, to a large extent. The main sources of variation are:

- **Wrong labels.** Only 39–65% of "vulnerable" labels in popular datasets are correct [risse2025top, p. 8], and 48.8% in a PHP benchmark [tashenova2026real, p. 11].
- **Test data copied from training data.** Up to 18.9% exact copies [ding2025vulnerability, p. 4]. Even when a PHP dataset was split by plugin, duplicate and contradictory-label groups still crossed the splits before cleaning [tashenova2026real, p. 4].
- **Balanced vs realistic test sets.** The class balance flips rankings [shu2026evaluating, p. 40; deininger2026open, p. 8]. LLM F1 falls from about 49–65% to 5.41–12.21% when pairs are dropped [yildiz2025benchmarking, p. 7].
- **Trivial baselines rarely reported.** Where they are reported, they are often competitive: a one-feature token-count rule matches learned models in PHP [tashenova2026real, p. 8], and "flag everything" scores within reach of most LLMs [deininger2026open, p. 7; lu2025lab, p. 14].
- **Time and dataset shift.** Detectors trained on one dataset or period collapse on another [lu2025lab, p. 12; lu2025lab, p. 13].
- **Prompt wording and decision threshold.** Either can change the winner [deininger2026open, p. 11; deininger2026open, p. 20].
- **Synthetic vs real code.** LLM accuracy is 10.5% lower on real code [khare2025understanding, p. 5].

- **Function-level vs line-level scoring.** Line-level scoring halves F1 [ahmed2026secvuleval, p. 8].
- **Custom datasets with inconsistent splits** [kaniewski2026systematic, p. 35].

Even a word-count classifier reaches F1 0.86 on BigVul [risse2025top, p. 11], so high scores on that dataset say little about real understanding. The same model on the same named dataset can differ by 28 F1 points between papers [lu2025lab, p. 12; ding2025vulnerability, p. 7].

Evidence strength: strong for label noise and for the collapse of scores under realistic evaluation, which independent primary studies and two systematic reviews agree on [kaniewski2026systematic, p. 39; sheng2025llms, p. 20]. **Moderate** for prompt and threshold sensitivity and for trivial baselines, which rest on fewer studies.

What would change it. A controlled study showing that rankings stay stable across clean and noisy versions of the same benchmark, across prompts and across thresholds.

RQ3 — Do LLM + static-analysis hybrids beat either family alone, and at what cost?

Answer. It depends on the type of hybrid.

- **LLM as alarm filter.** This is the most consistent benefit in the corpus, but it is a trade-off.
- An industrial study removes 94–98% of false alarms while keeping 75–88% of real bugs, at $0.0011–$0.12 per alarm [du2026reducing, p. 8]. This is the authors' own method at Tencent, and it covers general bug types, not only security bugs.
- An agent study cuts the false-positive rate on OWASP from 98.3% to 6.3% [xiong2026sifting, p. 12] but misses 22.25% of real vulnerabilities [xiong2026sifting, p. 18].
- **Strength: moderate.**
- **LLM inside the analyser** (IRIS, RepoAudit). The tool authors' own evaluations look strong:
- IRIS found 55 of 120 Java CVEs vs CodeQL's 27 [li2024iris, p. 8].
- RepoAudit reports 78.43% precision [guo2025repoaudit, p. 8].

An independent project-scale study does not reproduce these gains [li2026llm, p. 7; li2026llm, p. 12]. IRIS's gain also depends on GPT-4, and smaller LLMs make it worse [li2024iris, p. 10]. **Strength: weak and contested.**

- **Cost.** LLM costs range from cents per alarm to $1,247.66 for one project, while SAST takes minutes with no token cost [li2026llm, p. 16].
- The only *cost per true positive* (the cost of each real bug found) comes from a non-hybrid study: USD 0.003–0.212 and 3.7–119.6 kJ of energy per true positive at 1:44 prevalence [deininger2026open, p. 8].
- These are compute costs only. The analyst time needed to check the 25.6–41.9 false alarms that come with each real bug is not included [deininger2026open, p. 8], and part of the energy values are estimates from operation counts (FLOP estimates), not measurements [deininger2026open, p. 14].
- Small open models can match a frontier model at 1/66 of the cost [deininger2026open, p. 17]. **Strength: moderate, single study.**

- **Explainability.** LLMs sometimes point at the wrong location [gnieciak2025large, p. 11] **[A]**, and a fluent explanation does not mean the verdict is correct [tan2026think, p. 9].

What would change it. An independent, blinded replication of IRIS or RepoAudit (gap G4). For filtering, results on real security alerts rather than the synthetic OWASP suite (gap G3).

RQ4 — Which vulnerability types, languages and code sizes suit each family?

Answer.

- **By vulnerability type.** LLMs do best on self-contained injection-type weaknesses. They do worst on weaknesses that depend on wider context or on cryptography policy, and on locating memory-safety bugs [khare2025understanding, p. 7; xiong2026sifting, p. 18; shu2026evaluating, p. 30]. For example, the filtering agent misses only 0.41–2.38% of injection bugs but 77.17–84.50% of weak-crypto and weak-hash bugs [xiong2026sifting, p. 18]. CodeQL is better on integer overflow (CWE-190) [khare2025understanding, p. 8].
- **By code size.** LLM performance drops as functions or files get longer [ahmed2026secvuleval, p. 13; li2026llm, p. 9].
- **By project.** One DL detector (LineVul trained on BigVul) finds 76% of the 2025 Chromium CVEs but none in Linux, while all LLMs cover Linux better [lu2025lab, p. 13].
- **By language.** Evidence is concentrated on C/C++ and Java [kaniewski2026systematic, p. 28]. The only PHP study tests no LLM. In it, learned models detect cross-site scripting (XSS, F1 up to 0.500) far better than SQL injection (≤ 0.146) or remote code execution (≤ 0.085) [tashenova2026real, p. 9].

Evidence strength: weak. The per-type counts are small, studies disagree, and PHP rests on a single study. For example, khare finds LLMs beat CodeQL on CWE-476 [khare2025understanding, p. 8], while ahmed reports that LLMs over-flag it [ahmed2026secvuleval, p. 13].

What would change it. Larger per-type samples, and LLM-vs-established-SAST studies in PHP and Go (gap G5).

RQ5 — What open problems remain?

Answer. The analysed papers repeatedly ask for:

- verified, realistic, repository-level benchmarks [zhou2025large, p. 19; sheng2025llms, p. 25];
- trivial baselines and realistic prevalence in every evaluation [tashenova2026real, p. 13; deininger2026open, p. 8];
- label quality and control of *contamination*, i.e. test code that the LLM already saw during training [tashenova2026real, p. 12; lu2025lab, p. 17];
- same-benchmark three-family comparisons. Two studies run all three: khare, with SAST on synthetic data only [khare2025understanding, p. 7], and deininger, with lightweight function-level SAST on pre-2023 CVEs [deininger2026open, p. 10; deininger2026open, p. 24];
- independent human labelling and statistical testing, including how sensitive results are to the prompt [li2026llm, p. 11; deininger2026open, p. 20];

- cost reporting that includes human triage and states where energy figures come from [deininger2026open, p. 8];
- limiting the loss of real bugs during filtering [xiong2026sifting, p. 19];
- integration into developer workflows [zhou2025large, p. 19].

deininger's authors judge that no method they tested (LLM, SAST or fine-tuned DL) is ready for independent function-level triage [deininger2026open, p. 26] **[A]**.

Evidence strength: moderate [I] as a description of recurring limitations. Six of these became novelty-checked gaps (§6).

4. Key papers to read first

1. **deininger2026open** (*Computers* 2026) is the only study that runs LLMs, SAST tools and fine-tuned DL detectors on the same real functions with the same metrics, and the only one that reports cost and energy per true positive [deininger2026open, p. 8; deininger2026open, p. 10]. It also uses proper significance tests [deininger2026open, p. 6] and shows how a reworded prompt or a default threshold can change the winner [deininger2026open, p. 11; deininger2026open, p. 20]. Be aware of its limits: lightweight SAST without CodeQL, CVEs from 2008–2022 that the LLMs may have seen in training, and single functions without context [deininger2026open, p. 10; deininger2026open, p. 24].
2. **li2026llm** (TSE 2026) is the only independent, project-scale comparison of LLM detectors, agents and SAST tools that also reports cost. Read it to understand C2, C3 and C6 [li2026llm, p. 7; li2026llm, p. 16]. Be aware that it labels warnings partly with an LLM that it also evaluates [li2026llm, p. 11].
3. **lu2025lab** (JSS 2026) builds a test set of Linux and Chromium vulnerabilities disclosed from May 2025, after the evaluated models were released [lu2025lab, p. 4]. On it, a DL detector that scores F1 91.00 on its own benchmark drops to 6.87 [lu2025lab, p. 12; lu2025lab, p. 13], and the best open LLM reaches F1 5.36 [lu2025lab, p. 13]. It is the clearest picture of what happens on genuinely new code. It has no SAST arm and runs each LLM once [lu2025lab, p. 8].
4. **risse2025top** (FSE 2025) shows that common benchmark labels are often wrong and that trivial classifiers score highly. It is the backbone of the "setup explains results" argument [risse2025top, p. 8; risse2025top, p. 11].
5. **kaniewski2026systematic** (TOSEM 2026) is the most current systematic review (263 studies). You must position your review against it [kaniewski2026systematic, p. 8].

If you have time for two more: **ding2025vulnerability** (ICSE 2025, PrimeVul) audits label noise and duplication and introduces the vulnerable/patched pair test that defeats every model [ding2025vulnerability, p. 4; ding2025vulnerability, p. 9]; **khare2025understanding** (ICST 2025) was the first study in our corpus to run LLMs, SAST and DL together, with SAST on synthetic data only [khare2025understanding, p. 7; khare2025understanding, p. 9].

5. Where studies disagree

We found eight contradictions (evidence matrix C1–C8; C7 and C8 are new in pass 2). Each can be traced, at least in part, to a difference in setup. The explanations are our interpretation **[I]**.

- **C1. Fine-tuned models vs GPT-4.**
- What disagrees: on Big-Vul, fine-tuned models far exceed prompted GPT-4 (F1 94% and 87% vs 29%) [fu2023chatgpt, p. 4]. On PrimeVul pairs, GPT-4 beats every fine-tuned model (12.94% vs ≤3.01%) [ding2025vulnerability, p. 9]. Pass 2: on single, unpaired PrimeVul functions, a fine-tuned CodeBERT again beats all eight LLMs (balanced accuracy 0.765 vs ≤ 0.711) [deininger2026open, p. 11; deininger2026open, p. 17].
- Likely reason: the metric, the data split and the dataset cleaning differ. On pairs, every model is below the 22.70% random reference, so the reversal reflects the benchmark more than the model family. The threshold matters too: with the default rule, deininger's fine-tuned detector looks like an "always safe" classifier although it ranks functions well (ROC-AUC 0.845) [deininger2026open, p. 11]. Unpaired and pair results must not be pooled.
- **C2. RepoAudit: own paper vs independent study.**
- What disagrees: the authors report 78.43% precision and all 21 known bugs reproduced [guo2025repoaudit, p. 8]. The independent study finds 0 of 110 known C/C++ vulnerabilities and 92.77% false warnings [li2026llm, p. 7; li2026llm, p. 11].
- Likely reason: the targets, code versions, metric, labeller and scale all differ. The tool's dependence on built-in API models probably matters, because API modelling explains all 108 analysed misses [li2026llm, p. 7]. Unresolved.
- **C3. LLM vs SAST.**
- What disagrees: LLMs win on small programs [gnieciak2025large, p. 10; dubniczky2025castle, p. 11]. CodeQL wins on synthetic suites [khare2025understanding, p. 9]. At repository level, LLMs detect more but flag up to 77.4% of functions (preprint) [zhou2024comparison, p. 7]. Pass 2: on the same PrimeVul functions, LLMs lead on balanced accuracy, but Flawfinder and Semgrep are more precise at 1:44, while Cppcheck sits in the LLM range [deininger2026open, p. 8; deininger2026open, p. 10].
- Likely reason: synthetic suites match SAST rules and small programs fit an LLM's context window. Scoring was also more lenient for LLMs in two studies [gnieciak2025large, p. 6; dubniczky2025castle, p. 9]. The two families also fail in opposite directions: LLMs over-flag, SAST finds little. So the ranking depends on the metric.
- **C4. Does "reasoning" reduce false alarms?**
- What disagrees: CASTLE says reasoning models produce fewer false positives [dubniczky2025castle, p. 14] **[A]**, yet its own table shows DeepSeek R1 with more false positives than GPT-4o (166 vs 116) [dubniczky2025castle, p. 11]. A controlled factorial study finds that thinking mode raises recall, not accuracy [tan2026think, p. 6]. Pass 2: reasoning helps the frontier models but lowers DeepSeek-V3.2's *specificity* (the share of safe code correctly left alone) from 0.52 to 0.37 [deininger2026open, p. 17; deininger2026open, p. 20].
- Likely reason: the effect depends on the model; the claim holds for some models in some settings only.
- **C5. Class balance flips rankings.**

- What disagrees: fine-tuned GPT-4o wins on a balanced test set, but smaller pre-trained models win when only about 17% of samples are vulnerable [shu2026evaluating, p. 17; shu2026evaluating, p. 40]. Pass 2: deininger's precision ranking at 1:44 differs from its balanced-accuracy ranking [deininger2026open, p. 8], and in the PHP study, changing how safe samples are drawn flips the direction in which the same model fails [tashenova2026real, p. 9].
- Likely reason: balanced sets reward models that flag a lot.
- **C6. IRIS: own paper vs independent study.**
- What disagrees: IRIS beats CodeQL on known Java CVEs [li2024iris, p. 8] but has 97.80% false warnings vs CodeQL's 77.00% on new Java projects [li2026llm, p. 12].
- Likely reason: the comparison is not like-for-like. Open.
- **C7. Same model, same benchmark, different score (new).**
- What disagrees: a CodeBERT-based detector trained and tested on BigVul scores F1 91.00 in one paper [lu2025lab, p. 12] and 62.88 in another [ding2025vulnerability, p. 7]. Neither paper explains the 28-point gap.
- Likely reason: different splits and preprocessing; lu used the detector's original pipeline [lu2025lab, p. 8]. In-distribution BigVul numbers cannot be compared across papers.
- **C8. Does a fine-tuned detector survive on later code? (new)**
- What disagrees: a CodeBERT trained on PrimeVul keeps balanced accuracy 0.765 on PrimeVul's chronologically later test split [deininger2026open, p. 11], while DL detectors trained on BigVul, ICVul, Devign or Juliet fall to F1 3.47–8.65 on 2025 CVEs [lu2025lab, p. 13].
- Likely reason: the test data (2008–2022 PrimeVul CVEs vs 2025 CVEs), training data quality, threshold policy and metric all differ. Unresolved.

Where results do agree, the agreement is useful. CodeQL's results on OWASP replicate across two groups within about 2 points [khare2025understanding, p. 9; xiong2026sifting, p. 11]. Realistic evaluation lowers every family's scores [chakraborty2024revisiting, p. 7; ding2025vulnerability, p. 7; lu2025lab, p. 13]. And where trivial baselines are reported, they are competitive [tashenova2026real, p. 8; deininger2026open, p. 7; shu2026evaluating, p. 11].

6. Research gaps and possible contributions

All gaps passed a recorded novelty check on 2026-11-12 and were re-checked in pass 2. "No study found" means we found none in those searches, not that none exists. Gaps are ranked by evidence strength and by how feasible they are for a beginner; **the top three for you are G1, G2 and G3**.

Gap	After pass 2	Confidence	Suggested study	Beginner feasibility
G1. Published LLM, SAST and DL results are not re-scored *across studies*	unchanged (premise strengthened)	medium–high	Re-compute MCC, ΔF1 vs flag-all, and precision at 1%, 5% and 17% prevalence from the	**easy** (spreadsheet or ~50 lines of

against trivial baselines ("flag everything") or realistic prevalence; no shared evaluation protocol exists			published tables of the 20 primary studies; check the formulas against the studies that report their own baselines	Python, 2–4 weeks, no compute)
G2. No same-benchmark LLM vs *semantic* SAST vs fine-tuned DL comparison on real code released after the models' training cutoff (*post-cutoff* code), with file or repository context	narrowed: a function-level, pre-cutoff C/C++ three-family study now exists (deininger2026open)	medium	~100 post-cutoff + ~100 pre-cutoff Python or Java CVEs; Semgrep and CodeQL, a fine-tuned UniXcoder or CodeBERT, 2–3 open LLMs, plus flag-all. Cheaper variant: reuse the public 2025-CVE set from lu2025lab [lu2025lab, p. 2] and add a SAST arm, an LLM arm with file context and a flag-all baseline	moderate (6–10 weeks)
G3. Loss of real bugs in LLM alarm filtering is not controlled per vulnerability type on real security alerts	unchanged	low–medium	Filter only the CWE types with low miss rates; tune on half of OWASP, test on the other half and on ~100 real alerts	easy–moderate (4–6 weeks)
G4. No independent, blinded replication of IRIS / RepoAudit that separates target, metric and labeller effects	unchanged	medium	2 × 2 design: original vs new targets × original vs false-discovery metric, labelled by humans blinded to the tool	moderate–hard (an IRIS-only version on 20–30 CVEs: about 8 weeks)
G5. No real-CVE comparison of LLMs with *established* SAST tools for PHP and Go	narrowed: LLM-only WordPress evidence (Neef et al. 2026) and function-level Python/Java/JavaScript three-family evidence (Zhang et al. 2025) exist, both abstract only	low–medium	Reuse tashenova2026real's expert-verified PHP subset (125 vulnerable, 247 safe functions) [tashenova2026real, p. 11] or ~60 Go CVEs; Semgrep plus a PHP or Go taint tool, 2 open LLMs, token-count and flag-all baselines	moderate

G6. No cost per *confirmed* vulnerability that includes analyst triage time and a costed SAST and DL arm on the same targets	narrowed: compute cost per true positive for 8 LLMs now exists (deininger2026open)	low–medium	Add to G2 as a secondary outcome: time two raters triaging 50 warnings per tool	moderate (piggybacks on G2)

Why G1 matters and why it is the easiest contribution. On a 50/50 test set, flagging everything already gives F1 0.667. Several published best results sit at or just above that line: a dummy classifier at 0.6693 vs a best model at 0.7075 [shu2026evaluating, p. 17], best F1 values of 0.665–0.666 [tan2026think, p. 7] and GPT F1 values of 48–66 [yildiz2025benchmarking, p. 6]. Pass 2 shows the same at other prevalences:

- LLM F1 0.405–0.615 against 0.523 for "always vulnerable", and at 1:44 LLM precision only just above flag-all [deininger2026open, p. 7; deininger2026open, p. 8];
- a one-feature token-count rule matching learned PHP detectors [tashenova2026real, p. 8];
- 7 of 12 LLM F1 values on 2025 CVEs below the flag-all level, by our calculation **[I]** [lu2025lab, p. 14].

These studies apply baselines only to their own results; none re-scores other studies' published numbers on a common scale **[I]**. Statistics are also rare: among the first 16 primary studies only tan2026think applies significance tests to its main comparison [tan2026think, p. 6] [I, our count]; of the four pass-2 papers, deininger2026open does [deininger2026open, p. 6]. G1 needs only published tables and a spreadsheet, and the studies that report their own baselines give you a built-in check that your formulas are right. Credit those within-study baselines; the contribution is the *systematic cross-study* re-scoring.

Positioning of the review (medium confidence). We found no review that combines (a) a synthesis across LLM, SAST and DL plus both hybrid types with (b) a setup-based explanation of specific contradictions. Reviews in the corpus are either LLM-only [kaniewski2026systematic, p. 8], predate most hybrid and project-scale work [zhou2025large, p. 18], or are narrative [sheng2025llms, p. 25]. The closest competitor, qin2026review, has now been read in full. It shares our "evaluation reliability" framing [qin2026review, p. 10] **[A]**, but it reports no search protocol, puts SAST explicitly out of scope [qin2026review, p. 4], and its numbers re-report papers already in our corpus [qin2026review, p. 12; qin2026review, p. 15], so it is not independent evidence **[I]**. It strengthens rather than weakens our novelty claim, but reviewers will likely point to it and to kaniewski2026systematic, so the claim must stay narrow. Several other reviews outside the corpus were checked from their abstracts only (gap-report.md).

7. Limitations of this review

- **Size.** 24 papers (20 primary studies, 4 reviews). This is a focused critical review, not a complete systematic review. Several relevant papers were found during gap checks but not analysed, including a second three-family comparison (Zhang et al. 2025) that the pass-1 searches returned but we missed (§8).

- **Speed of the field.** Searches ran on 2026-11-12. Many near-miss studies appeared in 2026; pass 2 already narrowed G2, G5 and G6, and G3 could close soon too.
- **Search coverage.** The arXiv service timed out on 6 of the 9 pass-2 novelty searches (Q27–Q29 and Q31–Q33). We covered arXiv instead with one further arXiv search (Q34) and by reading the abstract pages of the two papers flagged for checking. Some recent arXiv preprints may still have been missed.
- **Corrections made in pass 2.** Some pass-1 gap notes were based on abstracts and turned out wrong once the full texts were read: deininger2026open *does* run SAST tools (the pass-1 notes said it did not), tashenova2026real is now read in full rather than known from its abstract, and lu2025lab turned out to be published in a journal. The gap report and this report now follow the full texts. **[I]** Abstract-based judgements about the papers still outside the corpus (§8) could be wrong in the same way.
- **Energy and cost figures.** Part of deininger2026open's energy per true positive comes from estimates based on counted operations (FLOP estimates), not from measured energy [deininger2026open, p. 14], and that estimator overstates measured batched energy by 5.4–16.9× [deininger2026open, p. 14]. Its cost figures also leave out human triage [deininger2026open, p. 8]. Treat these numbers as rough orders of magnitude.
- **Preprints and versions.**
- zhou2024comparison is not peer-reviewed. Its per-language results rest on about 8–10 test CVEs [zhou2024comparison, p. 6] **[I]**.
- Fifteen published papers, plus lu2025lab (arXiv v4 accepted manuscript), were read from arXiv copies. du2026reducing came from an early version with a placeholder header [du2026reducing, p. 2], so it carries the highest risk that its numbers changed.
- qin2026review's venue does not document peer review [qin2026review, p. 1]; we use it for positioning only.
- One paper's identity (ahmed2026secvuleval) is marked "unverified" by rlab because its title differs from the arXiv version. A reviewer confirmed it manually by its authors.
- **Independence.**
- Four studies share the PrimeVul dataset family (ding, yildiz, tan, deininger) **[I]**.
- One of lu2025lab's four training datasets (ICVul) is the authors' own [lu2025lab, p. 3].
- qin2026review re-reports papers already in our corpus, so it adds no independent support [qin2026review, p. 12].
- zhou2025large's only LLM-vs-SAST study is its own authors' preprint [zhou2025large, p. 22].
- The independent project-scale study uses an LLM as labeller [li2026llm, p. 11].
- **Single runs and prompts.** Most primary studies run each model once. deininger2026open shows that rewording the prompt can move balanced accuracy by up to 0.088 and change the winner [deininger2026open, p. 19; deininger2026open, p. 20], so small differences elsewhere should not be over-read **[I]**.
- **No pooling.** Metrics are too varied for a meta-analysis (a statistical combination of results), so our answers are narrative.
- **Language bias.** The evidence comes mostly from C/C++ and Java [kaniewski2026systematic, p. 28]; PHP rests on one study with no LLM arm [tashenova2026real, p. 13].

- **Abstract-only material.** Statements about papers outside the corpus (§6, §8) are based on abstracts only.
- **Record keeping.** The metadata.json files of three pass-2 papers carry a wrong, shared reason for inclusion; the correct reasons are in search/screening.md. This does not affect any finding.

8. Recommended next steps

(a) Turn this into the comparative review paper.

1. Frame it as a **critical comparative review**, not a systematic review. With 24 papers, reviewers would criticise a "comprehensive" framing. State two distinct contributions:
   - a synthesis across all three families and both hybrid types;
   - an explanation of each contradiction (C1–C8) through the study setup, together with a comparability audit that says which numbers may be compared across papers (methodology comparison §3).
1. Suggested structure:
   - introduction and positioning against kaniewski2026systematic, zhou2025large, sheng2025llms and qin2026review;
   - method (searches, screening, appraisal);
   - results by family (methodology comparison §2);
   - comparable results (§3);
   - contradictions (§4);
   - field-wide weaknesses (§5);
   - gaps and guidance;
   - threats to validity (this report §7).
1. Include **G1 as a table inside the review**. Re-scoring the published numbers against flag-all baselines and realistic prevalence needs only the published tables and a spreadsheet, so it adds a concrete result at no compute cost. Extract twice and spot-check against the PDFs. Rows without enough data to rebuild a confusion matrix go in a separate list.
2. Cite deininger2026open as the current best same-function evidence, but state its limits (lightweight SAST, pre-2023 CVEs, single functions), and cite qin2026review only for positioning.

(b) Decide whether to run a pass 3 (your decision). The pass-2 gap check recommends adding further papers, which would raise the corpus to about 26. They would be added with rlab add, fetched, analysed and critically reviewed like the others. In order of value:

- **Zhang et al. 2025**, *Benchmarking Large Language Models for Multi-Language Software Vulnerability Detection* (arXiv 2503.01449; preprint, check for a published version). High priority: from its abstract, it is a second same-benchmark comparison of LLMs, fine-tuned small models and SAST, and the only one in Python, Java and JavaScript. It bears on RQ1, G2 and G5.
- **Neef et al. 2026**, *Evaluating LLMs for Real-World Web Vulnerability Detection* (arXiv 2606.21397; AI&CCPS Workshop at ARES 2026). It tests six LLMs on real

WordPress-plugin vulnerabilities and is the closest work to G5. Its full text must show whether it includes any tool baseline and at what level (function, file or plugin) it works.
- Optional: the SoK *LLMs in Security Code Review and Testing* (doi:10.5070/sr3.62177) as a second positioning competitor, and an *Information and Software Technology* 2026 paper (doi:10.1016/j.infsof.2026.108213) whose title suggests an LLM-vs-traditional comparison but whose abstract we could not retrieve, so screen it first.

If you decline, the report stands as is, but §6 must keep describing G2 and G5 as "narrowed, pending these papers".

(c) Before submission.

- Re-check every key number against the version of record, starting with du2026reducing, the other 14 papers read from arXiv, and lu2025lab's JSS version.
- Re-run the novelty searches for G1, G2 and the positioning claim, including the 6 searches where arXiv timed out.

A first concrete step this week: start the G1 extraction sheet with the three studies that report their own trivial baselines (shu, deininger, tashenova), because they let you check your formulas. Then add the studies whose best results sit near the flag-all line (tan, yildiz, khare, lu), then the remaining primary studies.

Also pending: the plain-English reading guide (Google Doc) you approved, updated for all 24 papers.

9. Glossary

- **Vulnerability:** a security bug that an attacker could exploit.
- **CVE:** a public ID for one known, real-world vulnerability.
- **CWE:** a category of vulnerability type, e.g. CWE-89 for SQL injection.
- **XSS (cross-site scripting):** a web vulnerability where an attacker gets their script to run in other users' browsers.
- **SAST (static application security testing):** tools such as CodeQL or Semgrep that scan code with hand-written rules without running it.
- **Lightweight vs semantic SAST:** lightweight tools (e.g. Flawfinder, Cppcheck) match patterns in the text of the code; semantic tools (e.g. CodeQL) build a model of the whole program and track how data flows through it, which usually requires building the project.
- **Taint analysis:** a SAST technique that follows untrusted input ("tainted" data) to see whether it reaches a dangerous operation.
- **DL detector:** a neural network (e.g. LineVul) trained on examples of vulnerable and safe code.
- **LLM (large language model):** a very large text model such as GPT-4 that can read and reason about code.
- **Prompting / zero-shot / few-shot:** asking an LLM directly, with no examples (zero-shot) or a few examples (few-shot) in the request.
- **Fine-tuning:** further training a model on task-specific labelled data.
- **Agent:** an LLM that can call tools and take several steps, e.g. open other files.
- **Hybrid:** a system that combines an LLM with a SAST tool.

- **Precision:** the share of the tool's warnings that are real vulnerabilities.
- **Recall:** the share of real vulnerabilities that the tool finds.
- **F1:** a single score that balances precision and recall. *Macro F1* averages the F1 of the "vulnerable" and "safe" classes.
- **False-positive rate (FPR):** the share of safe code wrongly flagged as vulnerable.
- **Specificity:** the share of safe code that the tool correctly leaves alone (1 − FPR).
- **Balanced accuracy:** the average of recall and specificity, so a tool cannot score well just by always giving the majority answer.
- **ROC-AUC:** how well a model ranks vulnerable code above safe code across all possible thresholds; 0.5 is chance, 1.0 is perfect.
- **Decision threshold:** the score above which a model's output is counted as "vulnerable"; changing it trades recall against precision.
- **Cost per true positive:** the money or energy spent for each real vulnerability a tool finds.
- **FLOP estimate:** an energy figure calculated from the number of computing operations a model needs, rather than measured on the hardware.
- **SFDR (sampled false discovery rate):** the share of warnings that are false, estimated from a sample.
- **MCC:** a score that stays fair when vulnerable code is rare.
- **Prevalence / class balance:** the share of vulnerable samples in a test set. *Balanced* means 50%.
- **Trivial baseline:** a deliberately dumb method, e.g. "flag everything", that any real tool should beat.
- **Pairwise correctness:** the share of vulnerable/fixed code pairs where both versions are judged correctly.
- **Label noise:** wrong "vulnerable"/"safe" labels in a dataset.
- **Data leakage:** test samples that also appear in the training data, which inflates scores.
- **Contamination / training cutoff:** an LLM may have seen benchmark code during training. Code released after its training cutoff (*post-cutoff* code) avoids this.
- **Temporally separated / out-of-distribution test:** testing on data from a later period or a different source than the training data, which is closer to real use.
- **Synthetic vs real-world code:** code written to demonstrate a bug (e.g. the Juliet, OWASP and CASTLE suites) vs code from real projects.
- **Repository level:** analysing a whole project rather than single functions.
- **Benchmark:** a shared test dataset used to compare tools.
- **Systematic literature review (SLR):** a review that searches for and assesses all studies on a question using a fixed protocol.
- **Narrative review:** a review that summarises a field without a fixed search and selection protocol; it is easier to write but less reliable.
- **Critical comparative review:** a focused review that compares approaches across a chosen set of studies and appraises them critically, without claiming to cover everything.
- **Grey literature:** material outside the normal peer-reviewed channels, e.g. reports or papers in venues that do not document peer review.
- **Meta-analysis:** statistically combining the results of many studies.

- **Peer review / preprint:** expert checking before publication. A preprint has not been through it yet.
- **Version of record:** the final published version of a paper.
- **Novelty check:** targeted searches to confirm that a gap has not already been filled.
- **Significance test:** a statistical test of whether a difference could be due to chance.

Appendix B: Methodology Comparison

- Project: 2026-11-12-llm-based-vs-traditional-static-analysis
- Date: 2026-11-12
- Inputs: 24 analyses (status: complete, all full text), evidence-matrix.md, literature-matrix.md, critical reviews A–D of 2026-11-12. Updated pass 2, 2026-11-12: added qin2026review, lu2025lab, tashenova2026real and deininger2026open.
- Labels: **[V]** verified in the paper's own data, **[A]** author claim, **[I]** our interpretation. Unlabelled numeric statements are **[V]** from the cited page.
- Source tiers: four items are recorded by rlab as arXiv preprints. IRIS [li2024iris, p. 1] and RepoAudit [guo2025repoaudit, p. 1] are the ICLR 2025 and ICML 2025 camera-ready texts. lu2025lab was read as the arXiv v4 accepted manuscript [lu2025lab, p. 1]. The orchestrator confirmed via Crossref that it is published in the Journal of Systems and Software (2026, doi 10.1016/j.jss.2026.113185). We treat it as peer-reviewed; its page refs follow arXiv v4. Only zhou2024comparison is a true preprint without a known peer-reviewed version (**not peer-reviewed**) [zhou2024comparison, p. 1]. Of the other 20 papers, 15 were read from arXiv copies rather than the version of record. Five were read in publisher form: tan2026think, yildiz2025benchmarking, deininger2026open, tashenova2026real and qin2026review (see §5, W9). qin2026review appears in a low-visibility venue (Zeus Press) that does not document peer review [qin2026review, p. 1]. We give it the weight of grey literature and use it for positioning only.

A short glossary for readers new to the area:

- *SAST* (static application security testing) tools such as CodeQL or Semgrep scan code with hand-written rules.
- *DL detectors* (e.g. LineVul, Devign) are neural networks trained on labelled vulnerable code.
- *Prevalence* is the share of vulnerable samples in a test set. A *balanced* test set has 50% vulnerable samples; real code has far fewer (about 2% in two pass-2 studies).
- A *trivial baseline* is a classifier that ignores the code, for example "always vulnerable", "always safe" (majority class) or a rule based only on function length.
- *Balanced accuracy* is the mean of the recall on vulnerable and on safe samples, so every trivial baseline scores 0.5.
- *Pairwise correctness (P-C / pAcc)* is the share of vulnerable/patched function pairs where both versions are classified correctly. lu2025lab calls the same idea *patch pair accuracy (PACC)*.
- *SFDR* is the sampled false discovery rate, i.e. the share of a tool's warnings that are false.
- *Out-of-distribution (OOD)* test data come from a different source or time period than the training data.

1. Overview table

Paper	Tier / text read	Design	Data, N, unit	Metrics	Comparators	Statistics / runs	Data & code
ahmed2026secvuleval [ahmed2026secvuleval, p. 6; ahmed2026secvuleval, p. 8]	AIware 2026; arXiv v1	Benchmark + 5 LLMs in a multi-agent pipeline	SecVulEval 25,440 C/C++ functions; detection on 300 functions (~120 vulnerable)	P/R/F1 at function and statement level (manual judging)	none outside LLMs	one run; Mann-Whitney only for function length [ahmed2026secvuleval, p. 13]	dataset + code public [ahmed2026secvuleval, p. 3]
chakraborty2024revisiting [chakraborty2024revisiting, p. 4]	TSE; arXiv author copy	Replication of 4 DL detectors + 2 fine-tuned LLMs on new realistic data	Real-Vul: 5,528 vulnerable + ~1.68M "uncertain" functions, 10 C/C++ projects, time split	Acc, P, R, F1, AUC	DeepWukong, LineVul, ReVeal, IVDetect; CodeLlama, Mixtral	no CIs; Mann-Whitney inside analyses only [chakraborty2024revisiting, p. 10]	Zenodo [chakraborty2024revisiting, p. 2]
deininger2026open [deininger2026open, p. 4]	Computers 2026; publisher PDF	Benchmark: 8 zero-shot LLMs vs 3 SAST tools vs 2 fine-tuned CodeBERT detectors on identical functions; cost and energy	PrimeVul test split, C/C++; 1549 functions (all 549 vulnerable + 1000 safe; 35.4% vulnerable), re-scored at the natural 1:44 prevalence	balanced accuracy, MCC, F1, precision at 1:44, FP per TP, USD and kJ per TP	Flawfinder, Cppcheck, Semgrep; CodeBERT-Devign, CodeBERT-PrimeVul; always-safe / always-vulnerable [deininger2026open, p. 6; deininger2026open, p. 7]	paired bootstrap with Holm correction; one generation at T = 0 [deininger2026open, p. 6]	Zenodo [deininger2026open, p. 26]
ding2025vulnerability [ding2025v	ICSE 2025; arXiv v2	Label/duplication audit + PrimeVul benchmark	6,968 vulnerable / 228,800 benign C/C++	Acc, F1, VD-S, P-C	fine-tuned code LMs vs prompted	runs/seeds not reported	public artifact [ding2025vulnerabili

ulnerability, p. 6]		+ 7 code LMs	functions, 755 projects; 564 test pairs for GPT models		/fine-tuned GPT-3.5/4; random		ty, p. 3]
du2026reducing [du2026reducing, p. 4]	ICSE-SEIP 2026; arXiv v1	Industrial study: interviews + offline evaluation of alarm triage	433 alarms (328 FP / 105 TP) from Tencent's BkCheck, mostly C/C++	Acc, P, R, F1, false-alarm removal; seconds and USD per alarm	prompting LLMs, hybrid LLM4SA/LLM4PFA, 4 DL models	3 runs, majority vote; no tests [du2026reducing, p. 6]	data confidential; replication package [du2026reducing, p. 2]
dubniczky2025castle [dubniczky2025castle, p. 6]	LNCS chapter; arXiv v2	Micro-benchmark; 13 SAST + 2 formal tools vs 10 LLMs	CASTLE: 250 hand-written C programs (150 vulnerable / 100 clean), 25 CWEs	TP/FP/FN, P, R, CASTLE score	each other	repeats "<3%" deviation, number not reported [dubniczky2025castle, p. 14]	public [dubniczky2025castle, p. 1]
fu2023chatgpt [fu2023chatgpt, p. 3]	APSEC 2023; arXiv v1	Prompted ChatGPT vs fine-tuned LMs, 4 tasks	Big-Vul (C/C++), random 80/10/10 split, 1:20 ratio	F1, P, R, top-10 line accuracy	AIBugHunter, CodeBERT, GraphCodeBERT	single point estimates	data, prompts public [fu2023chatgpt, p. 1]
gnieciak2025large [gnieciak2025large, p. 6]	IEEE Access; arXiv v1	Head-to-head, 3 SAST vs 3 LLMs	10 author-prepared C# projects, 63 seeded vulnerabilities	macro P/R/F1, runtime	SonarQube, CodeQL, Snyk Code	single run; no tests	tool code public [gnieciak2025large, p. 1]
guo2025repoaudit [guo2025repoaudit, p. 7]	ICML 2025 (rlab: preprint)	Agent tool paper + ablations	15 C/C++ projects (avg 251 KLoC), 21 known NPD/MLK/UAF bugs; 9 field projects	TP, FP, precision; reproduction of known bugs; tokens, USD, time	Meta Infer, Amazon CodeGuru, CoT prompting, LLMDFA	one run per setting [guo2025repoaudit, p. 17]	code public [guo2025repoaudit, p. 1]
kaniews	TOSE	Systematic	263 studies (78	taxono	n/a	none	artifacts

ki2026systematic [kaniewski2026systematic, p. 8]	M; arXiv v2	literature review (SLR)	arXiv preprints), 2020–Nov 2025	my counts, dataset statistics			public [kaniewski2026systematic, p. 3]
khare2025understanding [khare2025understanding, p. 3]	ICST 2025; arXiv v3	16 LLMs × 3 prompts vs SAST and DL	5,000 balanced function snippets: OWASP, Juliet Java/C, CVEFixes Java/C	Acc, P, R, F1	CodeQL (synthetic only), DeepDFA, LineVul [khare2025understanding, p. 7]	one run, T = 0; no tests	prompts given; code not linked [khare2025understanding, p. 14]
li2024iris [li2024iris, p. 7]	ICLR 2025 (rlab: preprint)	Hybrid tool: LLM-inferred taint specs + CodeQL + LLM triage	CWE-Bench-Java: 120 real Java CVEs (avg 300K SLOC); 30 latest-version projects	detections, AvgFDR, AvgF1	CodeQL, Infer, SpotBugs, Snyk; 7 LLMs	runs not reported (T = 0, fixed seed) [li2024iris, p. 21]	code + data public [li2024iris, p. 2]
li2026llm [li2026llm, p. 5]	TSE; arXiv v2	Project-scale comparison + FP taxonomy + cost	265 known vulnerabilities (110 C/C++, 155 Java), 8 CWEs; 24 real-world projects	recall, precision, SFDR, tokens/USD/minutes	5 specialised LLM detectors, Codex, Claude Code vs CodeQL, Semgrep, CSA, SpotBugs	no tests between tools; CI only for labelling accuracy [li2026llm, p. 11]	artifacts public [li2026llm, p. 2]
lu2025lab [lu2025lab, p. 7]	JSS 2026; arXiv v4 accepted manuscript	4×4 within/cross-dataset DL matrix + temporally separated OOD test of DL and open LLMs	Juliet, Devign, BigVul, ICVul (C/C++); VentiVul: 13,326 functions, 200 Linux/Chromium CVEs disclosed from May 2025, 263 pairs; prevalence 1.97% **[I]** [lu2025lab, p. 4]	Acc, P, R, F1, CVE detection rate, PACC	ReVeal, LineVul (trained on 4 datasets) vs 4 open LLMs ≤ 32B, zero/few-shot/CoT; no SAST, no trivial baseline	none; one deterministic LLM run; DL runs not reported [lu2025lab, p. 8]	GitHub [lu2025lab, p. 2]
qin2026r	Comp	Narrative	32 references,	none	n/a	none	no study

eview [qin2026review, p. 1]	uters and AI (Zeus Press), peer review not documented; publisher PDF	review, single author, no search protocol	2019–2026 [qin2026review, p. 14]	(qualitative)			list
risse2025top [risse2025top, p. 6]	PACMSE (FSE 2025); arXiv v2	81-paper survey + expert labelling + word-count probe	300 vulnerable- + 90 secure-labelled functions (BigVul, Devign, DiverseVul), 2 annotators	label accuracy, Cohen's κ, F1	published SOTA scores	κ only	public [risse2025top, p. 13]
sheng2025llms [sheng2025llms, p. 8]	ACM CSUR; arXiv v2	Narrative survey	58 studies, 2019–2024	counts, F1 listing	n/a	none	paper list [sheng2025llms, p. 1]
shu2026evaluating [shu2026evaluating, p. 7]	EMSE; arXiv v2	8 fine-tuned PLMs vs 5 LLMs (zero/few-shot, instruction tuning), function + line level	REEF, 7 languages, balanced test 2,026 functions; imbalanced and 2024–25 time-split variants	Acc, P, R, F1, MCC, AUC, cost	CodeT5P etc.; dummy classifiers [shu2026evaluating, p. 11]	none; ~30 configurations on one split	public [shu2026evaluating, p. 48]
tan2026think [tan2026think, p. 3]	ASE 2026; publisher PDF	Factorial study, 64 configurations	PrimeVul Pair: 870 functions (435 pairs), 4 open LLMs	Acc, P, R, F1, P-C, FPR; energy, USD	instruct vs thinking; 4 orchestrations; call-matched self-revision	Wilcoxon + Cohen's d; single runs at T = 0 [tan2026think, p. 6]	Zenodo [tan2026think, p. 12]
tasheneva2026real	BDCC 2026; publis	Benchmark + plugin-grou	WordPress plugins, PHP: 1757 CVEs in	macro-F1 (fold mean,	unnamed intra-procedural	paired tests describe	labels, folds, predictio

[tashenova2026real, p. 5]	her PDF	ped 5-fold CV; in-house static taint analyser vs classical ML vs LSTMs vs 3 trivial baselines; **no LLM**	1281 plugins [tashenova2026real, p. 3]; 15,294 (Variant A) / 24,917 (Variant B) functions after leakage control	95% CI), balanced accuracy, MCC, PR-AUC, vulnerable-class FNR	lexical taint analyser; RF, SVM; Bi-LSTM; majority, source/sink rule, token count [tashenova2026real, p. 5]	d; one reported (plugin-level Wilcoxon p = 0.295) [tashenova2026real, p. 9]; single seed [tashenova2026real, p. 7]	ns on GitHub/Zenodo; plugin code re-downloaded [tashenova2026real, p. 14]
xiong2026sifting [xiong2026sifting, p. 6]	PACMSE (ISSTA 2026); arXiv v3	3 agent frameworks × 3 backbones vs vanilla prompting for SAST false-positive filtering	OWASP v1.2 (2,740 cases); 50 Vul4J alerts; 50 post-cutoff OSS-Fuzz alerts	residual FPR, FP recall/precision/F1, TP miss rate, USD	vanilla LLM; 4 SAST tools	no tests; κ only	Zenodo [xiong2026sifting, p. 21]
yildiz2025benchmarking [yildiz2025benchmarking, p. 5]	ACL 2025	Benchmark + plain / dependency-augmented / ReAct LLMs	JITVul: 879 PrimeVul CVEs → 1,758 paired samples, 91 CWEs	F1, pairwise accuracy	LLM designs only	runs not reported; no tests	public [yildiz2025benchmarking, p. 2]
zhou2024comparison [zhou2024comparison, p. 3]	arXiv preprint, **not peer-reviewed**	15 SAST vs 12 open LLMs at repository level	270 CVEs in 134 repos (Java, C, Python); ~8–10 test CVEs per language **[I]** [zhou2024comparison, p. 6]	detection ratio S1/S2, marked-function ratio (no precision/F1)	each other	single run	"public", no link [zhou2024comparison, p. 10]
zhou2025large [zhou2025large, p. 7]	TOSEM; arXiv v3	SLR with quality scoring	58 studies, 2018–Mar 2024	counts	n/a	none	keywords only [zhou2025large, p. 5]

[I] There are 19 primary detection studies (all except the four reviews and risse2025top). Only tan2026think and deininger2026open apply significance tests to their main comparison [tan2026think, p. 6; deininger2026open, p. 6]. tashenova2026real describes paired tests but reports only one [tashenova2026real, p. 9]. Seven report monetary cost: du, guo, li2026llm, shu, tan, xiong and deininger [du2026reducing, p. 8; guo2025repoaudit, p. 7; li2026llm, p. 16; shu2026evaluating, p. 42; tan2026think,

p. 10; xiong2026sifting, p. 17; deininger2026open, p. 7]. Only three report trivial-classifier baselines next to their headline metric: shu, deininger and tashenova [shu2026evaluating, p. 11; deininger2026open, p. 7; tashenova2026real, p. 5] (§5, W1).

2. Approach families

Family	Papers	Typical strengths	Typical weaknesses
A. Head-to-head LLM vs SAST	gnieciak2025large, dubniczky2025castle, khare2025understanding, zhou2024comparison, li2026llm, deininger2026open	Same targets for both families; directly answers RQ1	Small or synthetic targets, SAST on default rules or lightweight tools only, unequal scoring, single runs
B. LLM vs fine-tuned DL / PLM	fu2023chatgpt, ding2025vulnerability, chakraborty2024revisiting, khare2025understanding, shu2026evaluating, zhou2024comparison, du2026reducing, deininger2026open, lu2025lab	Large N; standard metrics; trivial baselines sometimes reported	Noisy function-level C/C++ datasets; F1 on balanced data; no SAST arm except khare, zhou2024 and deininger
B2. SAST vs classical ML/DL, no LLM	tashenova2026real	Real CVEs in a new language (PHP); leakage-controlled; trivial baselines	Single bespoke SAST tool; no LLM; about half of positive labels wrong
C1. Hybrid: LLM inside the analyser (spec inference, dataflow)	li2024iris, guo2025repoaudit; independently re-evaluated in li2026llm	Real projects; builds on mature analysers; new bugs found	Evaluated by tool authors on chosen targets; recall rarely measured
C2. Hybrid: LLM filters SAST alarms	du2026reducing, xiong2026sifting	Large false-alarm reduction; cost per alarm measured	True vulnerabilities also filtered; small real-world samples
D. Agentic / reasoning LLMs	tan2026think, yildiz2025benchmarking, ahmed2026secvuleval, li2026llm (general agents), xiong2026sifting	Repository context and tool use; controlled factorial designs (tan)	Cost; gains often explained by more inference or more flagging
E. Benchmark	chakraborty2024revisiting, ding2025vulnerability,	Audit labels and leakage; temporal	Mostly C/C++; some audits are second-hand

-realism studies	risse2025top, ahmed2026secvuleval, yildiz2025benchmarking, dubniczky2025castle, li2024iris, lu2025lab, tashenova2026real, deininger2026open	and realistic-prevalence testing; trivial baselines; build better benchmarks	or single-annotator
F. Surveys and reviews	kaniewski2026systematic, sheng2025llms, zhou2025large, qin2026review	Field-wide view of datasets and methods	No performance meta-analysis; overlap with primary studies; qin is narrative and not independent

A. Head-to-head LLM vs SAST. Results depend on the unit of analysis. On small, self-contained targets, prompted frontier LLMs score above SAST: macro F1 0.75–0.80 vs 0.26–0.55 on C# projects [gnieciak2025large, p. 10]; CASTLE score 977 for GPT-o3-mini vs 661 for the best non-LLM tool [dubniczky2025castle, p. 11]. On function-level synthetic suites, CodeQL has higher accuracy than the best LLM on all three datasets [khare2025understanding, p. 9]. At repository level, LLMs detect more CVEs but mark up to 77.4% of functions [zhou2024comparison, p. 7]. General coding agents reach far higher recall than SAST [li2026llm, p. 7; li2026llm, p. 9]. deininger2026open scores both families on the same 1549 PrimeVul functions. Seven of eight zero-shot LLMs (balanced accuracy 0.604–0.676) score above the best lightweight SAST configuration (Flawfinder 0.589, Cppcheck up to 0.592; Semgrep 0.501–0.514) [deininger2026open, p. 7; deininger2026open, p. 10]. At the natural 1:44 prevalence, however, Flawfinder and Semgrep are more precise than every LLM [deininger2026open, p. 8; deininger2026open, p. 10]. Its SAST arm has no CodeQL, because isolated functions cannot be built [deininger2026open, p. 10]. **[I]** The family's main weakness is that each study uses its own targets and scoring, so the papers measure different things (see §3.5).

B. LLM vs fine-tuned DL/PLM. On random-split Big-Vul, fine-tuned small models clearly beat prompted ChatGPT (F1 94% vs 29%) [fu2023chatgpt, p. 4]. Under realistic splits all models collapse [ding2025vulnerability, p. 7; chakraborty2024revisiting, p. 7]. The only multilingual comparison finds that the winner depends on class balance [shu2026evaluating, p. 17; shu2026evaluating, p. 40]. Two pass-2 studies add:

- On unpaired PrimeVul functions, a fine-tuned 125M CodeBERT beats all eight zero-shot LLMs (balanced accuracy 0.765 vs ≤ 0.711) [deininger2026open, p. 11; deininger2026open, p. 17]. With a default 0.5 decision threshold, the same detector looks like an always-safe classifier [deininger2026open, p. 11].
- On 2025 CVEs (VentiVul), trained DL detectors reach F1 3.47–8.65 and four open LLMs ≤ 32B reach conservative F1 0.79–5.36 [lu2025lab, p. 13; lu2025lab, p. 14]. **[I]** Both are close to the all-positive F1 of about 3.87%.

[I] This family inherits the label noise and leakage of Big-Vul-type data (§5, W2). Its results also depend on the decision threshold and the test prevalence (C1, C5, C8).

B2. SAST vs classical ML/DL without an LLM. tashenova2026real is the only PHP study in the corpus. On real WordPress plugin CVEs, an unnamed in-house intra-procedural lexical taint analyser detects 1.7–1.8% of vulnerabilities [tashenova2026real, p. 9]. The authors say this low

recall reflects "that implementation rather than taint analysis in general" [tashenova2026real, p. 13] **[A]**. The best learned model reaches macro-F1 0.601, and no neural model leads in any of six settings [tashenova2026real, p. 8]. **[I]** The paper says nothing about LLMs and cannot stand for established PHP SAST tools.

C1. Hybrid detection (LLM inside the analyser). IRIS doubles CodeQL's detections on CWE-Bench-Java (55 vs 27 of 120), with AvgFDR 84.82% vs 90.03% [li2024iris, p. 8]. Only GPT-4 lowers FDR below CodeQL; smaller LLMs worsen it [li2024iris, p. 10]. RepoAudit reports 78.43% precision [guo2025repoaudit, p. 8]. Both look much weaker in li2026llm's independent re-evaluation (§4, C2 and C6).

C2. Hybrid false-positive filtering. The industrial LLM4PFA hybrid removes 94–98% of false alarms while keeping 75–88% of real bugs [du2026reducing, p. 8]. The best agent configuration cuts OWASP FPR from 98.3% to 6.3% [xiong2026sifting, p. 12] but drops 22.25% of true vulnerabilities [xiong2026sifting, p. 18]. **[I]** This is the most consistent benefit in the corpus, but it is a trade-off, not a free gain (§3.9, G7).

D. Agentic/reasoning LLMs. Thinking mode raises F1 mainly through recall (0.31 → 0.41) at 2.7–4.4× the energy [tan2026think, p. 6; tan2026think, p. 10]. A four-call self-revision matches the multi-agent pipeline on P-C, so much of the agent gain is extra inference budget [tan2026think, p. 11] **[A]**. ReAct agents give the best pairwise accuracy with GPT-4o models, but it stays ≤20.17% [yildiz2025benchmarking, p. 6].

E. Benchmark realism. Manual audits find only 39–65% of vulnerable labels correct in BigVul, Devign and DiverseVul [risse2025top, p. 8], and 24–36% in CodeXGLUE/Devign and VulnPatchPairs [ding2025vulnerability, p. 4]. A word-count classifier reaches F1 0.86 on BigVul [risse2025top, p. 11]. Statement-level scoring halves LLM F1 [ahmed2026secvuleval, p. 8]. The pass-2 papers add three things:

- **Label audit outside C/C++.** An expert review finds only 48.8% of cleaned positive WordPress-PHP labels genuine (single annotator, n = 400) [tashenova2026real, p. 11]. A one-feature token-count baseline matches or beats TF-IDF Random Forest in 4 of 6 settings [tashenova2026real, p. 8].
- **Temporal separation.** VentiVul uses CVEs disclosed from May 2025, with the sample size fixed before collection [lu2025lab, p. 4].
- **Realistic prevalence.** deininger re-scores all detectors at PrimeVul's natural 1:44 ratio [deininger2026open, p. 8].

F. Surveys and reviews. Two SLRs and one narrative survey describe a field dominated by function-level C/C++ classification on noisy datasets [kaniewski2026systematic, p. 26; zhou2025large, p. 18; sheng2025llms, p. 20]. A second narrative review, qin2026review, is discussed below. None of the four pools effect sizes. kaniewski2026systematic excludes studies that only post-process tool output [kaniewski2026systematic, p. 8], so it does not cover family C2.

qin2026review is a single-author narrative review with 32 references and no search protocol [qin2026review, p. 14]. Its quantitative claims re-report ding2025vulnerability and gnieciak2025large [qin2026review, p. 12], so it is not independent support for either. It concludes that SAST is "usually more stable" within rule coverage [qin2026review, p. 12] **[A]**. **[I]** Its only head-to-head source,

however, found LLMs above SAST on F1 [gnieciak2025large, p. 10]. We use qin2026review only to position our review.

3. Comparable results

Numbers are compared only where dataset, metric and setup match.

Comparison	Dataset / metric	Verdict
3.1 CodeQL on OWASP Benchmark (khare vs xiong)	OWASP Java; P/R/F1	**Comparable** (different sample size)
3.2 PrimeVul pairs (ding, tan, yildiz)	PrimeVul-derived pairs; P-C / pAcc	**Partly comparable**
3.3 Big-Vul F1 (fu, ding, chakraborty, risse, lu)	"Big-Vul"; F1	**Not comparable**
3.4 CodeQL on CWE-Bench-Java (li2024iris vs li2026llm)	Java CVEs; recall	**Partly comparable**
3.5 LLM-vs-SAST headline scores	different units and metrics	**Not comparable**
3.6 LLM filtering of SAST alarms (du vs xiong)	different alarms and metrics	**Not numerically comparable**
3.7 Unpaired PrimeVul functions (deininger) vs PrimeVul pairs (ding, tan, yildiz)	same source dataset; different unit, prevalence and metric	**Not comparable; do not pool**
3.8 Detection at realistic prevalence (deininger at 1:44, lu at 1.97%)	different datasets; each scored against its own trivial baseline	**Not numerically comparable; same direction**

3.1 CodeQL on OWASP (comparable). khare reports CodeQL P 0.58, R 0.96, F1 0.72 on a 1,000-snippet balanced OWASP sample [khare2025understanding, p. 9]. xiong reports P 60.3%, R 97.0%, F1 74.4% on the full 2,740-case OWASP v1.2 [xiong2026sifting, p. 11]. **[I]** Two independent groups agree within about 2 points. This is the only cross-paper SAST figure we can treat as replicated. Both describe a high-recall, low-precision tool (FPR 68.2% [xiong2026sifting, p. 11]) whose F1 is near the always-vulnerable value of 0.667 on balanced data.

3.2 PrimeVul pairs (partly comparable). All three studies use pairs of vulnerable and patched functions drawn from PrimeVul:

- ding: GPT-4 CoT P-C 12.94% on 564 test pairs, below a random reference of 22.70% [ding2025vulnerability, p. 9].
- tan: best non-agent P-C 22.5% on 435 pairs with open models [tan2026think, p. 6]; self-revision 24.7 on 193 pairs [tan2026think, p. 11].
- yildiz: GPT-4o/4o-mini pAcc 1.02–20.17% on 879 CVE pairs from JITVul, with repository context [yildiz2025benchmarking, p. 6].

The pair subsets, models, context and dates differ. **[I]** The direction agrees: GPT-class and open LLMs stay at or below the ~25% ceiling that independent random labelling can reach. One outlier exists: DeepSeek-Coder-V2 with dependency augmentation reaches pAcc 69.24, reported in an appendix only [yildiz2025benchmarking, p. 15]. All three use one dataset family, so this is related evidence, not independent replication. deininger2026open also uses PrimeVul but scores single, unpaired functions, so it is kept out of this comparison (§3.7). lu2025lab's PACC values on VentiVul pairs (all < 6%) [lu2025lab, p. 13] come from a different dataset. **[I]** They point the same way: a random labeller would reach up to 25% PACC.

3.3 Big-Vul F1 (not comparable). Reported F1 on "Big-Vul" varies widely:

- CodeBERT: 87% [fu2023chatgpt, p. 4] vs 62.88% [ding2025vulnerability, p. 7].
- Replicated LineVul: 90% [chakraborty2024revisiting, p. 6].
- LineVul with its original pipeline: 91.00 [lu2025lab, p. 12]. lu used the original authors' code and recommended settings [lu2025lab, p. 8].
- Literature SOTA (DeepDFA): 96.46% [risse2025top, p. 7].
- Word-count baseline: 0.86 [risse2025top, p. 11].

Splits, preprocessing and de-duplication differ. Neither lu nor ding explains the 28-point gap between their in-distribution CodeBERT/LineVul values (contradiction C7). **[I]** A spread of about 25–28 points for one model on one named dataset means Big-Vul F1 values should be cited only as a range. Because a word-count baseline reaches 0.86, high Big-Vul F1 is not evidence of vulnerability understanding.

3.4 CodeQL on CWE-Bench-Java (partly comparable). CodeQL detects 27 of 120 CVEs (22.5%) in the IRIS paper [li2024iris, p. 8] and has 14.19% recall on li2026llm's Java set [li2026llm, p. 9]. That set mixes a Maven-only CWE-Bench-Java subset with 50 JLeaks resource leaks [li2026llm, p. 5]. **[I]** Both values are low, and the gap is plausibly due to the different CWE mix and CodeQL versions.

3.5 LLM-vs-SAST headline numbers (not comparable). gnieciak counts LLM findings as true positives even when the line or column is wrong, which SAST tools are not allowed [gnieciak2025large, p. 6]. dubniczky uses a severity-weighted score [dubniczky2025castle, p. 10]. khare uses function-level accuracy on balanced sets [khare2025understanding, p. 9]. zhou2024 uses CVE detection ratio and marked-function ratio [zhou2024comparison, p. 5]. li2026llm uses recall on known CVEs and SFDR on new code [li2026llm, p. 11]. deininger2026open scores LLMs and SAST on identical functions with identical metrics (balanced accuracy and precision at 1:44) [deininger2026open, p. 10]. Within that one study, the comparison is like-for-like. Its SAST arm is lightweight, though: Flawfinder, Cppcheck and Semgrep, with no CodeQL [deininger2026open, p. 10]. **[I]** These six studies cannot be pooled or ranked against each other. tashenova2026real reports macro-F1 on PHP for a bespoke analyser [tashenova2026real, p. 8] and does not add to this comparison.

3.6 False-positive filtering (not numerically comparable). du measures removal of false alarms on 433 industrial alarms of general bug types [du2026reducing, p. 8]. xiong measures residual FPR and TP miss rate on OWASP and 100 sampled alerts [xiong2026sifting, p. 12; xiong2026sifting, p. 18]. Only the qualitative pattern can be compared (§3.9, G7).

3.7 Unpaired PrimeVul functions vs PrimeVul pairs (not comparable; do not pool). The two designs differ as follows:

- **deininger** samples 1549 single functions (35.4% vulnerable) from the PrimeVul test split [deininger2026open, p. 4]. It reports balanced accuracy 0.511–0.711 for LLMs and 0.765 for fine-tuned CodeBERT [deininger2026open, p. 7; deininger2026open, p. 11; deininger2026open, p. 17].
- **ding, tan and yildiz** score both members of each vulnerable/patched pair together (§3.2).

The unit, prevalence and metric all differ. **[I]** Balanced accuracy on single functions and pair-wise correctness answer different questions. The first asks whether a detector ranks vulnerable functions above safe ones. The second asks whether it notices the fix. So "fine-tuned CodeBERT beats all LLMs" (deininger) and "GPT-4 beats fine-tuned models on pairs" (ding) are not a numerical contradiction; C1 discusses them. Even inside one study, yildiz's F1 drops from ~49–65% on balanced pairs to 5.41–12.21% on non-paired PrimeVul [yildiz2025benchmarking, p. 7]. deininger's PrimeVul test split (24,788 functions, 549 vulnerable) also differs from the split recorded by ding (695 vulnerable / 25,216 benign) [deininger2026open, p. 4; ding2025vulnerability, p. 6]. deininger explains this only as "after loading".

3.8 Detection at realistic prevalence (not numerically comparable; same direction). Two studies score detectors at about 2% prevalence against a trivial baseline:

- **deininger** (PrimeVul, 1:44 ≈ 2.2%): every LLM's precision is 2.3–3.8% in the first epoch and 2.8–4.6% in the budget-matched epoch, vs 2.21% for flag-all [deininger2026open, p. 8; deininger2026open, p. 17].
- **lu** (VentiVul, 1.97% **[I]**): trained DL reaches F1 3.47–8.65, and 7 of 12 LLM conservative F1 values fall below the all-positive F1 of about 3.87% [lu2025lab, p. 13; lu2025lab, p. 14].

The datasets, metrics (precision vs F1) and models differ (frontier and open LLMs vs open LLMs ≤ 32B). **[I]** Both show LLMs only slightly above, or below, a classifier that flags everything.

3.9 Agreements / converging evidence

Numbering follows the "Agreements" section of evidence-matrix.md.

- **G1. Realistic evaluation collapses reported performance (strong).** Several studies show it:
- DL precision falls to 1–10% on Real-Vul [chakraborty2024revisiting, p. 7].
- Code-LM F1 falls from 62.88–68.26% (BigVul) to 18.05–21.43% within PrimeVul and 1.91–5.82% cross-dataset [ding2025vulnerability, p. 7].
- LLM accuracy is 10.5% lower on real-world than synthetic data [khare2025understanding, p. 5].
- GPT-4o line-level F1 drops from 0.6641 to 0.3450 on 2024–25 code [shu2026evaluating, p. 39].
- Statement-level scoring halves F1 [ahmed2026secvuleval, p. 8].
- Pass 2: on post-May-2025 CVEs at 1.97% prevalence, DL F1 is 3.47–8.65, and 7 of 12 LLM conservative F1 values fall below the all-positive F1 of 3.87% [lu2025lab, p. 13; lu2025lab, p. 14].

- Pass 2: at 1:44 prevalence, LLM precision is 2.3–4.6% vs 2.21% for flag-all [deininger2026open, p. 8; deininger2026open, p. 17].
- Pass 2: on real WordPress PHP CVEs, the best macro-F1 is 0.601 [tashenova2026real, p. 8].

The surveys agree [kaniewski2026systematic, p. 39; sheng2025llms, p. 20].

- **G2. Benchmark labels are noisy (strong).** Two independent audits find 39–65% [risse2025top, p. 8] and 24–36% [ding2025vulnerability, p. 4] of vulnerable labels correct. ReVeal has 15% label inconsistency [chakraborty2024revisiting, p. 3]. The 25% BigVul figure is Chen et al.'s, reproduced by Ding [ding2025vulnerability, p. 4]; it is not an independent estimate. Pass 2 adds a third audit, in a new language: only 48.8% of cleaned positive WordPress-PHP labels are genuine (single annotator, n = 400) [tashenova2026real, p. 11].
- **G3. Prompted LLMs over-flag and cannot separate vulnerable from patched code (moderate).**
- GPT-4 two-shot calls both versions vulnerable in 71.63% of pairs [ding2025vulnerability, p. 9].
- "Both vulnerable" accounts for 40–95% of LLM failures [yildiz2025benchmarking, p. 8].
- The dual-agent design flags 88.2% of functions [tan2026think, p. 7].
- Several of khare's best F1 rows are "always vulnerable" outputs [khare2025understanding, p. 16].
- Frontier models over-flag CWE-476/416/190 [ahmed2026secvuleval, p. 13] **[A]**.

[I] ding, yildiz and tan share PrimeVul, so this is partly one dataset family.

- **G4. SAST recall is low on real code and high only on synthetic suites (moderate).** On real code:
- Best single SAST tools detect 30.0–44.4% of CVEs, and many detect 0% [zhou2024comparison, p. 6].
- CodeQL recall is 6.00–34.29% on C/C++ and 14.19% on Java [li2026llm, p. 7; li2026llm, p. 9].
- CodeQL finds 27/120 on CWE-Bench-Java [li2024iris, p. 8].
- On CASTLE, CodeQL reaches 29% and the SAST average is 23% [dubniczky2025castle, p. 12].

On synthetic suites, CodeQL reaches accuracy 0.94 on Juliet Java [khare2025understanding, p. 9].

- **G5. CodeQL on OWASP replicates across groups (moderate).** See §3.1 [khare2025understanding, p. 9; xiong2026sifting, p. 11].
- **G6. Missing context is the shared failure cause (moderate).**
- All 151 truly vulnerable audited functions need context beyond the function [risse2025top, p. 9].
- LLMs identify required context in ≤41.33% of cases per context type [ahmed2026secvuleval, p. 9].
- Global context is the main failure cause on real C/C++ [khare2025understanding, p. 7] **[A]**.
- Incomplete API modelling explains 108/108 RepoAudit (GPT-5.4) misses [li2026llm, p. 7], and 14.60% of all FPs come from truncated inter-procedural context [li2026llm, p. 15].
- **G7. LLM false-alarm filtering trades large FP reduction for lost true positives (moderate).** Evidence: du2026reducing reaches 94–98% false-alarm removal with real-bug recall of 0.75–0.88 [du2026reducing, p. 8]; xiong2026sifting cuts FPR to 6.3% but misses 22.25% of true vulnerabilities [xiong2026sifting, p. 12; xiong2026sifting, p. 18]. du covers

non-security bug classes, evaluates its authors' own method and has Tencent-affiliated authors [du2026reducing, p. 1]. Strength is therefore moderate.

- **G8. Prompt-only LLMs trail adapted models (moderate).**
- Prompted ChatGPT lags fine-tuned models on Big-Vul [fu2023chatgpt, p. 4].
- GPT-3.5 prompting detects 3.7–23.2% vs 89.2% for fine-tuned CodeBERT [zhou2024comparison, p. 10].
- Prompt-only LLMs are near chance on REEF, and only fine-tuned GPT-4o with few-shot stands out [shu2026evaluating, p. 17].
- **G9. LLM cost far exceeds SAST cost and varies by orders of magnitude (moderate).** SAST runs in minutes with no token cost, while RepoAudit averages $172.77 per project [li2026llm, p. 16]. Alarm filtering costs cents per alarm [du2026reducing, p. 8; xiong2026sifting, p. 17]. Instruction tuning GPT-4o costs about 15× a local PLM training run [shu2026evaluating, p. 42].
- **G10. Trivial baselines are competitive (moderate).** Three studies show this:
- A one-feature token-count baseline matches or beats TF-IDF Random Forest in 4 of 6 settings and is within 0.006 of the best neural model [tashenova2026real, p. 8].
- Always-vulnerable F1 is 0.523, against LLM F1 of 0.405–0.615 [deininger2026open, p. 7].
- The all-positive F1 of 3.87% is above most LLM F1 values on VentiVul [lu2025lab, p. 14].

shu2026evaluating also reports all-vulnerable and all-clean dummy baselines [shu2026evaluating, p. 11]. **[I]** Few papers report such baselines, and most headline numbers are never compared with one (§5, W1).

- **G11. Cost per true positive (moderate, single study).** Each true positive costs USD 0.003 (Gemma-3-4B) to 0.212 (Claude Sonnet 5) and 3.7–119.6 kJ at 1:44 prevalence [deininger2026open, p. 8]. This excludes human triage of the 25.6–41.9 false positives that come with each true one [deininger2026open, p. 8]. The kJ per TP for Gemma, Qwen and Llama come from FLOP estimates, not measured energy [deininger2026open, p. 14] (§5, W6).

4. Contradictions

C1. Fine-tuned models vs GPT-4: fu2023chatgpt vs ding2025vulnerability.

- Claim A: fine-tuned AIBugHunter (F1 94%) and CodeBERT (87%) far outperform prompted gpt-4 (29%) [fu2023chatgpt, p. 4].
- Claim B: GPT-4 CoT has a higher P-C (12.94%) than every fine-tuned model (≤3.01%) [ding2025vulnerability, p. 9].
- Differences:
- metric: F1 vs pairwise correctness;
- data: noisy random-split Big-Vul at 1:20 vs de-duplicated, time-split PrimeVul pairs;
- time: 2023 vs 2024 GPT versions.
- Ding's own BigVul rows (F1 62.88–68.26%) agree with Fu's direction [ding2025vulnerability, p. 7].
- **[I]** Explanation: fine-tuned models probably exploit dataset artefacts that random splits reward and paired, de-duplicated tests remove [risse2025top, p. 11]. On pairs, every model

is below the random reference (22.70%), so neither side shows real detection skill there, and the reversal says more about the benchmark than about the model families.

- Pass 2: on 1549 unpaired PrimeVul functions, a fine-tuned CodeBERT-PrimeVul beats all 8 zero-shot LLMs (balanced accuracy 0.765 vs ≤ 0.711) [deininger2026open, p. 11; deininger2026open, p. 17]. With an argmax decision rule, the same detector looks like an always-safe classifier (0.500) despite ROC-AUC 0.845 [deininger2026open, p. 11].
- **[I]** A conservative threshold would fit Ding's fine-tuned models labelling both pair members benign in 84–88% of pairs [ding2025vulnerability, p. 9]. Unpaired and pair-wise results must not be pooled (§3.7).

C2. RepoAudit: guo2025repoaudit vs li2026llm.

- Claim A: 40 TP / 11 FP (78.43% precision) on 15 C/C++ projects, all 21 known bugs reproduced, $2.54 and 0.44 h per project [guo2025repoaudit, p. 7; guo2025repoaudit, p. 8].
- Claim B: in li2026llm, RepoAudit (Sonnet 3.5) detects 0 of 110 known C/C++ vulnerabilities [li2026llm, p. 7]. On real projects it has 92.77% SFDR [li2026llm, p. 11] and costs $172.77 on average, up to $1,247.66 per project [li2026llm, p. 16].
- Differences:
- targets: prior tools' bug reports vs ReposVul CVEs;
- code version: older commits vs current HEAD (the five overlapping Linux/ImageMagick modules are in li's real-world set only);
- metric: precision on own reports vs recall/SFDR;
- labeller: authors vs Codex;
- scale: avg 251 KLoC vs multi-million-LOC modules.
- **[I]** Explanation: RepoAudit depends on source/sink API coverage. li2026llm records zero-token runs and 108/108 misses due to API modelling [li2026llm, p. 7; li2026llm, p. 16], so the tool may perform well only on targets matching its built-in specifications. Neither paper alone settles this.

C3. "LLMs beat SAST" vs "SAST beats LLMs" vs repository level.

- Claim A: LLMs win on 10 small C# projects [gnieciak2025large, p. 10] and on CASTLE C micro-programs [dubniczky2025castle, p. 11].
- Claim B: CodeQL beats the best LLM in accuracy on OWASP, Juliet Java and Juliet C/C++ [khare2025understanding, p. 9].
- Claim C (repository level): LLMs detect more but mark up to 77.4% of functions [zhou2024comparison, p. 7]. General agents reach the highest recall, while specialised LLM detectors and SAST both have near-zero recall on some CWEs and 75–100% SFDR [li2026llm, p. 7; li2026llm, p. 11].
- Differences: language (C#, C, Java), unit (project, program, function, repository), scoring (lenient LLM location rule in gnieciak [gnieciak2025large, p. 6]; the CASTLE prompt reveals the scoring rule to the LLM [dubniczky2025castle, p. 9]), and synthetic vs real code.
- **[I]** Explanation: synthetic suites match SAST rule sets, while small self-contained programs fit inside an LLM's context. Real repositories defeat both families through missing API and context models (G4, G6). The answer depends on setting and metric, so there is no single winner.
- Pass 2: deininger scores LLMs and SAST on the same 1549 PrimeVul functions:

- LLMs lead on balanced accuracy [deininger2026open, p. 7; deininger2026open, p. 10].
- At 1:44 prevalence, Flawfinder (6.2–13.4%) and Semgrep (26.0%, CI 5.7–66.9) are more precise than every LLM (2.3–4.6%) [deininger2026open, p. 8; deininger2026open, p. 10].
- Cppcheck's precision is within the LLM range [deininger2026open, p. 10].
- **[I]** The two families fail in opposite directions: LLMs over-flag, while SAST finds little. Which family "wins" therefore depends on the metric. This fits the pattern "LLMs higher recall, lower precision".

C4. Does reasoning reduce false positives?

- Claim A: reasoning models "consistently produced fewer false positives" [dubniczky2025castle, p. 14]. RepoAudit precision rises to 82.35–88.46% with reasoning backbones [guo2025repoaudit, p. 16].
- Claim B:
- In CASTLE's own table, DeepSeek R1 has 166 FP vs GPT-4o's 116 [dubniczky2025castle, p. 11].
- Thinking mode raises F1 through recall with unchanged accuracy, and its F1 benefit turns negative under dual- and multi-agent designs [tan2026think, p. 6; tan2026think, p. 8].
- Reasoning LLMs are near chance on REEF (Acc 0.52–0.53) [shu2026evaluating, p. 37].
- Differences: OpenAI o-series vs open reasoning models; micro-benchmark and repository agent vs balanced function pairs; FP count vs F1 and accuracy.
- **[I]** Explanation: the claim holds for some proprietary reasoning models in some settings and does not generalise to "reasoning" as such.
- Pass 2, against: reasoning lowers DeepSeek-V3.2's specificity from 0.52 to 0.37 [deininger2026open, p. 17; deininger2026open, p. 20]. In the same study it raises the balanced accuracy of the frontier models [deininger2026open, p. 17]. **[I]** The effect of reasoning on false positives is model-dependent.

C5. Class balance flips the ranking (shu2026evaluating, internal).

- Claim A: on balanced REEF, fine-tuned GPT-4o + few-shot beats CodeT5P (Acc 0.7196 vs 0.6037; MCC 0.4360 vs 0.2860) [shu2026evaluating, p. 17].
- Claim B: on the imbalanced (~17% vulnerable) test, PLMs win (UniXcoder F1 0.4626, CodeT5P MCC 0.3627 vs GPT-4o F1 0.4010, MCC 0.2774) [shu2026evaluating, p. 40; shu2026evaluating, p. 41].
- yildiz shows the same sensitivity: F1 falls from ~49–65% on balanced pairs to 5.41–12.21% on non-paired PrimeVul [yildiz2025benchmarking, p. 7].
- Pass 2 adds two more cases:
- deininger's precision ranking at 1:44 differs from its balanced-accuracy ranking [deininger2026open, p. 8].
- In tashenova, changing how negatives are drawn flips the majority class and the direction in which the same Random Forest fails [tashenova2026real, p. 9].
- **[I]** Explanation: LLMs flag positives more liberally, which balanced sets reward. Rankings from balanced benchmarks should not be carried over to deployment, where vulnerable code is rare.

C6. Hybrid IRIS: original vs independent evaluation.

- Claim A: IRIS+GPT-4 beats CodeQL on 120 Java CVEs (55 vs 27 detected; AvgFDR 84.82% vs 90.03%) [li2024iris, p. 8].
- Claim B: on 15 real-world Java projects, IRIS (GPT-4) has 97.80% SFDR vs 77.00% for CodeQL [li2026llm, p. 12].
- Differences: known CVEs vs new code, detections vs SFDR, CodeQL-protocol FDR vs Codex-labelled sampling.
- **[I]** Explanation: hybrids may raise recall on benchmark CVEs while adding noise on unseen projects. The comparison is not like-for-like, so the tension stays open.

C7. Same model, same benchmark, different in-distribution score: lu2025lab vs ding2025vulnerability.

- Claim A: LineVul (CodeBERT) trained and tested on BigVul reaches F1 91.00 [lu2025lab, p. 12].
- Claim B: CodeBERT trained and tested on BigVul reaches F1 62.88 [ding2025vulnerability, p. 7].
- Differences: neither paper explains the 28-point gap. lu used LineVul's original pipeline and recommended settings [lu2025lab, p. 8], whereas ding trains its own set of code LMs [ding2025vulnerability, p. 7].
- **[I]** Explanation: likely split and preprocessing differences. In-distribution BigVul numbers are not comparable across papers (§3.3).

C8. Does a time-split fine-tuned detector survive? deininger2026open vs lu2025lab.

- Claim A: CodeBERT-PrimeVul keeps balanced accuracy 0.765 on PrimeVul's chronologically later test split [deininger2026open, p. 3; deininger2026open, p. 11]. At 1:44 prevalence this corresponds to F1 ≈ 0.15 (our calculation from reported TPR and precision).
- Claim B: DL trained on BigVul, ICVul, Devign or Juliet falls to F1 3.47–8.65 on post-May-2025 VentiVul [lu2025lab, p. 13].
- Differences:
- test data: PrimeVul functions with CVEs from 2008–2022 [deininger2026open, p. 24] vs VentiVul 2025 CVEs scored per function across whole files [lu2025lab, p. 4];
- training data: PrimeVul's own training split vs older, noisier benchmarks;
- threshold policy: validation-tuned [deininger2026open, p. 11] vs the original implementations' settings [lu2025lab, p. 8];
- metric: balanced accuracy vs F1.
- **[I]** Explanation: unresolved. A detector trained on cleaner data from the same source, with a tuned threshold, may hold up better than detectors trained on other benchmarks. Neither study tests this directly.

5. Methodological weaknesses across the field

- **W1. Trivial baselines rarely reported.** On a 50/50 set, "flag everything" scores F1 0.667, so balanced-data F1 near that value shows no skill. Examples:
- shu's DummyClf_vul scores 0.6693 against a best of 0.7075 [shu2026evaluating, p. 17].
- tan's best F1 is 0.665–0.666 [tan2026think, p. 7; tan2026think, p. 11].

- All of yildiz's GPT F1 values (48–66) are at or below it [yildiz2025benchmarking, p. 6].
- khare's best-per-dataset F1s include always-vulnerable rows [khare2025understanding, p. 16].

Pass 2 shows the same problem at other prevalences (G10):

- deininger: the best LLM F1 (0.615) is 0.092 above always-vulnerable (0.523), and Qwen3-Coder-30B (0.405) is below it [deininger2026open, p. 7].
- tashenova: a token-count baseline is within 0.006 of the best neural model [tashenova2026real, p. 8].
- lu reports no trivial baseline. **[I]** By our calculation, 7 of its 12 LLM conservative F1 values fall below the all-positive F1 of about 3.87% [lu2025lab, p. 14].

[I] Only shu, deininger and tashenova report trivial-classifier baselines (3 of 19 primary studies). ding reports a random reference only for pair-wise correctness [ding2025vulnerability, p. 9]. tashenova's authors argue that detectors should be required to clear such baselines [tashenova2026real, p. 13] **[A]**.

- **W2. Label quality and leakage.** Problems include:
- **Exact-copy leakage** from test to train reaches 12.7% (BigVul) and 18.9% (CVEFixes) [ding2025vulnerability, p. 4]. Even plugin-grouped PHP folds contained duplicate groups spanning folds (166 / 151 groups) and 730 / 456 groups with contradictory labels before cleaning [tashenova2026real, p. 4].
- **Label quality** is low wherever it is audited: 39–65% [risse2025top, p. 8], 24–36% [ding2025vulnerability, p. 4] and 48.8% for PHP [tashenova2026real, p. 11] of positive labels are correct. **[I]** tashenova's evaluated corpus also contains lower-tier labels, so its true label precision is probably lower still. Its authors argue that label quality, not architecture, is "the more plausible binding constraint" [tashenova2026real, p. 12] **[A]**.
- **Contamination.** Benchmarks probably sit in LLM pre-training data [kaniewski2026systematic, p. 30] **[A]**. deininger finds recall 7–15 points higher on pre-2020 than on 2021–2022 CVEs for better-balanced LLMs, which is consistent with memorisation [deininger2026open, p. 24].
- **Post-cutoff testing** is rare. Only xiong, li2026llm and lu test explicitly on code disclosed after the evaluated models' release [xiong2026sifting, p. 15; li2026llm, p. 5; lu2025lab, p. 5]. lu notes that release-date separation cannot fully exclude contamination [lu2025lab, p. 17]. shu adds a time split [shu2026evaluating, p. 39].
- **W3. LLM-as-labeller.** li2026llm labels real-world warnings with Codex, a tool it also evaluates (94.36% accurate on 621 validated warnings) [li2026llm, p. 11]. tan's LLM judge rates explanations higher in ways humans corroborate only for informativeness [tan2026think, p. 8]. **[I]** Self-labelling can favour the labelling model.
- **W4. Unequal scoring between LLMs and SAST.** gnieciak applies a lenient location rule only to LLMs [gnieciak2025large, p. 6]. CASTLE's LLM prompt states the reward and penalty scheme [dubniczky2025castle, p. 9]. SAST tools run on default configurations [gnieciak2025large, p. 8], and khare tests CodeQL only on synthetic data [khare2025understanding, p. 7]. **[I]** These choices favour LLMs in two studies and SAST in one.
- **W5. Single runs, no statistics.** Most studies report one run of non-deterministic models without variance (§1). Examples:

- zhou2024's results rest on about 8–10 test CVEs per language **[I]** [zhou2024comparison, p. 6].
- xiong's real-world samples are 50 alerts each [xiong2026sifting, p. 13].
- shu chooses the best of ~30 configurations post hoc [shu2026evaluating, p. 17].
- lu runs each LLM once and does not report DL run counts [lu2025lab, p. 8].
- tashenova uses a single seed [tashenova2026real, p. 7].

deininger shows why this matters. A reworded prompt moves balanced accuracy by up to 0.088 and changes which model wins [deininger2026open, p. 19; deininger2026open, p. 20]. Absolute scores shift by up to 0.06 between measurement epochs seven weeks apart [deininger2026open, p. 20].

- **W6. Little cost reporting; energy figures of mixed provenance.** Twelve of 19 primary studies report no monetary cost (§1). Where cost is measured it is decisive: up to $1,247.66 per project [li2026llm, p. 16]. Only deininger reports cost per true positive (G11), and it excludes human triage [deininger2026open, p. 8]. Its energy figures mix three sources:
- Frontier-model energy is a FLOP estimate that assumes 100B active parameters [deininger2026open, p. 7].
- The kJ per TP for Gemma, Qwen and Llama come from FLOP estimates (64, 40 and 944 J per task), not from measured energy [deininger2026open, p. 8; deininger2026open, p. 14].
- The FLOP estimator overstates measured batched energy by 5.4–16.9× [deininger2026open, p. 14].

[I] With measured single-request energy, our recalculation gives about 4.9 (Gemma), 8.8 (Qwen) and 34.9 (Llama) kJ per TP. The ordering among open models changes, but the gap to frontier models remains. Energy numbers should always state whether they are measured or estimated.

- **W7. Narrow scope.** Data are C/C++-biased and long-tailed, and 555 mappable CWEs appear in no analysed dataset [kaniewski2026systematic, p. 34]. 87 studies build custom datasets and only nine datasets are reused in more than ten studies [kaniewski2026systematic, p. 35]. Only one of 40 detection studies to March 2024 worked at repository level [zhou2025large, p. 18]. PHP is covered by a single function-level study, which has no LLM arm and only a bespoke analyser [tashenova2026real, p. 13].
- **W8. Three-way comparisons are rare. [I]** Two studies evaluate LLMs, SAST and DL together:
- khare, whose SAST arm covers synthetic data only [khare2025understanding, p. 7];
- deininger, on identical PrimeVul functions with lightweight SAST and no CodeQL [deininger2026open, p. 10], using CVEs from 2008–2022 that predate the models' training cutoffs [deininger2026open, p. 24].

zhou2024 compares fine-tuned and prompted LLMs with SAST but no classic DL detector [zhou2024comparison, p. 4]. lu compares DL with LLMs but has no SAST arm, and tashenova compares SAST with ML/DL but has no LLM arm [lu2025lab, p. 7; tashenova2026real, p. 13].

- **W9. Preprint text vs version of record.** Fifteen published papers were analysed from arXiv copies (xiong2026sifting's copy is the formatted published article). lu2025lab was read as the arXiv v4 accepted manuscript of its JSS 2026 version [lu2025lab, p. 1]. du2026reducing comes from a v1 with a placeholder header [du2026reducing, p. 2], so it has

the highest risk that numbers changed. zhou2025large's only LLM-vs-SAST study is the authors' own zhou2024comparison [zhou2025large, p. 22], so it is not independent support.

- **W10. Independence. [I]** Several sources overlap:
- guo and li2026llm share five C/C++ modules.
- ding, yildiz, tan and deininger all draw on PrimeVul.
- Duplication figures reused elsewhere trace back to ding alone (critical review B).
- qin2026review's quantitative claims re-report ding and gnieciak, so they add no independent support [qin2026review, p. 12].
- ICVul, one of lu's four training benchmarks, is the authors' own dataset [lu2025lab, p. 3].
- **W11. Test prevalence rarely realistic.** Most test sets are balanced or close to it (W1), while vulnerable code is rare in practice. The pass-2 studies show what this hides:
- Re-scored at PrimeVul's natural 1:44 ratio, LLMs raise 25.6–41.9 false positives per true one [deininger2026open, p. 8].
- At BigVul's 6% prevalence, DL accuracy of 87–94% hides F1 as low as 0.19 [lu2025lab, p. 11].
- tashenova draws negatives only from vulnerable plugins [tashenova2026real, p. 3]. **[I]** Its prevalence is therefore 74.5% (Variant A) and 46.2% (Variant B), and the false-alarm rate in deployment is unknown.

[I] Results from balanced sets should be re-scored at realistic prevalence before any deployment claim.

6. What this means for our research questions

Updated pass 2, 2026-11-12 (24 papers). The same answers appear in RESEARCH-QUESTIONS.md.

RQ1 — effectiveness head-to-head. No family wins across settings.

- **LLMs vs SAST.** Frontier LLMs beat SAST on small, self-contained code [gnieciak2025large, p. 10; dubniczky2025castle, p. 11]. CodeQL beats LLMs on synthetic function-level suites [khare2025understanding, p. 9]. At project scale, general coding agents reach much higher recall than SAST (59.55% vs 0–34.29% on C/C++) [li2026llm, p. 7], but most LLM detectors and SAST tools produce mostly false warnings [li2026llm, p. 11].
- **Same functions, all three families.** On 1549 unpaired PrimeVul functions, seven of eight zero-shot LLMs beat lightweight SAST on balanced accuracy [deininger2026open, p. 7; deininger2026open, p. 10]. At the natural 1:44 prevalence, though, Flawfinder and Semgrep are more precise than every LLM. LLM precision there (2.3–4.6%) is barely above flag-all (2.21%) [deininger2026open, p. 8; deininger2026open, p. 17].
- **LLMs vs fine-tuned DL.** The ranking depends on dataset, class balance and decision threshold [fu2023chatgpt, p. 4; shu2026evaluating, p. 40; deininger2026open, p. 11]. A fine-tuned CodeBERT beats all eight LLMs on the same unpaired PrimeVul functions (balanced accuracy 0.765 vs ≤ 0.711) [deininger2026open, p. 11; deininger2026open, p. 17]. All families are near or below chance on PrimeVul pairs [ding2025vulnerability, p. 9; tan2026think, p. 6].

- **Recent code.** On temporally separated 2025 CVEs at 1.97% prevalence, trained DL (F1 3.47–8.65) and four open LLMs ≤ 32B (0.79–5.36) both sit near the all-positive F1 of about 3.87% **[I]** [lu2025lab, p. 13; lu2025lab, p. 14].
- **PHP.** No LLM has been tested. A bespoke lexical taint analyser detects 1.7–1.8% of vulnerabilities, and ML reaches macro-F1 ≤ 0.601 [tashenova2026real, p. 8; tashenova2026real, p. 9].

Strength: weak to moderate. Most results come from single-run studies with incompatible metrics and no pooling. Only one study compares all three families on identical real functions, and its SAST arm is lightweight. The most robust pattern is "LLMs higher recall, lower precision" **[I]**. A second pattern **[I]** is that at realistic prevalence, LLMs show only a small margin over trivial baselines.

RQ2 — evaluation setups. Setup explains much of the disagreement. Sources of variation:

- label noise: 39–65% [risse2025top, p. 8] and 48.8% for PHP [tashenova2026real, p. 11] of positive labels correct;
- leakage, including duplicates and contradictory labels across folds [ding2025vulnerability, p. 4; tashenova2026real, p. 4];
- balanced vs realistic class ratios [shu2026evaluating, p. 40; yildiz2025benchmarking, p. 7; deininger2026open, p. 8];
- missing trivial baselines, which are often competitive [tashenova2026real, p. 8; deininger2026open, p. 7; lu2025lab, p. 14];
- temporal and cross-dataset shift [lu2025lab, p. 12; lu2025lab, p. 13];
- prompt wording and decision threshold, each of which can change the winner [deininger2026open, p. 11; deininger2026open, p. 20];
- synthetic vs real code [khare2025understanding, p. 5];
- function vs statement granularity [ahmed2026secvuleval, p. 8];
- custom datasets with inconsistent splits [kaniewski2026systematic, p. 35]. The same model on the same named dataset can differ by 28 F1 points between papers [lu2025lab, p. 12; ding2025vulnerability, p. 7].

Strength: strong for label noise and the collapse under realistic evaluation, which are consistent across independent primary studies and two SLRs. **Moderate** for prompt and threshold sensitivity and for trivial baselines, which rest on fewer studies.

RQ3 — hybrids, cost, explainability.

- **Alarm filtering.** LLM filtering of SAST alarms removes most false alarms at under $0.20 per alarm but loses 12–25% of real bugs [du2026reducing, p. 8; xiong2026sifting, p. 18]. **Strength: moderate.**
- **Hybrid detection.** IRIS and RepoAudit beat SAST in the authors' own evaluations [li2024iris, p. 8; guo2025repoaudit, p. 8] but not in an independent project-scale study [li2026llm, p. 7; li2026llm, p. 12]. The gain depends on a strong LLM [li2024iris, p. 10]. **Strength: weak/contested.**
- **Cost.** Costs range from cents to $1,247.66 per project, against minutes and no token cost for SAST [li2026llm, p. 16]. deininger, which is not a hybrid study, gives the only cost per true positive at 1:44 prevalence: USD 0.003–0.212 and 3.7–119.6 kJ [deininger2026open, p. 8]. These are compute costs only. They exclude human triage of the

25.6–41.9 false positives per true one [deininger2026open, p. 8], and part of the energy values are FLOP estimates [deininger2026open, p. 14]. Small open models can match a frontier model at a fraction of the cost (1/66) [deininger2026open, p. 17]. Reasoning mode costs 4.5–12.1× more per task [deininger2026open, p. 18]. **Strength: moderate (single study)** for cost per TP.

- **Explainability.** LLMs mislocate findings [gnieciak2025large, p. 11] **[A]**, and fluent explanations do not signal correctness [tan2026think, p. 9].

RQ4 — CWEs, languages, scale.

- **CWEs.** LLMs do best on self-contained injection-style CWEs and worst on context-dependent and crypto/policy CWEs and on localising memory-safety bugs [khare2025understanding, p. 7; xiong2026sifting, p. 18; shu2026evaluating, p. 30]. CodeQL is better on CWE-190 [khare2025understanding, p. 8].
- **Scale.** LLM performance falls with function or file length [ahmed2026secvuleval, p. 13; li2026llm, p. 9].
- **Projects.** Results also vary by project: LineVul (BigVul) finds 76% of Chromium CVEs but none in Linux, while all LLMs cover Linux better [lu2025lab, p. 13].
- **Languages.** Evidence is concentrated on C/C++ and Java [kaniewski2026systematic, p. 28]. The only PHP evidence comes from tashenova, which tests no LLM and uses a bespoke analyser. There, learned models detect XSS (F1 up to 0.500) far better than SQLi (≤ 0.146) or RCE (≤ 0.085) [tashenova2026real, p. 9].

Strength: weak. Per-CWE cells are small and per-CWE results diverge across studies. For example, khare finds LLMs beat CodeQL on CWE-476 [khare2025understanding, p. 8], while ahmed reports over-flagging of it [ahmed2026secvuleval, p. 13].

RQ5 — open problems. Recurring problems:

- verified, realistic, repository-level benchmarks with context [zhou2025large, p. 19; sheng2025llms, p. 25];
- trivial baselines and realistic prevalence (W1, W11);
- label quality, leakage and contamination control (W2) [tashenova2026real, p. 12; lu2025lab, p. 17];
- same-benchmark three-way comparisons beyond lightweight SAST and pre-cutoff function-level code (W8);
- independent labelling and statistics, including prompt sensitivity (W3, W5);
- cost reporting that includes triage, with clear energy provenance (W6);
- true-positive loss in filtering [xiong2026sifting, p. 19];
- workflow integration: none of 58 reviewed studies was integrated into developer workflows [zhou2025large, p. 19].

deininger's authors conclude that no method they tested (LLM, SAST or fine-tuned DL) is ready for independent function-level triage [deininger2026open, p. 26] **[A]**. qin2026review lists similar open problems [qin2026review, p. 13], but it is not independent of our sources.

Strength: moderate as a description of recurring limitations. **[I]** These are candidate gaps only. Novelty checks and gap specifics are in gap-report.md (standards §7).

Appendix C: Evidence Matrix

One row per piece of evidence that bears on a research question. Source is [<key>, p. N] (checked by rlab cite-check). Label: **VERIFIED** (checked in the paper's own data/results), **AUTHOR-CLAIM** (stated by authors, not independently checkable), **INTERPRETATION** (our reasoning). Strength: strong / moderate / weak (see docs/research-standards.md).

RQ	Claim / finding	Source	Label	Design & N	Strength	Notes
RQ1	Best LLM (Claude-3.7-Sonnet, multi-agent) reaches 23.83% statement-level F1 and 53.89% function-level F1 on real C/C++ CVEs; function-level precision ≤47.41% for all 5 LLMs; no SAST/DL baseline	[ahmed2026secvuleval, p. 8]	VERIFIED	Benchmark construction + empirical evaluation of 5 LLMs in a multi-age; N: 25,440 functions (10,998 vulnerable / 14,442 non-vulnerable	moderate	
RQ1	Even trained on balanced Real-Vul, DL detectors reach only F1 2–36% and AUC 51–52% on realistic test data	[chakraborty2024revisiting, p. 9]	VERIFIED	Replication + re-evaluation of 4 DL detectors (and 2 fine-tuned LLMs) ; N: 5,528 vulnerable functions (4,418 train / 1,110 test) + 1,68	moderate	
RQ1	Fine-tuned CodeLlama/Mixtral F1 42%/46%, AUC 67%/71% vs augmented LineVul F1 46%, AUC 81.8% (LLM test-set composition unclear)	[chakraborty2024revisiting, p. 12]	VERIFIED	Replication + re-evaluation of 4 DL detectors (and 2 fine-tuned LLMs) ; N: 5,528 vulnerable functions (4,418 train / 1,110 test) + 1,68	weak	
RQ1	Dead-code augmentation raises LineVul F1 20 → 46% and AUC 51.4 →	[chakraborty2024revisiting, p. 10]	VERIFIED	Replication + re-evaluation of 4 DL detectors (and 2 fine-tuned	weak	

	81.8% on Real-Vul			LLMs) ; N: 5,528 vulnerable functions (4,418 train / 1,110 test) + 1,68		
RQ1	On 1549 PrimeVul C/C++ functions (35.4% vulnerable), zero-shot LLM balanced accuracy 0.511–0.676 (first epoch) and 0.568–0.711 (budget-matched epoch); best DeepSeek-V3.2	[deininger 2026open, p. 7; deininger 2026open, p. 17]	VERIFIED	empirical benchmark; zero-shot prompting of 8 LLMs (3 frontier, 5 open; N: 1549 functions (549 vulnerable, 35.4%); 8 LLMs x several configuration	moderate	single dataset, prompt-sensitive
RQ1	Fine-tuned CodeBERT-PrimeVul (125M, validation-tuned threshold) beats every LLM: bal. acc. 0.765, MCC 0.526, precision at 1:44 8.23% vs best LLM 0.711 / 0.403 / 4.62% (budget-matched epoch)	[deininger 2026open, p. 10; deininger 2026open, p. 11; deininger 2026open, p. 17]	VERIFIED	empirical benchmark; zero-shot prompting of 8 LLMs (3 frontier, 5 open; N: 1549 functions (549 vulnerable, 35.4%); 8 LLMs x several configuration	moderate	in-distribution vs zero-shot
RQ1	Same functions, SAST: Flawfinder bal. acc. 0.589 (risk≥1), Cppcheck 0.548–0.592, Semgrep 0.501–0.514 (recall 0.002–0.031). At 1:44 only Flawfinder (6.2–13.4%) and Semgrep (26.0% [5.7–66.9]) are more precise than every LLM (2.3–3.8% first	[deininger 2026open, p. 7; deininger 2026open, p. 8; deininger 2026open, p. 10; deininger 2026open, p. 11; deininger 2026open, p. 17]	VERIFIED	empirical benchmark; zero-shot prompting of 8 LLMs (3 frontier, 5 open; N: 1549 functions (549 vulnerable, 35.4%); 8 LLMs x several configuration	moderate	lightweight SAST only; no CodeQL; Semgrep precision rests on 2 FP

	epoch; 2.8–4.6% budget-matched); Cppcheck (2.46–3.77%) is in the LLM range; Semgrep's CI overlaps CodeBERT's (8.23% [6.9–9.7])					
RQ1	Trivial baselines: always-vulnerable F1 0.523 vs LLM F1 0.405–0.615; at 1:44 every LLM's precision 2.3–3.8% (first epoch; 2.8–4.6% budget-matched) vs always-flag 2.21%, 25.6–41.9 FP per TP (first epoch)	[deininger2026open, p. 7; deininger2026open, p. 8; deininger2026open, p. 17]	VERIFIED	empirical benchmark; zero-shot prompting of 8 LLMs (3 frontier, 5 open; N: 1549 functions (549 vulnerable, 35.4%); 8 LLMs x several configuration	moderate	
RQ1	At FPR ≤ 0.5%, PrimeVul-trained code LMs miss 88.78–91.51% of vulnerabilities (VD-S)	[ding2025vulnerability, p. 7]	VERIFIED	Dataset audit (label accuracy, duplication) + new benchmark (PrimeVul); N: PrimeVul 6,968 vulnerable + 228,800 benign functions (755 pr	moderate	
RQ1	Pair-wise correct (P-C) on 564 PrimeVul pairs: GPT-4 CoT 12.94%, GPT-4 two-shot 5.14%, GPT-3.5 CoT 6.21%, GPT-3.5 fine-tuned 1.24%, fine-tuned 7B models ≤3.01%, random 22.70%	[ding2025vulnerability, p. 9]	VERIFIED	Dataset audit (label accuracy, duplication) + new benchmark (PrimeVul); N: PrimeVul 6,968 vulnerable + 228,800 benign functions (755 pr	moderate	
RQ1	GPT-4 two-shot predicts both versions of a pair vulnerable in	[ding2025vulnerability, p. 9]	VERIFIED	Dataset audit (label accuracy, duplication) + new benchmark	moderate	

	71.63% of pairs; fine-tuned models predict both benign in 84–88%			(PrimeVul); N: PrimeVul 6,968 vulnerable + 228,800 benign functions (755 pr		
RQ1	Class weights and contrastive learning do not reduce VD-S below ~88.5%	[ding2025 vulnerabili ty, p. 8]	VERIFI ED	Dataset audit (label accuracy, duplication) + new benchmark (PrimeVul); N: PrimeVul 6,968 vulnerable + 228,800 benign functions (755 pr	moderat e	
RQ1	Off-the-shelf DL detectors (Devign, ReGVD, LLMAO, LineVul) fail at alarm triage (accuracy 0.36–0.44; LineVul labels all as FP)	[du2026re ducing, p. 7; du2026re ducing, p. 8]	VERIFI ED	Industrial empirical study at Tencent: interviews + offline evaluation; N: 433 alarms (328 FP, 105 TP)	weak	
RQ1	Top LLMs outscore all static and FV tools on CASTLE: GPT-o3-Mini 977, GPT-o1 962, DeepSeek R1 956, GPT-4o 954 vs ESBMC 661, CodeQL 634	[dubniczk y2025cast le, p. 11]	VERIFI ED	Benchmark paper; N: 250 programs (150 vulnerable, 100 non-vulnerable), 10,392 li	moderat e	
RQ1	LLM precision/recall: GPT-o3-Mini 63%/78%, GPT-4o 54%/76%, DeepSeek R1 47%/90%; small open models 20–28% precision	[dubniczk y2025cast le, p. 11]	VERIFI ED	Benchmark paper; N: 250 programs (150 vulnerable, 100 non-vulnerable), 10,392 li	weak	
RQ1	SAST recall is low: CodeQL 29%	[dubniczk y2025cast le, p. 11;	VERIFI ED	Benchmark paper; N: 250	moderat e	

	(45 TP; 45/150 would be 30%, see RQ2 denominator row), SAST average 23%; high-precision tools have very low recall (Clang 87%/9%, CBMC 100%/12%)	dubniczky 2025castle, p. 12]		programs (150 vulnerable, 100 non-vulnerable), 10,392 li		
RQ1	Some SAST tools are unusable from noise: Splint 1,027 FP and CodeThreat 1,101 FP, giving negative CASTLE scores	[dubniczky2025castle, p. 11; dubniczky 2025castle, p. 13]	VERIFIED	Benchmark paper; N: 250 programs (150 vulnerable, 100 non-vulnerable), 10,392 li	moderate	
RQ1	On Big-Vul (random split), prompted gpt-4 F1 29% and gpt-3.5-turbo F1 10% vs fine-tuned AIBugHunter 94% and CodeBERT 87% for function-level detection	[fu2023chatgpt, p. 4]	VERIFIED	Short empirical comparison; N: ~188k C/C++ functions (Big-Vul) + 5.5k repair pairs	moderate	
RQ1	Line-level top-10 accuracy: gpt-4 65%, gpt-3.5-turbo 25% vs AIBugHunter/CodeBERT 99%	[fu2023chatgpt, p. 4]	VERIFIED	Short empirical comparison; N: ~188k C/C++ functions (Big-Vul) + 5.5k repair pairs	moderate	
RQ1	ChatGPT produced 0% exact-match repairs vs 7–30% for fine-tuned models	[fu2023chatgpt, p. 5]	VERIFIED	Short empirical comparison; N: ~188k C/C++ functions (Big-Vul) + 5.5k repair pairs	weak	
RQ1	Macro mean F1: GPT-4.1 0.797, Mistral Large 0.753, DeepSeek	[gnieciak2025large, p. 10]	VERIFIED	Head-to-head benchmark, 3 SAST tools (SonarQube,	weak	

	V3 0.750 vs Snyk Code 0.546, CodeQL 0.386, SonarQube 0.260 on 10 C# projects (63 vulns)			CodeQL, Snyk Code) vs; N: 10 projects, 63 vulnerabilities, 3-10 files and 3,145-5,317		
RQ1	Macro mean recall: LLMs 0.783–0.877 vs SAST 0.187–0.523; macro precision LLMs 0.723–0.783 vs SAST 0.567–0.686	[gnieciak2025large, p. 10]	VERIFIED	Head-to-head benchmark, 3 SAST tools (SonarQube, CodeQL, Snyk Code) vs; N: 10 projects, 63 vulnerabilities, 3-10 files and 3,145-5,317	weak	
RQ1	Pooled from raw counts, SAST precision 0.74–0.87 vs LLM 0.74–0.79 and Snyk F1 ≈ 0.70; macro averaging with zero-precision imputation widens the gap	[gnieciak2025large, p. 8; gnieciak2025large, p. 9]	INTERPRETATION	Head-to-head benchmark, 3 SAST tools (SonarQube, CodeQL, Snyk Code) vs; N: 10 projects, 63 vulnerabilities, 3-10 files and 3,145-5,317	weak	
RQ1	RepoAudit (Claude 3.5 Sonnet) finds 40 TPs / 11 FPs (precision 78.43%) on 15 C/C++ projects for NPD/MLK/UAF	[guo2025repoaudit, p. 7; guo2025repoaudit, p. 8]	VERIFIED	tool paper with empirical evaluation; N: 15 benchmark projects (21 previously reported bugs)	weak	
RQ1	Meta Infer finds 7 TPs / 2 FPs on the 8 projects it could analyse; Amazon CodeGuru 0 TPs / 18 FPs	[guo2025repoaudit, p. 8; guo2025repoaudit, p. 15]	VERIFIED	tool paper with empirical evaluation; N: 15 benchmark projects (21 previously reported bugs)	weak	
RQ1	With reasoning models RepoAudit precision rises to 82.35–88.46% (42–46 TPs)	[guo2025repoaudit, p. 8; guo2025repoaudit, p. 16]	VERIFIED	tool paper with empirical evaluation; N: 15 benchmark projects (21	weak	

				previously reported bugs)		
RQ1	This 263-study SLR does not synthesise effectiveness; it presents SAST tools as rule-dependent with high false-positive rates, and LLMs as rule-free but unproven outside curated benchmarks (no pooled comparison)	[kaniewski2026systematic, p. 1; kaniewski2026systematic, p. 12; kaniewski2026systematic, p. 13; kaniewski2026systematic, p. 40]	AUTHOR-CLAIM	Systematic literature review (Kitchenham guidelines) with taxonomy and; N: 263 primary studies (185 peer-reviewed, 78 arXiv preprints)	weak	
RQ1	Best LLM+prompt per dataset averages 62.8% accuracy on balanced function-level data; real-world accuracy 10.5% lower than synthetic	[khare2025understanding, p. 5]	VERIFIED	Empirical study, 16 prompted LLMs x 3 prompts vs CodeQL (synthetic onl; N: 5,000 function-level snippets (1,000 per dataset, 500 vulner	moderate	
RQ1	CodeQL beats the best LLM in accuracy on OWASP (0.63 vs 0.60), Juliet Java (0.94 vs 0.76) and Juliet C/C++ (0.72 vs 0.65); F1: CodeQL 0.72/0.94/0.61 vs LLM 0.65/0.79/0.72. Caveat **[I]**: on OWASP both are near the always-vulnerable baseline (A 0.50, F1 0.667)	[khare2025understanding, p. 9]	VERIFIED	Empirical study, 16 prompted LLMs x 3 prompts vs CodeQL (synthetic onl; N: 5,000 function-level snippets (1,000 per dataset, 500 vulner	moderate	
RQ1	On CVEFixes	[khare202	VERIFI	Empirical study,	moderat	

	C/C++, Qwen-2.5-32B (prompted) A 0.56 / F1 0.65 vs DL tools F1 0.02–0.64 (in-distribution LineVul A 0.59 / F1 0.61, higher accuracy than Qwen); LineVul trained on BigVul collapses to F1 0.02. Caveat **[I]**: Qwen and all DL F1s are below the always-vulnerable F1 of 0.667 on this 50/50 set	5understanding, p. 10]	ED	16 prompted LLMs x 3 prompts vs CodeQL (synthetic onl; N: 5,000 function-level snippets (1,000 per dataset, 500 vulner	e	
RQ1	Model scale does not reliably improve detection; Qwen-2.5-14B/32B beat GPT-4 on real-world data	[khare2025understanding, p. 5]	AUTHOR-CLAIM	Empirical study, 16 prompted LLMs x 3 prompts vs CodeQL (synthetic onl; N: 5,000 function-level snippets (1,000 per dataset, 500 vulner	moderate	
RQ1	Other SAST tools on CWE-Bench-Java: Infer 0, SpotBugs 4, Snyk 23 of 120	[li2024iris, p. 8]	VERIFIED	Tool proposal + benchmark evaluation (LLM-inferred taint specs + CodeQ; N: 120 vulnerabilities (one per project)	moderate	
RQ1	On 110 known C/C++ vulns, Codex/Claude Code average recall 59.55% vs CodeQL 6.00–34.29%, Semgrep and CSA 0%; KNighter 0, RepoAudit (Sonnet 3.5) 0	[li2026llm, p. 7]	VERIFIED	comparative empirical study; N: 265 known vulnerabilities (110 C/C++, 155 Java)	moderate	

	and RepoAudit (GPT-5.4) 2 detections					
RQ1	On 155 known Java vulns, Claude Code recall 81.94% and Codex 63.87% vs CodeQL 14.19%, Semgrep 6.67%, SpotBugs 14.84%	[li2026llm, p. 9]	VERIFIED	comparative empirical study; N: 265 known vulnerabilities (110 C/C++, 155 Java)	moderate	
RQ1	On 24 real-world projects, SFDR is high for most tools: IRIS 97.80%, INFERROI 96.40%, RepoAudit 92.77%, CodeQL 77.00–86.01%, Semgrep 92.71–100%, CSA 85.51%, SpotBugs 86.21%; Codex lowest at 9.09%/31.82% but on ~22 warnings each	[li2026llm, p. 11; li2026llm, p. 12]	VERIFIED	comparative empirical study; N: 265 known vulnerabilities (110 C/C++, 155 Java)	moderate	
RQ1	On temporally separated VentiVul (CVEs from May 2025, prevalence 1.97%), trained DL F1 3.47–8.65 (best ReVeal-BigVul 8.65, P 5.25%); LineVul 91.00–94.52 in-distribution → 4.39–6.87	[lu2025lab, p. 12; lu2025lab, p. 13]	VERIFIED	empirical evaluation; within/cross-dataset DL training + temporally se; N: 4 DL models x 4 datasets; VentiVul 13,326 functions, 200 CVEs, 263 vul	moderate	peer-reviewed; CVE sample size fixed before collection and manually verified (p. 4); 13,326 functions / 263 positives; but single study, no CIs, 2 DL families; consistent with ding2025vulnerability

						BigVul→PrimeVul collapse (F1 1.91–5.82); published JSS 2026 (doi 10.1016/j.jss.2026.113185); pages = arXiv v4 accepted manuscript
RQ1	On VentiVul (not identical sample sets: ReVeal on its processable subset, p. 7; few-shot excludes demonstration pairs and valid-only uses parseable outputs only, p. 8), 4 open-weight LLMs (zero/few-shot/CoT) reach conservative F1 0.79–5.36 (best Mistral-24B few-shot), PACC < 6%; zero-shot CDR 5–34%, below most DL configs. **[I]** Relative to chance, best DL F1 8.65 ≈ 2.5× its all-positive level (≈ 3.5%) vs best LLM 5.36 ≈ 1.4× (≈ 3.87%)	[lu2025lab, p. 7; lu2025lab, p. 8; lu2025lab, p. 13; lu2025lab, p. 14]	VERIFIED	empirical evaluation; within/cross-dataset DL training + temporally se; N: 4 DL models x 4 datasets; VentiVul 13,326 functions, 200 CVEs, 263 vul	moderate	label VERIFIED (ratio: INTERPRETATION); peer-reviewed; deterministic, fully specified decoding (p. 8); but one run, 4 open-weight models ≤ 32B, no frontier or fine-tuned LLMs; published JSS 2026 (doi 10.1016/j.jss.2026.113185); pages = arXiv v4 accepted manuscript
RQ1	The evaluated DL detectors and four open-weight LLMs ≤ 32B score Whole-File F1 within 4.8 points of an all-positive	[lu2025lab, p. 4; lu2025lab, p. 9; lu2025lab, p. 13; lu2025lab, p. 14]	INTERPRETATION	empirical evaluation; within/cross-dataset DL training + temporally se; N: 4 DL models x 4 datasets;	moderate	our arithmetic from reported counts and Tables 7–8 of a

	classifier (≈ 3.87% at p = 1.97%; 7 of 12 LLM conservative F1 below it), and their PACC (≤ 5.75%) is far below the random-labeller level q(1−q) (up to 25%): they mostly give vulnerable and patched versions the same label			VentiVul 13,326 functions, 200 CVEs, 263 vul		peer-reviewed study; consistent with ding2025vulnerability random-guess pair accuracy 22.70% vs far lower model values [ding2025vulnerability, p. 9]; scope limited to the evaluated models; published JSS 2026 (doi 10.1016/j.jss.2026.113185); pages = arXiv v4 accepted manuscript
RQ1	SAST more stable within rule coverage; LLMs better at semantic summarisation/explanation; unified benchmarks support complementarity, not replacement (no numbers; its only head-to-head source found LLMs above SAST on F1 [gnieciak2025large, p. 10])	[qin2026review, p. 12]	AUTHOR-CLAIM	narrative (non-systematic) literature review, single author; no search; N: 32 references (no stated corpus of primary studies)	weak	narrative review; not independent of ding/gnieciak/sheng
RQ1	The survey offers no quantified LLM-vs-SAST/DL comparison; it states that for	[sheng2025llms, p. 20]	AUTHOR-CLAIM	Narrative (non-systematic) survey with keyword-driven, iterative venue;	weak	

	full-codebase analysis LLMs typically assist static analysis tools or fuzzing rather than performing standalone analysis			N: 58 highly relevant studies (from ~500-600 screened		
RQ1	On balanced REEF (7 languages), fine-tuned GPT-4o + BM25 few-shot beats best PLM CodeT5P at function level (Acc 0.7196 vs 0.6037; MCC 0.4360 vs 0.2860; F1 tied 0.7069 vs 0.7075)	[shu2026 evaluating, p. 17]	VERIFIED	Empirical comparison of 8 fine-tuned/embedding PLMs vs 5 LLMs (zero-sh; N: Function level 20,165 functions (test 2,026, balanced)	moderate	
RQ1	Prompt-only LLMs (zero-/few-shot, incl. GPT-4o, GRACE, DeepSeek-R1, QwQ-plus) are near chance at function level (Acc 0.36-0.53, MCC ≤0.07)	[shu2026 evaluating, p. 17; shu2026evaluating, p. 37]	VERIFIED	Empirical comparison of 8 fine-tuned/embedding PLMs vs 5 LLMs (zero-sh; N: Function level 20,165 functions (test 2,026, balanced)	moderate	
RQ1	Line level: GPT-4o ITFSP F1 0.6641 vs CodeT5P 0.4841 (but CodeT5P AUC 0.8644 > 0.8116)	[shu2026 evaluating, p. 23; shu2026evaluating, p. 26]	VERIFIED	Empirical comparison of 8 fine-tuned/embedding PLMs vs 5 LLMs (zero-sh; N: Function level 20,165 functions (test 2,026, balanced)	moderate	
RQ1	Thinking mode raises non-agent mean F1 from 0.31 to 0.41 across 4 open models (Wilcoxon p = 0.023, d =	[tan2026think, p. 6]	VERIFIED	controlled factorial empirical study; N: 870 functions (435 pairs) x 64 configurations	moderate	

	1.37), mainly via recall; accuracy unchanged (0.518 vs 0.523)					
RQ1	Best function-level F1 on PrimeVul Pair is 0.620 (Nemotron-Super-49B thinking) and best non-agent P-C 22.5%, i.e. weak discrimination of vulnerable vs patched functions	[tan2026think, p. 6]	VERIFIED	controlled factorial empirical study; N: 870 functions (435 pairs) x 64 configurations	moderate	
RQ1	On 24,917 real WordPress PHP functions (Variant B binary, plugin-grouped, de-duplicated), best macro-F1 0.601 (nested stacking); Bi-LSTM 0.573, SVM 0.562; no neural model leads in any of 6 settings	[tashenova2026real, p. 7; tashenova2026real, p. 8]	VERIFIED	empirical benchmark; plugin-grouped 5-fold CV (de-duplicated, clone-co; N: 15,294 (Variant A) and 24,917 (Variant B) PHP functions after leakage	weak–moderate	single study, single seed, ≤48.8% label precision, overlapping CIs
RQ1	An unnamed, unversioned intra-procedural lexical taint analyser detects 1.7–1.8% of PHP vulnerabilities (FNR 0.982; macro-F1 0.366 vs majority 0.350; MCC 0.035)	[tashenova2026real, p. 5; tashenova2026real, p. 8; tashenova2026real, p. 9; tashenova2026real, p. 13]	VERIFIED	empirical benchmark; plugin-grouped 5-fold CV (de-duplicated, clone-co; N: 15,294 (Variant A) and 24,917 (Variant B) PHP functions after leakage	weak	for SAST in general (single bespoke tool
RQ1	No LLM or pre-trained transformer evaluated (CodeBERT not run: no GPU/transformers)	[tashenova2026real, p. 13]	VERIFIED	empirical benchmark; plugin-grouped 5-fold CV (de-duplicated, clone-co; N: 15,294 (Variant A) and 24,917	n/a	scope

				(Variant B) PHP functions after leakage		
RQ1	SAST alone on OWASP v1.2: CodeQL P 60.3% R 97.0% FPR 68.2%; Semgrep FPR 74.8%; SonarQube FPR 94.6%; Joern R 8.2%	[xiong2026sifting, p. 11]	VERIFIED	Comparative empirical study: 3 LLM agent frameworks x 3 backbones vs v; N: OWASP 2,740 cases (1,325 negatives / 1,415 positives)	moderate	
RQ1	GPT-4o/4o-mini plain, dependency-augmented and ReAct detectors reach F1 48–66% but pairwise accuracy only 1.02–20.17% on 879 CVE pairs	[yildiz2025benchmarking, p. 6]	VERIFIED	benchmark paper + comparative evaluation of plain LLM, dependency-augm; N: 1,758 paired samples (879 vulnerable, 879 benign) from 879 C	moderate	
RQ1	ReAct agents give the highest pAcc for every prompting strategy with GPT-4o/4o-mini (gains up to 16.61 points)	[yildiz2025benchmarking, p. 7]	VERIFIED	benchmark paper + comparative evaluation of plain LLM, dependency-augm; N: 1,758 paired samples (879 vulnerable, 879 benign) from 879 C	weak	
RQ1	Best single SAST tools detect 37.5% (Java, Horusec), 44.4% (C, CodeQL), 30.0% (Python, DevSkim) of test CVEs (S1), marking 0.2–5.2% of functions; many SAST tools detect 0%	[zhou2024comparison, p. 6; zhou2024comparison, p. 7]	VERIFIED	Empirical comparison, 15 SAST tools vs 12 open-source LLMs (fine-tuned; N: 270 CVEs in 134 repos (87 Java, 85 C, 98 Python)	weak	preprint (strength downgraded)

RQ1	LLMs reach up to 100% (Java, C) and 90% (Python) S1 detection but mark 3.5–77.4% of functions	[zhou2024comparison, p. 6; zhou2024comparison, p. 7]	VERIFIED	Empirical comparison, 15 SAST tools vs 12 open-source LLMs (fine-tuned; N: 270 CVEs in 134 repos (87 Java, 85 C, 98 Python)	weak	preprint (strength downgraded)
RQ1	LoRA-fine-tuned large LLMs beat zero-shot/CoT/few-shot prompting: Java 73.4% S1 with 19.4% marked vs 52.1–55.0% marked for prompting	[zhou2024comparison, p. 7]	VERIFIED	Empirical comparison, 15 SAST tools vs 12 open-source LLMs (fine-tuned; N: 270 CVEs in 134 repos (87 Java, 85 C, 98 Python)	weak	preprint (strength downgraded)
RQ1	GPT-3.5 prompting detects only 3.7–23.2% (S1) on a 1:100 sample, far below fine-tuned CodeBERT (89.2%) and Llama3 (67.4%)	[zhou2024comparison, p. 10]	VERIFIED	Empirical comparison, 15 SAST tools vs 12 open-source LLMs (fine-tuned; N: 270 CVEs in 134 repos (87 Java, 85 C, 98 Python)	weak	preprint (strength downgraded)
RQ1	Only one study in the 58-study corpus compares LLMs with SAST tools (repo level). It reports that SAST tools have low detection rates with few false positives, while LLMs detect more but with high false-positive rates. Second-hand, and the study is the survey authors' own	[zhou2025large, p. 18; zhou2025large, p. 22]	AUTHOR-CLAIM	Systematic literature review (manual top-venue search + 7-database aut; N: 58 primary studies (40 detection studies in Table 2, 19 repa	weak	

RQ1/RQ3	Industrial SAT alone: 76% of alarms false (328/433); hybrid LLM4PFA removes 94–98% of false alarms, keeping 75–88% of real bugs (precision 0.83–0.93)	[du2026reducing, p. 6; du2026reducing, p. 8]	VERIFIED	Industrial empirical study at Tencent: interviews + offline evaluation; N: 433 alarms (328 FP, 105 TP)	moderate	
RQ1/RQ3	IRIS+GPT-4 detects 55/120 CVEs vs CodeQL 27/120; AvgFDR 84.82% vs 90.03%; AvgF1 0.177 vs 0.076	[li2024iris, p. 8]	VERIFIED	Tool proposal + benchmark evaluation (LLM-inferred taint specs + CodeQ; N: 120 vulnerabilities (one per project)	moderate	
RQ1/RQ3	Real-world Java (50 CodeQL alerts): up to 93.3% FP recall, best F1 83.3%	[xiong2026sifting, p. 13]	VERIFIED	Comparative empirical study: 3 LLM agent frameworks x 3 backbones vs v; N: OWASP 2,740 cases (1,325 negatives / 1,415 positives)	weak	
RQ2	Requiring correct vulnerable statements + reasoning cuts F1 by more than half vs. function-level labels (e.g. GPT-4.1 52.25% → 22.38%)	[ahmed2026secvuleval, p. 8]	VERIFIED	Benchmark construction + empirical evaluation of 5 LLMs in a multi-age; N: 25,440 functions (10,998 vulnerable / 14,442 non-vulnerable	moderate	
RQ2	SecVulEval: 25,440 de-duplicated C/C++ functions, 5,867 CVEs, 145 CWEs, statement-level + context labels;	[ahmed2026secvuleval, p. 3; ahmed2026secvuleval, p. 6]	VERIFIED	Benchmark construction + empirical evaluation of 5 LLMs in a multi-age; N: 25,440 functions (10,998	moderate	

	prior datasets have up to 18.9% duplicates			vulnerable / 14,442 non-vulnerable		
RQ2	Statement-level recall appears computed without counting wrong-reasoning cases as FN; under the stated definition Claude's recall would be ~27.7% (not 53.23%) and F1 ~19.8%	[ahmed2026secvuleval, p. 7; ahmed2026secvuleval, p. 8; ahmed2026secvuleval, p. 13]	INTERPRETATION	Benchmark construction + empirical evaluation of 5 LLMs in a multi-age; N: 25,440 functions (10,998 vulnerable / 14,442 non-vulnerable	moderate	
RQ2	Evaluated on whole-project, time-split Real-Vul, DL detectors' precision drops by up to 95 pp (to 1–10%) and F1 by up to 91 pp vs their original benchmarks	[chakraborty2024revisiting, p. 7]	VERIFIED	Replication + re-evaluation of 4 DL detectors (and 2 fine-tuned LLMs) ; N: 5,528 vulnerable functions (4,418 train / 1,110 test) + 1,68	moderate	
RQ2	15% of vulnerable samples in the ReVeal dataset also appear as uncertain (label inconsistency)	[chakraborty2024revisiting, p. 3]	VERIFIED	Replication + re-evaluation of 4 DL detectors (and 2 fine-tuned LLMs) ; N: 5,528 vulnerable functions (4,418 train / 1,110 test) + 1,68	weak	
RQ2	Prompt paraphrase moves bal. acc. up to 0.088 and changes the winner (ρ vs anchor 0.381); safe-pool redraw ≤0.015; temperature-0 verdict agreement 88.3–100%	[deininger2026open, p. 19; deininger2026open, p. 20]	VERIFIED	empirical benchmark; zero-shot prompting of 8 LLMs (3 frontier, 5 open; N: 1549 functions (549 vulnerable, 35.4%); 8 LLMs x several configuration	moderate	

RQ2	Recall on pre-2020 CVEs 7–15 points higher than 2021–2022 for better-balanced LLMs (memorisation signal)	[deininger2026open, p. 24]	VERIFIED	empirical benchmark; zero-shot prompting of 8 LLMs (3 frontier, 5 open; N: 1549 functions (549 vulnerable, 35.4%); 8 LLMs x several configuration	weak–moderate	no significance test reported
RQ2	Argmax decision rule made the fine-tuned detector look like always-safe (bal. acc. 0.500) despite ROC-AUC 0.845; threshold choice can flip LLM-vs-DL conclusions	[deininger2026open, p. 11]	VERIFIED	empirical benchmark; zero-shot prompting of 8 LLMs (3 frontier, 5 open; N: 1549 functions (549 vulnerable, 35.4%); 8 LLMs x several configuration	moderate	
RQ2	Ding's own manual audit: only 24% (CodeXGLUE/Devign) and 36% (VulnPatchPairs) of vulnerable labels correct; PrimeVul 86–92%	[ding2025vulnerability, p. 4]	VERIFIED	Dataset audit (label accuracy, duplication) + new benchmark (PrimeVul); N: PrimeVul 6,968 vulnerable + 228,800 benign functions (755 pr	moderate	
RQ2	BigVul 25%, CrossVul 47.8%, CVEFixes 51.7%, DiverseVul 60% label accuracy (Chen et al., reproduced in Table I)	[ding2025vulnerability, p. 4]	AUTHOR-CLAIM	Dataset audit (label accuracy, duplication) + new benchmark (PrimeVul); N: PrimeVul 6,968 vulnerable + 228,800 benign functions (755 pr	moderate	
RQ2	Exact-copy test-to-train leakage: BigVul 12.7%, CVEFixes 18.9%, DiverseVul 3.3%,	[ding2025vulnerability, p. 4]	VERIFIED	Dataset audit (label accuracy, duplication) + new benchmark (PrimeVul); N: PrimeVul 6,968	moderate	

	CodeXGLUE 0.6%; PrimeVul 0.0%			vulnerable + 228,800 benign functions (755 pr		
RQ2	Fine-tuned code LMs: F1 62.88–68.26% on BigVul vs 1.91–5.82% when tested on PrimeVul and 18.05–21.43% when trained+tested on PrimeVul	[ding2025 vulnerabili ty, p. 7]	VERIFI ED	Dataset audit (label accuracy, duplication) + new benchmark (PrimeVul); N: PrimeVul 6,968 vulnerable + 228,800 benign functions (755 pr	moderat e	
RQ2	Scope: general bug classes (NPD/OOB/DBZ), proprietary C/C++ data, author-built winning method	[du2026re ducing, p. 4; du2026re ducing, p. 10]	INTERP RETATI ON	Industrial empirical study at Tencent: interviews + offline evaluation; N: 433 alarms (328 FP, 105 TP)	weak	
RQ2	Benchmark is 250 hand-written single-file C programs (mean 42 lines), balanced 150/100, line-labelled; differs strongly from real-world repo-level setups	[dubniczk y2025cast le, p. 6]	VERIFI ED	Benchmark paper; N: 250 programs (150 vulnerable, 100 non-vulnerable), 10,392 li	strong	
RQ2	Table 3 TP+FN exceeds the 150 vulnerable programs for all 10 LLMs (154–179) and 4 SAST tools (155–157), so reported recall/accuracy use inconsistent denominators (e.g., CodeQL "29% (45/150)" = 45/157)	[dubniczk y2025cast le, p. 11; dubniczky 2025castl e, p. 12]	INTERP RETATI ON	Benchmark paper; N: 250 programs (150 vulnerable, 100 non-vulnerable), 10,392 li	moderat e	

RQ2	Evaluation used an 80/10/10 Big-Vul split (1:20 ratio) with no de-duplication or temporal split reported	[fu2023chatgpt, p. 3]	VERIFIED	Short empirical comparison; N: ~188k C/C++ functions (Big-Vul) + 5.5k repair pairs	moderate	
RQ2	LLM findings counted as TP despite wrong line/column; SAST tools on default configs; single run; 10 author-prepared small projects	[gnieciak2025large, p. 5; gnieciak2025large, p. 6; gnieciak2025large, p. 8]	VERIFIED	Head-to-head benchmark, 3 SAST tools (SonarQube, CodeQL, Snyk Code) vs; N: 10 projects, 63 vulnerabilities, 3-10 files and 3,145-5,317	moderate	
RQ2	Recall is measured only as reproduction of 21 previously reported bugs; overall recall unknown	[guo2025repoaudit, p. 17]	VERIFIED	tool paper with empirical evaluation; N: 15 benchmark projects (21 previously reported bugs)	moderate	
RQ2	Tension with li2026llm (0/110 known C/C++ vulns, 92.77% SFDR): overlapping projects only in li's real-world set, at newer commits and partly different bug types; precision vs recall/SFDR; different labellers (see §9)	[guo2025repoaudit, p. 7; guo2025repoaudit, p. 8]	INTERPRETATION	tool paper with empirical evaluation; N: 15 benchmark projects (21 previously reported bugs)	weak	
RQ2	Most-used benchmarks are old function-level C/C++ datasets (Devign 69, Big-Vul 63, ReVeal 40 studies) with vulnerable shares	[kaniewski2026systematic, p. 26; kaniewski2026systematic, p. 28]	VERIFIED	Systematic literature review (Kitchenham guidelines) with taxonomy and; N: 263 primary studies (185 peer-reviewed, 78 arXiv	strong	

	from 0.7% (ReposVul) to 100% (SecurityEval) across Table 5, or 1.4% (D2A) to 50.0% (PairVul) among the function-level C/C++ benchmarks			preprints)		
RQ2	87 studies use or introduce custom datasets, only nine datasets are reused in more than ten studies, and few datasets have predefined splits, so cross-study comparison is unreliable	[kaniewski2026systematic, p. 35; kaniewski2026systematic, p. 39]	VERIFIED	Systematic literature review (Kitchenham guidelines) with taxonomy and; N: 263 primary studies (185 peer-reviewed, 78 arXiv preprints)	strong	
RQ2	Widely used benchmarks are likely in LLM pre-training corpora (data leakage), so high scores may reflect memorisation	[kaniewski2026systematic, p. 30; kaniewski2026systematic, p. 38; kaniewski2026systematic, p. 39]	AUTHOR-CLAIM	Systematic literature review (Kitchenham guidelines) with taxonomy and; N: 263 primary studies (185 peer-reviewed, 78 arXiv preprints)	moderate	
RQ2	Many LLM configurations behave as "always vulnerable" (P 0.50, R 1.00, F1 0.67) on 50/50 data, so F1 ≈ trivial baseline	[khare2025understanding, p. 16]	INTERPRETATION	Empirical study, 16 prompted LLMs x 3 prompts vs CodeQL (synthetic onl; N: 5,000 function-level snippets (1,000 per dataset, 500 vulner	moderate	
RQ2	CodeQL evaluated only on synthetic datasets; best	[khare2025understanding, p. 5; khare202	VERIFIED	Empirical study, 16 prompted LLMs x 3 prompts vs	moderate	

	LLM configuration chosen post hoc from 16 models × 3 prompts	5understanding, p. 7; khare2025understanding, p. 8]		CodeQL (synthetic onl; N: 5,000 function-level snippets (1,000 per dataset, 500 vulner		
RQ2	New benchmark CWE-Bench-Java: 120 compilable, manually vetted real-world Java CVEs, avg 300K SLOC	[li2024iris, p. 7; li2024iris, p. 19]	VERIFIED	Tool proposal + benchmark evaluation (LLM-inferred taint specs + CodeQ; N: 120 vulnerabilities (one per project)	moderate	
RQ2	Specialized LLM detectors show much lower recall than in their original papers; authors attribute this to narrow original evaluations (e.g. RepoAudit on 4 intra-function Linux cases)	[li2026llm, p. 7]	AUTHOR-CLAIM	comparative empirical study; N: 265 known vulnerabilities (110 C/C++, 155 Java)	moderate	
RQ2	Real-world TP/FP labels come from Codex (94.36% accuracy on 621 validated warnings), a tool that is also evaluated	[li2026llm, p. 11]	VERIFIED	comparative empirical study; N: 265 known vulnerabilities (110 C/C++, 155 Java)	moderate	
RQ2	Tension with guo2025repoaudit: RepoAudit 40 TP / 11 FP and $2.54 avg per project there vs 0/110 known C/C++ vulns, 92.77% SFDR and $172.77 avg ($1,247.66 max) here; targets, commits, bug-type	[li2026llm, p. 7; li2026llm, p. 11; li2026llm, p. 15; li2026llm, p. 16]	INTERPRETATION	comparative empirical study; N: 265 known vulnerabilities (110 C/C++, 155 Java)	weak	

	mapping, scale, metric and labeller differ (see §9)					
RQ2	In-distribution DL F1 is high (LineVul 94.52 Juliet, 91.00 BigVul; VulBERTa 94.14 Juliet) but drops cross-dataset (LineVul-BigVul → 0 / 1.90 / 13.05; LineVul-Juliet → 53.95 / 10.73 / 51.22)	[lu2025lab, p. 11; lu2025lab, p. 12]	VERIFIED	empirical evaluation; within/cross-dataset DL training + temporally se; N: 4 DL models x 4 datasets; VentiVul 13,326 functions, 200 CVEs, 263 vul	moderate	peer-reviewed (JSS 2026); full 4×4 train/test matrix with deduplication and fixed test sets (p. 7); but a single study, 4 models, DL run count not reported; direction consistent with ding2025vulnerability and chakraborty2024revisiting; published JSS 2026 (doi 10.1016/j.jss.2026.113185); pages = arXiv v4 accepted manuscript
RQ2	High accuracy hides failure at 6% prevalence: BigVul accuracy 87.45/94.09/94.39 with F1 23.58/16.90/0.19 (ReVeal/Devign/VulBERTa)	[lu2025lab, p. 11]	VERIFIED	empirical evaluation; within/cross-dataset DL training + temporally se; N: 4 DL models x 4 datasets; VentiVul 13,326 functions, 200 CVEs, 263 vul	moderate	table values from a peer-reviewed study; the accuracy–F1 gap follows arithmetically from the 6% class ratio; single study; published JSS 2026 (doi 10.1016/j.jss.2026.113185); pages =

						arXiv v4 accepted manuscript
RQ2	High CVE-level coverage does not mean patch sensitivity: CDR 94.59–100% with PACC 0–0.76% for Devign/ICVul-trained models; all DL PACC < 5.5%	[lu2025lab, p. 13]	VERIFIED	empirical evaluation; within/cross-dataset DL training + temporally se; N: 4 DL models x 4 datasets; VentiVul 13,326 functions, 200 CVEs, 263 vul	moderate	directly in Table 7 of a peer-reviewed study; PACC uses manually verified pairs (p. 17); single study, 263 pairs; published JSS 2026 (doi 10.1016/j.jss.2026.113185); pages = arXiv v4 accepted manuscript
RQ2	Prompting effects are model- and metric-dependent: few-shot helps only Mistral; CoT raises Qwen2.5 valid-only F1 but lowers PACC	[lu2025lab, p. 13]	VERIFIED	empirical evaluation; within/cross-dataset DL training + temporally se; N: 4 DL models x 4 datasets; VentiVul 13,326 functions, 200 CVEs, 263 vul	weak	4 models × 3 prompts, one fixed demonstration set, single run; differences are a few points; published JSS 2026 (doi 10.1016/j.jss.2026.113185); pages = arXiv v4 accepted manuscript
RQ2	Benchmark gains are inflated by duplicates, label noise, random splits, temporal leakage and model-version changes; random-split F1 =	[qin2026review, p. 1; qin2026review, p. 10]	AUTHOR-CLAIM	narrative (non-systematic) literature review, single author; no search; N: 32 references (no stated corpus of primary studies)	weak	narrative review; narrative review; not independent of ding/gnieciak/sheng

	"within-benchmark performance" only					
RQ2	Re-reports PrimeVul Table V: five code LMs F1 62.88–68.26% on BigVul vs 1.91–5.82% BigVul→PrimeVul vs 18.05–21.43% PrimeVul→PrimeVul; all ranges match the primary source [ding2025vulnerability, p. 7] — cite ding directly	[qin2026review, p. 12]	AUTHOR-CLAIM	narrative (non-systematic) literature review, single author; no search; N: 32 references (no stated corpus of primary studies)	weak	secondary re-report; not independent of ding2025vulnerability; narrative review; not independent of ding/gnieciak/sheng
RQ2	LLM methods (GRACE, DLAP, context enhancement, M2CVD) cannot be ranked because datasets, model versions and protocols differ	[qin2026review, p. 12]	AUTHOR-CLAIM	narrative (non-systematic) literature review, single author; no search; N: 32 references (no stated corpus of primary studies)	weak	narrative review; not independent of ding/gnieciak/sheng
RQ2	88% of 81 ML4VD papers (2020–2024, top venues) frame detection as function-level binary classification; 53/81 use BigVul, Devign or ReVeal	[risse2025top, p. 5]	VERIFIED	Literature survey (81 ML4VD papers) + manual expert labelling study + ; N: 300 vulnerable-labelled + 90 secure-labelled functions manua	moderate	
RQ2	Only 39% (BigVul), 47% (Devign) and 65% (DiverseVul) of sampled vulnerable-labelled functions are actually vulnerable (n=100 each)	[risse2025top, p. 8]	VERIFIED	Literature survey (81 ML4VD papers) + manual expert labelling study + ; N: 300 vulnerable-labelled + 90 secure-labelled functions manua	moderate	
RQ2	All 151 truly	[risse202	VERIFI	Literature	moderat	

	vulnerable functions required context beyond the function (mainly external functions 43–52% and arguments 40–47%)	5top, p. 9]	ED	survey (81 ML4VD papers) + manual expert labelling study + ; N: 300 vulnerable-labell ed + 90 secure-labelled functions manua	e	
RQ2	A word-count Gradient Boosting classifier reaches F1 0.86 on BigVul (SOTA 0.96) and 0.62 on Devign (SOTA 0.68), but only 0.11 on DiverseVul (SOTA 0.48)	[risse202 5top, p. 11]	VERIFI ED	Literature survey (81 ML4VD papers) + manual expert labelling study + ; N: 300 vulnerable-labell ed + 90 secure-labelled functions manua	moderat e	
RQ2	About 40 studies (83% as stated) evaluate isolated code snippets, where LLM performance exceeds real-world detection performance	[sheng20 25llms, p. 20]	AUTHO R-CLAI M	Narrative (non-systematic) survey with keyword-driven, iterative venue; N: 58 highly relevant studies (from ~500-600 screened	weak	
RQ2	Existing datasets are narrowly scoped, mislabelled and affected by data leakage that inflates metrics; repository-level datasets are scarce (2 of 42 catalogued)	[sheng20 25llms, p. 3; sheng202 5llms, p. 13; sheng202 5llms, p. 25]	AUTHO R-CLAI M	Narrative (non-systematic) survey with keyword-driven, iterative venue; N: 58 highly relevant studies (from ~500-600 screened	moderat e	
RQ2	The same model family scores very differently by benchmark (e.g. CodeBERT 0.099 F1 on PrimeVul vs 0.66 on Choi2017, second-hand)	[sheng20 25llms, p. 19]	AUTHO R-CLAI M	Narrative (non-systematic) survey with keyword-driven, iterative venue; N: 58 highly relevant studies (from ~500-600	weak	

				screened		
RQ2	Under realistic imbalance (~17% vulnerable), PLMs outperform the best LLM (UniXcoder F1 0.4626 / CodeT5P MCC 0.3627 vs GPT-4o ITFSP F1 0.4010 / MCC 0.2774) — conclusion flips with class ratio	[shu2026 evaluating, p. 40; shu2026e valuating, p. 41]	VERIFIED	Empirical comparison of 8 fine-tuned/embedding PLMs vs 5 LLMs (zero-sh; N: Function level 20,165 functions (test 2,026, balanced)	moderate	
RQ2	Time-split (2024-2025) GPT-4o ITFSP keeps function-level Acc 0.7874 but line-level F1 drops to 0.3450 from 0.6641	[shu2026 evaluating, p. 39]	VERIFIED	Empirical comparison of 8 fine-tuned/embedding PLMs vs 5 LLMs (zero-sh; N: Function level 20,165 functions (test 2,026, balanced)	moderate	
RQ2	Random function-level split with paired vulnerable/fixed twins and BM25 few-shot retrieval from train; only exact-match dedup — possible leakage favouring ITFSP. Counter-evidence: time-split function-level Acc stays 0.7874 (retrieval pool for that run not stated); line-level drop to 0.3450 still compatible	[shu2026 evaluating, p. 6; shu2026e valuating, p. 7; shu2026e valuating, p. 12; shu2026e valuating, p. 38; shu2026e valuating, p. 39; shu2026e valuating, p. 44]	INTERPRETATION	Empirical comparison of 8 fine-tuned/embedding PLMs vs 5 LLMs (zero-sh; N: Function level 20,165 functions (test 2,026, balanced)	weak	
RQ2	Line-level dummy precision 0.0267 vs ~14% vulnerable lines in	[shu2026 evaluating, p. 8; shu2026e valuating,	INTERPRETATION	Empirical comparison of 8 fine-tuned/embedding PLMs vs 5	weak	

	dataset suggests metrics over padded 113-line slots, inflating accuracy/FPR	p. 22; shu2026evaluating, p. 23]		LLMs (zero-sh; N: Function level 20,165 functions (test 2,026, balanced)		
RQ2	Dual-agent design reaches the highest F1 (0.634) by flagging 88.2% of functions (FPR 0.880) and has the worst pairwise accuracy (P-C 6.2)	[tan2026think, p. 7]	VERIFIED	controlled factorial empirical study; N: 870 functions (435 pairs) x 64 configurations	moderate	
RQ2	F1 on balanced data and pairwise correctness rank designs differently: MA best on P-C in 11/16 settings, DA in 0/16 despite best F1	[tan2026think, p. 7; tan2026think, p. 8]	VERIFIED	controlled factorial empirical study; N: 870 functions (435 pairs) x 64 configurations	moderate	
RQ2	Plugin-grouped folds still had exact-duplicate groups spanning folds: 166 / 151 groups holding 1086 (A) / 676 (B) records (test-side share not reported; authors call them test copies, p. 10); 730/456 groups had contradictory labels; conflict removal + dedup + clone collapse cut the corpus by 50.4% / 19.3% (collapse alone 11,123 / 3714 records)	[tashenova2026real, p. 4; tashenova2026real, p. 5]	VERIFIED	empirical benchmark; plugin-grouped 5-fold CV (de-duplicated, clone-co; N: 15,294 (Variant A) and 24,917 (Variant B) PHP functions after leakage	moderate	
RQ2	A one-feature token-count	[tashenova2026real, p. 8]	VERIFIED	empirical benchmark;	moderate	

	baseline matches or beats TF-IDF RF in 4/6 settings; on Variant B binary it scores 0.572 vs Bi-LSTM 0.573 and best neural (uni-LSTM) 0.578			plugin-grouped 5-fold CV (de-duplicated, clone-co; N: 15,294 (Variant A) and 24,917 (Variant B) PHP functions after leakage		
RQ2	Expert review: only 48.8% of cleaned positive diff-derived labels are genuine (73.0% HIGH, 44.7% MEDIUM, 0.7% LOW; single annotator)	[tashenova2026real, p. 11]	VERIFIED	empirical benchmark; plugin-grouped 5-fold CV (de-duplicated, clone-co; N: 15,294 (Variant A) and 24,917 (Variant B) PHP functions after leakage	moderate	n=400, 1 annotator
RQ2	In-sample stacking inflates macro-F1 by +1.27 pp on average (range −1.62 to +2.95)	[tashenova2026real, p. 10]	VERIFIED	empirical benchmark; plugin-grouped 5-fold CV (de-duplicated, clone-co; N: 15,294 (Variant A) and 24,917 (Variant B) PHP functions after leakage	moderate	
RQ2	Re-scored vs trivial predictors: always-vulnerable macro-F1 ≈0.316 (B) / 0.427 (A, = majority); best detectors beat majority by +0.251 (B) / +0.136 (A) and always-vulnerable by +0.285 (B); MCC ≤0.203 (Variant B only; A not reported)	[tashenova2026real, p. 5; tashenova2026real, p. 8]	INTERPRETATION	empirical benchmark; plugin-grouped 5-fold CV (de-duplicated, clone-co; N: 15,294 (Variant A) and 24,917 (Variant B) PHP functions after leakage	moderate	our pooled calculation from Table 2 counts vs fold-mean scores
RQ2	Abstract "over 92%" vs results	[xiong2026sifting, p. 1;	INTERPRETATI	Comparative empirical study:	weak	

	98.3% initial FPR; reduction reported as pp, described as %	xiong2026sifting, p. 12; xiong2026sifting, p. 19]	ON	3 LLM agent frameworks x 3 backbones vs v; N: OWASP 2,740 cases (1,325 negatives / 1,415 positives)		
RQ2	F1 is distribution-sensitive: GPT-4o-mini F1 drops from ~49–65% (balanced pairs) to 5.41–12.21% on non-paired PrimeVul (695 vul / 25,216 ben)	[yildiz2025benchmarking, p. 6; yildiz2025benchmarking, p. 7]	VERIFIED	benchmark paper + comparative evaluation of plain LLM, dependency-aug m; N: 1,758 paired samples (879 vulnerable, 879 benign) from 879 C	moderate	
RQ2	LLM methods label >90% of samples vulnerable in some settings, with precision ~50%, inflating F1	[yildiz2025benchmarking, p. 7]	VERIFIED	benchmark paper + comparative evaluation of plain LLM, dependency-aug m; N: 1,758 paired samples (879 vulnerable, 879 benign) from 879 C	moderate	
RQ2	With DeepSeek-Coder-V2, Dep-Aug LLM reaches F1 82.09 / pAcc 69.24, contradicting the agent advantage seen with GPT models	[yildiz2025benchmarking, p. 15]	VERIFIED	benchmark paper + comparative evaluation of plain LLM, dependency-aug m; N: 1,758 paired samples (879 vulnerable, 879 benign) from 879 C	weak	
RQ2	Repo-level task with 0.008–0.18% vulnerable functions vs 4.5–9.2% in function-level datasets;	[zhou2024comparison, p. 3; zhou2024comparison, p. 5]	VERIFIED	Empirical comparison, 15 SAST tools vs 12 open-source LLMs (fine-tuned; N: 270 CVEs in	weak	preprint (strength downgraded)

	precision/F1 not computed, marked-function ratio used as FPR proxy			134 repos (87 Java, 85 C, 98 Python)		
RQ2	Test sets of roughly 8–10 CVEs per language (inferred from 12.5/11.1/10-point steps in Table 3); no variance reported	[zhou2024comparison, p. 6]	INTERPRETATION	Empirical comparison, 15 SAST tools vs 12 open-source LLMs (fine-tuned; N: 270 CVEs in 134 repos (87 Java, 85 C, 98 Python)	weak	preprint (strength downgraded)
RQ2	Most LLM detection studies work at function level (7 at line level, 0 at class level, 1 at repo level)	[zhou2025large, p. 17; zhou2025large, p. 18]	VERIFIED	Systematic literature review (manual top-venue search + 7-database aut; N: 58 primary studies (40 detection studies in Table 2, 19 repa	moderate	
RQ2	Detection datasets mostly use heuristic labels from vulnerability-fixing commits, which prior work has shown to be noisy	[zhou2025large, p. 18]	AUTHOR-CLAIM	Systematic literature review (manual top-venue search + 7-database aut; N: 58 primary studies (40 detection studies in Table 2, 19 repa	moderate	
RQ2	Benchmarks may overlap with LLM pre-training corpora (data contamination); contamination-free benchmarks are recommended	[zhou2025large, p. 20]	AUTHOR-CLAIM	Systematic literature review (manual top-venue search + 7-database aut; N: 58 primary studies (40 detection studies in Table 2, 19 repa	moderate	

RQ2/RQ3	Post-cutoff C/C++ (50 alerts): SWE-agent F1 95.5% vs vanilla 51.6%, LLM4SA-style 70.8%	[xiong2026sifting, p. 15]	VERIFIED	Comparative empirical study: 3 LLM agent frameworks x 3 backbones vs v; N: OWASP 2,740 cases (1,325 negatives / 1,415 positives)	weak	
RQ2/RQ5	Data contamination of public CVEs is not addressed	[li2024iris, p. 7; li2024iris, p. 10]	INTERPRETATION	Tool proposal + benchmark evaluation (LLM-inferred taint specs + CodeQ; N: 120 vulnerabilities (one per project)	weak	
RQ3	Multi-agent pipeline with open 22-33B models costs ~5-8 min per function on 4 x L40S	[ahmed2026secvuleval, p. 17]	VERIFIED	Benchmark construction + empirical evaluation of 5 LLMs in a multi-age; N: 25,440 functions (10,998 vulnerable / 14,442 non-vulnerable	moderate	
RQ3	Cost per task: open USD 0.00004–0.00048 vs frontier 0.00139–0.00448; Gemma-3-4B matches Claude-Sonnet-5 quality at 1/66 cost; reasoning costs 4.5–12.1× more	[deininger2026open, p. 7; deininger2026open, p. 17; deininger2026open, p. 18]	VERIFIED	empirical benchmark; zero-shot prompting of 8 LLMs (3 frontier, 5 open; N: 1549 functions (549 vulnerable, 35.4%); 8 LLMs x several configuration	moderate	list prices, provider-dependent up to 14×
RQ3	Cost and energy per true positive at 1:44: USD 0.003 (Gemma) to 0.212 (Claude); 3.7 to 119.6 kJ as reported, with	[deininger2026open, p. 7; deininger2026open, p. 8; deininger2026open, p. 14]	VERIFIED	empirical benchmark; zero-shot prompting of 8 LLMs (3 frontier, 5 open; N: 1549 functions (549	moderate	compute only; mixed energy provenance

	Gemma, Qwen and Llama kJ/TP computed from FLOP estimates (64, 40, 944 J), not measured energy; human triage of 25.6–41.9 FP per TP not costed			vulnerable, 35.4%); 8 LLMs x several configuration		
RQ3	Re-computed with measured concurrency-1 energy, kJ/TP ≈ 4.9 (Gemma), 8.8 (Qwen), 34.9 (Llama), still far below frontier 78.3–119.6 (estimated)	[deininger2026open, p. 8; deininger2026open, p. 14]	INTERPRETATION	empirical benchmark; zero-shot prompting of 8 LLMs (3 frontier, 5 open; N: 1549 functions (549 vulnerable, 35.4%); 8 LLMs x several configuration	moderate	our calculation from Table 5 Found and Tables 8–9
RQ3	Energy per task: measured 88 J (Qwen3-Coder-30B) and 738 J (Llama-3.3-70B); frontier FLOP-estimated 1332–2489 J (range 333–9955); batching cuts measured energy 10–22×	[deininger2026open, p. 7; deininger2026open, p. 14]	VERIFIED	empirical benchmark; zero-shot prompting of 8 LLMs (3 frontier, 5 open; N: 1549 functions (549 vulnerable, 35.4%); 8 LLMs x several configuration	weak–moderate	frontier estimated
RQ3	Hybrid (LLM4SA, LLM4PFA; accuracy 0.86–0.94) beats prompting-only LLMs (accuracy 0.40–0.72) on the same alarms	[du2026reducing, p. 8]	VERIFIED	Industrial empirical study at Tencent: interviews + offline evaluation; N: 433 alarms (328 FP, 105 TP)	moderate	
RQ3	Cost per alarm: LLM4PFA 12.6–58.3 s, $0.0025–$0.12; all LLM methods 2.1–109.5 s, $0.0011–$0.12 vs	[du2026reducing, p. 6; du2026reducing, p. 8]	VERIFIED	Industrial empirical study at Tencent: interviews + offline evaluation; N: 433 alarms (328	moderate	

	10–20 min manual			FP, 105 TP)		
RQ3	Combining ESBMC and CodeQL raises the CASTLE score to 814 (+23% vs ESBMC); overlapping tools lose score through extra FPs; no LLM+tool hybrid tested	[dubniczky2025castle, p. 10; dubniczky2025castle, p. 13]	VERIFIED	Benchmark paper; N: 250 programs (150 vulnerable, 100 non-vulnerable), 10,392 li	weak	
RQ3	Reasoning LLMs (o1, o3-mini) produce fewer FPs than GPT-4o/4o-mini; **[!]** not true for DeepSeek R1 (166 FP vs GPT-4o 116), so the claim holds for the OpenAI o-series only	[dubniczky2025castle, p. 11; dubniczky2025castle, p. 14]	AUTHOR-CLAIM	Benchmark paper; N: 250 programs (150 vulnerable, 100 non-vulnerable), 10,392 li	weak	
RQ3	Runtime per project: CodeQL ~186–213 s, SonarQube ~30–64 s, Snyk ~14–29 s, GPT-4.1 ~3.5–18 s, Mistral ~10–67 s, DeepSeek ~6–30 s	[gnieciak2025large, p. 9]	VERIFIED	Head-to-head benchmark, 3 SAST tools (SonarQube, CodeQL, Snyk Code) vs; N: 10 projects, 63 vulnerabilities, 3-10 files and 3,145-5,317	weak	
RQ3	Authors recommend a hybrid: LLMs for early broad triage, deterministic SAST for high-assurance audits (no hybrid tested)	[gnieciak2025large, p. 1; gnieciak2025large, p. 11; gnieciak2025large, p. 12]	AUTHOR-CLAIM	Head-to-head benchmark, 3 SAST tools (SonarQube, CodeQL, Snyk Code) vs; N: 10 projects, 63 vulnerabilities, 3-10 files and 3,145-5,317	weak	
RQ3	LLMs always mislocate	[gnieciak2025large, p. 11;	AUTHOR-CLAI	Head-to-head benchmark, 3	weak	

	line/column and hallucinate outdated-dependency findings, limiting explainability and SARIF integration	[gnieciak2025large, p. 12]	M	SAST tools (SonarQube, CodeQL, Snyk Code) vs; N: 10 projects, 63 vulnerabilities, 3-10 files and 3,145-5,317		
RQ3	Average cost $2.54, 0.44 h and ~615K input tokens per project; $0.95 per true bug; o3-mini $0.30, DeepSeek R1 $0.57	[guo2025repoaudit, p. 7; guo2025repoaudit, p. 8; guo2025repoaudit, p. 16; guo2025repoaudit, p. 17]	VERIFIED	tool paper with empirical evaluation; N: 15 benchmark projects (21 previously reported bugs)	moderate	
RQ3	Cost contradiction: li2026llm measures RepoAudit (Sonnet 3.5) at avg $172.77, max $1,247.66 and >800 min per project vs $2.54 / 0.44 h here; different projects and sizes (avg 251 KLoC here vs Linux modules up to 3.9M LOC, gpac, bitlbee in li)	[guo2025repoaudit, p. 7; guo2025repoaudit, p. 8]	INTERPRETATION	tool paper with empirical evaluation; N: 15 benchmark projects (21 previously reported bugs)	weak	
RQ3	Validator removal raises FPs by 245.45%; abstraction removal cuts TPs by 47.50% and raises FPs by 181.82%	[guo2025repoaudit, p. 8; guo2025repoaudit, p. 15]	VERIFIED	tool paper with empirical evaluation; N: 15 benchmark projects (21 previously reported bugs)	weak	
RQ3	Compilation-dependent analyzers fail on some projects (2 build failures, 5 Infer	[guo2025repoaudit, p. 15]	VERIFIED	tool paper with empirical evaluation; N: 15 benchmark projects (21	moderate	

	crashes of 15); RepoAudit needs no build			previously reported bugs)		
RQ3	In LLM-based detectors, SAST output is used as auxiliary input, or the LLM verifies/refines static-analysis findings; studies that only post-process tool output are excluded from the SLR	[kaniewski2026systematic, p. 7; kaniewski2026systematic, p. 8; kaniewski2026systematic, p. 14; kaniewski2026systematic, p. 18]	VERIFIED	Systematic literature review (Kitchenham guidelines) with taxonomy and; N: 263 primary studies (185 peer-reviewed, 78 arXiv preprints)	moderate	
RQ3	Authors recommend SAST + LLM synergy (SAST detects then LLM filters false positives; LLM proposes then static analysis verifies) as a pathway to industrial adoption; not quantified	[kaniewski2026systematic, p. 42]	AUTHOR-CLAIM	Systematic literature review (Kitchenham guidelines) with taxonomy and; N: 263 primary studies (185 peer-reviewed, 78 arXiv preprints)	weak	
RQ3	CWE-DF (dataflow-style) prompt raises real-world F1 by up to 0.18; LLMs correctly identify sources/sinks/sanitizers in 16–18 of 20 inspected samples even when the verdict is wrong	[khare2025understanding, p. 5; khare2025understanding, p. 6]	VERIFIED	Empirical study, 16 prompted LLMs x 3 prompts vs CodeQL (synthetic onl; N: 5,000 function-level snippets (1,000 per dataset, 500 vulner	weak	
RQ3	Authors propose neuro-symbolic hybrids: LLMs infer specifications, static analysers reason over them	[khare2025understanding, p. 8; khare2025understanding, p. 11]	AUTHOR-CLAIM	Empirical study, 16 prompted LLMs x 3 prompts vs CodeQL (synthetic onl; N: 5,000	weak	

				function-level snippets (1,000 per dataset, 500 vulner		
RQ3	Hybrid gain depends on the LLM: only GPT-4 lowers FDR below CodeQL; L3 8B/DSC 7B FDR ~95%; LLM triage hurts small models	[li2024iris, p. 8; li2024iris, p. 10]	VERIFIED	Tool proposal + benchmark evaluation (LLM-inferred taint specs + CodeQ; N: 120 vulnerabilities (one per project)	moderate	
RQ3	Both LLM-inferred sources and sinks are needed (ablation 27 / 36 / 24 / 55)	[li2024iris, p. 10]	VERIFIED	Tool proposal + benchmark evaluation (LLM-inferred taint specs + CodeQ; N: 120 vulnerabilities (one per project)	moderate	
RQ3	Runtime 43 s to 4 h per project (Llama-3 8B); dollar cost not reported	[li2024iris, p. 22; li2024iris, p. 24]	VERIFIED	Tool proposal + benchmark evaluation (LLM-inferred taint specs + CodeQ; N: 120 vulnerabilities (one per project)	moderate	
RQ3	On real-world SFDR, IRIS (GPT-4, LLM+CodeQL) is worse than plain CodeQL on Java (97.80% vs 77.00%; IRIS GPT-5.4 78.73%) and KNighter (GPT-5.4) is worse than CSA/CodeQL (95.56% vs 85.51%/86.01%); KNighter (o3-mini) is lower at 81.25% but from 48 warnings on one	[li2026llm, p. 9; li2026llm, p. 11; li2026llm, p. 12]	VERIFIED	comparative empirical study; N: 265 known vulnerabilities (110 C/C++, 155 Java)	moderate	

	module (linux/drivers/net). INFERROI beats CodeQL/SpotBugs on CWE-772 recall (66% vs 10%/6%)					
RQ3	LLM cost per project: RepoAudit up to $1,247.66 and 225M input tokens; LLMDFA avg $61.60 and 2,000 min; Codex $5.38/14.8 min; SAST runs in minutes with no token cost	[li2026llm, p. 16]	VERIFIED	comparative empirical study; N: 265 known vulnerabilities (110 C/C++, 155 Java)	moderate	
RQ3	Recommends SAST candidates → LLM ranking/explanation → human confirmation; judge by audit time, false negatives, explanation consistency, not F1	[qin2026review, p. 8; qin2026review, p. 13]	AUTHOR-CLAIM	narrative (non-systematic) literature review, single author; no search; N: 32 references (no stated corpus of primary studies)	weak	no quantitative hybrid evidence; narrative review; not independent of ding/gnieciak/sheng
RQ3	Costs (API calls, GPU memory, latency, version drift, prompt maintenance) should be reported; none quantified	[qin2026review, p. 9; qin2026review, p. 12]	AUTHOR-CLAIM	narrative (non-systematic) literature review, single author; no search; N: 32 references (no stated corpus of primary studies)	weak	narrative review; not independent of ding/gnieciak/sheng
RQ3	Authors argue function-level static analysis faces the same context problem and propose combining static analysis with ML	[risse2025top, p. 12]	AUTHOR-CLAIM	Literature survey (81 ML4VD papers) + manual expert labelling study + ; N: 300 vulnerable-labelled + 90 secure-labelled functions manua	weak	

RQ3	Code-processing techniques (AST/CFG/DFG, RAG, slicing) are used in 41.3% of studies with only modest gains that shrink for cross-file vulnerabilities	[sheng2025llms, p. 16]	VERIFIED	Narrative (non-systematic) survey with keyword-driven, iterative venue; N: 58 highly relevant studies (from ~500-600 screened	weak	
RQ3	Neuro-symbolic approaches (CodeQL/Bear + LLM) show promise on large projects but need better taint modelling and efficiency	[sheng2025llms, p. 3]	AUTHOR-CLAIM	Narrative (non-systematic) survey with keyword-driven, iterative venue; N: 58 highly relevant studies (from ~500-600 screened	weak	
RQ3	GPT-4o instruction tuning costs $86.07 (function) vs ≈$5.59 per local A6000 PLM training run (~15x cheaper); authors recommend PLMs under privacy/limited hardware	[shu2026evaluating, p. 42]	VERIFIED	Empirical comparison of 8 fine-tuned/embedding PLMs vs 5 LLMs (zero-sh; N: Function level 20,165 functions (test 2,026, balanced)	moderate	
RQ3	Four-call self-revision matches or beats the multi-agent pipeline on P-C (24.7 vs 21.9) on 193 pairs: much of the agent gain is extra inference budget	[tan2026think, p. 11]	VERIFIED	controlled factorial empirical study; N: 870 functions (435 pairs) x 64 configurations	weak	
RQ3	Thinking uses 2.7–4.4× more energy and 2.5–3.6× more output tokens; MA-thinking uses ~25.5K tokens per	[tan2026think, p. 10]	VERIFIED	controlled factorial empirical study; N: 870 functions (435 pairs) x 64 configurations	moderate	

	function; per-configuration cost SA $27.5, DA $37.1, MA $52.7					
RQ3	Thinking's F1 benefit turns negative under DA (−0.054) and MA (−0.066)	[tan2026think, p. 8]	VERIFIED	controlled factorial empirical study; N: 870 functions (435 pairs) x 64 configurations	weak	
RQ3	Agent post-filtering cuts union-of-SAST FPR 98.3% → 6.3% (SWE-agent+Claude Sonnet 4) vs 23.0% vanilla Claude	[xiong2026sifting, p. 12]	VERIFIED	Comparative empirical study: 3 LLM agent frameworks x 3 backbones vs v; N: OWASP 2,740 cases (1,325 negatives / 1,415 positives)	moderate	
RQ3	Same best config wrongly filters 22.25% of true vulnerabilities (314/1,411)	[xiong2026sifting, p. 18]	VERIFIED	Comparative empirical study: 3 LLM agent frameworks x 3 backbones vs v; N: OWASP 2,740 cases (1,325 negatives / 1,415 positives)	moderate	
RQ3	Agent benefit is backbone-dependent: DeepSeek vanilla (11.2%) beats its agents (13.1–15.8%)	[xiong2026sifting, p. 12]	VERIFIED	Comparative empirical study: 3 LLM agent frameworks x 3 backbones vs v; N: OWASP 2,740 cases (1,325 negatives / 1,415 positives)	moderate	
RQ3	Cost per alert $0.0028–$0.1867, 11k–269k tokens depending on agent/model	[xiong2026sifting, p. 17]	VERIFIED	Comparative empirical study: 3 LLM agent frameworks x 3 backbones vs v; N: OWASP 2,740 cases	moderate	

				(1,325 negatives / 1,415 positives)		
RQ3	Union of SAST tools raises detection by 25.2–100% relative (e.g., Python S1 30.0% → 50.0%) at small marked-ratio cost (4.5% → 7.7%)	[zhou2024comparison, p. 8]	VERIFIED	Empirical comparison, 15 SAST tools vs 12 open-source LLMs (fine-tuned; N: 270 CVEs in 134 repos (87 Java, 85 C, 98 Python)	weak	preprint (strength downgraded)
RQ3	Voting over 12 fine-tuned LLMs cuts the marked ratio by 40.9–74.6%; >80% voting gives 0.8–9.2% marked but 0–37.5% S2 detection	[zhou2024comparison, p. 9]	VERIFIED	Empirical comparison, 15 SAST tools vs 12 open-source LLMs (fine-tuned; N: 270 CVEs in 134 repos (87 Java, 85 C, 98 Python)	weak	preprint (strength downgraded)
RQ3	No SAST+LLM hybrid is evaluated, although abstract/conclusion claim that ensembling SAST tools and LLMs mitigates drawbacks	[zhou2024comparison, p. 1; zhou2024comparison, p. 8; zhou2024comparison, p. 9; zhou2024comparison, p. 11]	INTERPRETATION	Empirical comparison, 15 SAST tools vs 12 open-source LLMs (fine-tuned; N: 270 CVEs in 134 repos (87 Java, 85 C, 98 Python)	weak	preprint (strength downgraded)
RQ3/RQ5	On real-world Java, all four GPT-5 settings (incl. vanilla) filter out ≈47–67% of the true vulnerabilities (≈14–19 TPs per configuration; derived from Table 4 percentages)	[xiong2026sifting, p. 13]	INTERPRETATION	Comparative empirical study: 3 LLM agent frameworks x 3 backbones vs v; N: OWASP 2,740 cases (1,325 negatives / 1,415 positives)	weak	
RQ4	All 5 LLMs detect short functions and miss long	[ahmed2026secvuleval, p. 13]	VERIFIED	Benchmark construction + empirical	moderate	

	ones (median 14-33 vs. 64-71 LoC; Mann-Whitney p ≤ 0.0168)			evaluation of 5 LLMs in a multi-age; N: 25,440 functions (10,998 vulnerable / 14,442 non-vulnerable		
RQ4	GPT-4.1 and Claude over-flag CWE-476, CWE-416 and CWE-190 (e.g. null deref despite explicit null check)	[ahmed2026secvuleval, p. 13]	AUTHOR-CLAIM	Benchmark construction + empirical evaluation of 5 LLMs in a multi-age; N: 25,440 functions (10,998 vulnerable / 14,442 non-vulnerable	weak	
RQ4	DL detectors detect information-leak and tainted-input CWE clusters best (F1 up to 33.6%) and path-traversal/predictability/misc clusters worst (none detected for SFP 907)	[chakraborty2024revisiting, p. 11]	VERIFIED	Replication + re-evaluation of 4 DL detectors (and 2 fine-tuned LLMs) ; N: 5,528 vulnerable functions (4,418 train / 1,110 test) + 1,68	moderate	
RQ4	High-severity vulnerabilities get the lowest F1 (9.06–10.89%) across all four models	[chakraborty2024revisiting, p. 11]	VERIFIED	Replication + re-evaluation of 4 DL detectors (and 2 fine-tuned LLMs) ; N: 5,528 vulnerable functions (4,418 train / 1,110 test) + 1,68	moderate	
RQ4	Best on Divide-by-Zero, worst on Null Pointer Dereference; failures tied to long functions and	[du2026reducing, p. 8; du2026reducing, p. 9]	VERIFIED	Industrial empirical study at Tencent: interviews + offline evaluation; N: 433 alarms (328	weak	

	many conditionals			FP, 105 TP)		
RQ4	FV tools miss non-formal CWEs (SQL injection, path traversal, hard-coded credentials) but have near-zero FPs; no per-CWE numbers reported	[dubniczky2025castle, p. 12]	AUTHOR-CLAIM	Benchmark paper; N: 250 programs (150 vulnerable, 100 non-vulnerable), 10,392 li	weak	
RQ4	LLM accuracy declines and FPs rise on a 400+ line merged program; most LLMs missed a single hidden vulnerability	[dubniczky2025castle, p. 14]	AUTHOR-CLAIM	Benchmark paper; N: 250 programs (150 vulnerable, 100 non-vulnerable), 10,392 li	weak	
RQ4	CWE-ID classification accuracy: ChatGPT 13–20% vs fine-tuned models 62–65% (C/C++, Big-Vul)	[fu2023chatgpt, p. 4]	VERIFIED	Short empirical comparison; N: ~188k C/C++ functions (Big-Vul) + 5.5k repair pairs	moderate	
RQ4	Evidence is for C# web-style vulnerabilities (SQLi, XSS, secrets, command injection, weak crypto, outdated deps); no per-category results	[gnieciak2025large, p. 5]	VERIFIED	Head-to-head benchmark, 3 SAST tools (SonarQube, CodeQL, Snyk Code) vs; N: 10 projects, 63 vulnerabilities, 3-10 files and 3,145-5,317	weak	
RQ4	Evaluated only on C/C++ memory bugs (CWE-476/401/416); call-chain depth limited to 4 functions	[guo2025repoaudit, p. 7; guo2025repoaudit, p. 9]	VERIFIED	tool paper with empirical evaluation; N: 15 benchmark projects (21 previously reported bugs)	moderate	
RQ4	Datasets are C/C++-biased and long-tailed (Top-25 CWEs ≈80% of labels);	[kaniewski2026systematic, p. 28; kaniewski2026syste	VERIFIED	Systematic literature review (Kitchenham guidelines) with taxonomy and;	strong	

	555 mappable CWEs are absent from all analysed datasets; logic/interaction pillars are under-represented	matic, p. 32; kaniewski 2026systematic, p. 34]		N: 263 primary studies (185 peer-reviewed, 78 arXiv preprints)		
RQ4	LLMs beat CodeQL on CWE-22, CWE-78, CWE-476, CWE-416 (e.g., +24% on CWE-416); CodeQL better on CWE-190 (+11–21%)	[khare2025understanding, p. 8]	VERIFIED	Empirical study, 16 prompted LLMs x 3 prompts vs CodeQL (synthetic onl; N: 5,000 function-level snippets (1,000 per dataset, 500 vulner	moderate	
RQ4	LLMs do well on self-contained CWEs (78, 476, 89, 79, Java 125/787) and poorly on context-dependent ones (CWE-862, C/C++ out-of-bounds with structs/pointers)	[khare2025understanding, p. 7]	VERIFIED	Empirical study, 16 prompted LLMs x 3 prompts vs CodeQL (synthetic onl; N: 5,000 function-level snippets (1,000 per dataset, 500 vulner	moderate	
RQ4	Per-CWE: CWE-94 0→8, CWE-79 4→13, CWE-22 22→31, CWE-78 1→3 (hard for all tools)	[li2024iris, p. 8]	VERIFIED	Tool proposal + benchmark evaluation (LLM-inferred taint specs + CodeQ; N: 120 vulnerabilities (one per project)	moderate	
RQ4	Vulnerabilities missed by general agents involve larger files; IRIS/INFERROI less affected by file size	[li2026llm, p. 9; li2026llm, p. 10]	VERIFIED	comparative empirical study; N: 265 known vulnerabilities (110 C/C++, 155 Java)	weak	
RQ4	Project-specific API modelling drives misses in	[li2026llm, p. 8]	VERIFIED	comparative empirical study; N: 265 known	moderate	

	both families; perfect API specs raise CWE-401 recall of CodeQL 6→56% and RepoAudit 0→48%			vulnerabilities (110 C/C++, 155 Java)		
RQ4	Results vary by project and CWE: LineVul-BigVul CDR 76% Chromium vs 0% Linux; all LLMs higher CDR on Linux (LLMs: zero-shot, valid-only, p. 13); best per-CWE PACC 18.18% (CWE-472) on small cells	[lu2025lab, p. 13; lu2025lab, p. 14]	VERIFIED	empirical evaluation; within/cross-dataset DL training + temporally se; N: 4 DL models x 4 datasets; VentiVul 13,326 functions, 200 CVEs, 263 vul	weak	per-project/per-CWE cells are small (100 CVEs per project; CWE counts only in Fig. 7) and there is a single run; published JSS 2026 (doi 10.1016/j.jss.2026.113185); pages = arXiv v4 accepted manuscript
RQ4	50% of studies target C/C++, 21.1% Java, 11.8% Solidity	[sheng2025llms, p. 11]	VERIFIED	Narrative (non-systematic) survey with keyword-driven, iterative venue; N: 58 highly relevant studies (from ~500-600 screened	weak	
RQ4	Memory-related vulnerabilities are detected more accurately than logic vulnerabilities (second-hand claim; the CWE-787 vs CWE-862 example is cited to a mismatched reference [36])	[sheng2025llms, p. 3; sheng2025llms, p. 22]	AUTHOR-CLAIM	Narrative (non-systematic) survey with keyword-driven, iterative venue; N: 58 highly relevant studies (from ~500-600 screened	weak	

RQ4	GPT-4o ITFSP per-language function Acc: Go 0.8082 (best) to Python 0.6626 (worst); CodeT5P beats it in JavaScript (0.7062 vs 0.6642)	[shu2026 evaluating, p. 21]	VERIFIED	Empirical comparison of 8 fine-tuned/embedding PLMs vs 5 LLMs (zero-sh; N: Function level 20,165 functions (test 2,026, balanced)	moderate	
RQ4	Line-level F1 by language: JavaScript 0.7815 best, C# 0.4348 worst (CodeT5P better in C#, n = 13)	[shu2026 evaluating, p. 8; shu2026evaluating, p. 26]	VERIFIED	Empirical comparison of 8 fine-tuned/embedding PLMs vs 5 LLMs (zero-sh; N: Function level 20,165 functions (test 2,026, balanced)	weak	
RQ4	Memory-safety CWEs are hardest to localise: line-level correct rate CWE-787 6.25%, CWE-119 21.74%, CWE-125 27.27% (best model)	[shu2026 evaluating, p. 30]	VERIFIED	Empirical comparison of 8 fine-tuned/embedding PLMs vs 5 LLMs (zero-sh; N: Function level 20,165 functions (test 2,026, balanced)	weak	
RQ4	LLM-only correct functions are more complex/longer (median CC 4.0 vs 2.0; 135.5 vs 96 tokens) than PLM-only correct ones; PLM handles deeper nesting	[shu2026 evaluating, p. 34]	VERIFIED	Empirical comparison of 8 fine-tuned/embedding PLMs vs 5 LLMs (zero-sh; N: Function level 20,165 functions (test 2,026, balanced)	weak	
RQ4	PHP per-class F1 (Variant B 4-class): XSS up to 0.500, SQLi ≤0.146, RCE ≤0.085; RF never predicts RCE	[tashenova2026real, p. 9]	VERIFIED	empirical benchmark; plugin-grouped 5-fold CV (de-duplicated, clone-co; N: 15,294 (Variant	weak	rare classes

				A) and 24,917 (Variant B) PHP functions after leakage		
RQ4	Injection CWEs filtered reliably (miss ≤2.38%); crypto/policy CWEs poorly (miss 50–84.5%)	[xiong2026sifting, p. 18]	VERIFIED	Comparative empirical study: 3 LLM agent frameworks x 3 backbones vs v; N: OWASP 2,740 cases (1,325 negatives / 1,415 positives)	moderate	
RQ4	Best approach differs by language: combined LLMs best for Java, combined SAST best for C and Python	[zhou2024comparison, p. 9]	AUTHOR-CLAIM	Empirical comparison, 15 SAST tools vs 12 open-source LLMs (fine-tuned; N: 270 CVEs in 134 repos (87 Java, 85 C, 98 Python)	weak	preprint (strength downgraded)
RQ4	CWE-119 best detected (LLM group 100%), CWE-476 second; CWE-835 and CWE-89 missed by both SAST and LLM groups	[zhou2024comparison, p. 8]	VERIFIED	Empirical comparison, 15 SAST tools vs 12 open-source LLMs (fine-tuned; N: 270 CVEs in 134 repos (87 Java, 85 C, 98 Python)	weak	preprint (strength downgraded)
RQ4	LLMs reportedly struggle more with less frequent CWE types and with inter-procedural and multi-unit vulnerabilities	[zhou2025large, p. 20]	AUTHOR-CLAIM	Systematic literature review (manual top-venue search + 7-database aut; N: 58 primary studies (40 detection studies in Table 2, 19 repa	weak	
RQ5	LLMs identify required context	[ahmed2026secvuleval, p. 9]	VERIFIED	Benchmark construction +	moderate	

	poorly (≤41.33% per type; function arguments ≤5.31%; environment context 0%)			empirical evaluation of 5 LLMs in a multi-age; N: 25,440 functions (10,998 vulnerable / 14,442 non-vulnerable		
RQ5	Authors call for evaluating static and dynamic analysis tools on Real-Vul (not done)	[chakraborty2024revisiting, p. 13]	AUTHOR-CLAIM	Replication + re-evaluation of 4 DL detectors (and 2 fine-tuned LLMs) ; N: 5,528 vulnerable functions (4,418 train / 1,110 test) + 1,68	weak	
RQ5	Authors: no method tested (LLM, SAST, fine-tuned DL) is ready for independent function-level triage	[deininger2026open, p. 26]	AUTHOR-CLAIM	empirical benchmark; zero-shot prompting of 8 LLMs (3 frontier, 5 open; N: 1549 functions (549 vulnerable, 35.4%); 8 LLMs x several configuration	moderate	
RQ5	Authors recommend broader inter-procedural context and hybrid LM + program-analysis systems	[ding2025vulnerability, p. 10]	AUTHOR-CLAIM	Dataset audit (label accuracy, duplication) + new benchmark (PrimeVul); N: PrimeVul 6,968 vulnerable + 228,800 benign functions (755 pr	weak	
RQ5	Recall below 90% enterprise threshold; LLMs can only filter, not find bugs beyond analyser coverage	[du2026reducing, p. 9; du2026reducing, p. 10]	AUTHOR-CLAIM	Industrial empirical study at Tencent: interviews + offline evaluation; N: 433 alarms (328 FP, 105 TP)	moderate	

RQ5	Open problems: LLM scalability to large codebases, line-level localisation, JSON reliability of small models, benchmark overfitting	[dubniczky2025castle, p. 8; dubniczky2025castle, p. 14]	AUTHOR-CLAIM	Benchmark paper; N: 250 programs (150 vulnerable, 100 non-vulnerable), 10,392 li	weak	
RQ5	Authors argue prompted LLMs lack security domain knowledge and need fine-tuning for vulnerability tasks	[fu2023chatgpt, p. 5]	AUTHOR-CLAIM	Short empirical comparison; N: ~188k C/C++ functions (Big-Vul) + 5.5k repair pairs	weak	
RQ5	Open problems: localisation precision, hallucinated findings, false-positive noise, prompt sensitivity	[gnieciak2025large, p. 11; gnieciak2025large, p. 12]	AUTHOR-CLAIM	Head-to-head benchmark, 3 SAST tools (SonarQube, CodeQL, Snyk Code) vs; N: 10 projects, 63 vulnerabilities, 3-10 files and 3,145-5,317	weak	
RQ5	Field study: 185 new bugs in 9 projects, 174 confirmed or fixed	[guo2025repoaudit, p. 8; guo2025repoaudit, p. 9]	AUTHOR-CLAIM	tool paper with empirical evaluation; N: 15 benchmark projects (21 previously reported bugs)	weak	
RQ5	Seven open limitation themes: granularity, label quality/leakage, comparability, outdated knowledge, representation robustness, explainability, workflow integration/cost	[kaniewski2026systematic, p. 37; kaniewski2026systematic, p. 40]	AUTHOR-CLAIM	Systematic literature review (Kitchenham guidelines) with taxonomy and; N: 263 primary studies (185 peer-reviewed, 78 arXiv preprints)	moderate	
RQ5	Few studies	[kaniewski2026syst	AUTHO	Systematic	moderat	

	evaluate in realistic development settings; models often underperform outside curated benchmarks	ematic, p. 39; kaniewski 2026systematic, p. 40]	R-CLAIM	literature review (Kitchenham guidelines) with taxonomy and; N: 263 primary studies (185 peer-reviewed, 78 arXiv preprints)	e	
RQ5	Missing global/inter-procedural context is the main failure cause on real-world C/C++; multi-file samples excluded	[khare2025understanding, p. 5; khare2025understanding, p. 7]	AUTHOR-CLAIM	Empirical study, 16 prompted LLMs x 3 prompts vs CodeQL (synthetic onl; N: 5,000 function-level snippets (1,000 per dataset, 500 vulner	moderate	
RQ5	Authors: many vulnerabilities still undetected; many LLM calls raise cost; Java only	[li2024iris, p. 10]	AUTHOR-CLAIM	Tool proposal + benchmark evaluation (LLM-inferred taint specs + CodeQ; N: 120 vulnerabilities (one per project)	moderate	
RQ5	63.33% of 5,896 FPs come from incorrect program-point classification and 14.60% from truncated interprocedural context, across LLM and SAST tools	[li2026llm, p. 15]	VERIFIED	comparative empirical study; N: 265 known vulnerabilities (110 C/C++, 155 Java)	moderate	
RQ5	Scalability (tokens, hours to days per project) remains a bottleneck for CI use of project-scale LLM detection	[li2026llm, p. 17; li2026llm, p. 18]	AUTHOR-CLAIM	comparative empirical study; N: 265 known vulnerabilities (110 C/C++, 155 Java)	moderate	

RQ5	Temporal separation by release date cannot exclude LLM training contamination; unrelated-function labels carry residual noise	[lu2025lab, p. 17]	AUTHOR-CLAIM	empirical evaluation; within/cross-dataset DL training + temporally se; N: 4 DL models x 4 datasets; VentiVul 13,326 functions, 200 CVEs, 263 vul	moderate	author-stated limitations that follow directly from undisclosed pretraining corpora and the labelling rule (p. 4, p. 17); untested, so magnitude unknown; published JSS 2026 (doi 10.1016/j.jss.2026.113185); pages = arXiv v4 accepted manuscript
RQ5	Open problems: out-of-time + project-isolated testing, data cards, uncertainty calibration, fixed-FP-budget recall, explanation verification, audit-time studies	[qin2026review, p. 13]	AUTHOR-CLAIM	narrative (non-systematic) literature review, single author; no search; N: 32 references (no stated corpus of primary studies)	weak	narrative review; not independent of ding/gnieciak/sheng
RQ5	Authors argue there is currently no evidence that ML4VD techniques can identify vulnerabilities at function level	[risse2025top, p. 2]	AUTHOR-CLAIM	Literature survey (81 ML4VD papers) + manual expert labelling study + ; N: 300 vulnerable-labelled + 90 secure-labelled functions manua	moderate	
RQ5	Main open challenges: limited problem scope, complex vulnerability semantics and context, LLM	[sheng2025llms, p. 20; sheng2025llms, p. 25]	AUTHOR-CLAIM	Narrative (non-systematic) survey with keyword-driven, iterative venue; N: 58 highly relevant studies	moderate	

	robustness and explanation consistency, lack of high-quality datasets			(from ~500-600 screened		
RQ5	Blinded explanation quality is indistinguishable between correct and incorrect predictions (all p > 0.3); fluent explanations do not signal correctness	[tan2026think, p. 9]	VERIFIED	controlled factorial empirical study; N: 870 functions (435 pairs) x 64 configurations	moderate	
RQ5	Authors call for repository-level evaluation with dependency resolution, retrieval and tools	[tan2026think, p. 11]	AUTHOR-CLAIM	controlled factorial empirical study; N: 870 functions (435 pairs) x 64 configurations	weak	
RQ5	Authors: label quality, not architecture, is the binding constraint; detectors should be required to clear trivial baselines	[tashenova2026real, p. 12; tashenova2026real, p. 13]	AUTHOR-CLAIM	empirical benchmark; plugin-grouped 5-fold CV (de-duplicated, clone-co; N: 15,294 (Variant A) and 24,917 (Variant B) PHP functions after leakage	moderate	
RQ5	Agents unsuitable for automatic suppression; human-in-the-loop needed	[xiong2026sifting, p. 19]	AUTHOR-CLAIM	Comparative empirical study: 3 LLM agent frameworks x 3 backbones vs v; N: OWASP 2,740 cases (1,325 negatives / 1,415 positives)	moderate	
RQ5	Most frequent failure is labelling both vulnerable and patched	[yildiz2025benchmarking, p. 8; yildiz2025benchmar	VERIFIED	benchmark paper + comparative evaluation of	moderate	

	versions vulnerable (40–95% of LLM failures); agents analyse pairs inconsistently	king, p. 9]		plain LLM, dependency-augm; N: 1,758 paired samples (879 vulnerable, 879 benign) from 879 C		
RQ5	High LLM marked-function ratios need filtering, repository context, better prompts or ensembles; repo-level evaluation recommended over function-level	[zhou2024comparison, p. 10]	AUTHOR-CLAIM	Empirical comparison, 15 SAST tools vs 12 open-source LLMs (fine-tuned; N: 270 CVEs in 134 repos (87 Java, 85 C, 98 Python)	weak	preprint (strength downgraded)
RQ5	Building a high-quality vulnerability detection benchmark remains an open challenge; a curated, living, manually verified test set is proposed	[zhou2025large, p. 19; zhou2025large, p. 22]	AUTHOR-CLAIM	Systematic literature review (manual top-venue search + 7-database aut; N: 58 primary studies (40 detection studies in Table 2, 19 repa	moderate	
RQ5	None of the 58 studies is integrated into developer workflows, and almost none supports developer interaction or explanations	[zhou2025large, p. 19]	VERIFIED	Systematic literature review (manual top-venue search + 7-database aut; N: 58 primary studies (40 detection studies in Table 2, 19 repa	moderate	
RQ5	Directions still under-explored as of Mar 2024: repo-level detection, LLM agents, external tool use, LLMs customised for	[zhou2025large, p. 22; zhou2025large, p. 23]	AUTHOR-CLAIM	Systematic literature review (manual top-venue search + 7-database aut; N: 58 primary studies (40	moderate	

	vulnerabilities			detection studies in Table 2, 19 repa		

Contradictions

<!-- Findings that disagree: which papers, what differs (data, method, population, metric), likely explanation (INTERPRETATION). Full discussion: reports/methodology-comparison.md §4. -->

- **C1 Fine-tuned vs GPT-4.** Fine-tuned AIBugHunter (F1 94%) and CodeBERT (87%) far exceed prompted gpt-4 (29%) on Big-Vul [fu2023chatgpt, p. 4], but GPT-4 CoT beats every fine-tuned model on pair-wise correctness on 564 PrimeVul pairs (12.94% vs ≤3.01%) [ding2025vulnerability, p. 9]. Differs in metric (F1 vs P-C), data (noisy random-split Big-Vul 1:20 vs de-duplicated time-split pairs) and model versions; Ding's own BigVul rows (F1 62.88–68.26%) agree with Fu [ding2025vulnerability, p. 7]. **[I]** Fine-tuned models exploit dataset artefacts rewarded by random splits [risse2025top, p. 11]; on pairs all are below the 22.70% random reference, so the reversal reflects the benchmark more than the model family. Pass 2: on 1549 unpaired PrimeVul functions a fine-tuned CodeBERT-PrimeVul beats all 8 zero-shot LLMs (balanced accuracy 0.765 vs ≤0.711) [deininger2026open, p. 11; deininger2026open, p. 17]; with an argmax decision rule the same detector looks like always-safe (0.500) despite ROC-AUC 0.845 [deininger2026open, p. 11]. **[I]** A conservative threshold is consistent with Ding's fine-tuned models labelling both pair members benign in 84–88% of pairs [ding2025vulnerability, p. 9]; unpaired and pair-wise results must not be pooled.
- **C2 RepoAudit.** Own paper: 40 TP / 11 FP (78.43% precision), all 21 known bugs reproduced, $2.54 and 0.44 h per project [guo2025repoaudit, p. 7; guo2025repoaudit, p. 8]. Independent study: 0/110 known C/C++ vulnerabilities [li2026llm, p. 7], 92.77% SFDR on real-world projects [li2026llm, p. 11], average $172.77 per project, max $1,247.66 [li2026llm, p. 16]. Differs in targets (prior tools' bug reports vs ReposVul CVEs), version (old commits vs current HEAD; the 5 overlapping modules only in li's real-world set), metric (precision vs recall/SFDR), labeller (authors vs Codex), bug types and scale. **[I]** Dependence on source/sink API coverage (zero-token runs; API modelling explains 108/108 misses [li2026llm, p. 7; li2026llm, p. 16]). Unresolved.
- **C3 LLM vs SAST.** LLMs win on 10 small C# projects [gnieciak2025large, p. 10] and CASTLE C micro-programs [dubniczky2025castle, p. 11]; CodeQL beats the best LLM's accuracy on OWASP, Juliet Java and Juliet C/C++ [khare2025understanding, p. 9]; at repository level LLMs detect more but flag up to 77.4% of functions [zhou2024comparison, p. 7] (preprint), and general agents lead recall while specialised LLM detectors and SAST show near-zero recall on some CWEs and 75–100% SFDR [li2026llm, p. 7; li2026llm, p. 11]. Differs in language, unit, synthetic vs real code, and scoring leniency (location rule only for LLMs [gnieciak2025large, p. 6]; CASTLE prompt reveals scoring [dubniczky2025castle, p. 9]). **[I]** Synthetic suites match SAST rules, small programs fit LLM context, real repositories defeat both: no single winner. Pass 2: on the same 1549 PrimeVul functions LLMs lead on balanced accuracy, but at 1:44 prevalence Flawfinder (6.2–13.4%) and Semgrep (26.0%, CI 5.7–66.9) are more precise than every LLM (2.3–4.6%), while Cppcheck sits

in the LLM range [deininger2026open, p. 8; deininger2026open, p. 10]. **[I]** The families fail in opposite directions (LLMs over-flag, SAST finds little), so the ranking depends on the metric.

- **C4 Does reasoning reduce false positives?** For: reasoning models "consistently produced fewer false positives" [dubniczky2025castle, p. 14]; RepoAudit precision 82.35–88.46% with reasoning backbones [guo2025repoaudit, p. 16]. Against: DeepSeek R1 166 FP vs GPT-4o 116 in CASTLE's own table [dubniczky2025castle, p. 11]; thinking raises F1 via recall with unchanged accuracy and turns negative under DA/MA [tan2026think, p. 6; tan2026think, p. 8]; reasoning LLMs near chance on REEF [shu2026evaluating, p. 37]. **[I]** Holds for OpenAI o-series in some settings only. Pass 2, against: reasoning lowers DeepSeek-V3.2 specificity from 0.52 to 0.37 [deininger2026open, p. 17; deininger2026open, p. 20], while it helps frontier models' balanced accuracy. **[I]** Model-dependent.
- **C5 Class balance flips rankings.** Balanced REEF: fine-tuned GPT-4o + few-shot beats CodeT5P (Acc 0.7196 vs 0.6037) [shu2026evaluating, p. 17]; imbalanced test: PLMs win (UniXcoder F1 0.4626 vs GPT-4o 0.4010) [shu2026evaluating, p. 40; shu2026evaluating, p. 41]. yildiz F1 drops from ~49–65% on balanced pairs to 5.41–12.21% on non-paired PrimeVul [yildiz2025benchmarking, p. 7]. **[I]** Balanced sets reward liberal flagging; balanced rankings do not transfer to deployment. Pass 2: deininger's precision ranking at 1:44 differs from its balanced-accuracy ranking [deininger2026open, p. 8]; in tashenova changing how negatives are drawn flips the majority class and the direction in which the same Random Forest fails [tashenova2026real, p. 9].
- **C6 IRIS hybrid.** IRIS+GPT-4 beats CodeQL on 120 Java CVEs (55 vs 27; AvgFDR 84.82% vs 90.03%) [li2024iris, p. 8], but on real-world Java IRIS SFDR is 97.80% vs CodeQL 77.00% [li2026llm, p. 12]. Not like-for-like (known CVEs vs new code; detections vs SFDR; labelling). **[I]** Hybrids may raise recall on benchmark CVEs while adding noise on unseen code. Open.
- **C7 Same model, same benchmark, different in-distribution score.** LineVul (CodeBERT) trained and tested on BigVul: F1 91.00 [lu2025lab, p. 12] vs CodeBERT BV→BV F1 62.88 [ding2025vulnerability, p. 7]. Neither paper explains the 28-point gap. **[I]** Likely split and preprocessing differences (lu used LineVul's original pipeline [lu2025lab, p. 8]); in-distribution BigVul numbers are not comparable across papers.
- **C8 Does a time-split fine-tuned detector survive?** CodeBERT-PrimeVul on PrimeVul's chronologically later test split keeps balanced accuracy 0.765 (F1 ≈ 0.15 at 1:44, our calculation) [deininger2026open, p. 11], while BigVul/ICVul/Devign/Juliet-trained DL on post-May-2025 VentiVul falls to F1 3.47–8.65 [lu2025lab, p. 13]. **[I]** Differences in dataset (VentiVul 2025 CVEs, Whole-File units), training data quality (PrimeVul vs BigVul) and threshold policy; unresolved.

Agreements / converging evidence

- **G1 Realistic evaluation collapses performance (strong).** DL precision falls to 1–10% on Real-Vul [chakraborty2024revisiting, p. 7]; code-LM F1 62.88–68.26% → 18.05–21.43% within PrimeVul and 1.91–5.82% cross-dataset [ding2025vulnerability, p. 7]; LLM accuracy 10.5% lower on real-world than synthetic [khare2025understanding, p. 5]; line-level F1 0.6641

→ 0.3450 on 2024–25 code [shu2026evaluating, p. 39]; statement-level scoring halves F1 [ahmed2026secvuleval, p. 8]. Surveys concur [kaniewski2026systematic, p. 39; sheng2025llms, p. 20]. Pass 2: on post-May-2025 CVEs at 1.97% prevalence, DL F1 3.47–8.65 and 7 of 12 LLM conservative F1 values fall below the all-positive F1 of 3.87% [lu2025lab, p. 13; lu2025lab, p. 14]; at 1:44, LLM precision 2.3–4.6% vs flag-all 2.21% [deininger2026open, p. 8; deininger2026open, p. 17]; on real WordPress PHP CVEs best macro-F1 0.601 [tashenova2026real, p. 8].

- **G2 Benchmark labels are noisy (strong).** Independent audits: 39–65% of vulnerable labels correct [risse2025top, p. 8]; 24–36% [ding2025vulnerability, p. 4]; ReVeal 15% inconsistency [chakraborty2024revisiting, p. 3]. (The 25% BigVul figure is Chen et al. via Ding — not independent.) Pass 2: only 48.8% of cleaned positive WordPress-PHP labels are genuine (single annotator, n = 400) [tashenova2026real, p. 11].
- **G3 Prompted LLMs over-flag and cannot separate vulnerable from patched code (moderate).** GPT-4 two-shot labels both pair members vulnerable in 71.63% of pairs [ding2025vulnerability, p. 9]; "both vulnerable" is 40–95% of failures [yildiz2025benchmarking, p. 8]; dual-agent flags 88.2% [tan2026think, p. 7]; always-vulnerable rows among khare's best F1 [khare2025understanding, p. 16]. Partly one dataset family (PrimeVul).
- **G4 SAST recall is low on real code, high only on synthetic suites (moderate).** Best single tools 30.0–44.4%, many 0% [zhou2024comparison, p. 6]; CodeQL recall 6.00–34.29% C/C++ and 14.19% Java [li2026llm, p. 7; li2026llm, p. 9]; CodeQL 27/120 on CWE-Bench-Java [li2024iris, p. 8]; CASTLE CodeQL 29%, SAST average 23% [dubniczky2025castle, p. 12]; vs CodeQL accuracy 0.94 on Juliet Java [khare2025understanding, p. 9].
- **G5 CodeQL on OWASP replicates across groups (moderate).** P 0.58 / R 0.96 / F1 0.72 [khare2025understanding, p. 9] vs P 60.3% / R 97.0% / F1 74.4% [xiong2026sifting, p. 11].
- **G6 Missing context is the shared failure cause (moderate).** All 151 truly vulnerable audited functions need external context [risse2025top, p. 9]; context-type identification ≤41.33% [ahmed2026secvuleval, p. 9]; API modelling explains 108/108 RepoAudit (GPT-5.4) misses [li2026llm, p. 7]; 14.60% of FPs from truncated inter-procedural context [li2026llm, p. 15].
- **G7 LLM filtering of SAST alarms trades large FP reduction for lost true positives (moderate).** du2026reducing removes 94–98% of false alarms with real-bug recall 0.75–0.88 [du2026reducing, p. 8] (own method, Tencent authors, non-security bugs); xiong2026sifting FPR 6.3% with 22.25% miss rate [xiong2026sifting, p. 12; xiong2026sifting, p. 18].
- **G8 Prompt-only LLMs trail adapted models (moderate).** Fine-tuned models beat prompted ChatGPT on Big-Vul [fu2023chatgpt, p. 4]; GPT-3.5 prompting detects 3.7–23.2% vs 89.2% fine-tuned CodeBERT [zhou2024comparison, p. 10]; prompt-only LLMs near chance on REEF [shu2026evaluating, p. 17].
- **G9 LLM cost far exceeds SAST and varies by orders of magnitude (moderate).** RepoAudit averages $172.77 per project [li2026llm, p. 16]; alarm filtering < $0.20 per alarm [du2026reducing, p. 8; xiong2026sifting, p. 17]; GPT-4o instruction tuning ~15× a PLM run [shu2026evaluating, p. 42].

- **G10 Trivial baselines are competitive (moderate).** A one-feature token-count baseline matches or beats TF-IDF RF in 4/6 settings and is within 0.006 of the best neural model [tashenova2026real, p. 8]; always-vulnerable F1 0.523 vs LLM F1 0.405–0.615 [deininger2026open, p. 7]; all-positive F1 3.87% vs most LLM F1 on VentiVul [lu2025lab, p. 14]; shu2026evaluating also reports all-vulnerable and all-clean dummy baselines [shu2026evaluating, p. 11]. **[I]** Few papers report such baselines; most headline numbers are never compared to one.
- **G11 Cost per true positive (moderate, single study).** USD 0.003 (Gemma-3-4B) to 0.212 (Claude Sonnet 5) and 3.7–119.6 kJ per true positive at 1:44, excluding human triage of 25.6–41.9 false positives per true one [deininger2026open, p. 8]; Qwen/Llama kJ/TP use FLOP estimates, not measured energy [deininger2026open, p. 14].